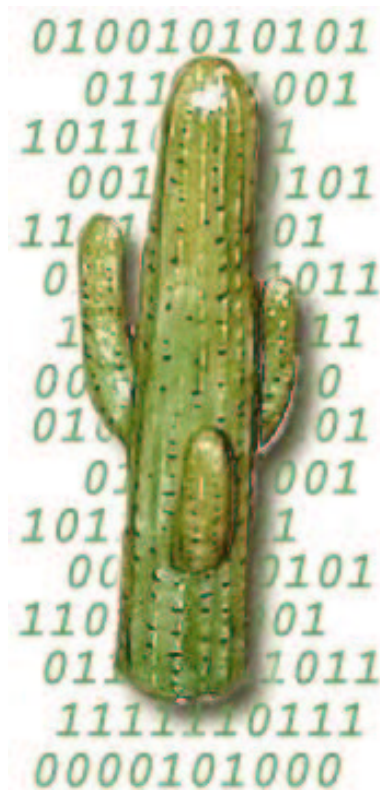

Cactus 4.0

Users' Guide



Revision: 1.41

Date: 2003/07/17 15:09:50

Contents

A	Installation and Running	A1
A1	Installation	A2
A1.1	Required software	A2
A1.2	Supported architectures	A3
A1.3	Checkout procedure	A4
A1.4	Directory structure	A4
A1.5	Getting help	A5
A2	Compilation	A6
A2.1	Creating a configuration	A6
A2.1.1	Configuration options	A6
A2.1.2	Available options	A7
A2.1.3	Compiling with extra packages	A9
A2.1.4	File layout	A11
A2.2	Building and Administering a configuration	A12
A2.2.1	gmake targets for building and administering configurations	A12
A2.2.2	Compiling in thorns	A13
A2.2.3	Notes and Caveats	A14
A2.2.4	gmake options for building configurations	A14
A2.3	Other gmake targets	A14
A2.4	Testing	A15
A3	Running Cactus	A16
A3.1	Command Line Options	A16
A3.2	Parameter File Syntax	A17
A3.3	Thorn Documentation	A18
A4	Getting and looking at output	A20
A4.1	Screen output	A20
A4.2	Output	A21
A4.3	Checkpointing/Recovery	A21
B	Application thorn writing	B1
B1	Overview	B2
B2	Thorn concepts	B3
B2.1	Thorns	B3
B2.2	Arrangements	B3
B2.3	Implementations	B3
B3	Anatomy of a thorn	B5
B3.1	Thorns	B5
B3.2	Creating a thorn	B5
B3.3	Configuring your thorn	B6
B3.3.1	General syntax of CCL files	B6

B3.3.2	The <code>interface.ccl</code>	B6
B3.3.3	The <code>param.ccl</code>	B7
B3.3.4	The <code>schedule.ccl</code>	B9
B3.4	Naming Conventions for Source Files	B12
B3.5	Adding source files	B12
B3.5.1	<code>make.code.defn</code> based thorn building	B13
B3.5.2	<code>Makefile</code> based thorn building	B13
B3.5.3	Other makefile variables	B13
B4	Cactus Variables	B14
B4.1	Data Type	B14
B4.2	Group Types	B15
B4.3	Timelevels	B15
B4.4	Size and Distrib	B16
B4.5	Ghost Size	B16
B4.6	Staggering	B17
B4.7	Information about Grid Variables	B18
B4.7.1	Group Information	B18
B5	Cactus Parameters	B20
B5.1	Types and Ranges	B20
B5.1.1	Integer	B20
B5.1.2	Real	B21
B5.1.3	Keyword	B21
B5.1.4	Boolean	B21
B5.1.5	String	B21
B5.2	Scope	B21
B6	Scheduling	B22
B6.1	Schedule Bins	B22
B6.2	Groups	B23
B6.3	Schedule Options	B23
B6.4	The Schedule Block	B23
B6.5	How Cactus Calls Scheduled Functions	B24
B7	Writing a Thorn	B25
B7.1	Thorn Programming Languages	B25
B7.2	What the Flesh provides	B25
B7.2.1	Fortran Routines	B25
B7.2.2	C Routines	B28
B7.2.3	Cactus Variables	B30
B7.2.4	Cactus Data Types	B31
B7.2.5	Staggering	B32
B7.3	Parallelisation	B34
B8	Cactus Application Interfaces	B35
B8.1	Coordinates	B35
B8.1.1	Registering Coordinates and Coordinate Properties	B35
B8.1.2	Using Coordinates	B36
B8.2	IO	B38
B8.3	Interpolation Operators	B39
B8.4	Reduction Operators	B40
B9	Completing a Thorn	B47
B9.1	Commenting Source Code	B47
B9.2	Providing Runtime Information	B47
B9.3	Error handling, warnings and code termination	B48
B9.4	Adding documentation	B49

B9.4.1	README	B49
B9.4.2	Contribution to ThornGuide	B50
B9.5	Adding a test suite	B51
B10	Advanced Thorn Writing	B52
B10.1	Using Cactus Timers	B52
B10.1.1	What are timers?	B52
B10.1.2	Timing calls	B52
B10.1.3	The <code>cTimerData</code> Structure	B53
B10.1.4	How to insert timers in your code	B53
B10.2	Include Files	B54
B10.3	Memory Tracing	B56
B10.3.1	Activating Memory Tracing	B56
B10.3.2	Using Memory Tracing	B56
B10.4	Calls to different language	B57
B10.4.1	Calling C routines from FORTRAN	B57
B10.4.2	String Arguments from Fortran	B58
B10.4.3	Calling FORTRAN routines from C	B60
B10.5	Function aliasing	B60
B10.5.1	Using an aliased function	B60
B10.5.2	Providing a function	B61
B10.5.3	Conventions and restrictions	B61
B10.5.4	Examples	B61
B10.6	Naming conventions	B62
B10.7	General Naming Conventions	B62
B10.8	Data Types and Sizes	B62
B10.8.1	Fortran Thorn Writers	B63
B11	Telling the makesystem what to do	B64
B11.1	Basic Recipe	B64
B11.2	Make Concepts	B64
B11.3	The Four Files	B64
B11.4	How your code is built	B64
C	Utility Routines	C1
C1	Introduction	C2
C2	Key/Value Tables	C3
C2.1	Motivation	C3
C2.2	The Basic Idea	C3
C2.3	A Simple Example	C4
C2.4	Arrays as Table Values	C5
C2.5	Character Strings	C6
C2.6	Convenience Routines	C6
C2.7	Table Iterators	C7
C2.8	Multithreading and Multiprocessor Issues	C7
C2.9	Metadata about All Tables	C8
D	Infrastructure Thorn Writer's Guide	D1
D1	Introduction	D2
D2	Concepts and Terminology	D3
D2.1	Overloading and Registration	D3
D2.1.1	Overloading	D3

D2.1.2	Registration	D3
D2.2	GH Extensions	D3
D2.3	IO Methods	D3
D3	GH Extensions	D4
D4	Overloadable and Registerable Functions in Main	D5
D5	Overloadable and Registerable Functions in Comm	D6
D6	Overloadable and Registerable Functions in IO	D7
D7	Adding a Driver	D8
D7.1	Anatomy	D8
D7.2	Startup	D8
D7.3	The GH Extension	D9
D7.4	Memory Functions	D12
D7.4.1	En/Disable Group Storage	D12
D7.4.2	En/Disable Group Comm	D13
D8	Adding an I/O Method	D14
D8.1	I/O Method Registration	D14
D8.2	Periodic Output of Grid Variables	D15
D8.3	Triggered Output of Grid Variables	D15
D8.4	Unconditional Output of Grid Variables	D15
D9	Adding a Checkpointing/Recovery Method	D16
D9.1	Invocation of Checkpointing Routines	D16
D9.2	Invocation of Recovery Routines	D16
D10	Adding a timer	D18
E	FunctionReference	E1
E1	Cactus Functions	E2
E1.1	Functions Alphabetically	E2
E1.2	Full Description of Functions	E8
E2	Utility Functions	E157
E2.1	Functions Alphabetically	E157
E2.1.1	String Functions	E157
E2.1.2	Table Functions	E157
E2.2	Full Descriptions of String Functions	E159
E2.3	Full Descriptions of Table Functions	E167
F	Appendices	F1
F1	Glossary	F2
F2	Configuration file syntax	F5
F2.1	General Concepts	F5
F2.2	interface.ccl	F5
F2.2.1	Header block	F5
F2.2.2	Include files	F6
F2.2.3	Function aliasing	F6
F2.2.4	Variable blocks	F7
F2.3	param.ccl	F8

F2.3.1	Parameter data scoping items	F8
F2.3.2	Parameter object specification items	F9
F2.4	schedule.ccl	F10
F2.4.1	Assignment Statements	F10
F2.4.2	Schedule Blocks	F10
F2.4.3	Conditional Statements	F12
F3	Schedule bins	F13
F4	Flesh parameters	F15
F4.1	Private parameters	F15
F4.2	Restricted parameters	F15
F5	Using GNATS	F17
F6	Using CVS	F18
F6.1	Essential CVS commands	F18
F6.2	CVS Options	F19
F6.3	CVS Examples	F20
F6.4	Checking out flesh and thorns with CVS	F21
F7	Using TAGS	F23
F7.1	TAGS with emacs	F23
F7.2	TAGS with vi	F24
F8	ThornLists	F25

Preface

This document will eventually be a complete guide to using and developing with the Cactus Code. However, it is currently under development, so please be patient if you can't find what you need. Please report omissions, errors, or suggestions to and of our contact addresses below, and we will try and fix them as soon as possible.

Overview of documentation

This guide covers the following topics

Part A: Installation and Running.

We give an overview of required hardware and software and talk you through the installation of a working Cactus tool kit. You will be able to verify the correct installation with our test suite technology.

Part B: Application Thorn Writing.

We introduce thorn concepts and describe all you need to know for creating, writing and maintaining application thorns. You will learn how to use the programming interface to take advantage of parallelism and modularity.

Part C: Utility Routines.

This part discusses low-level utility routines available in Cactus, such as key/value tables.

Part D: Infrastructure Thorn Writer's Guide.

In this more advanced part, we talk about user supplied infrastructure routines such as *additional output routines*, *drivers*, *etc.* [This part is not yet complete, and is currently under construction.]

Part E: Function Reference.

Here all Cactus flesh functions which are available to thorn writers in C and Fortran are described.

Part F: Appendices.

These contain a description of the Cactus Configuration Language, and other odds and ends, such as how to use GNATS or TAGS.

Other topics to be discussed in separate documents include:

Computational Thorn Guide

This will contain details about the arrangements and thorns making up the standard Cactus Computational Tool Kit.

Relativity Thorn Guide

This will contain details about the arrangements and thorns making up the Cactus Relativity Tool Kit, one of the major motivators, and still the driving force, for the Cactus Code.

Flesh Maintainers Guide

This will contain all the gruesome details about the inner workings of Cactus, for all those who want or need to expand or maintain the core of Cactus.

Typographical Conventions

Typewriter	Is currently used for everything you type, for program names, and code extracts.
< ... >	Indicates a compulsory argument.
[...]	Indicates an optional argument.
	Indicates an exclusive or.

How to Contact Us

Please let us know of any errors or omissions in this guide, as well as suggestions for future editions. These can be reported via our bug tracking system at www.cactuscode.org, or via email to cactusmaint@cactuscode.org. Alternatively, write to us at

The Cactus Team
Albert Einstein Institute
Max Planck Institute for Gravitational Physics
Am Mühlenberg 1
D-14476 Golm
Germany

Acknowledgements

Hearty thanks to all those who have helped with documentation for the Cactus Code. Special thanks to those who struggled with the earliest sparse versions of this guide and sent in mistakes and suggestions, in particular John Baker, Carsten Gundlach, Ginny Hudak-David, Sai Iyer, Paul Lamping, Nancy Tran and Ed Seidel.

Part A

Installation and Running

Chapter A1

Installation

A1.1 Required software

In general, Cactus *requires* the following set of software to function in single processor mode. Please refer to the architecture section [A1.2](#) for architecture specific items.

Perl5.0	Perl is used extensively during the Cactus thorn configuration phase. Perl is available for nearly all operating systems known to man and can be obtained at http://www.perl.org
GNU make	The make process works with the GNU make utility (referred to as gmake henceforth). While other make utilities may also work, this is not guaranteed. Gmake can be obtained from your favorite GNU site or from www.gnu.org
C	C compiler. For example, the GNU compiler. This is available for most supported platforms. Platform specific compilers should also work.
CPP	C Pre-processor. For example, the GNU CPP. These are normally provided on most platforms, and many C compilers have an option to just run as a preprocessor.
CVS	The “ <i>Concurrent Versioning System</i> ” is not needed to run/compile Cactus, but you are strongly encourage to install this software to take advantage of the update procedures. It can be downloaded from your favorite GNU site. Tar files of each release are also available.

To use Cactus, with the default driver¹ (CactusPUGH/PUGH) on multiple processors you also need:

MPI	the <i>Message Passing Interface (MPI)</i> which provides inter-processor communication. Supercomputing sites often supply a native MPI implementation with which Cactus is very likely to be compatible. Otherwise there are various freely available ones available, e.g. the MPICH version of MPI is available for various architectures and operating systems at http://www-unix.mcs.anl.gov/mpi/ .
-----	--

If you are using any thorns containing routines written in C++ you also need

C++	C++ compiler. For example, the GNU compiler. This is available for most supported platforms. Platform specific compilers should also work. Note that if a C++
-----	---

¹For help with unfamiliar terms, please consult the glossary, Appendix [F1](#).

compiler is available then the *main* routine in the flesh is compiled with C++ to allow static class initialisations.

If you are using any thorns containing routines written in **FORTRAN** you also need

F90/F77 For routines written in F77, either an F90 or an F77 compiler can be used. For routines written in F90 a F90 compiler is obviously required. There is a very limited set of free F90 compilers available for the different architectures.

While not required for compiling or running Cactus, for thorn development it is useful to install

ctags/etags The program Tags enables you browse through the calling structure of a program by help of a function call database. Navigating the flesh and arrangements becomes very easy. Emacs and vi both support this method. See [F7](#) for a short guide to “tags”.

A1.2 Supported architectures

Cactus runs on many machines, under a large number of operating systems. Here we list the machines we have compiled and verified Cactus on, including some architecture specific notes. A complete list of architectures supported by Cactus, along with more notes, can be found at

<http://www.cactuscode.org/Documentation/Architectures.html>.

SGI 32 or 64 bit running Irix.

Cray T3E

Compaq Alpha Compaq operating system and Linux. Single processor mode and MPI supported. The Alphas need to have the GNU C/C++ compilers installed.

IA32 running Linux or Windows 2000/NT. Single processor mode and MPI (MPICH and LAM) supported.
On Windows Cactus compiles with Cygwin. MPI (WMPI, HPVM, and MPIPro) supported. Please read doc/README.NT for details.

IA64 running Linux.

Macintosh PowerPC
(MacOS X and Linux PPC)

IBM SP2 32 or 64 bit running AIX.

Hitachi SR8000-F1

Sun Solaris

Fujitsu

The following machines are only partially supported

HP Exemplar

NEC SX-5

A1.3 Checkout procedure

Cactus is distributed, extended, and maintained using the free CVS software (*Concurrent Versioning System*: <http://www.cvshome.org>). CVS allows many people to work on a large software project together without getting into a tangle. Since Cactus thorns are distributed from several repositories on the main CVS site, and from a growing number of user sites, we provide a script, described below, on our website for checking out the flesh and thorns. The Cactus website also provides a form interface for direct download.

CVS experts who want to use raw CVS commands are directed to Appendix F6 for full instructions. For CVS novices, we also summarize in this appendix basic CVS commands.

The space required for an installation depends on the arrangements and thorns used. The flesh on its own requires less than 5 MB.

The script for checking out the flesh and thorns, **GetCactus**, is available from the website at

<http://www.cactuscode.org/Download/GetCactus>

The script takes as an argument the name of a file containing a *ThornList*, that is a list of thorns with the syntax

```
<arrangement name>/<thorn name>
```

If no filename is given, only the flesh is checked out. Optional directives in the *ThornList* indicate which CVS repository to fetch thorns from. The default is to take the thorns from the same repository as the flesh. A full description of *ThornList* syntax is provided in Appendix F8. *ThornLists* for example applications are provided on the Cactus website.

The same script can be used to checkout additional thorns.

A1.4 Directory structure

A fresh checkout creates a directory **Cactus** with the following subdirectories:

CVS	the CVS book-keeping directory, present in every subdirectory
doc	Cactus documentation
lib	contains libraries
src	contains the source code for Cactus
arrangements	contains the Cactus arrangements. The arrangements (the actual “physics”) are not supplied by checking out just Cactus. If the arrangements you want to use are standard Cactus arrangements, or reside on our CVS repository (cvs.cactuscode.org), they can be checked out in similar way to the Flesh.

When Cactus is first compiled it creates a new directory **Cactus/configs**, which will contain all the source code, object files and libraries created during the build process. Disk space may be a problem on supercomputers where home directories are small.

A workaround is to first create a configs directory on scratch space, say **scratch/cactus_configs/** (where **scratch/** is your scratch directory), and then either

- set the environment variable `CACTUS_CONFIGS_DIR` to point to this directory

or

- soft link this directory (`ln -s scratch/cactus_configs Cactus/configs/`) to the Cactus directory, if your file-system supports soft-links.

Configurations are described in detail in section [A2.1](#).

A1.5 Getting help

For tracking problem reports and bugs we use GNATS, which is a bugtracking system published under the GNU license. We have set up a web interface at <http://www.cactuscode.org> which allows easy submission and browsing of problem reports.

A description of the GNATS categories which we use is provided in the appendix [F5](#).

Chapter A2

Compilation

Cactus can be built in different configurations from the same copy of the source files, and these different configurations coexist in the **Cactus/configs** directory. Here are several instances in which this can be useful:

1. Different configurations can be for *different architectures*. You can keep executables for multiple architectures based on a single copy of source code, shared on a common file system.
2. You can compare different *compiler options, debug-modes*. You might want to compile different communication protocols (e.g. MPI or GLOBUS) or leave them out all together.
3. You can have different configurations for *different thorn collections* compiled into your executable.

Once a configuration has been created, by **gmake <config>** as described in detail in the next section, a single call to **gmake <config>** will compile the code. The first time it generates a compile **ThornList**, and gives you the chance to edit it before continuing.

A2.1 Creating a configuration

At its simplest, this is done by **gmake <config>**¹. This generates a configuration with the name **config**, doing its best to automatically determine the default compilers and compilation flags suitable for the current architecture.

There are a number of additional command line arguments which may be supplied to override some parts of the procedure.

A2.1.1 Configuration options

There are three ways to pass options to the configuration process.

1. Create a file **~/.cactus/config**.

All available configuration options may be set in the file **~/.cactus/config**, any which are not set will take a default value. The file should contain lines of the form:

¹A note on the Cactus make system — if at any point it prompts you to enter something, the default value, which will be assumed if you simply press enter, is shown in parentheses.

`<option> [=] ...`

The equals sign is optional.

2. Add the options to a configuration file and use,

```
gmake <config>-config options=<filename>
```

The options file has the same format as `~/.cactus/config`.

3. Pass the options individually on the command line,

```
gmake <configuration name>-config <option name>=<chosen value>, ...
```

Not all configuration options can be set on the command line. Those that can be set are indicated in the table below.

They are listed here in order of increasing precedence, e.g. options set on the command line will take priority over (potentially conflicting) options set in `~/.cactus/config`.

It is important to note that these methods cannot be used to, for example, add options to the default values for `CFLAGS`. Setting any variable in the configuration file or the command line will overwrite completely the default values.

A2.1.2 Available options

There is a plethora of available options.

- **Compiled Thorns**

These specify the chosen set of thorns for compilation. If the thorn choice is not provided during configuration, a list containing all thorns in the `arrangements` directory is automatically created, and the users prompted for any changes.

THORNLIST	Name of file containing a list of thorns with the syntax <code><arrangement name>/<thorn name></code> , lines beginning with <code>#</code> or <code>!</code> are ignored.
THORNLIST_DIR	Location of directory containing THORNLIST. This defaults to the current working directory.

- **Compiler and tool specification**

These are used to specify which compilers and other tools to use. Entries followed by `*` may be specified on the command line.

CC	* The C compiler.
CXX	The C++ compiler.
F90	* The Fortran 90 compiler.
F77	* The FORTRAN 77 compiler.
CPP	The preprocessor used to generate dependencies and to preprocess Fortran code.
LD	* The linker.
AR	The archiver used for generating libraries.
RANLIB	The archive indexer to use.
MKDIR	The program to use to create a directory.
PERL	The name of the perl executable.

- **Compilation and tool flags**

Flags which are passed to the compilers and the tools.

CFLAGS	Flags for the C compiler.
CXXFLAGS	Flags for the C++ compiler.
F90FLAGS	* Flags for the Fortran 90 compiler.
F77FLAGS	* Flags for the FORTRAN 77 compiler.
CPPFLAGS	Flags for the preprocessor (used to generate compilation dependencies and preprocess Fortran code).
MKDIRFLAGS	Flags for MKDIR so that no error is given if the directory exists.
LDFLAGS	* Flags for the linker.
ARFLAGS	Flags for the archiver.
DEBUG	* Specifies what type of debug mode should be used, the default is no debugging. Current options are yes , no , or memory . The option yes switches on all debugging features, whereas memory just employs memory tracing (B10.3).
OPTIMISE, OPTIMIZE	* Specifies what type of optimisation should be used. The only options currently available are yes and no . The default is to use optimisation. Note that the British spelling OPTIMISE will be checked first and, if set, will override any setting of the American-spelled OPTIMIZE .
C_OPTIMISE_FLAGS	Optimisation flags for the C compiler, their use depends on the type of optimisation being used.
CXX_OPTIMISE_FLAGS	Optimisation flags for the C++ compiler, their use depends on the type of optimisation being used.
F90_OPTIMISE_FLAGS	Optimisation flags for the FORTRAN 90 compiler, their use depends on the type of optimisation being used.
F77_OPTIMISE_FLAGS	Optimisation flags for the FORTRAN 77 compiler, their use depends on the type of optimisation being used.
C_WARN_FLAGS	Warning flags for the C compiler, their use depends on the type of warnings used during compilation (A2.2.4).
CXX_WARN_FLAGS	Warning flags for the C++ compiler, their use depends on the type of warnings used during compilation (A2.2.4).
F90_WARN_FLAGS	Warning flags for the FORTRAN 90 compiler, their use depends on the type of warnings used during compilation (A2.2.4).
F77_WARN_FLAGS	Warning flags for the Fortran 77 compiler, their use depends on the type of warnings used during compilation (A2.2.4).
• Architecture-specific flags	
IRIX_BITS=32 64	For Irix SGI systems: whether to build a 32- or 64-bit configuration.
AIX_BITS=32 64	For IBM SP systems: whether to build a 32- or 64-bit configuration.
• Library flags	
Used to specify auxiliary libraries and directories to find them in.	
LIBS	The additional libraries.
LIBDIRS	Any other library directories.
• Extra include directories	
SYS_INC_DIRS	Used to specify any additional directories for system include files.

- Precision options

Used to specify the precision of the default real and integer data types, specified as the number of bytes the data takes up. Note that not all values will be valid on all architectures.

REAL_PRECISION * Allowed values are 16, 8, 4.

INTEGER_PRECISION * Allowed values are 8, 4 and 2.

- Executable name

EXEDIR The directory in which to place the executable.

EXE The name of the executable.

- Extra packages

Compiling with extra packages is described fully in Section [A2.1.3](#), which should be consulted for the full range of configuration options.

MPI * The MPI package to use, if required. Supported values are CUSTOM, NATIVE, MPICH or LAM.

HDF5 Supported values are *yes*, *no*. A blank value is taken as *no*.

LAPACK Supported values are *yes*, *no*. A blank value is taken as *no*.

PTHREADS Supported values are *yes*.

- Miscellaneous

PROMPT Setting this to 'no' turns off all prompts from the make system.

SILENT Setting this to 'no' instructs **gmake** to print the commands that it is executing.

A2.1.3 Compiling with extra packages

MPI: Message Passing Interface

MPI (the *Message Passing Interface*) provides inter-processor communication. It can either be implemented natively on a machine (this is usual on most supercomputers), or through a standard package such as MPICH, LAM, WMPI, or PACX.

To compile with MPI, the configure option is

MPI = <MPI_TYPE>

where <MPI_TYPE> can take the values (entries followed by * may be specified on the configuration command line):

CUSTOM For a custom MPI configuration set the variables

MPI_LIBS * libraries.

MPI_LIB_DIRS * library directories.

MPI_INC_DIRS * include file directories.

NATIVE Use the native MPI for this machine, as indicated in the **known-architectures** directory (**lib/make/known-architectures**).

MPICH Use MPICH (<http://www-unix.mcs.anl.gov/mpi/mpich>). This is controlled by the options

	MPICH_ARCH *	machine architecture.
	MPICH_DIR *	directory in which MPICH is installed. If this option is not defined it will be searched for.
	MPICH_DEVICE *	the device used by MPICH . If not defined, the configuration process will search for this in a few defined places. Supported devices are currently ch_p4 , ch_shmem , globus and myrinet . For versions of MPICH prior to 1.2.0 the devices are searched for in this order, for 1.2.0 you may need to specify MPICH_DEVICE , depending on the installation.
	If MPICH_DEVICE is chosen to be globus , (www.globus.org), an additional variable must be set	
	GLOBUS_LIB_DIR *	directory in which Globus libraries are installed.
	If MPICH_DEVICE is chosen to be ch_gm , (www.myri.com), an additional variable must be set	
	MYRINET_DIR *	directory in which Myrinet libraries are installed.
LAM	Use LAM (Local Area Multicomputer, http://www.lam-mpi.org/). This is controlled by the variables	
	LAM_DIR *	directory in which LAM is installed. This will be searched for in a few provided places if not given.
	if the LAM installation splits libraries and include files into different directories, instead of setting LAM_DIR set the two variables	
	LAM_LIB_DIR *	directory in which LAM libraries are installed.
	LAM_INC_DIR *	directory in which LAM include files are installed.
WMPI	Use WMPI (Win32 Message Passing Interface, http://dsg.dei.uc.pt/w32mpi/intro.html). This is controlled by the variable	
	WMPI_DIR *	directory in which WMPI is installed.
HPVM	Use HPVM (High Performance Virtual Machine, http://www-csag.ucsd.edu/projects/hpvm.html). This is controlled by the variable	
	HPVM_DIR *	directory in which HPVM is installed.
MPIPro	Use MPIPro (http://www.mpi-softtech.com/).	
PACX	Use the PACX Metacomputing package (PArallel Computer eXtension, http://www.hlr.de/structure/organisation/par/projects/pacx-mpi/). This is controlled by the variables	
	PACX_DIR *	directory in which PACX is installed. If this option is not defined it will be searched for.
	PACX_MPI *	the MPI package PACX uses for node-local communication. This can be any of the above MPI packages.

Note that the searches for libraries etc. mentioned above use the locations given in the files in `lib/make/extras/MPI`.

HDF5: Hierarchical Data Format version 5

To compile with HDF5 (<http://hdf.ncsa.uiuc.edu/whatishdf5.html>), the configure options are

```
HDF5 = yes/no [HDF5_DIR = <dir>] [LIBZ_DIR = <dir>]
```

If HDF5_DIR is not given the configuration process will search for an installed HDF5 package in some standard places (defined in `lib/make/extras/HDF5`). If the found HDF5 library was compiled with `libz.a`, the configuration process also searches for that library and adds it to the linker flags. You may also point directly to an installation of `libz.a` by setting `LIBZ_DIR`.

LAPACK: Linear Algebra PACKage

To compile with LAPACK (<http://www.netlib.org/lapack/>), the configure options are

```
LAPACK = yes | no | <blank>
[ LAPACK_DIR = <dir> | none ]
[ LAPACK_EXTRA_LIBS_DIRS = <dir> ]
[ LAPACK_LIBS = <libs> ]
[ LAPACK_EXTRA_LIBS = <libs> ]
```

If LAPACK_DIR is not given the configuration process will search for a LAPACK library *liblapack.{a,so}* in some standard places (defined in `lib/make/extras/LAPACK`). If LAPACK_DIR is set to *none* the LAPACK library path is assumed to be installed in a standard system location (eg. `1/usr/lib/`) and thus the library path will not be added to the linker's command line.

Because LAPACK doesn't come as a standardized system installation, there are additional configuration variables to set the name of the lapack library (`LAPACK_LIBS`) as well as the name (`LAPACK_EXTRA_LIBS`) and location (`LAPACK_EXTRA_LIBS_DIRS`) of extra libraries that are required by LAPACK itself.

PTHREADS: POSIX threads

To enable multithreading support within Cactus using POSIX threads the configure option is

```
PTHREADS = yes
```

The configuration process will check if a re-entrant C library is available and adds it to the linker flags. It will also search for the system's Pthreads library (either `libpthread` or `libpthreads`) and set preprocessor defines necessary for compiling multithreaded code.

A2.1.4 File layout

The configuration process sets up various subdirectories and files in the `configs` directory to contain the configuration specific files, these are placed in a directory with the name of the configuration.

<code>config-data</code>	contains the files created by the configure script: The most important ones are <code>make.config.defn</code> contains compilers and compilation flags for a configuration.
--------------------------	---

<code>make.extra.defn</code>	contains details about extra packages used in the configuration.
<code>cctk_Config.h</code>	The main configuration header file, containing architecture specific definitions.
<code>cctk_Archdefs.h</code>	An architecture specific header file containing things which cannot be automatically detected, and have thus been hand-coded for this architecture.

These are the first files which should be checked or modified to suit any peculiarities of this configuration.

In addition the following files may be informative:

<code>fortran_name.pl</code>	A perl script used to determine how the Fortran compiler names subroutines. This is used to make some C routines callable from Fortran, and Fortran routines callable from C.
<code>make.config.deps</code>	Initially empty. Can be edited to add extra architecture specific dependencies needed to generate the executable.
<code>make.config.rule</code>	Make rules for generating object files from source files.

Finally, autoconf generates the following files.

<code>config.log</code>	A log of the autoconf process.
<code>config.status</code>	A script which may be used to regenerate the configuration.
<code>config.cache</code>	An internal file used by autoconf.

<code>lib</code>	An empty directory which will contain the libraries created for each thorn.
<code>build</code>	An empty directory which will contain the object files generated for this configuration, and preprocessed source files.
<code>config-info</code>	A file containing information about the configuration.
<code>bindings</code>	A directory which contains all the files generated by the CST from the <code>.cc1</code> files.
<code>scratch</code>	A scratch directory which is used to accomodate Fortran 90 modules.

A2.2 Building and Administering a configuration

Once you have created a new configuration, the command

```
gmake <configuration name>
```

will build an executable, prompting you along the way for the thorns which should be included. There is a range of gmake targets and options which are detailed in the following sections.

A2.2.1 gmake targets for building and administering configurations

A target for `gmake` can be naively thought of as an argument that tells it which of several things listed in the `Makefile` it is to do. The command `gmake help` lists all gmake targets:

```
gmake <config>      builds a configuration. If the configuration doesn't exist it will create it.
gmake <config>-clean removes all object and dependency files from a configuration.
```

`gmake <config>-cleandeps`
removes all dependency files from a configuration.

`gmake <config>-cleanobjs`
removes all object files from a configuration.

`gmake <config>-config`
creates a new configuration or reconfigures an old one.

`gmake <config>-cvsupdate`
update the Flesh and Thorns for a configuration using CVS

`gmake <config>-delete`
deletes a configuration (`rm -r configs/<config>`).

`gmake <config>-editthorns`
edits the ThornList.

`gmake <config>-examples`
copies all the example parameter files relevant for this configuration to the directory **examples** in the Cactus home directory. If a file of the same name is already there, it will not overwrite it.

`gmake <config>-realclean`
removes from a configuration all object and dependency files, as well as files generated from the CST (stands for Cactus Specification Tool, which is the perl scripts which parse the thorn configuration files). Only the files generated by configure and the ThornList file remain.

`gmake <config>-rebuild`
rebuilds a configuration (reruns the CST).

`gmake <config>-testsuite`
runs the test programs associated with each thorn in the configuration. See section [A2.4](#) for information about the testsuite mechanism.

`gmake <config>-thornlist`
regenerates the ThornList for a configuration.

`gmake <config>-utils [UTILS=<list>]`
builds all utility programs provided by the thorns of a configuration. Individual utilities can be selected by giving their names in the UTILS variable.

`gmake <config>-ThornGuide`
builds documentation for the thorns in this configuration.

`gmake <config>-configinfo`
displays the options used to build the configuration.

`gmake <config>-cvsupdate`
updates the Flesh and this configuration's Thorns from the CVS repositories.

A2.2.2 Compiling in thorns

Cactus will try to compile all thorns listed in `configs/<config>/ThornList`. The `ThornList` file is simply a list of the form `<arrangement>/<thorn>`. All text after a `#` or `!` sign on a line is treated as a comment and ignored. The first time that you compile a configuration, if you did not specify a `ThornList` already during configuration, you will be shown a list of all the thorns in your arrangement directory, and asked if you wish to edit them. You can regenerate this list at anytime by typing

```
gmake <config>-thornlist
```

or you can edit it using

```
gmake <config>-editthorns
```

Instead of using the editor to specify the thorns you want to have compiled, you can *edit* the *ThornList* outside the make process. It is located in `configs/<config>/ThornList`, where `<config>` refers to the name of your configuration. The directory, `./configs`, exists *after* the very first make phase for the first configuration.

A2.2.3 Notes and Caveats

- If during the build you see the error “missing separator” you are probably not using GNU make.
- *The EDITOR environment variable.* You may not be aware of this, but this thing very often exists and may be set by default to something scary like `vi`. If you don’t know how to use `vi` or wish to use your favorite editor instead, reset this environment variable. (To exit `vi` type `<ESC> :q!`)

A2.2.4 gmake options for building configurations

An *option* for gmake can be thought of as an argument which tells it how it should make a **target**. Note that the final result is always the same.

```
gmake <target> PROMPT=no
                        turns off all prompts from the make system.
gmake <target> SILENT=no
                        print the commands that gmake is executing.
gmake <target> WARN=yes
                        show compiler warnings during compilation.
gmake <target> FJOBS=<number>
                        compile in parallel, across files within each thorn.
gmake <target> TJOBS=<number>
                        compile in parallel, across thorns.
```

Note that with more modern versions of gmake, it is sufficient to pass the normal `-j <number>` flag to gmake to get parallel compilation.

A2.3 Other gmake targets

<code>gmake help</code>	lists all make options.
<code>gmake checkout</code>	allows you to easily checkout Cactus arrangements and thorns. For example it can checkout all the thorns in any thornlist file found in the <code>thornlists</code> subdirectory of the Cactus root directory.
<code>gmake cvsdiff</code>	differences between checked out version of Cactus and that in the CVS repositories.
<code>gmake cvsstatus</code>	status of checked out version of Cactus, reporting which files have been modified or need updating.
<code>gmake cvsupdate</code>	update Flesh and all Thorns from CVS repositories.
<code>gmake default</code>	creates a new configuration with a default name.
<code>gmake distclean</code>	delete your <code>configs</code> directory and hence all your configurations.

<code>gmake downsize</code>	removes non-essential files as documents and testsuites to allow for minimal installation size.
<code>gmake newthorn</code>	creates a new thorn, prompting for the necessary information and creating template files.
<code>gmake TAGS</code>	creates an Emacs style TAGS file. See section F7 for using TAGS within Cactus.
<code>gmake tags</code>	creates a vi style tags file. See section F7 for using TAGS within Cactus.
<code>gmake UsersGuide</code>	runs LaTeX to produce a copy of the Users' Guide.
<code>gmake ThornGuide</code>	runs LaTeX to produce a copy of the Thorn Guide, for all the thorns in the arrangements directory.
<code>gmake MaintGuide</code>	runs LaTeX to produce a copy of the Maintainers' Guide.
<code>gmake configinfo</code>	prints configuration options for every configuration found in user's <code>configs</code> subdirectory.

A2.4 Testing

Some thorns come with a testsuite, consisting of example parameter files and the output files generated by running these. To run the testsuites for the thorns you have compiled use

```
gmake <configuration>-testsuite
```

These testsuites serve the dual purpose of

Regression testing	i.e. making sure that changes to the thorn or the flesh don't affect the output from a known parameter file.
Portability testing	i.e. checking that the results are independent of the architecture — this is also of use when trying to get Cactus to work on a new architecture.

Chapter A3

Running Cactus

Cactus executables always run from a parameter file (which may be a physical file or taken from standard input), which specifies which Thorns to use and set the values of any parameters which are different from the default values. Any accepted filename can be used for the name of the parameter file, although standard convention is to use the file extension `.par`. Optional command line arguments can be used to customise runtime behaviour, and to provide information about the Thorns used in the executable. The general syntax for running Cactus from a physical parameter file is then

```
./cactus-<config> <parameter file> [command line options]
```

or if the parameter file should be taken from standard input

```
./cactus-<config> [command line options] -
```

The remainder of this chapter covers all aspects for running your Cactus executable. These include: command line options, parameter file syntax, understanding screen output, environment variables, and creating thorn documentation.

A3.1 Command Line Options

The cactus executable accepts numerous command line arguments:

Short Version	Long Version
-O[v]	-describe-all-parameters
-o<param>	-describe-parameter=<param>
-T	-list-thorns
-t<arrangement/thorn>	-test-thorn-compiled=<arrangement/thorn>
-h,-?	-help
-v	-version
-W<level>	-warning-level=<level>
-E<level>	-error-level=<level>
-r[o e oe eo]	-redirect=[o e oe eo]
-i	-ignore-next
	-parameter-level=<level>

-O or -describe-all-parameters

Produces a full list of all parameters from all thorns which were compiled, along with descriptions and allowed values. This can take an optional extra parameter v

(i.e. `-Ov` to give verbose information about all parameters).

- `-o<param>` or `-describe-parameter=<param>`
 Produces the description and allowed values for a given parameter — takes one argument.
- `-T` or `-list-thorns` Produces a list of all the thorns which were compiled in.
- `-t<arrangement or thorn>` or `-test-thorn-compiled=<arrangement or thorn>`
 Checks if a given thorn was compiled in — takes one argument.
- `-h`, `-?` or `-help` Produces a help message.
- `-v` or `-version` Produces version information of the code.
- `-W<level>` or `-warning-level=<level>`
 Sets the warning level of the code. All warning messages are given a level — the lower the level the greater the severity. This parameter controls the level of messages to be seen, with all warnings of value `<level>` or higher printed to standard out. The default is a warning level of 1, with 0 indicating that only those messages which are (by default) fatal should be seen.
- `-E<level or -error-level=<level>`
 This works in concert with `-W` — it controls which warning level is treated as a fatal error. This cannot be set to a higher value than `-W`. The default value is zero.
- `-r[o|e|oe|eo]` or `-redirect=[o|e|oe|eo]`
 This redirects the standard output ('o') and/or standard error ('e') of each processor to a file. By default the standard outputs from processors other than processor 0 are discarded.
- `-i` or `-ignore-next` Ignore the next argument on the command line.
- `-parameter-level=<level>`
 Set the level of parameter checking to be used, either **strict**, **normal** (the default), or **relaxed**. See Section [A3.2](#).

A3.2 Parameter File Syntax

The parameter file is used to control the behaviour of the code at runtime. It is a text file with lines which are either comments, denoted by a '#' or '!', or parameter statements. A parameter statement consists of one or more parameter names, followed by an '=', followed by the value(s) for this (these) parameter(s). Note that all string parameters are case insensitive.

The **first parameter** in any parameter file should be **ActiveThorns**. This is a special parameter which tells the code which *thorns* are to be activated. Only parameters from active thorns can be set (and only those routines *scheduled* by active thorns are run). By default all thorns are inactive. For example, the first entry in a parameter file which is using just the two thorns **CactusPUGH/PUGH** and **CactusBase/CartGrid3D** should be

```
ActiveThorns = 'PUGH CartGrid3D'
```

Parameters following the **ActiveThorns** parameter all have names whose syntax depends on the scope of the parameter:

Global parameters Just the name of the parameter itself. Global parameters are avoided, and there are none in the **Flesh** and **Cactus Toolkits**.

Restricted parameters

The name of the *implementation* which defined the parameter, two colons, and the name of the parameter — e.g. `driver::global_nx`.

Private parameters

The name of the *thorn* which defined the parameter, two colons, and the name of the parameter — e.g. `wavetoyF77::amplitude`.

This notation is not strictly enforced currently in the code. It is sufficient to specify the first part of the parameter name using either the implementation name, or the thorn name. However, it is suggested that the convention above is followed.

The Cactus Flesh performs checks for consistency and range of parameters, the severity of these checks is controlled by the command line argument `-parameter-level` which can take the following values

relaxed	Cactus will issue a level 0 warning (that is the default behaviour will be to terminate) if • The specified parameter value is outside of the allowed range.
normal	This is the default, and provides the same warnings as the relaxed level, with in addition a level 0 warning issued for • An implementation and/or thorn <code>foo</code> is active, but the parameter <code>foo::bar</code> was not defined. • The parameter <code>foo::bar</code> was successfully set for both an active implementation <code>foo</code> not implemented by a thorn <code>foo</code>, and to a thorn <code>foo</code>.
strict	This provides the same warnings as the normal level, with in addition a level 0 warning issued for • The parameter <code>foo::bar</code> is specified in the parameter file, but no implementation or thorn with the name <code>bar</code> is active.

Notes:

- You can obtain lists of the parameters associated with each thorn using the command line options `-o` and `-O` (Section [A3.1](#)).
- The parameter file is read *sequentially* from top to bottom, this means that if you set the value of a parameter twice in the parameter file, the second value will be used. (This is why the **ActiveThorns** parameter is always first in the file).
- Some parameters are *steerable* and can be changed during the execution of a code using parameter steering interfaces (for example, thorn `CactusConnect/HTTDP`, or using a parameter file when recovering from a checkpoint file).
- For examples of parameter files, look in the **par** directory which can be found in most thorns.

A3.3 Thorn Documentation

The Cactus make system provides a mechanism for generating a *Thorn Guide* containing separate chapters for each thorn and arrangement in your configuration. The provided documentation for an individual thorn obviously depends on what the thorn authors added, but the Thorn Guide is a good place to first look for special instructions on how to run and interpret the output from a Thorn. Details about parameters, grid variables and scheduling are automatically included in from a thorns CCL files into the Thorn Guide. To construct a Thorn Guide for the configuration `<config>` use

```
gmake <config>-ThornGuide
```

or to make a Thorn Guide for all the thorns in the `arrangements` directory

`gmake <config>.`

Chapter A4

Getting and looking at output

A4.1 Screen output

As your Cactus executable runs, standard output and standard error are usually written to the screen. Standard output provides you with information about the run, and standard error reports warnings and errors from the flesh and thorns.

As the program runs, the normal output provides the following information:

Active thorns	A report is made as each of the thorns in the <code>ActiveThorns</code> parameters from the parameter file is attempted to be activated. This report shows whether the activation was successful, and if successful gives the thorn's implementation. For example <pre>Activating thorn idscalarwave...Success -> active implementation idscalarwave</pre>
Failed parameters	If any of the parameters in the parameter file does not belong to any of the active thorns, or if the parameter value is not in the allowed range, an error is registered. For example, if the parameter is not recognised <pre>Unknown parameter time::ddtfac</pre> or if the parameter value is not in the allowed range <pre>Unable to set keyword CartGrid3D::type - ByMouth not in any active range</pre>
Scheduling information	A complete list of all scheduled routines is given, in the order that they will be executed. For example <pre>----- Startup routines Cactus: Register banner for Cactus CartGrid3D: Register GH Extension for GridSymmetry CartGrid3D: Register coordinates for the Cartesian grid IOASCII: Startup routine IOBasic: Startup routine IOUtil: IOUtil startup routine</pre>

```

PUGH: Startup routine
WaveToyC: Register banner

Parameter checking routines
CartGrid3D: Check coordinates for CartGrid3D
IDScalarWave: Check parameters

Initialisation
CartGrid3D: Set up spatial 3D Cartesian coordinates on the GH
PUGH: Report on PUGH set up
Time: Set timestep based on speed one Courant condition
WaveToyC: Schedule symmetries
IDScalarWave: Initial data for 3D wave equation

do loop over timesteps
WaveToyC: Evolution of 3D wave equation
t = t+dt
if (analysis)
endif
enddo
-----

```

Thorn banners

Any banners registered from the thorns are displayed.

A4.2 Output

Output methods in Cactus are all provided by thorns. Any number of output methods can be used for each run. The behaviour of the output thorns in the standard arrangements are described in those thorns' documentation.

In general, these thorns decide what to output by parsing a string parameter containing the names of those grid variables, or groups of variables, for which output is required. The names should be fully qualified with the `implementation` and `group` or `variable` names.

There is usually a parameter for each method to denote how often, in evolution iterations, this output should be performed. There is also usually a parameter to define the directory in which the output should be placed, defaulting to the directory from which the executable is run.

A4.3 Checkpointing/Recovery

Checkpointing is defined as saving the current state of a run (parameter settings, contents of grid variables, and other relevant information) to a file. At a later time, this run can then be restarted from that state by recovering all the data from the checkpoint file.

Checkpointing/recovery methods in Cactus are provided by thorns.

In general, these thorns decide how often to generate a checkpoint. They also register their recovery routines with the flesh which then get called during initialization to perform the recovery of parameters and grid variables if requested in the parameter file.

Part B

Application thorn writing

Chapter B1

Overview

This chapter goes into the nitty-gritty of writing a thorn. It introduces key concepts for thorns, then goes on to give a brief outline of how to configure a thorn. There is then some detail about concepts introduced by the configuration step, followed by discussion of code which you must put into your files in order to use Cactus functionality, and details of utility functions you may use to gain extra functionality.

Chapter B2

Thorn concepts

B2.1 Thorns

A thorn is the basic working module within Cactus. All user-supplied code goes into thorns, which are, by and large, independent of each other. Thorns communicate with each other via calls to the flesh API, plus, more rarely, custom APIs of other thorns.

The connection from a thorn to the flesh or to other thorns is specified in configuration files which are parsed at compile time and used to generate glue code which encapsulates the external appearance of a thorn.

Thorn names must be (case independently) unique, must start with a letter, can only contain letters, numbers or underscores, and must contain 27 characters or less. In addition, a thorn cannot have the name `doc`, this is reserved for arrangement documentation. Arrangement names which start with a `#`, or finish with `~` or `.bak` will be ignored.

B2.2 Arrangements

Thorns are grouped into *arrangements*. This is a logical grouping of thorns which is purely for organisational purposes. For example, you might wish to keep all your initial data thorns in one arrangement, and all your evolution thorns in another arrangement, or you may want to have separate arrangements for your developments, private and shared thorns.

The arrangements live in the `arrangements` directory off the main Cactus directory. Arrangement names must be (case independently) unique, must start with a letter, and can only contain letters, numbers or underscores. Arrangement names which start with a `#`, or finish with `~` or `.bak` will be ignored.

Inside an arrangement directory there are directories for each thorn belonging to the arrangement.

B2.3 Implementations

One of the key concepts for thorns is the concept of the **implementation**. Relationships among thorns are all based upon relationships among the **implementations** they provide. In principle it should be possible to swap one thorn providing an implementation with another thorn providing that implementation, without affecting any other thorn.

An **implementation** defines a group of variables and parameters which are used to implement some functionality. For example the thorn **CactusPUGH/PUGH** provides the implementation *driver*. This implementation is responsible for providing memory for grid variables and for communication. Another thorn can also implement **driver**, and both thorns can be compiled in *at the same time*. At runtime, the user can decide which thorn providing **driver** is used. No other thorn should be affected by this choice.

When a thorn decides it needs access to a variable or a parameter provided by another thorn, it defines a relationship between itself and the other thorn's **implementation**, not explicitly with the other **thorn**. This allows the transparent replacement, at compile or runtime, of one thorn with another thorn providing the same functionality as seen by the other thorns.

Chapter B3

Anatomy of a thorn

B3.1 Thorns

A thorn consists of a subdirectory of an arrangement containing three administrative files

<code>interface.ccl</code>	the cactus interface, which defines the grid functions, variables, etc. See F2.2 .
<code>param.ccl</code>	the parameters introduced by this thorn, and the parameters needed from other thorns. See F2.3 .
<code>schedule.ccl</code>	scheduling information for routines called by the flesh. See F2.4 .

Thorns can also contain

- a subdirectory called `src`, which should hold source files and compilation instructions for the thorn
- a subdirectory `src/include` for include files
- a `README` containing a brief description of the thorn
- a `doc` directory for documentation
- a `par` directory for example parameter files
- a `test` subdirectory may also be added, to hold the thorn's testsuite. See [B9.5](#) for details.

B3.2 Creating a thorn

To simplify the creation of a thorn, a make target `gmake newthorn` has been provided. When this is run:

1. You will be prompted for the name of the new thorn.
2. You will be prompted for the name of the arrangement you would like to include your thorn in. Either enter a new arrangement name or pick one from the list of available arrangements that are shown.

B3.3 Configuring your thorn

The interaction of a thorn with the flesh and other thorns is controlled by various configuration files.

These consist of:

<code>interface.ccl</code>	This defines the implementation (Section B2.3) the thorn provides, and the variables the thorn needs, along with their visibility to other implementations.
<code>param.ccl</code>	This defines the parameters that are used to control the thorn, along with their visibility to other implementations.
<code>schedule.ccl</code>	This defines which functions from the thorn are called and when they are called. It also handles memory and communication assignment for grid variables.

B3.3.1 General syntax of CCL files

Cactus Configuration Language (CCL) files are simple text files used to define configuration information for a thorn. CCL files are case independent, and may contain comments introduced by the ‘#’ character, which indicates that the rest of the line is a comment. If the last non-blank character of a line in a CCL file is a backslash \, the following line is treated as a continuation of the current line.

B3.3.2 The `interface.ccl`

The `interface.ccl` file is used to declare

- the implementation provided by the thorn,
- the variables provided by the thorn
- the include files provided by the thorn
- the functions provided by the thorn (in development)

The implementation is declared by a single line at the top of the file

```
implements: <name>
```

Where <name> can be any combination of alphanumeric characters and underscores, and is case independent. There are three different access levels available for variables

Public	Can be ‘inherited’ by other implementations (see below).
Protected	Can be shared with other implementations which declare themselves to be friends of this one (see below).
Private	Can only be seen by this thorn.

Corresponding to the first two access levels there are two relationship statements that can be used to get variables from other thorns.

- Inherits:** `<name>` This gets all **Public** variables from implementation `<name>`, and all variables that `<name>` has in turn inherited. An implementation may inherit from any number of other implementations.
- Friend:** `<name>` This gets all **Protected** variables from implementation `<name>`, but, unlike **inherits**, it is symmetric and also defines a transitive relation by pushing its own implementations **Protected** variables onto implementation `<name>`. This keyword is used define to a group of implementations which all end up with the same **Protected** variables.

So, for example, an `interface.ccl` starting

```
implements: wavetoy
inherits:   grid
friend:     wave_extract
```

declares that the thorn provides an implementation called ‘wavetoy’, gets all the **public** variables declared by an implementation called ‘grid’, and shares all **protected** variables with `wave_extract` and its friends.

Cactus variables, described in Chapter B4 are placed in groups with homogeneous attributes, where the attributes describe properties such as the data type, group type, dimension, ghostsize, number of timelevels, type of staggering and distribution.

For example, a group, called `realfields` of 5 real grid functions (`phi`, `a`, `b`, `c`, `d`), on a 3D grid, would be defined by

```
CCTK_REAL realfields type=GF TimeLevels=3 Dim=3
{
  phi
  a,b,c,d
} "Example grid functions"
```

or, for a group called `intfields` consisting of just one distributed 2D array of integers,

```
CCTK_INT intfields type=ARRAY size=xsize,ysize ghostsize=gxsize,gysize dim=2
{
  anarray
} "My 2D arrays"
```

where `xsize`, `ysize`, `gxsize`, `gysize` are all parameters defined in the thorn’s `param.ccl`.

By default all groups are **private**, to change this an access specification of the form **public:** or **protected:** (or **private:** to change it back) may be placed on a line by itself. This changes the access level for any group defined in the file from that point on.

All variables seen by any one thorn must have distinct names.

B3.3.3 The `param.ccl`

Users control the operation of thorns via parameters given in a file at runtime. The `param.ccl` is used to specify the parameters used to control an individual thorn, and to specify the values these parameters are allowed to take. When the code is run it reads a parameter file and sets the parameters if they fall within the allowed values. If a parameter is not assigned in a parameter file, it is given its default value.

There are three access levels available for parameters:

Global	These parameters are seen by all thorns.
Restricted	These parameters may be used by other implementations if they so desire.
Private	These are only seen by this thorn.

A parameter specification consists of:

- the parameter type (each may have an optional CCTK_ in front)

REAL	
INT	
KEYWORD	A distinct string with only a few known allowed values.
STRING	An arbitrary string, which must conform to a given regular expression.
BOOLEAN	A boolean type which can take values 1, 't', 'true', 'yes' or 0, 'f', 'false', 'no'.
- the parameter name
- an optional size (in square brackets) – if this is present the parameter is a “parameter array”, i.e. will actually be an array of parameters each of which has the same properties but a different value. Such arrays appear as normal arrays in C or Fortran, 0-based in C and 1-based in Fortran. In the parameter file the value of each element is specified with square brackets and is 0-based. The size must be an integer.
- a description of the parameter
- an allowed value block – This consists of a brace delimited block of lines¹ describing the allowed values of the parameter. Each range may have a description associated with it by placing a :: on the line and putting the description afterwards.
- the default value – This must be one of the allowed values.

For the numeric types INT and REAL, a range consists of a string of the forms lower-bound:upper-bound:step where a missing number or a * denotes anything (i.e. infinite bounds or an infinitesimal step).

For example

```
REAL Coeff "Important coefficient"
{
0:3.14 :: "Range has to be from zero to Pi, default is zero"
} 0.0

#No need to define a range for BOOLEAN
BOOLEAN nice "Nice weather ?"
{
}"yes"

# A example for a set of keywords and its default (which has to be
# defined in the body)
KEYWORD confused "Are we getting confused ?"
{
"yes"      :: "absolutely positively"
"perhaps"  :: "we are not sure"
"never"    :: "never"
```

¹The beginning brace ({) must sit on a line by itself; the ending brace (}) must be preceded by a carriage return.

```

} "never"

REAL Length[2] "Length in each direction"
{
0:* :: "Range has to be from zero to infinity, default is one"
} 1.0

```

defines a REAL parameter, a BOOLEAN parameter, a KEYWORD and an array of REAL parameters.

By default all parameters are **private**, to change this an access specification of the form **global:** or **restricted:** (or **private:** to change it back) may be placed on a line by itself. This changes the access level for any parameter defined in the file from that point on.

To access **restricted** parameters from another implementation, a line containing **shares: <name>** declares that all parameters mentioned in the file from now until the next access specification originate in implementation **<name>**. (Note that only one implementation can be specified on each **shares:** line.) Each of these parameters must be qualified by the initial token **USES** or **EXTENDS**, where

USES	indicates that the parameters range remains unchanged.
EXTENDS	indicates that the parameters range is going to be extended.

In contrast to parameter declarations in other access blocks, the default value must be omitted — it is impossible to set the default value of any parameter not originating in this thorn. For example, the following block adds possible values to the keyword **initial_data** originally defined in the implementation **einstein**, and uses the real parameter **speed**.

```

shares:einstein

EXTENDS KEYWORD initial_data
{
  "bl_bh"      :: "Brill Lindquist black holes"
  "misner_bh"  :: "Misner black holes"
  "schwarzschild" :: "One Schwarzschild black hole"
}

USES CCTK_REAL speed

```

Note that you must compile at least one thorn which implements **einstein**.

B3.3.4 The `schedule.ccl`

By default no routine of a thorn will be run. The `schedule.ccl` file defines those that should be run and when and under which conditions they should be run.

The specification of routine scheduling is via a `schedule` block which consists of lines of the form

```

schedule <name> at <time bin> [other options]
{
  LANG:      <FORTRAN|C>
  STORAGE:   [group list with timelevels]

```

```

    TRIGGERS: [group list]
    SYNC:      [group list]
} "A description"

```

where `<name>` is the name of the routine, and `<time bin>` is the name of a schedule bins (the `CCTK_` prefix is optional). A list of the most useful schedule bins for application thorns is given here, a complete and more descriptive list is provided in Appendix F3:

<code>CCTK_STARTUP</code>	For routines run before the grid hierarchy is set up, for example function registration.
<code>CCTK_PARAMCHECK</code>	For routines which check parameter combinations, routines registered here only have access to the grid size and the parameters.
<code>CCTK_BASEGRID</code>	Responsible for setting up coordinates etc.
<code>CCTK_INITIAL</code>	For generating initial data.
<code>CCTK_POSTINITIAL</code>	Tasks which must be applied after initial data is created.
<code>CCTK_PRESTEP</code>	Stuff done before the evolution step.
<code>CCTK_EVOL</code>	The evolution step.
<code>CCTK_POSTSTEP</code>	Stuff done after the evolution step.
<code>CCTK_ANALYSIS</code>	For analysing data.

The `other options` allow finer grained control of the scheduling. It is possible to state that the routine must run `BEFORE` or `AFTER` another routine. It is also possible to schedule the routine under an alias name by using `AS <alias_name>`.

<code>LANG</code>	The <code>LANG</code> keyword specifies the linkage of the scheduled routine which determines how to call it from the scheduler. C and Fortran linkage are possible here. C++ routines should be defined as <code>extern "C"</code> and registered as <code>LANG: C</code> .
<code>STORAGE</code>	The <code>STORAGE</code> keyword specifies any groups for which memory should be allocated for the duration of the routine. The storage status reverts to its previous status after the routine returns. The format of the <code>STORAGE</code> statement includes specifying the number of timelevels of each group for which storage should be activated. <pre>STORAGE: <group1>[timelevels1], <group2>[timelevels2]</pre> This number can range from one to the maximum number of timelevels for the group as specified in the group definition in its <code>interface.cc1</code> file. If this maximum number is one, the timelevel specification can be omitted from the <code>STORAGE</code> statement.
<code>TRIGGERS</code>	<code>TRIGGERS</code> is used when the routine is registered at <code>ANALYSIS</code> — this is a special time bin, a routine registered here will only be called if one of the variables from a group in <code>TRIGGERS</code> is due for output.
<code>SYNC</code>	The keyword <code>SYNC</code> specifies groups of variables which should be synchronised (that is, their ghostzones should be exchanged between processors) on exit from the routine. Specifying synchronisation of grid variables in <code>schedule.cc1</code> is an alternative to calling the function <code>CCTK_SyncGroup()</code> from inside a routine. Using the <code>SYNC</code> keyword in the <code>schedule.cc1</code> is the preferred method, since it provides the Flesh with more information about the behaviour of your code, and in particular is a requirement for using a driver with adaptive mesh refinement.

As well as schedule blocks it's possible to embed C style `if/else` statements in the `schedule.ccl`. These can be used to schedule things based upon the value of a parameter.

Example I:

If the parameter `evolve_hydro` is positively set, the Fortran routine `hydro_predictor` is scheduled to run in the *evolution* loop, after the routine `metric_predictor` and before `metric_corrector`. The routine names `metric_predictor` and `metric_corrector` may either be real routine names from the same or a different thorn, or they may be aliased routine names (see the next example).

Before entry to `hydro_predictor` storage will be allocated for one timelevel for the group of grid variables `hydro_variables`, on exit from the routine this storage will be deallocated and the contents of the variables will be lost.

```
if(CCTK_Equals(evolve_hydro,"yes"))
{
  SCHEDULE hydro_predictor AT evol AFTER metric_predictor BEFORE metric_corrector
  {
    LANG:      FORTRAN
    STORAGE:   hydro_variables[1]
  } "Do a predictor step on the hydro variables"
}
```

If the parameter `evolve_hydro` is set negatively, the `hydro_predictor` routine will not be called by the scheduler. Note that if the `evolve_hydro` parameter is `STEERABLE` it can be dynamically scheduled and de-scheduled during a run if a steering interface is available.

Example II:

The thorns `WaveToy77` and `WaveToyC` each provide a routine to evolve the 3D wave equation: `WaveToyF77_Evolution` and `WaveToyC_Evolution`. The routine names have to be different, so that both thorns can be compiled at the same time, their functionality is identical though. Either one of them can then be activated at run time in the parameter file via `ActiveThorns`.

Since each evolution routine provides the same functionality, it makes sense to schedule them under the common alias `WaveToy_Evolution` to allow relative scheduling (`BEFORE/AFTER`) independent of the actual routine name (which may change depending on the activation in the parameter file).

In both cases the group of variables `scalarfield` are synchronised across processes when the routine is exited.

```
schedule WaveToyF77_Evolution AS WaveToy_Evolution AT evol
{
  LANG: Fortran
  STORAGE: scalartmps
  SYNC: scalarfield
} "Evolution of 3D wave equation"

schedule WaveToyC_Evolution AS WaveToy_Evolution AT evol
{
  LANG: C
  STORAGE: scalartmps
  SYNC: scalarfield
} "Evolution of 3D wave equation"
```


The thorn `IDScalarWave` schedules the routine `WaveBinary` after the alias `WaveToy_Evolution`. It is scheduled independently of the C or Fortran routine name.

```
schedule WaveBinary AT evol AFTER WaveToy_Evolution
{
  STORAGE: wavetoy::scalarevolve
  LANG:    Fortran
} "Provide binary source during evolution"
```

Storage Outside of Schedule Blocks

The keyword `STORAGE` can also be used outside of the schedule blocks to indicate that storage for these groups should be switched on at the start of the run. Note that the storage is only allocated in this way at the start, a thorn could explicitly switch the storage off (although this is not recommended practise). As for the `STORAGE` statement in schedule blocks, each group must also specify how many timelevels to activate storage for.

B3.4 Naming Conventions for Source Files

The make system uses file extensions to designate coding language. The following extensions can be handled:

Extension	Coding Language
<code>.F</code>	Fortran90 fixed form
<code>.f</code>	(no preprocessing) Fortran90 fixed form
<code>.F90</code>	Fortran90 free form
<code>.f90</code>	(no preprocessing) Fortran90 free form
<code>.F77</code>	Fortran77
<code>.f77</code>	(no preprocessing) Fortran77
<code>.c</code>	C
<code>.cc</code> or <code>.C</code>	C++

The following restrictions apply to file names:

- For portability across all operating systems, the base names for any particular extension should not depend on the operating system being case sensitive (e.g. having `MyFile.c` and `MYFILE.f77` is alright, but `MyFile.c` and `MYFILE.c` could cause problems).
- Currently all source files in different subroutines within a thorn must have distinct names. We hope to relax this in future. Different thorns can have files with the same names.

B3.5 Adding source files

By default the CCTK looks in the `src` directory of the thorn for source files.

There are two ways in which to specify the sources. The easiest is to use the `make.code.defn` based method in which the CCTK does all the work, but you may instead put a `Makefile` in the `src` directory and do everything yourself.

B3.5.1 `make.code.defn` based thorn building

This is the standard way to compile your thorn's source files. The Cactus make system looks for a file called `make.code.defn` in that directory (if there is no file called `Makefile` in the `src` directory). At its simplest, this file contains two lines

- `SRCS = <list of all source files in this directory>`
- `SUBDIRS = <list of all subdirectories, including subdirectories of subdirectories>`

Each subdirectory listed should then have a `make.code.defn` file containing just a `SRCS =` line, a `SUBDIRS =` line will be ignored.

In addition, each directory can have a `make.code.deps` file, which, for files in that directory, can contain additional make rules and dependencies for files in that directory. See the GNU Make documentation for complete details of the syntax.

B3.5.2 `Makefile` based thorn building

This method gives you the ultimate responsibility. The only requirement is that a library called `$NAME` be created by the `Makefile`.

The makefile is passed the following variables

<code>\$(CCTK_HOME)</code>	the main Cactus directory
<code>\$(TOP)</code>	the configuration directory
<code>\$(SRCDIR)</code>	the directory in which the source files can be found
<code>\$(CONFIG)</code>	the directory containing the configuration files
<code>\$(THORN)</code>	the thorn name
<code>\$(SCRATCH_BUILD)</code>	the scratch directory where Fortran90 module files should end up if they need to be seen by other thorns.
<code>\$(NAME)</code>	the name of the library to be built

and has a working directory of `<config>/build/<thorn_name>` .

B3.5.3 Other makefile variables

- `CC`
- `CXX`
- `F90`
- `F77`
- `CFLAGS`
- `CXXFLAGS`
- `F90FLAGS`
- `F77FLAGS`
- `LD`

Chapter B4

Cactus Variables

Cactus variables are sorted into **groups**. All variables in a group are of the same type, and have the same attributes, and most Cactus operations act on a group as a whole.

Cactus variables are used instead of local variables for a number of reasons:

- Cactus variables can be made visible to other thorns, allowing thorns to communicate and share data,
- Cactus variables can be distributed and communicated among processors, allowing parallel computation,
- A database of Cactus variables, and their attributes, is held by the Flesh, and this information is used by thorns, for example for obtaining a list of variables for checkpointing,
- Many Cactus APIs and tools can only be used with Cactus variables,
- Cactus provides features for error checking based on Cactus variables and their attributes.

The specification for a group declaration (fully described in Appendix [F2.2](#)) is,

```
<data_type> <group_name> [TYPE=<group_type>] [DIM=<dim>] [TIMELEVELS=<num>] \  
  [SIZE=<size in each direction>] [DISTRIB=<distribution_type>] \  
  [GHOSTSIZE=<ghostsize>] [STAGGER=<stagger-specification>]  
{  
  [ <variable_name>[,]<variable_name>  
    <variable_name> ]  
} ["<group_description>"] ]
```

Currently, the names of groups and variables must be distinct.

B4.1 Data Type

Cactus supports integer, real, complex and character variables, in various different sizes. (Sizes in the following refer to the number of bytes the type takes).

INTEGER	CCTK_INT, CCTK_INT2, CCTK_INT4, CCTK_INT8. (CCTK_INT defaults to being CCTK_INT4).
---------	--

REAL	CCTK_REAL, CCTK_REAL4, CCTK_REAL8, CCTK_REAL16. (CCTK_REAL defaults to being CCTK_REAL8).
COMPLEX	CCTK_COMPLEX, CCTK_COMPLEX8, CCTK_COMPLEX16, CCTK_COMPLEX32. (CCTK_COMPLEX defaults to being CCTK_COMPLEX16).
BYTE	This is a 1 byte data type.

Normally a thorn should use the default types — CCTK_INT, CCTK_REAL, CCTK_COMPLEX — rather than explicitly setting the size, as this gives maximum portability. Also, the defaults can be changed at configuration time (see [A2.1.2](#)), and this allows people to compile the code with different precisions to test for round-off effects, or to run more quickly with a lower accuracy.

B4.2 Group Types

Groups can be either **scalars**, **grid functions** (GFs), or **grid arrays**. Different attributes are relevant for the different group types.

SCALAR	This is just a single number, e.g. the total energy of some field. These variables aren't communicated between processors — what would be the result of such communication?
GF	This is the most common group type. A GF is an array with a specific size, set at run-time in the parameter file, which is distributed across processors. All GFs have the same size, and the same number of ghostzones. Groups of GFs can also specify a dimension, number of timelevels, and stagger type.
ARRAY	This is a more general form of the GF. Each group of arrays can have a distinct size and number of ghostzones, in addition to dimension, number of timelevels and stagger type. The drawback of using an array over a GF is that a lot of data about the array can only be determined by function calls, rather than the quicker methods available for GFs.

B4.3 Timelevels

These are best introduced by an example using finite differencing. Consider the 1-D wave equation

$$\frac{\partial^2 \phi}{\partial t^2} = \frac{\partial^2 \phi}{\partial x^2} \quad (\text{B4.1})$$

To solve this by partial differences, one discretises the derivatives, to get an equation relating the solution at different times. There are many ways to do this, one of which produces the following difference equation

$$\phi(t + \Delta t, x) - 2\phi(t, x) + \phi(t - \Delta t, x) = \frac{\Delta t^2}{\Delta x^2} \{ \phi(t, x + \Delta x) - 2\phi(t, x) + \phi(t, x - \Delta x) \} \quad (\text{B4.2})$$

which relates the three timelevels $t + \Delta t$, t , and $t - \Delta t$.

Obviously the code could just use three variables, one for each timelevel. This turns out, however, to be inefficient, because as soon as the time is incremented to $t + \Delta t$, it would be necessary to copy data from the t variable to the $t - \Delta t$ variable and from the $t + \Delta t$ variable to the t variable.

To remove this extraneous copy, Cactus allows you to specify the number of timelevels used by your numerical scheme. It then generates variables with the base name (e.g. `phi`) suffixed by a qualifier for which timelevel is being referred to — no suffix for the next timelevel, and `_p` for each previous timelevel.

The timelevels are rotated (by the driver thorn) at the start of each evolution step, that is:

```
initial
poststep
analysis

loop:
  rotate timelevels
  t = t + dt
  it = it + 1
  prestep
  evolve
  poststep
  analysis
```

Timelevel rotation means that, for example, `phi_p` now holds the values of the former `phi`, and `phi_p_p` the values of the former `phi_p`, etc. Note that after rotation `phi` is undefined, and it's values should not be used until they have been updated.

All timelevels, except the current level, should be considered **read-only** during evolution, that is their values should not be changed by thorns. The exception to this rule is for function initialisation, when the values at the previous timelevels do need to be explicitly filled out.

B4.4 Size and Distrib

A Cactus grid function or array has a size set at runtime by parameters. This size can either be the global size of the array across all processors (`DISTRIB=DEFAULT`), or, if `DISTRIB=CONSTANT` the specified size on **each** processor. If the size is split across processors, the driver thorn is responsible for assigning the size on each processor.

B4.5 Ghost Size

Cactus is based upon a **distributed computing** paradigm. That is, the problem domain is split into blocks, each of which is assigned to a processor. For hyperbolic and parabolic problems the blocks only need to communicate at the edges.

To illustrate this, take the example of the wave equation again. Equation B4.2 describes a possible time-evolution scheme. On examination you can see that to generate the data at the point $(t + \Delta t, x)$ we need data from the four points (t, x) , $(t - \Delta t, x)$, $(t, x + \Delta x)$ and $(t, x - \Delta x)$ only. Ignoring the points at x , which are obviously always available on a given processor, you can see that the algorithm requires a point on either side of the point x , i.e. this algorithm has **stencil size 1**. Similarly algorithms requiring two points on either side have stencil size 2, etc.

Now, if you evolve the above scheme, it becomes apparent that at each iteration the number of grid points you can evolve decreases by one at each edge (see figure B4.1).

At the outer boundary of the physical domain the data for the boundary point can be generated by the boundary conditions, however at internal boundaries the data has to be copied from the adjacent processor. It would be inefficient to copy each point individually, so instead, a number of **ghostzones** are created at the internal boundaries. A ghostzone consists of a copy of the whole plane (in 3d, line in 2d, point in 1d) of the data from the adjacent processor. That is, the array on each processor is augmented with copies of points from the adjacent processors, thus allowing the algorithm to proceed **on the points**

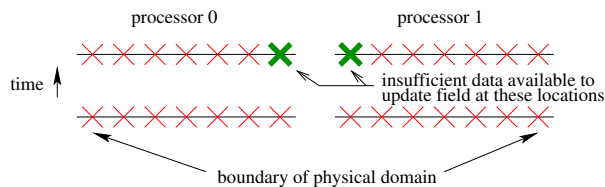


Figure B4.1: Distributed wave equation with no ghostzones

owned by this processor without having to worry about copying data. Once the data has been evolved one step, the data in the ghostzones can be exchanged (or **synchronised**) between processors in one fell swoop before the next evolution step. (See figure B4.2.) Note that you should have at least as many ghostzones as your stencil-size requires.

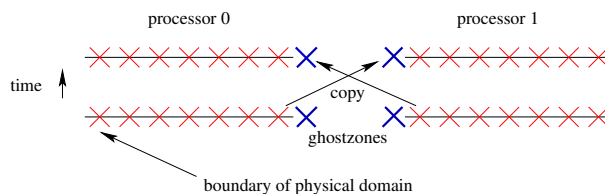


Figure B4.2: Distributed wave equation with ghostzones

B4.6 Staggering

The staggering of a gridfunction or array describes the *physical* placement of that gridfunction relative to the supporting grid structure. For example, a gridfunction does not have to be placed at the intersection of the “grid lines”. It can be moved by half a grid spacing in any or all dimensions. In the latter case, it will be placed in the center of a cell.

The staggering of a grid function is a pure *physical* property: the values will be calculated at a different position in physical space. Still the indexing (or bookkeeping) is kept the same for all types of staggers: the indexing of the default unstaggered grids is used.

Specifying the staggetype

The type of staggering applied to a gridfunction can be specified in the `interface.ccl` file by the attribute `stagger` (see F2.2). Cactus supports three kinds of staggering per dimension. The physical location of a gridfunction is shifted relative to the default position by adding the following values to the `stagger` attribute:

M	no staggering, default. Refers to the “minus” face relative to the default gridpoint.
C	centre staggering. The physical location is offset by half of the grid spacing in the positive direction (or to the right).
P	full staggered. P refers to plus. The physical location is offset by a full gridspacing in the positive direction (or the right).

For multi dimensional gridfunctions you concatenate the code characters in xyz order. In Figure B4.3 we show four different staggers of a two dimensional grid function. The solid black grid circles show the default location of the grid function at the intersections of the grid lines. In (A) we show an additional

grid function of type `stagger=MC`: no staggering in x direction, center staggered in y direction. In (B) we have `stagger=CM` and staggering each direction (`stagger=CC`) is shown in (C). The full staggering in (D) (`stagger=PP`) obeys the same rules, but is rather unusual; it is included here for completeness.

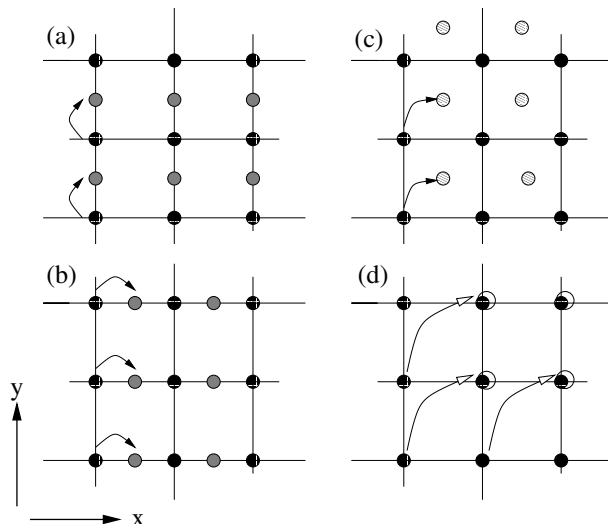


Figure B4.3: **Staggered gridpoints in 2D** for several staggerings. (a) : MC, (b): CM, (c): CC, (d): PP. Note that the staggering of gridfunctions does not change its index. The staggered gridpoints and the corresponding unstaggered points (arrows) are accessed by the same indices.

B4.7 Information about Grid Variables

The Flesh holds a database with information related to grid variables, and provides a set of querying APIs.

B4.7.1 Group Information

Fundamental information about grid functions (e.g. local grid size and location, number of ghostzones) is passed through the argument list of scheduled routines (see B7.2.3). To obtain similar information from non-scheduled routines, or for general grid variables, a set of functions are provided, the last two letters of which specify whether the information is requested using a group name (GN) or index (GI), or a variable name (VN) or index (VI).

- `CCTK_Group1sh[GN|GI|VN|VI]` An array of integers with the local grid size on this processor.
- `CCTK_Groupgsh[GN|GI|VN|VI]` An array of integers with the global grid size.
- `CCTK_Groupbbox[GN|GI|VN|VI]` An array of integers which indicate whether the boundaries are internal boundaries (e.g. between processors), or physical boundaries. A value of 1 indicates a physical (outer) boundary at the edge of the computational grid, and 0 indicates an internal boundary.
- `CCTK_Groupnghostzones[GN|GI|VN|VI]` An array of integers with the number of ghostzones used in each direction.
- `CCTK_Group1bnd[GN|GI|VN|VI]` An array of integers containing the lowest index (in each direction) of the local grid, as seen on the global grid. Note that these indices start from zero, so you need to add one when using them in Fortran thorns.

- `CCTK_Groupubnd[GN|GI|VN|VI]` An array of integers containing the largest index (in each direction) of the local grid, as seen on the global grid. Note that these indices start from zero, so you need to add one when using them in Fortran thorns.

Chapter B5

Cactus Parameters

Parameters are the means by which the user specifies the run-time behaviour of the code. The user specifies values for parameters in the parameter file, and then the flesh validates these values against the ranges the thorn writers have specified to be valid. Once validated, parameter values are fixed, and cannot be changed (unless the parameter is specified to be steerable, see below). For a detailed discussion of the `param.ccl` syntax see Appendix [F2.3](#).

The full specification for a parameter decalaration is

```
[EXTENDS|USES] <parameter_type>[[<size>]] <parameter name> "<parameter description>"
{
    <PARAMETER_RANGES>
} <default value>
```

B5.1 Types and Ranges

Parameters can be of several types:

Integer	Can take any integral value
Real	Can take any floating point value
Keyword	Can have a value consisting of one of a choice of strings
Boolean	Can be true or false (1, 't', 'true', or 0, 'f', 'false').
String	Can have any string value

Each parameter can be validated against a set of allowed values or ranges, each of which has a description associated with it. The type of range is determined by the type of parameter.

B5.1.1 Integer

The range specification is of the form

`lower:upper:stride`

where *lower* and *upper* specify the lower and upper allowed range, and *stride* allows numbers to be missed out. e.g.

`1:21:2`

means the value must be an odd number between one and twenty-one (inclusive).

A missing end of range (or a ‘*’) implies negative or positive infinity, and the default stride is one.

B5.1.2 Real

The range specification is of the form

`lower:upper`

where *lower* and *upper* specify the lower and upper allowed range. A missing end of range (or a ‘*’) implies negative or positive infinity. The above is inclusive of the end-points. A ‘(’ (or ‘)’) before (or after) the lower (or upper) range specifies an open end-point.

The numbers written in a param.ccl file are interpreted as C code. To express a number in ‘scientific notation’, use e.g. “1e-10”, which is a double precision constant in C. (If the floating precision of the variable to which it is assigned is not double, then C will typecast appropriately. If you *really* want to specify a single precision floating constant, or a long double constant, append the number with `f` or `l` respectively.)

B5.1.3 Keyword

The range specification consists of a string, which will be matched in a case-insensitive manner.

B5.1.4 Boolean

There is no range specification for this type of parameter.

B5.1.5 String

The range is a POSIX regular expression. On some machines you may be able to use extended regular expressions, but this is not guaranteed to be portable.

B5.2 Scope

Parameters can be *GLOBAL*, *RESTRICTED*, or *PRIVATE*. Global parameters are visible to all thorns. Restricted parameters are visible to any thorn which chooses to *USE* or *EXTEND* it. A private parameter is only visible to the thorn which declares it.

Chapter B6

Scheduling

Cactus contains a rule-based scheduling system, which determines which routines from which thorns are run in which order. The scheduler determines if the specifications are inconsistent, but does allow the user to schedule a routine with respect to another routine which may not exist. For a detailed discussion of the `schedule.ccl` syntax see Appendix [F2.4](#).

A usual simple specification for a schedule declaration is

```
schedule <function name> AT <schedule bin>
{
  LANG: <language>
  [STORAGE:      <group>,<group>...]
} "Description of function"
```

The full specification for a schedule declaration is

```
schedule [GROUP] <function|schedule group name> AT|IN <schedule bin|group name> \
          [BEFORE|AFTER <item>] [WHILE <variable>] [AS <alias>]
{
  LANG: <language>
  [STORAGE:      <group>,<group>...]
  [TRIGGER:      <group>,<group>...]
  [SYNC:         <group>,<group>...]
  [OPTIONS:      <option>,<option>...]
} "Description of function or schedule group"
```

This full schedule specification consists of a mandatory part, a set of options, and the main body limited by braces, referred to as the `schedule block`.

Each schedule item is scheduled either *AT* a particular *scheduling bin*, or *IN* a *schedule group*.

B6.1 Schedule Bins

These are the main times at which scheduled functions are run, from fixed points in the Flesh and Driver thorn (schedule bins can easily be traversed from any thorn, although this is not usual). When a schedule bin is *traversed*, all the functions scheduled in that particular are called, in the manner described in [B6.5](#) and respecting the requested ordering of functions [B6.3](#). In the absence of any ordering, functions in a particular schedule bin will be called in an undetermined order.

The schedule bins are described in [B3.3.4](#). Note that the preceeding `CCTK_` is optional for the use of the bin names in the `schedule.ccl` file.

B6.2 Groups

If the optional `GROUP` specifier is used, the item is a schedule group rather than a normal function. Schedule groups are effectively new, user-defined, schedule bins. Functions or groups may be scheduled *IN* these in the same way as they are scheduled *AT* the main schedule bins. (That is, groups may be nested.)

B6.3 Schedule Options

The options define various characteristics of the schedule item.

BEFORE or AFTER	These specify a function or group before or after which this item will be scheduled.
WHILE	This specifies a <code>CCTK_INT</code> grid scalar which is used to control the execution of this item. If the grid scalar has a non-zero value the schedule item will be executed, otherwise the item will be ignored. This allows some dynamic behaviour with scheduling.
AS	This assigns a new name to a function for scheduling purposes. This is used, for instance, to allow a thorn to schedule something before or after a routine from another implementation; two thorns providing this implementation can schedule a routine <i>AS</i> the same thing, thus allowing other thorns to operate independently of which one is active.

B6.4 The Schedule Block

The schedule block specifies further details of the scheduled function or group.

LANG	This specifies the language of the routine. Currently this is either C or Fortran. C++ routines should be defined as <code>extern 'C'</code> and registered as <code>LANG: C</code> .
STORAGE	The <code>STORAGE</code> keyword specifies groups for which memory should be allocated for the duration of the routine or schedule group. The storage status reverts to its previous status after completion of the routine or schedule group.
TRIGGER	This is only used for items scheduled at <code>CCTK_ANALYSIS</code> . The item will only be executed if output is due for at least one variable in one of the listed groups.
SYNC	On exit from this item the ghost zones of the listed groups will be exchanged.
OPTIONS	This is for miscellaneous options. The only currently supported option is <code>GLOBAL</code> which tells the driver that this routine does not use a local grid, but instead uses global operations to process data; such a routine should only be called once however many sub-grids the driver may have broken the problem into.

B6.5 How Cactus Calls Scheduled Functions

For each scheduled function called, the flesh performs a variety of jobs at entry and exit.

On entry to a scheduled routine, if the routine is being called at the ANALYSIS timebin first a check is made to see if the routine should actually be called on this timestep. For this, all grid variables in the trigger groups for the routine are checked with all registered output methods to determine if it is time to output any triggers. The routine will only be called if at least one is due to be output. Routines from all other timebins are always called.

Next storage is assigned for any required variables, remembering the original state of storage.

The routine is then called, and on exit, any required grid variables are first synchronised. Following synchronization, any required output methods are called for the triggers. Finally, the storage of grid variables is returned to the original state.

Chapter B7

Writing a Thorn

B7.1 Thorn Programming Languages

When you start writing a new thorn, the first decision to make is which programming language to use? The source code in Cactus thorns can be written in any mixture of `FORTRAN77`, `FORTRAN90`, `C` or `C++`. The following points should be considered when choosing a language to work in

- All functions designed for application thorn writers are available in all languages, however some interfaces for infrastructure thorn writing are only available from `C` or `C++`.
- Stick to `C` rather than `C++`, unless you really need features from `C++`, this will help you with portability.

Whatever language you choose, if you want your thorn to be portable, and compile and run on multiple platforms, stick to the standards and don't use machine dependent extensions.

B7.2 What the Flesh provides

The flesh provides for thorns:

Variables

Parameters

Cactus Functions	• driver (parallelisation) utilities • IO utilities • Coordinates utilities • Reduction utilities • Interpolation utilities • Information utilities
------------------	---

B7.2.1 Fortran Routines

Any source file using Cactus infrastructure should include the header file `cctk.h` using the line

```
#include "cctk.h"
```

(Fortran programmers should not be put off by this being a C style header file, most Cactus files are run through a C preprocessor before compilation).

Variables

Any routine using Cactus argument lists (for example all routines called from the scheduler at time bins between `CCTK_STARTUP` and `CCTK_SHUTDOWN`) should include at the top of the file the header

```
#include "cctk_Arguments.h"
```

A Cactus macro `CCTK_ARGUMENTS` is defined for each thorn to contain:

- General information about the grid hierarchy, for example the number of grid points used. See Section [B7.2.3](#) for a complete list.
- All the grid variables defined in the thorn's `interface.ccl`
- All the grid variables required from other thorns as requested by the `inherits` and `friend` lines in the `interface.ccl`

These variables must be declared at the start of the routine using the macro `DECLARE_CCTK_ARGUMENTS`.

To pass the arguments to another routine in the same thorn use the macro `CCTK_PASS_FTOF` in the calling routine, and again the macro `CCTK_ARGUMENTS` in the receiving routine.

Note that you cannot use Cactus argument lists in routines scheduled at the `CCTK_STARTUP` and `CCTK_SHUTDOWN` time bins because at this time no grid hierarchy exists.

Parameters

All parameters defined in a thorn's `param.ccl` and all `global` parameters appear as local variables of the corresponding CCTK datatype in Fortran source code, ie. Booleans and Integers appear as `CCTK_INT` types (with non-zero/zero values for boolean `yes/no`), Reals as `CCTK_REAL`, and Keywords and String parameters as `CCTK_POINTER` (see also note below). These variables are *read only* and *changes should not be made to them*. The effect of changing a parameter is undefined (at best).

Any routine using Cactus parameters should include at the top of the file the header

```
#include "cctk_Parameters.h"
```

The parameters should be declared at the start of the routine using them with the macro `DECLARE_CCTK_PARAMETERS`.

In Fortran, special care should be taken with string valued parameters. These parameters are passed as C pointers, and can not be treated as normal Fortran strings. To compare a string valued parameter and Fortran string use the macro `CCTK_EQUALS()` or the function `CCTK_Equals()`. To print the value of a string valued parameter to screen, use the subroutine `CCTK_PrintString()`. A further function `CCTK_FortranString` provides a mechanism for converting a string parameter to a Fortran string. For example, if `operator` is a Cactus string parameter holding the name of a reduction operator whose handle you need to find, you cannot pass it directly into the subroutine `CCTK_ReductionHandle` which is expecting a Fortran string. Instead, the following is needed:

```

character*200 fortran_operator
CCTK_INT      fortran_operator_len
integer        handle

call CCTK_FortranString(fortran_operator_len,operator,fortran_operator)
call CCTK_ReductionHandle(handle,fortran_operator(1:fortran_operator_len))

```

Fortran Example

The Fortran routine `MyFRoutine` is scheduled in the `schedule.ccl` file, doesn't use Cactus parameters, and calls another routine, in the same thorn, `MyNewRoutine` which does use parameters. This routine needs to be passed an integer flag as well as the standard Cactus variables. The source file should look like

```

#include "cctk.h"
#include "cctk_Arguments.h"
#include "cctk_Parameters.h"

      subroutine MyFRoutine(CCTK_ARGUMENTS)

c      I'm very cautious, so I want to declare all variables
      implicit none

      DECLARE_CCTK_ARGUMENTS

      integer flag

      flag = 1
      call MyNewRoutine(CCTK_PASS_FTOF,flag)

      return
      end

      subroutine MyNewRoutine(CCTK_ARGUMENTS,flag)

      implicit none

      DECLARE_CCTK_ARGUMENTS
      DECLARE_CCTK_PARAMETERS
      integer flag

c      Main code goes here

      return
      end

```

Cactus Fortran Functions

Cactus Fortran functions, for example `CCTK_MyProc()` and `CCTK_Equals`, can all be declared by adding before any executable code, the declaration

```
DECLARE_CCTK_FUNCTIONS
```


Fortran 90 Modules

Fortran 90 modules should be included in a thorn's `make.code.deps` file ([B3.5.1](#)) to ensure they are compiled before the routines which use them. This is especially important for parallel building. For example, if a routine in `MyRoutine.F` uses a module in `MyModule.F` add the line:

```
$(SYS_OBJD)/MyRoutine.F.o:      $(SYS_OBJD)/MyModule.F.o
```

The MOD function

The intrinsic function `MOD` in Fortran takes two integer arguments, which should both be of the same type. This means that it may be necessary to cast the arguments to *e.g* `INT` for some architectures. This can occur in particular when a `CCTK_INT` parameter and the Cactus variable `cctk_iteration` (which is declared to be `INTEGER`) are used, in which case the correct code is

```
MOD(cctk_iteration,INT(MyParameter))
```

B7.2.2 C Routines

Any source file using Cactus infrastructure should include the header file `cctk.h` using the line

```
#include "cctk.h"
```

Variables

Any routine using Cactus argument lists (for example all routines called from the scheduler at time bins between `CCTK_STARTUP` and `CCTK_SHUTDOWN`) should include at the top of the file the header

```
#include "cctk_Arguments.h"
```

A Cactus macro `CCTK_ARGUMENTS` is defined for each thorn to contain

- General information about the grid hierarchy, for example the number of grid points on the processor. See Section [B7.2.3](#) for a complete list.
- All the grid variables defined in the thorn's `interface.ccl`
- All the grid variables required from other thorns as requested by the `inherits` and `friend` lines in the `interface.ccl`

These variables must be declared at the start of the routine using the macro `DECLARE_CCTK_ARGUMENTS`. This macro should always be the first line of the routine.

To pass the arguments to another routine in the same thorn use the macro `CCTK_PASS_CTOC` in the calling routine, and again the macro `CCTK_ARGUMENTS` in the receiving routine.

Note that you cannot use Cactus argument lists in routines scheduled at the `CCTK_STARTUP` and `CCTK_SHUTDOWN` time bins because at this time no grid hierarchy exists.

Parameters

All parameters defined in a thorn's `param.ccl` and all `global` parameters appear as local variables of the corresponding CCTK datatype in C source code, ie. Integers and Booleans appear as `CCTK_INT` types (with non-zero/zero values for boolean `yes/no`), Reals as `CCTK_REAL`, and Keywords and String parameters as `CCTK_STRING`. These variables are *read only* and *changes should not be made to them*. The effect of changing a parameter is undefined (at best).

Any routine using Cactus parameters should include at the top of the file the header

```
#include "cctk_Parameters.h"
```

The parameters should be declared as the last statement in the declaration part of the routine using them with the macro `DECLARE_CCTK_PARAMETERS`.

Example

The C routine “MyCRoutine” is scheduled in the `schedule.ccl` file, and uses Cactus parameters. The source file should look like

```
#include "cctk.h"
#include "cctk_Arguments.h"
#include "cctk_Parameters.h"

void MyCRoutine(CCTK_ARGUMENTS)
{
    DECLARE_CCTK_ARGUMENTS
    DECLARE_CCTK_PARAMETERS

    /* Here goes your code */
}
```

Complex variables

Cactus supports complex grid variables, and since there is no complex data type in C, Cactus provides a number of functions for manipulating complex numbers to mirror the functionality available in Fortran. These functions are `CCTK_Cmplx`, `CCTK_CmplxReal`, `CCTK_CmplxImag`, `CCTK_CmplxConjg`, `CCTK_CmplxAdd`, `CCTK_CmplxSub`, `CCTK_CmplxMul`, `CCTK_CmplxDiv`, `CCTK_CmplxExp`, `CCTK_CmplSin`, `CCTK_CmplxAbs`, `CCTK_CmplxLog`, and `CCTK_CmplSqrt`.

Specifically for C Programmers

Grid functions are held in memory as 1D C arrays. These are laid out in memory as in Fortran. This means that the first index should be incremented through most rapidly. This is illustrated in the example below.

Cactus provides macros to find the 1D index which is needed from the multidimensional indices which are usually used. There is a macro for each dimension of grid function. Below is an artificial example to demonstrate this using the 3D macro `CCTK_GFINDEX3D`:

```
for (k=0; k<cctk_lsh[2]; k++)
```

```

{
  for (j=0; j<cctk_lsh[1]; j++)
  {
    for (i=0; i<cctk_lsh[0]; i++)
    {
      My3D_GF[CCTK_GFINDEX3D(cctkGH,i,j,k)] = i*j*k;
    }
  }
}

```

Here, `CCTK_GFINDEX3D(cctkGH,i,j,k)` expands to

```
((i) + cctkGH->cctk_lsh[0]*((j)+cctkGH->cctk_lsh[1]*(k)))
```

B7.2.3 Cactus Variables

The Cactus variables which are passed through the macro `CCTK_ARGUMENTS` are

<code>cctkGH</code>	A C pointer identifying the grid hierarchy.
<code>cctk_dim</code>	An integer with the number of dimensions used for this grid hierarchy.
<code>cctk_lsh</code>	An array of <code>cctk_dim</code> integers with the local grid size on this processor.
<code>cctk_gsh</code>	An array of <code>cctk_dim</code> integers with the <i>global</i> grid size.
<code>cctk_iteration</code>	The current iteration number.
<code>cctk_delta_time</code>	A <code>CCTK_REAL</code> with the timestep.
<code>cctk_time</code>	A <code>CCTK_REAL</code> with the current time.
<code>cctk_delta_space</code>	An array of <code>cctk_dim</code> <code>CCTK_REALs</code> with the grid spacing in each direction.
<code>cctk_nghostzones</code>	An array of <code>cctk_dim</code> integers with the number of ghostzones used in each direction.
<code>cctk_origin_space</code>	An array of <code>cctk_dim</code> <code>CCTK_REALs</code> with the spatial coordinates of the global origin of the grid. The coordinates of the <i>i</i> th local grid point in the <i>x</i> direction can e.g. in C be calculated by <code>x = CCTK_ORIGIN_SPACE(0) + (cctk_lsh[0] + i) * CCTK_DELTA_SPACE(0)</code> .
<code>cctk_lssh</code>	This is an internal array used to hold array extents for staggering. One should use the macro <code>CCTK_LSSH(.)</code> to access its elements.

The following variables describe the location of the local grid (e.g. the grid treated on a given processor) within the global grid.

- `cctk_lbnd` An array of `cctk_dim` integers containing the lowest index (in each direction) of the local grid, as seen on the global grid. Note that these indices start from zero, so you need to add one when using them in Fortran thorns.
- `cctk_ubnd` An array of `cctk_dim` integers containing the largest index (in each direction) of the local grid, as seen on the global grid. Note that these indices start from zero, so you need to add one when using them in Fortran thorns.

- **cctk_bbox** An array of $2 \times \text{cctk_dim}$ integers (in the order $[\text{dim}_0^{\min}, \text{dim}_0^{\max}, \text{dim}_1^{\min}, \text{dim}_1^{\max}, \dots]$), which indicate whether the boundaries are internal boundaries (e.g. between processors), or physical boundaries. A value of 1 indicates a physical (outer) boundary at the edge of the computational grid, and 0 indicates an internal boundary.

The following variable is needed for grid refinement methods

- **cctk_levfac** An array of **cctk_dim** integer factors by which the local grid is refined in the corresponding direction with respect to the base grid.
- **cctk_levoff** and **cctk_levoffdenom** Two arrays of **cctk_dim** integers describing the distance by which the local grid is offset with respect to the base grid, measured in local grid spacings. The distance in direction **dir** is given by $1.0 * \text{cctk_levoff}[\text{dir}] / \text{cctk_levoffdenom}[\text{dir}]$.
- **cctk_timefac** The integer factor by which the time step size is reduced with respect to the base grid.

The following variable is used for identifying convergence levels. **NOTE:** Convergence is not currently implemented by Cactus, so that **cctk_convlevel** is currently always 0.

- **cctk_convlevel** The convergence level of this grid hierarchy. The base level is 0, and every level above that is currently coarsened by a factor of 2.

The variables **cctk_delta_space**, **cctk_delta_time**, and **cctk_origin_space** denote the grid spacings, time step size, and spatial origin on the *base* grid. If you are using a grid refinement method, you need to calculate these quantities on the grid you are on. There are Cactus macros provided for this, with the syntax **CCTK_DELTA_SPACE(dir)**, **CCTK_ORIGIN_SPACE(dir)**, and **CCTK_DELTA_TIME** for both C and Fortran. It is recommended that these macros are always used to provide the grid spacings, time step sizes, and spatial origins in your thorns. In doing so, you incorporate the effects of **cctk_levfac**, **cctk_levoff**, **cctk_levoffdenom**, and **cctk_timefac**, so that you do not explicitly have to take them into account.

B7.2.4 Cactus Data Types

The Cactus grid variables and parameters are defined and declared using Cactus data types, to provide portability across platforms. The most important of these data types are described below, for a full description see Section B10.8. These data types should be used to declare local variables where needed, and to declare Cactus grid variables or parameters that need declarations.

CCTK_INT	default size 4 bytes
CCTK_REAL	default size 8 bytes

Example

In the following example **MyScalar** is a grid scalar which is declared in the **interface.ccl** as **CCTK_REAL**.

```
subroutine InitialData(CCTK_ARGUMENTS)

  DECLARE_CCTK_ARGUMENTS
```

```

CCTK_REAL local_var

local_var = 1.0/3.0
MyScalar = local_var

return
end

```

Declaring `local_var` to have a non-Cactus data type, e.g. `REAL*4`, or using one of the other Cactus real data types described in Section B10.8 could give problems for different architectures or configurations.

B7.2.5 Staggering

Indexing, ghostzones, etc.

Note that staggering does not make any changes to the indexing of a gridfunction: the black solid circles in diagram B7.1 and their associated staggered gridfunctions (connected by arrows) have the same index!

Since the gridfunction does not “know” anything about the physical location (it’s only addressed by indices) why add staggering if the indexing is the same?

Indeed, you could roll your own, but there are compelling reasons: Readability and the fact that you are able to query the staggetype of a gridfunction. More important: in the way the grid is laid out, there is one grid point *less* for M and P staggered grid functions. This is illustrated in picture B7.1, which shows 15 gridpoints distributed across 3 processors. The solid black circles show the default location of the gridfunctions, the grey circles depict the ghostzones. Note that the number of center staggered gridpoints (fat crosses) corresponds to the number of default gridpoints on all processors but the last one. (The same is true for full staggered gridpoints).

Staggetypes

The string specifying the staggering is encoded in a number called the *staggerindex*. With the 3 supported staggetings, the string is converted into a base-3 number. Several routines exist, to extract the staggering in a specific direction, called *directional staggerindex*. E.g. `stagger = MCM`: *staggerindex* = 3, in the x-direction: *directional staggerindex* = `CCTK_STAGGER_M` (value 0), in the y-direction: *directional staggerindex* = `CCTK_STAGGER_C` (value 1).

<code>CCTK_STAGGER_M</code>	value used for M-type staggering
<code>CCTK_STAGGER_C</code>	value used for C-type staggering
<code>CCTK_STAGGER_P</code>	value used for P-type staggering
<code>CCTK_NO_STAGGER</code>	value to indicate no staggering
<code>CCTK_STAGGER</code>	value to indicate staggering
<code>CCTK_NSTAGGER</code>	number of coded staggetings (3)

When a thorn programmer uses staggered gridpoints, he has to be aware of this gridpoint anomaly. This can be done most easily by using the `CCTK_LSSH(<dir_staggetype>,<direction>)` macro. For a given staggetype and direction, this 2d array returns the local number of gridpoints, including ghostzones and the necessary change for the staggering on the outermost processor.

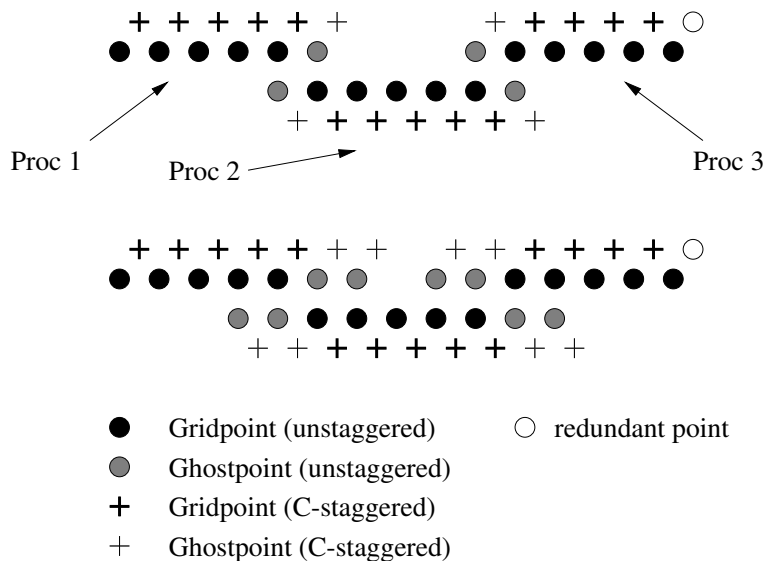


Figure B7.1: **Unstaggered and center-staggered gridpoints** with ghostzone size of one (above) and two (below). The points are distributed across three processors. Note that the number of center staggered gridpoints (fat crosses) is one less on the outermost grid. How to treat this case in a easy way is explained below.

`CCTL_LSSH(<dir_staggertype>, <direction>)`

for a given staggertype and a direction this macro returns the number of processor local gridpoints, including ghostzones.

- macro has to be in capital letters
- This macro is C/Fortran indexing aware: can specify the dimension in C ranging from 0... and in Fortan ranging from 1...

Several functions exist to derive the staggertype for a given group and for a certain direction.

`int CCTK_GroupStaggerIndexGI(int group_index)`

call `CCTK_GroupStaggerIndexGI(int staggerindex, int group_index)`
returns the *staggerindex* for a given group index.

`int CCTK_GroupStaggerIndexGN(char *group_name)`

call `CCTK_GroupStaggerIndexGN(int staggerindex, char *group_name)`
returns the *staggerindex* for a given group name.

`int CCTK_GroupStaggerIndexVI(int variable_index)`

call `CCTK_GroupStaggerIndexVI(int staggerindex, int variable_index)`
returns the *staggerindex* for a given variable index.

`int CCTK_GroupStaggerIndexVN(char *variable_name)`

```
call CCTK_GroupStaggerIndexVN(int staggerindex, char *variable_name)
    returns the staggerindex for a given variable name.

int CCTK_StaggerIndex(char *stagger_string)

call CCTK_StaggerIndex(int staggerindex, char *stagger_string)
    return the staggerindex for a given stagger string.

int CCTK_DirStaggerIndex(int direction, char *stagger_string)

call CCTK_DirStaggerIndex(int dir_staggerindex, int direction, char *stagger_string)
    returns the directional staggerindex for a given direction and stagger string.

int CCTK_DirStaggerIndexI(int direction, char *stagger_type)

call CCTK_DirStaggerIndexI(int dir_direction, char *stagger_type)
    returns the directional staggerindex for a given direction and staggerindex.
```

B7.3 Parallelisation

The flesh itself does not actually set up grid variables. This is done by a *driver* thorn. To allow the distribution of a grid over a number of processors, the driver thorn must also provide the grid decomposition, and routines to enable parallelisation. The method used to provide this parallelisation (e.g. MPI, PVM) is not usually important for the thorn writer since the driver thorn provides routines which are called by standard interfaces from the flesh. Here we describe briefly the most important of these routines for the application thorn writer. A more detailed description of these interfaces with their arguments, is given in the Reference Manual. A complete description of the routines a driver thorn must provide will be provided in the Infrastructure Thorn Writers guide (Part D). The standard driver thorn is currently PUGH in the CactusPUGH package, which is a parallel unigrid driver.

CCTK_nProcs	Returns the number of processors being used
CCTK_MyProc	Returns the processor number (this starts at processor number zero)
CCTK_SyncGroup	Synchronises a group of variables by exchanging the values held in each processor ghostzones with the physical values of their neighbours
CCTK_Barrier	Waits for all processors to reach this point before proceeding

Chapter B8

Cactus Application Interfaces

B8.1 Coordinates

The flesh provides utility routines for registering and querying coordinate information. The flesh does not provide any coordinates itself, these must be supplied by a thorn. Thorns are not required to register coordinates to the flesh, but registering coordinates provides a means for infrastructure thorns to make use of coordinate information.

Coordinate support is still being developed in the Cactus flesh. At the moment, it is assumed that coordinates will usually be grid functions.

Coordinates are grouped into *coordinate systems*, which have a specified dimension. Any number of coordinate systems can be registered with the flesh, and a coordinate system must be registered before any coordinates can be registered, since they must be associated with their corresponding system. Coordinates can be registered, with any chosen name, with an existing coordinate system, along with their direction or index in the coordinate system. Optionally, the coordinate can also be associated with a given grid variable. A separate call can register the global range for a coordinate on a given grid hierarchy.

Following conventions for coordinate system and coordinate names provides a means for other thorns to use the physical properties of coordinate systems, without being tied to a particular thorn.

A registered coordinate system can be referred to by both its name and an associated handle. Passing such an integer handle instead of the name string may be necessary for calling C routines from Fortran.

B8.1.1 Registering Coordinates and Coordinate Properties

Coordinate systems and their properties can be registered at any time with the Flesh. The registration utilities for thorns providing coordinates are:

CCTK_CoordRegisterSystem

Assigns a coordinate system with a chosen name and dimension. For example, a 3-dimensional Cartesian coordinate system could be registered with the name `cart3d` using the call from C

```
int ierr;  
int dim=3;  
ierr = CCTK_CoordRegisterSystem(dim,"cart3d");
```

CCTK_CoordRegisterData

Defines a coordinate in a given coordinate system, with a given direction and name,

and optionally associates it to a grid variable. The directions of the coordinates range from 1 to the dimension of the coordinate system. For example, to register the grid variable *grid::y3d* to have the coordinate name *y* in the **cart3d** system

```
int ierr;
int dir=2;
ierr = CCTK_CoordRegisterData(dir,"grid:y3d","y","cart3d");
```

CCTK_CoordRegisterRange

Assigns the global computational maximum and minimum for a coordinate on a grid hierarchy, that is in a **cctkGH**. At this time the maximum and minimum values have to be of type **CCTK_REAL**. For example, if the *y* coordinate for the **cart3d** system ranges between zero and one

```
CCTK_REAL lower=0;
CCTK_REAL upper=1;
int ierr;
ierr = CCTK_CoordRegisterRange(cctkGH, lower, upper, -1, "y", "cart3d");
```

Note that the API allows either the coordinate name or the direction to be used, so that the following is also valid

```
CCTK_REAL lower=0;
CCTK_REAL upper=1;
int ierr;
ierr = CCTK_CoordRegisterRange(cctkGH, lower, upper, 2, NULL, "cart3d");
```

CCTK_CoordRegisterPhysIndex

Implementing such things as symmetry properties for a grid leads to the need to know the details of the *physical* section of a grid. Such information is typically needed by IO thorns. The following call illustrates how to register the indices 3 and 25 as supplying the physical range of the *y* coordinate in the **cart3d** system

```
int loweri=3;
int upperi=25;
int ierr;
ierr = CCTK_CoordRegisterPhysIndex(cctkGH, loweri, upperi, -1, "y", "cart3d");
```

B8.1.2 Using Coordinates

The utilities for thorns using coordinates are:

CCTK_NumCoordSystems

Returns the number of coordinate systems registered with the **Flesh**. For example,

```
int num;
num = CCTK_NumCoordSystems();
```

CCTK_CoordSystemName

Provides the name of a registered coordinate system, given the integer handle (or index) for the system in the **Flesh**'s coordinate data base. Note that the handle ranges between zero and the number of coordinate systems minus one: $0 \leq \text{handle} \leq \text{CCTK_NumCoordSystems}() - 1$. It is important to remember that the handle given to a coordinate system depends on the order in which systems are registered, and can be different from one simulation to the next.

For example, to print the names of all registered coordinate systems:

```
for (i=0; i<CCTK_NumCoordSystems(); i++)
    printf("%s ", CCTK_CoordSystemName(i));
```

CCTK_CoordSystemDim

Provides the dimension of a coordinate system. For example, if the `cart3d` system was registered as having 3 dimensions, the variable `dim` below will now be set to 3,

```
int dim;
dim = CCTK_CoordSystemDim("cart3d");
```

CCTK_CoordSystemHandle

Provides the integer handle for a given coordinate system name. The handle describes the index for the coordinate system in the Flesh coordinate database, and its value will range between zero and the number of registered systems minus one. For example, the handle for the `cart3d` coordinate system can be found using

```
int handle;
handle = CCTK_CoordSystemHandle("cart3d");
```

CCTK_CoordSystemName

The inverse to the previous function call, this provides the name for a given coordinate system handle. For example to find the first coordinate system in the Flesh database

```
int handle = 0;
const char *name = CCTK_CoordSystemName(handle);
```

CCTK_CoordIndex

Provides the grid variable index for a given coordinate. Note that it is not necessary for a registered coordinate to have an associated grid variable, and if no such grid variable is found a negative integer will be returned. For example, to find the grid variable index associated with the `y` coordinate of the `cart3d` system, either of the two following calls could be made

```
int index;
index = CCTK_CoordIndex(2, NULL, "cart3d");

int index;
index = CCTK_CoordIndex(-1, "y", "cart3d");
```

CCTK_CoordDir

Provides the direction for a given coordinate. Directions are integers ranging from one to the number of dimensions for the coordinate system. For example, to return the direction of the `y` coordinate in the `cart3d` system

```
int dir;
dir = CCTK_CoordDir("y", "cart3d");
```

The return of a negative integer indicates that the coordinate direction could not be found.

CCTK_CoordRange

Provides the global range (that is, the minimum and maximum values across the complete grid) of a coordinate on a given grid hierarchy. Currently the minimum and maximum values must be of type `CCTK_REAL`. The coordinate can be specified either by name or by its direction. Note that this call returns the **addresses** or the minimum and maximum values. For example, the range of the `y` coordinate of the `cart3d` coordinate system can be found using

```
CCTK_REAL lower, upper;
int ierr;
ierr = CCTK_CoordRange(cctkGH, &lower, &upper, -1, "y", "cart3d"); or
alternatively, using the direction

CCTK_REAL lower, upper;
int ierr;
ierr = CCTK_CoordRange(cctkGH, &lower, &upper, 2, NULL, "cart3d");
```

CCTK_CoordLocalRange

Provides the local range of a coordinate on a processor for a given grid hierarchy. **WARNING:** This utility only currently works for regular cartesian grids. For example, the local processor range of the `y` coordinate of the `cart3d` coordinate system can be found using

```

CCTK_REAL lower, upper;
int ierr;
ierr = CCTK_CoordLocalRange(cctkGH, &lower, &upper, -1, "y", "cart3d");
or alternatively, using the direction
CCTK_REAL lower, upper;
int ierr;
ierr = CCTK_CoordLocalRange(cctkGH, &lower, &upper, 2, NULL, "cart3d");

```

CCTK_CoordRangePhysIndex

For a given coordinate, provides the indices describing the *physical* range of the coordinate. A negative return value signifies that no such range was registered for the coordinate.

This index range provides a mechanism for describing grid points which should not be considered part of the simulation results (for example, grid points used for different boundary conditions). The physical range of the y coordinate of the `cart3d` system can be found using

```

int ilower, iupper;
int ierr;
ierr = CCTK_CoordRangePhysIndex(cctkGH,&ilower,&iupper, -1, "y", "cart3d");
or using the coordinate direction
int ilower, iupper;
int ierr;
ierr = CCTK_CoordRangePhysIndex(cctkGH,&ilower,&iupper, 2, NULL, "cart3d");

```

CCTK_CoordSystemImplementation

This call returns the name of the implementation which registered a coordinate system. Note that there is no guarantee that a thorn which registered a coordinate system is the same thorn which registers each of the coordinates in the system, although this should usually be the case.

B8.2 IO

To allow flexible IO, the flesh itself does not provide any output routines, however it provides a mechanism for thorns to register different routines as IO methods (see chapter D8). Application thorns can interact with the different IO methods through the following function calls:

CCTK_OutputGH (const cGH *GH)

This call loops over all registered IO methods, calling the routine that each method has registered for `OutputGH`. The expected behaviour of any `OutputGH` routine is to loop over all GH variables outputting them if the IO method contains appropriate routines (that is, not all methods will supply routines to output all different types of variables) and if the method decides it is an appropriate time to output.

CCTK_OutputVar (const cGH *GH, const char *varname)

Output a variable `varname` looping over all registered IO methods. The output should take place if at all possible. If output goes into a file and the appropriate file exists the data is appended, otherwise a new file is created.

CCTK_OutputVarAs (const cGH *GH, const char *varname, const char *alias)

Output a variable `varname` looping over all registered IO methods. The output should take place if at all possible. If output goes into a file and the appropriate file exists the data is appended, otherwise a new file is created. Uses `alias` as the name of the variable for the purpose of constructing a filename.

`CCTK_OutputVarByMethod (const cGH *GH, const char *varname, const char *methodname)`

Output a variable `varname` using the IO method `methodname` if it is registered. The output should take place if at all possible. If output goes into a file and the appropriate file exists the data is appended, otherwise a new file is created.

`CCTK_OutputVarAsByMethod (const cGH *GH, const char *varname, const char *methodname, const char *alias)`

Output a variable `varname` using the IO method `methodname` if it is registered. The output should take place if at all possible. If output goes into a file and the appropriate file exists the data is appended, otherwise a new file is created. Uses `alias` as the name of the variable for the purpose of constructing a filename.

B8.3 Interpolation Operators

The flesh does not provide interpolation routines by itself. Instead it offers a general function API to thorns for the registration and invocation of interpolation operators.

There are several different flesh APIs for interpolation, depending on whether the data arrays are Cactus grid arrays or processor-local “ordinary programming-language” arrays, and on what assumptions are made about the topology and spacing of the grid (these descriptions are for 3-D, but the generalizations to other numbers of dimensions should be obvious):

`CCTK_InterpGV()` Interpolates Cactus grid arrays, with the topology of the grid implicitly specified by a Cactus coordinate system. (old API, will be phased out soon)

`CCTK_InterpGridArrays()` Interpolates Cactus grid arrays, with the topology of the grid implicitly specified by a Cactus coordinate system.

This API doesn’t provide an interpolation functionality itself, it only takes care of the interprocessor communication necessary when interpolating distributed grid arrays, and invokes the `CCTK_InterpLocalUniform()` API on the each processor’s local patch of the data.

`CCTK_InterpLocal()` Interpolate processor-local arrays (old API, will be phased out soon)

`CCTK_InterpLocalUniform()` Interpolates processor-local arrays with *uniformly* spaced data points, i.e. where the coordinates *xyz* are related to the integer array subscripts *ijk* by *linear* functions

$$\begin{aligned}x &= \text{origin}_x + \text{delta}_x i \\y &= \text{origin}_y + \text{delta}_y j \\z &= \text{origin}_z + \text{delta}_z k\end{aligned}$$

where the caller specifies the `origin` and `delta` values.

There are separate flesh routines to register interpolation operators for the APIs (note the calling sequences differ slightly from one registration routine to another!):

`CCTK_InterpRegisterOperatorGV()` Register a `CCTK_InterpGV()` interpolation operator (old API, will be phased out soon)

`CCTK_InterpRegisterOperatorLocal()` Register a `CCTK_InterpLocal()` interpolation operator (old API, will be phased out soon)

```
CCTK_InterpRegisterOpLocalUniform()
```

Register a CCTK_InterpLocalUniform() interpolation operator

These are described in detail in the Reference Manual.

Each operator is registered under a character-string name; at registration the name is mapped to a unique integer handle which may be used to refer to the operator. CCTK_InterpHandle() is used to get the handle corresponding to a given character-string name. In general each name/handle is actually associated with a *set* of interpolation operators, one for each of the interpolation APIs.¹ The combination of a name/handle and an API must be globally unique with a Cactus binary, and uniquely identifies the interpolation operator.

For example, suppose thorn A registers CCTK_InterpGV() and CCTK_InterpLocal() interpolation operators under the name "interpolator A", and thorn B registers CCTK_InterpLocal() and CCTK_InterpLocalUniform() interpolation operators under the name "B's new interpolator". Then any thorn may still register a CCTK_InterpLocalUniform() interpolation operator under the name "interpolator A", or a CCTK_InterpGV() interpolation operator under then name "B's new interpolator".

B8.4 Reduction Operators

A reduction operation can be defined as an operation on variables distributed across multiple processors resulting in a single number. Typical reduction operations are sum, minimum/maximum value, and boolean operations. A typical application is, for example, sending to each processor the maximum value of the grid function holding the truncation error.

The exchange of information across processors needs the functionality of a communication layer e.g. CactusPUGH/PUGH. For this reason, the reduction operation itself is not part of the flesh, instead Cactus (again) provides a registration mechanism for thorns to register routines they provide as reduction operators. The different operators are identified by their name and/or a unique number, called a *handle*. The registration mechanism gives the advantage of a common interface while hiding the individual communication calls in the layer.

In Cactus, reduction operators can in principle be applied to grid functions, arrays, and scalars, as well as to local (non CCTK-) arrays. Note that different implementations of reduction operators may be limited in the types of objects to which they can be applied. There is a fundamental difference between a reduction operation on CCTK variables (grid functions, arrays, scalars) and arbitrary (non-CCTK, local) arrays (which includes a single variable as a special case of a one element array).

The reduction interface is currently under revision, and in the future should closely resemble that of the new interpolator interface. See e.g. the description of CCTK_InterpLocalUniform() in the Reference Manual.

Obtaining the reduction handle

Before calling the routine which performs the reduction operation, the handle, which indentifies the operation, must be derived from its registered name. To obtain a handle for the reduction of a CCTK variable, use

```
int CCTK_ReductionHandle(const char *reduction_name);
```

```
call CCTK_ReductionHandle(reduction_handle, reduction_name)
integer      reduction_handle
character*(*) reduction_name
```

¹If (as is often the case) an operator doesn't support all the APIs, the unused ones should be be left unregistered.

(for C or Fortran respectively), while for a local, non-CCTK-array, use

```
int CCTK_ReductionArrayHandle(const char *reduction_name);

call CCTK_ReductionArrayHandle(reduction_handle, reduction_name)
integer      reduction_handle
character*(*) reduction_name
```

reduction_handle Each function returns a reduction handle. In Fortran the result will be stored in this parameter. A negative handle value indicates failure to identify the correct operator.

reduction_name is the name under which the operator has been registered by the providing thorn. The only thorn in the standard Computational Toolkit release which provides reduction operators is CactusPUGH/PUGHReduce.

(Note that although it would appear to be far more convenient to pass the name of the reduction operator directly to the following function call to `CCTK_Reduce` this causes problems with the translation of strings from FORTRAN to C with variable argument lists).

The general reduction interface

The main interfaces for reduction operations described here are quite powerful (and hence rather complicated). To ease the use of these main interfaces, wrappers designed for specific and more restricted use are described in the next section. If uncertain, you should use those simpler interfaces.

To reduce any CCTK variable, use

```
int CCTK_Reduce(  const cGH *GH,
                  int processor,
                  int operation_handle,
                  int num_out_vals,
                  int type_out_vals,
                  void *out_vals,
                  int num_in_fields,
                  ...);

call CCTK_Reduce( integer returnvalue,
                  CCTK_POINTER_TO_CONST cctkGH,
                  integer processor,
                  integer operation_handle,
                  integer num_out_vals,
                  integer type_out_vals,
                  CCTK_POINTER out_vals,
                  integer num_in_fields,
                  ... )
```

(for C or Fortran respectively). To reduce any non-CCTK (local) variable, use

```
int CCTK_ReduceArray(  const cGH *GH,
                       int processor,
                       int operation_handle,
                       int num_out_vals,
                       int type_out_vals,
```

	<pre> void *out_vals, int num_dims, int num_in_arrays, int type_in_arrays, ...) call CCTK_ReduceArray(integer returnvalue, CCTK_POINTER_TO_CONST cctkGH, integer processor, integer operation_handle, integer num_out_vals, integer type_out_arrays, CCTK_POINTER out_vals, integer num_dims, integer num_in_arrays, integer type_in_arrays, ...) </pre>
returnvalue	the return value of the operation. A negative value indicates a failure to perform the reduction. A zero indicates successful operation.
GH or cctkGH	the pointer to the grid hierarchy structure.
processor	the processor which collects the information; a negative value will distribute the data to all processors.
operation_handle	the reduction operation handle (integer). This is obtained by calling <code>CCTK_ReductionHandle</code> or <code>CCTK_ReductionArrayHandle</code> .
num_out_vals	the number of values which define the result of the reduction operation. If reducing to an array, this would be the number of elements in the array. If reducing to a single number, this would be one.
type_out_arrays, type_in_arrays	specifies the type of the data you are communicating. Use the values as specified in B10.8 . Note: Do not <i>mix</i> datatypes. For example, in Fortran do not declare a variable as <code>integer</code> and then specify the type <code>CCTK_VARIABLE_INT</code> in the reduction command. These types may not be the same on some architectures and will conflict.
out_vals	a pointer to the buffer which will hold the output values. Note that the input and output buffer must be distinct.
num_dims	(<code>CCTK_ReduceArray</code> only) the number of dimensions of the input and output arrays.
num_in_fields	(<code>CCTK_Reduce</code> only) specifies the number of input CCTK variables which will be specified in the variable argument list <code><...></code> .
num_in_arrays	(<code>CCTK_ReduceArray</code> only) specifies the number of input <i>arrays</i> (not the number of input <i>fields</i> , which would be <code>num_in_arrays * (num_dims + 1)</code> , see below) which will be specified in the variable argument list <code><...></code> .
...	indicates a variable argument list: for <code>CCTK_Reduce</code>: Specify a list of CCTK variable indices, one for each variable that is to be reduced. The number of specified variables must be the same as the value of the <code>num_in_fields</code> variable. for <code>CCTK_ReduceArray</code>: For each input array that is to be reduced, first specify the size of the array in each dimension, followed by (a pointer to) the array itself. The number of specified arrays must be the same as the value of the <code>num_in_arrays</code> variable.

Special reduction interfaces

These routines are designed for the purpose of reducing scalars, arrays and grid functions; they will work on CCTK variables as well as local arrays. They hide many of the options of the generic interface described above.

Reduction of scalars Use these routines to reduce a single variable across multiple processors. The result of the reduction operation can be placed on the specified processor or on all processors.

```
int CCTK_ReduceLocScalar (const cGH *GH,
                          int processor,
                          int operation_handle,
                          const void *in_scalar,
                          void *out_scalar,
                          int data_type)

call CCTK_ReduceLocScalar(integer returnvalue,
                          CCTK_POINTER_TO_CONST cctkGH,
                          integer processor,
                          integer operation_handle,
                          in_scalar,
                          out_scalar,
                          integer data_type)
```

<code>returnvalue</code>	the return value of the operation. A negative value indicates a failure to perform the reduction. A zero indicates successful operation.
<code>GH</code> or <code>cctkGH</code>	the pointer to the grid hierarchy structure.
<code>processor</code>	the processor which collects the information; a negative value will distribute the data to all processors.
<code>operation_handle</code>	the reduction operation handle (integer). This is obtained by calling <code>CCTK_ReductionHandle</code> or <code>CCTK_ReductionArrayHandle</code> .
<code>in_scalar</code>	the processor local variable with local value to be reduced. In Fortran, this can be of any (scalar) data type.
<code>out_scalar</code>	the reduction result: a processor local variable with the global value (same on all processors) if <code>processor</code> has been set to <code>-1</code> . Otherwise only processor <code>processor</code> will hold the reduction result. Note that the input and output scalar variable must be distinct.
<code>data_type</code>	specifies the type of the gridfunction you are communicating. Use the values as specified in B10.8 .

Reduction of 1d arrays Use these routines to reduce a 1d array on all processors to a 1d array. This reduction is carried out element by element. The arrays need to have the same size on all processors.

```
int CCTK_ReduceLocArrayToArray1D( const cGH *GH,
                                  int processor,
                                  int operation_handle,
                                  const void *in_array1d,
                                  void *out_array1d,
                                  int xsize)
                                  int data_type)
```



```
call CCTK_ReduceLocArrayToArray1D( integer returnvalue
                                   CCTK_POINTER_TO_CONST cctkGH,
                                   integer processor,
                                   integer operation_handle,
                                   in_array1d,
                                   out_array1d,
                                   integer xsize,
                                   integer data_type)
```

<code>returnvalue</code>	the return value of the operation. A negative value indicates a failure to perform the reduction. A zero indicates successful operation.
<code>GH</code> or <code>cctkGH</code>	the pointer to the grid hierarchy structure.
<code>processor</code>	the processor which collects the information; a negative value will distribute the data to all processors.
<code>operation_handle</code>	the reduction operation handle (integer). This is obtained by calling <code>CCTK_ReductionHandle</code> or <code>CCTK_ReductionArrayHandle</code> .
<code>in_array1d</code>	the one dimensional array to be reduced across all processors, element by element.
<code>out_array1d</code>	the array holding the reduction result. <code>out_array1d[1] = Reduction(in_array[1])</code> . Note that the input and output array must be distinct.
<code>xsize</code>	the size of the one dimensional array.
<code>data_type</code>	specifies the type of the gridfunction you are communicating. Use the values as specified in B10.8 .

Reduction of 2d arrays Use these routines to reduce a 2d array, element by element. The arrays need to have the same size on all processors.

```
int CCTK_ReduceLocArrayToArray2D( const cGH *GH,
                                  int processor,
                                  int operation_handle,
                                  const void *in_array_2d,
                                  void *out_array2d,
                                  int xsize,
                                  int ysize,
                                  int data_type)

call CCTK_ReduceLocArrayToArray2D( integer returnvalue
                                   CCTK_POINTER_TO_CONST cctkGH,
                                   integer processor,
                                   integer operation_handle,
                                   in_array2d,
                                   out_array2d,
                                   integer xsize,
                                   integer ysize,
                                   integer data_type)
```

<code>returnvalue</code>	the return value of the operation. A negative value indicates a failure to perform the reduction. A zero indicates successful operation.
--------------------------	--

GH or cctkGH	the pointer to the grid hierarchy structure.
processor	the processor which collects the information; a negative value will distribute the data to all processors.
operation_handle	the reduction operation handle (integer). This is obtained by calling <code>CCTK_ReductionHandle</code> or <code>CCTK_ReductionArrayHandle</code> .
in_array2d	two dimensional array, to be reduced across all processors, element by element.
out_array2d	two dimensional array to hold the reduction result. <code>out_array2d[i,j] = Reduction(in_array2d[i,j])</code> .
xsize	the size of the two dimensional array in x direction.
ysize	the size of the two dimensional array in y direction. Note that the input and output array must be distinct.
data_type	specifies the type of the gridfunction you are communicating. Use the values as specified in B10.8 .

Reduction of 3d arrays Use these routines to reduce a 3d array, element by element. The arrays need to have the same size on all processors.

```
int CCTK_ReduceLocArrayToArray3D(const cGH *GH,
                                int processor,
                                int operation_handle,
                                const void *in_array_3d,
                                void *out_array3d,
                                int xsize,
                                int ysize,
                                int zsize,
                                int data_type)
```

```
call CCTK_ReduceLocArrayToArray3D(integer returnvalue
                                CCTK_POINTER_TO_CONST cctkGH,
                                integer processor,
                                integer operation_handle,
                                in_array3d,
                                out_array3d,
                                integer xsize,
                                integer ysize,
                                integer zsize,
                                integer data_type)
```

returnvalue	the return value of the operation. A negative value indicates a failure to perform the reduction. A zero indicates successful operation.
GH or cctkGH	the pointer to the grid hierarchy structure.
processor	the processor which collects the information; a negative value will distribute the data to all processors.
operation_handle	the reduction operation handle (integer). This is obtained by calling <code>CCTK_ReductionHandle</code> or <code>CCTK_ReductionArrayHandle</code> .
in_array3d	the three dimensional array, to be reduced across all processors, element by element.
out_array3d	three dimensional array holding the reduction result. <code>out_array3d[i,j,k] = Reduction(in_array3d[i,j,k])</code> .

<code>xsize</code>	the size of the three dimensional array in x direction.
<code>ysize</code>	the size of the three dimensional array in y direction.
<code>ysize</code>	the size of the three dimensional array in z direction. Note that the input and output array must be distinct.
<code>data_type</code>	specifies the type of the gridfunction you are communicating. Use the values as specified in B10.8 .

Some brief examples:

Reduction of a scalar: A local error is reduced across all processors with the maximum operation. The variable `tmp` will hold the maximum of the error and is the same on all processors. This quantity can then be reassigned to `normerr`.

```

CCTK_REAL normerr, tmp
integer ierr, reduction_handle

call CCTK_ReductionArrayHandle(reduction_handle,"maximum")

if (reduction_handle.lt.0) then
  call CCTK_WARN(1,"Cannot get reduction handle for maximum operation.")
endif

call CCTK_ReduceLocScalar(ierr, cctkGH, -1,
.      reduction_handle,
.      normerr, tmp, CCTK_VARIABLE_REAL)
if ((ierr.ne.0) then
  call CCTK_WARN(1,"Reduction of norm failed!");
endif
normerr = tmp

```

Reduction of a 2d array: A two dimensional (2x3) array is reduced; the reduction result (array of same size: `bla_tmp`) is seen on all processors (-1 entry as the third argument). This example also demonstrates some simple error checking with the `CCTKi_EXPECTOK` macro.

```

CCTK_REAL bla(2,3),bla_tmp(2,3);
integer ierr, sum_handle

call CCTK_ReductionArrayHandle(sum_handle,"sum")
bla = 1.0d0
write (*,*) "BLA ",bla

call CCTK_ReduceLocArrayToArray2D(ierr, cctkGH, -1, sum_handle,
.  bla, bla_tmp, 2, 3, CCTK_VARIABLE_REAL)
call CCTKi_EXPECTOK(ierr, 0, 1, "2D Reduction failed")

bla = bla_tmp
write (*,*) "BLA ",bla

```

Note that the memory for the returned values must be allocated before the reduction call is made.

Chapter B9

Completing a Thorn

B9.1 Commenting Source Code

Note that since most source files (see Section B3.4 for exceptions) pass through a C preprocessor, C style comments can be used in Fortran code. (Note that C++ comments (that is `//`), should only be used in C++ source code).

The Flesh and the Cactus thorns use the `grdoc` Code Documenting System (<http://jean-luc.aei.mpg.de/Codes/grdoc/>) to document source code.

B9.2 Providing Runtime Information

To write from thorns to standard output (*i.e.* the screen) at runtime, use the macro `CCTK_INFO` or the function `CCTK_VInfo()`.

For example, from the Fortran thorn `MyThorn`,

```
call CCTK_INFO("Starting Tricky Calculation")
```

will write the line:

```
INFO (MyThorn): Starting Tricky Calculation
```

For a multiprocessor run, only runtime information from processor zero will be printed to screen by default. The standard output of other processors will usually be discarded unless the “`-r`” command line option is used (Section A3.1).

Note that the routine `CCTK_VInfo()` can only be called from C because Fortran doesn’t know about variable argument lists. So including variables in the info message using `CCTK_INFO` is currently more tricky, since you need to build the string to be output.

For example, in C you would just write

```
int myint;

CCTK_VInfo(CCTK_THORNSTRING, "The integer is %d", myint);
```

But in Fortran you have to do the following:

```
integer      myint
character*200 message

write (message, '("The integer is ",i4)') myint
call CCTK_INFO (message(1:len_trim(message)))
```

In Fortran 90, you can also do:

```
integer      myint
character(200) message

write (message, '("The integer is ",i4)') myint
call CCTK_INFO (trim(message))
```

Note that:

- CCTK_INFO is just a macro which expands to a call to the internal function CCTK_Info() and automatically includes the thorn name in function call.
- CCTK_INFO should be used rather than print statements, since it will give consistent behaviour on multiprocessors, and also provides a mechanism for switching the output to screen on and off, even on a thorn-by-thorn basis. (Although this is not yet implemented).

B9.3 Error handling, warnings and code termination

The Cactus function CCTK_VWarn() and its accompanying CCTK_WARN macro should be used to issue warning messages during code execution. Along with the warning message, an integer is given to indicate the severity of the warning. The warning severity indicates whether the message is actually printed to standard error and whether the code should be stopped. A level 0 warning indicates the highest severity, with higher numbers indicating lower severity.

By default, a Cactus run will abort on a level 0 warning and will report level 1 and severer warnings to screen. This behaviour can be amended using command line arguments, as described in [Section A3.1](#).

For example, to provide a warning which will be printed to standard error but which will not terminate the code for a run with default options, a level 1 warning should be used.

The syntax from Fortran is

```
call CCTK_WARN(1, "Your warning message")
```

and from C

```
CCTK_WARN(1, "Your warning message");
```

Note that CCTK_WARN is just a macro which expands to a call to the internal function CCTK_Warn(). The macro automatically includes the name of the thorn, the source file name and line number of the warning in the function call. (For this reason it is important for Fortran code that capital letters are always used in order to expand the macro).

If the flesh parameter `cctk_full_warnings` is set to true, then the source file name and line number will be printed to standard error along with the originating processor number, the thorn name and the warning message. The default is to omit the source file name and line number.

Note that the routine `CCTK_VWarn()` can only be called from C because Fortran doesn't know about variable argument lists. So including variables in the warning message using `CCTK_WARN` is currently more tricky, since you need to build the string to be output.

For example, in C you would just write

```
int    myint;
double myreal;

CCTK_VWarn(1, __LINE__, __FILE__, CCTK_THORNSTRING,
           "Your warning message, including %f and %d",
           myreal, myint);
```

But in Fortran you have to do the following:

```
integer      myint
real         myreal
character*200 message

write (message, '("Your warning message, including ",g12.7," and ",i8)') myreal, myint
call CCTK_WARN (1, message(1:len_trim(message)))
```

In Fortran 90, you can also do:

```
integer myint
real myreal
character(200) message

write (message, '("Your warning message, including ",g12.7," and ",i8)') myreal, myint
call CCTK_WARN (1, trim(message))
```

The flesh will be implementing standard error return codes which can be used by the thorns, although this is not yet ready. In general, thorns should attempt to handle errors without terminating, and warning messages should be liberally used.

B9.4 Adding documentation

Documentation is a vital part of your thorn, helping to ensure it's ease of use and longevity, not only for others but also for the thorn authors. Although any kind of independent documentation can be added to a thorn (ideally in the `doc` directory), there are two standard places for adding thorn documentation, a `README` and a file `doc/documentation.tex` for including in Thorn Guides.

B9.4.1 README

The `README`, in the top level of a thorn, should contain brief and essential details about the thorn, such as the authors, any copyright details, and a synopsis of what the thorn does.

B9.4.2 Contribution to ThornGuide

The latex file `doc/documentation.tex` is included in Thorn Guides built by the Cactus make system. (e.g. by `gmake <config>-ThornGuide`). Ideally this file should contain complete (and *up-to-date*) details about the thorn, exactly what is relevant is for the authors to decide, but remember that the Cactus make system automatically parses the thorn CCL files to include information about all parameters, variables and scheduling. Suggested sections include:

- **model** A description of the system which the thorn is modelling, including the equations etc. which are being solved or implemented.
- **numerical implementation** Details about how the model is numerically implemented in the thorn.
- **using the thorn** Any special details needed for using the thorn, tricky parameters, particular operating systems or additional required software, interactions with other thorns and examples of use.
- **history** Here is where you should describe why the thorn was written, any previous software or experience which was made use of, the authors of the thorn sourcecode and documentation, how to get hold of the thorn etc.
- **references** A bibliography can be included, referencing papers published using or about this thorn, or additional information about the model or numerics used.

A latex template for the Thorn Guide documentation can be found in the Flesh distribution at

`doc/ThornGuide/template.tex`,

this file is automatically copied to the correct location in a new thorn which is created with `gmake newthorn`.

Since Cactus scripts need to parse this documentation, and since the latex document should be consistent across all thorns included in a Thorn Guide, please follow the guidelines below when filling in the documentation:

- Use the Cactus Thorn Guide style file, located in the Flesh distribution at `doc/latex/cactus.sty`. This should be included using a relative link, so that updates to the style file are applied to all thorns.

```
\usepackage{../../../../../doc/latex/cactus}
```

- Aside from the `date`, `author`, and `title` fields, all of the documentation to be included in a Thorn Guide should be placed between the lines
`% START CACTUS THORNGUIDE`
and
`% END CACTUS THORNGUIDE`
- The command `\def` can be used to define macros, but all such definitions must lie between the `START` and `END` line. Do not redefine any standard latex command
- Do not use the `\appendix` command, instead include any appendices you have as standard sections.
- Currently we only support postscript (`.eps` or `.ps`) figures. Graphics figures should be included using the `includegraphics` command (not `epsffit`), with no file extension specified. For example,

```

\begin{figure}[ht]
  \begin{center}
    \includegraphics[width=6cm]{MyArrangement_MyThorn_MyFigure}
  \end{center}
  \caption{Illustration of this and that}
  \label{MyArrangement_MyThorn_MyLabel}
\end{figure}

```

- All **labels**, **citations**, **references**, and **graphic images** names should conform to the following guidelines: `ARRANGEMENT_THORN_LABEL`. For instance, if you arrangement is called `CactusWave`, your thorn `WaveToyC`, and your original image `blackhole.eps`, you should rename your image to be `CactusWave.WaveToyC_blackhole.eps`
- References should be formatted with the standard latex **bibitem** command, for example, a bibliography section should look like:

```

\begin{thebibliography}{9}
  \bibitem{MyArrangement_MyThorn_Author99}
    {J. Author, {\em The Title of the Book, Journal, or periodical}, 1 (1999),
    1--16. {\tt http://www.nowhere.com/}}
\end{thebibliography}

```

B9.5 Adding a test suite

To add a test suite to your thorn, devise a series of parameter files which use as many aspects of your thorn as possible. Make sure that the parameter files produce ascii output to files, and that these files are in the directory `./<parameter file base name>`.

Run Cactus on each of the parameter files, and move the parameter files, and the output directories they produced, to the `test` directory in your thorn.

Document carefully any situations or architectures in which your test suite does not give the correct answers.

For details on running the test suites, see Section [A2.4](#).

Chapter B10

Advanced Thorn Writing

B10.1 Using Cactus Timers

B10.1.1 What are timers?

Cactus Timers can only currently be used in C or C++ source code, although the API's for Fortran code are currently being added [FIXME].

Cactus provides a flexible mechanism for timing different sections of your thorns using various clocks which have been registered with the Flesh. By default, the Flesh provides a set of clocks (if they are available), measuring for example wall clock time and CPU time. Additional clocks can be implemented by thorns and registered with the Flesh

You can add any number of timers to your thorn source code, providing each with a chosen name, for example `TimeMyRoutine`, `TimeNextCalculation`, and then use Cactus functions to switch on the timers, stop or reset them, and recover timing information.

Note that we use the word *Clock* to describe the timing instrument itself, for example the hardware counters on a machine, and the word *Timer* to describe the calls in your code which collect results from the different *Clocks*.

B10.1.2 Timing calls

`CCTK_TimerCreate`, `CCTK_TimerCreateI`

Create a timer with a given name (`CCTK_TimerCreate`), or with no name (`CCTK_TimerCreateI`) and give back a timer index. If no name is provided for the timer, future calls should be made using the returned timer index.

`CCTK_TimerDestroy`, `CCTK_TimerDestroyI`

Destroy a timer using either the timer name or timer index.

`CCTK_TimerStart`, `CCTK_TimerStartI`

Start the given timer (identified by name or index), using all registered clocks.

`CCTK_TimerStop`, `CCTK_TimerStopI`

Stop the given timer (identified by name or index) on all registered clocks.

`CCTK_TimerReset`, `CCTK_TimerResetI`

Reset the given timer on all registered clocks.

`CCTK_TimerCreateData`, `CCTK_Timer`, `CCTK_TimerI`, `CCTK_TimerDestroyData`

Access the actual timing results, which are passed back as a structure, `cTimerData` described below, in `CCTK_Timer`. Since the timing data is dynamic, before it can be assessed, the structure must be allocated with a call to `CCTK_TimerCreateData`. A similar function is provided to destroy the structure

`CCTK_NumTimers` Return the number of created timers

`CCTK_TimerName` Provide the name of the timer for a given timer index

B10.1.3 The `cTimerData` Structure

The structure holding the timing data, `cTimerData`, contains the number of collected times (one measurement for each registered clock), and the measured time for each clock. The measured time is held in a structure of type `cTimerVal` which contains the data type of the measured value, a description of the clock, and the units used for the measurements. Hopefully additional interfaces will soon be added to avoid detailed knowledge of these structures, but for now here are the gory details:

```
typedef enum {val_none, val_int, val_long, val_double} cTimerValType;
```

```
typedef struct
{
    cTimerValType type;
    const char *heading;
    const char *units;
    union
    {
        int      i;
        long int  l;
        double    d;
    } val;
} cTimerVal;
```

```
typedef struct
{
    int n_vals;
    cTimerVal *vals;
} cTimerData;
```

B10.1.4 How to insert timers in your code

The function prototypes and structure definitions are contained in the include file `cctk.Timers.h`, which is included in the standard thorn header file `cctk.h`. At the moment the timer calls are only available from C.

The following example, which uses a timer called `TimeMyStuff` to instrument a section of code, illustrates how timers are used by application thorns. A working example is available in the thorn `CactusTest/TestTimers`.

Creating the `TimeMyStuff` timer

The first action for any timer is to create it, using

`CCTK_TimerCreate`. This can be performed at any time, as long as it precedes using the timer:

```
#include "cctk_Timers.h"
ierr = CCTK_TimerCreate('TimeMyStuff');
```

Instrumenting a section of code

Code sections are instrumented using the Start, Stop and Reset functions. These functions are applied to the chosen timer using all the registered clocks.

```
#include "cctk_Timers.h"
ierr = CCTK_TimerStart('TimeMyStuff');
/* Piece of code to time */
y = CalculateNewValue(y);
ierr = CCTK_TimerStop('TimeMyStuff');
```

Accessing the timer results

This is a little bit cumbersome at the moment, since the timer data can be of different types.

```
#include "cctk_Timers.h"
cTimerData *info;

info = CCTK_TimerCreateData();
ierr = CCTK_Timer('TimeMyStuff',info);

for (i = 0; i < info->n_vals; i++)
{
    switch (info->vals[i].type)
    {
        case val_int:
            printf("%s: %d %s", info->vals[i].heading,info->vals[i].val.i,
                    info->vals[i].units);
            break;

        case val_long:
            printf("%s: %d %s", info->vals[i].heading,(int) info->vals[i].val.l,
                    info->vals[i].units);
            break;

        case val_double:
            printf("%s: %.3f %s", info->vals[i].heading,info->vals[i].val.d,
                    info->vals[i].units);
            break;

        default:
            CCTK_WARN(1, "Unknown data type for timer info");
            break;
    }
}
ierr = CCTK_TimerDestroyData(info);
```

B10.2 Include Files

Cactus provides a mechanism for thorns to add code to include files which can be used by any other thorn. Such include files can contain executable source code, or header/declaration information. A difference is

made between these two cases, since included executable code is protected from being run if a thorn is compiled but not active by being wrapped by a call to `CCTK_IsThornActive`.

Any thorn which uses the include file must declare this in its `interface.ccl` with the line

```
USES INCLUDE [SOURCE|HEADER]: <file_name>
```

(If the optional `[SOURCE|HEADER]` is omitted, `HEADER` is assumed. Note that this can be dangerous, as included *source* code which is incorrectly assumed to be *header* code will be executed in another thorn *even if the providing thorn is inactive*. Thus it is recommended to always include the optional `[SOURCE|HEADER]` specification.) Any thorn which wishes to add to this include file, declares in its own `interface.ccl`

```
INCLUDE [SOURCE|HEADER]: <file_to_include> in <file_name>
```

Example

For an example of this in practice, for the case of Fortran code, consider thorn A which wants to gather terms for a calculation from any thorn which wishes to provide them. Thorn A could have the lines in its source code

```
c Get source code from other thorns
    allterms = 0d0
#include "AllSources.inc"
```

and would then add to `interface.ccl` the line

```
USES INCLUDE SOURCE: AllSources.inc
```

If thorn B wants to add terms for the calculation, it would create a file, say `Bterms.inc` with the lines

```
c Add this to AllSources.inc
    allterms = allterms + 1d0
```

and would add to its own `interface.ccl`

```
INCLUDE SOURCE: Bterms.inc in AllSources.inc
```

The final file for thorn A which is compiled will contain the code

```
c Get source code from other thorns
    allterms = 0d0
    if (CCTK_IsThornActive("B").ne.0) then
c Add this to AllSources.inc
    allterms = allterms + 1d0
    end if
```

Any Fortran thorn routines which include source code must include the declaration `DECLARE_CCTK_FUNCTIONS`.

B10.3 Memory Tracing

Cactus provides a mechanism for overriding the standard C memory allocation routines (`malloc`, `free`, ...) with Cactus specific routines, that track the amount of memory allocated and from where the allocation call was made. This information can be accessed by the user to provide an understanding of the memory consumption between two instances and to track down possible memory leaks. This feature is available in C only.

B10.3.1 Activating Memory Tracing

Memory tracing has to be activated at configure time. The standard `malloc` statements are overridden with macros (`CCTK_MALLOC`). To activate memory tracing use either

```
DEBUG=all           Enables all debug options (compiler debug flags, redefines malloc)
DEBUG=memory        Redefine malloc only.
```

The `CCTK_MALLOC` statements can also be used directly in the C code. But by employing them this way, only a fraction of the total memory consumption is traced. Also, they cannot be turned off at configure time. (this behavior might change). For example:

```
machine> gmake bigbuild DEBUG=yes
machine> gmake bigbuild-config DEBUG=memory
```

The new configuration `bigbuild` is configured with all debugging features turned on. The already existing configuration `bigbuild` is reconfigured with memory tracing only.

B10.3.2 Using Memory Tracing

You can request Cactus to store the memory consumption at a certain instance in the program flow and return the difference in memory allocation some time later.

```
int CCTK_MemTicketRequest(void)
    Request a ticket: save the current total memory to a database. Return an integer
    (ticket). Use the ticket to calculate the difference in memory allocation between
    the two instances in CCTK_MemTicketCash.

long int CCTK_MemTicketCash(int your_ticket)
    Cash in your ticket: return the memory difference between now and the time the
    ticket was requested. Tickets can be cashed in several times. See Example below.
    This only tracks the real data memory, which is the same as in undebug mode.
    It does not keep track of the internal allocations done to provide the database,
    motivation is that this is not allocated either if you compile undebugged.

int CCTK_MemTicketDelete(int your_ticket)
    Delete the memory ticket. The ticket-id will not be reused, since it's incremented
    with every ticket request, but the memory of the memory datastructure is deallo-
    cated.

unsigned long int CCTK_TotalMemory(void)
    Returns the total allocated memory (not including the tracing data structures).
```

`void CCTK_MemStat` Prints an info string, stating the current, past and total memory (in bytes) allocation between two successive calls to this routine, as well as the difference.

Sample C Code demonstrating the ticket handling. Two tickets are requested during malloc operations. The `CCTK_MALLOC` statement is used directly. They are cashed in and the memory difference is printed. Ticket 1 is cashed twice. The tickets are deleted at the end.

```
int ticket1;
int ticket2;

/* store current memstate, ticket: t1*/
t1 = CCTK_MemTicketRequest();

/* allocate data */
hi = (int*) CCTK_MALLOC(10*sizeof(int));

/* store current memstate, ticket: t2*/
t2 = CCTK_MemTicketRequest();

/* cash ticket t1, print mem difference */
printf("NOW1a: %+d \n",CCTK_MemTicketCash(t1));

/* allocte some more data */
wo = (CCTK_REAL*)CCTK_MALLOC(10*sizeof(CCTK_REAL));

/* cash ticket t1 and t2, print mem difference */
printf("NOW1b: %+d \n",CCTK_MemTicketCash(t1));
printf("NOW2 : %+d \n",CCTK_MemTicketCash(t2));

/* delete the tickets from the database */
CCTK_MemTicketDelete(t1);
CCTK_MemTicketDelete(t2);
```

B10.4 Calls between different programming languages

B10.4.1 Calling C routines from FORTRAN

To make the following C routine,

```
int <routine name>(<argument list>)
...

```

also callable from Fortran, a new routine must added, which is declared using the `CCTK_FCALL` and `CCTK_FNAME` macros:

```
void CCTK_FCALL CCTK_FNAME(<routine name>)(int *ierr, <argument list>)
<rewrite routine code, or call C routine itself>
```

The convention used in Cactus is that `<routine name>` be the same as any C-callable routine name, and that this is mixed-case. The macros change the case and number of underscores of the routine name to

match that expected by Fortran.

All arguments passed by Fortran to the routine (except strings) are pointers in C, e.g. a call from Fortran

```
CCTK_INT arg1
CCTK_REAL arg2
CCTK_REAL arg3(30,30,30)

...

call MyCRoutine(arg1,arg2,arg3)
```

should appear in C as

```
void CCTK_FCALL CCTK_FNAME(MyCRoutine)(CCTK_INT *arg1,
                                         CCTK_REAL *arg2,
                                         CCTK_REAL *arg3)
{
  ...
}
```

B10.4.2 String Arguments from Fortran

Fortran passes string arguments in a special, compiler-dependent, way. To facilitate this, the CCTK provides a set of macros to enable the translation to C strings. The macros are defined in `cctk.FortranString.h` which should be included in your C file.

String arguments *must always come last* in the argument list for these macros to be effective (some Fortran compilers automatically migrate the strings to the end so there is no portable workaround).

The macros to use depends upon the number of string arguments – we currently support up to three. The macros are `<ONE|TWO|THREE>_FORTSTRING_ARG`. Corresponding to each of these are two macros `<ONE|TWO|THREE>_FORTSTRING_CREATE` and `<ONE|TWO|THREE>_FORTSTRING_PTR` which take one,two,or three arguments depending on the number of strings. The latter set is only necessary if a string is to be modified. In more detail:

`<ONE|TWO|THREE>_FORTSTRING_ARG`

Used in the argument list of the C routine to which the Fortran strings are passed.

`<ONE|TWO|THREE>_FORTSTRING_CREATE`

Used in the declaration section of the C routine to which the Fortran strings are passed. These macros have one, two or three arguments which are the variable names you choose to use for the strings in the C routine, created by null-terminating the passed-in Fortran strings. The `CREATE` macros create new strings with the names you provide, and thus should be treated as read-only and freed after use.

`<ONE|TWO|THREE>_FORTSTRING_PTR`

These macros, used in the declaration section of the C routine *after* the `CREATE` macro, should be used if you need to modify one of the passed-in strings. They declare and define pointers to the passed-in strings.

`cctk_strlen<1|2|3>`

these integer variables, automatically defined by the `CREATE` macro, hold the lengths of the passed in Fortran strings.

The use of the macros is probably best explained with examples. For read-only access to the strings, only the first two macros are needed, the following example compares two strings passed in from Fortran.

```
#include <stdlib.h>
#include <string.h>
#include <cctk_FortranString.h>

int CCTK_FCALL CCTK_FNAME(CompareStrings)(TWO_FORTSTRING_ARG)
{
    int retval;

    /* Allocate and create C strings with \0 at end. */
    /* This makes variable declarations so must be before
       any executable statements.*/

    TWO_FORTSTRING_CREATE(arg1,arg2)

    /* Do some work with the strings */
    retval = strcmp(arg1,arg2);

    /* Important, these must be freed after use */
    free(arg1);
    free(arg2);

    return retval;
}
```

Since the null terminated strings may be copies of the strings passed from Fortran, they should be treated as read-only.

To change the data in a string passed from Fortran, you need to use the `FORTSTRING_PTR` macros, which declare and set up pointers to the strings passed from C. Note that this macro must be used *after* the `FORTSTRING.CREATE` macro. For example, the following routine copies the contents of the second string to the first string.

```
#include <stdlib.h>
#include <string.h>
#include <cctk_FortranString.h>

int CCTK_FCALL CCTK_FNAME(CopyStrings)(TWO_FORTSTRING_ARG)
{
    int retval;

    /* Allocate and create C strings with \0 at end. */
    /* This makes variable declarations so must be before
       any executable statements. */

    TWO_FORTSTRING_CREATE(arg1,arg2)
    TWO_FORTSTRING_PTR(farg1,farg2)

    /* Do some work with the strings */

    retval = strncpy(farg1,arg2,cctk_strlen1);
}
```



```

/* Important, these must be freed after use */
free(arg1);
free(arg2);

return retval;
}

```

Note that in the example above two new variables, pointers to the Fortran strings, were created. These are just pointers and *should not be freed*. The example also illustrates the automatically-created variables (e.g. `cctk_strlen1` which hold the sizes of original Fortran strings. When writing to a string its length should never be exceeded.

B10.4.3 Calling FORTRAN routines from C

To call a utility Fortran routine from C use

```
void CCTK_FCALL CCTK_FNAME(<Fortran routine name>)(<argument list>)
```

Note that Fortran expects all arguments (apart from strings) to be pointers, so any non-array data should be passed by address.

Currently we have no support for calling Fortran routines which expect strings from C.

B10.5 Function aliasing

As well as calling functions in a different language, Cactus offers a mechanism for calling a function in a different thorn where you need not know which thorn is actually providing the function, nor what language the function is provided in. The idea of *function aliasing* is similar to that of thorns; the routine that calls a function should not need to know anything about it except that the function exists.

Function aliasing is quite restrictive because of the problems involved in inter-language calling, as seen in the previous section. Function aliasing is also comparatively inefficient and should not be used in a part of your code where efficiency is important.

Function aliasing is language neutral, however the syntax is strongly based on C. The API documented here describes the current code, however it is temporary. In the future the function aliasing declarations will go into a new `functions.ccl` file, and will have a format more similar to that of variable group and parameter declarations.

B10.5.1 Using an aliased function

To use an aliased function you must first declare it in your `interface.ccl` file. Declare the prototype as, for example,

```
CCTK_REAL FUNCTION SumStuff(CCTK_REAL x, CCTK_REAL y)
```

and that this function will be used in your thorn by

```
USES FUNCTION SumStuff
```

A prototype of this function will be available to any C routine that includes the `cctk.h` header file.

However, in a Fortran file that uses the `implicit none` declaration the function will have to be explicitly declared; a definition will not be included in the `DECLARE_CCTK_FUNCTIONS` macro.

B10.5.2 Providing a function

To provide an aliased function you must again add the prototype to your `interface.ccl` file. A statement containing the name of the providing function and the language it is provided in must also be given. For example,

```
CCTK_REAL FUNCTION SumStuff(CCTK_REAL x, CCTK_REAL y)
PROVIDES FUNCTION SumStuff WITH AddItUp LANGUAGE C
```

The appropriate function must then be provided somewhere in this thorn.

It is necessary to specify the language of the providing function; no default will be assumed.

B10.5.3 Conventions and restrictions

Various restrictions are necessary to make function aliasing work. These are

- The return type of any function must be either `void` or one of the Cactus data types `CCTK_INT` or `CCTK_REAL`. Standard types such as `int` are not allowed.
- The type of an argument must be one of scalar types `CCTK_INT`, `CCTK_REAL`, `CCTK_STRING`, `CCTK_POINTER`, `CCTK_FPOINTER`, or an array or pointer type `CCTK_INT:ARRAY`, `CCTK_REAL:ARRAY`. The scalar types are assumed to be not modifiable. Any changes made to a scalar argument by a providing function may be silently lost, or may not; it is dependent on the language of the providing and calling function. If you wish to modify an argument then it must have the `ARRAY` suffix (and hence must be either a `CCTK_INT` or `CCTK_REAL`).
- The name of both the aliased and providing function are restricted. They must follow the standard C semantics (start with a letter, contain only letters, numbers or underscores). Additionally, they must be mixed case (that is, contain at least one uppercase and one lowercase letter) and be less than 21 characters long. The names of the aliased and providing functions must be distinct.
- If an argument is a function pointer then the syntax looks like

```
CCTK_REAL Integrate(CCTK_REAL CCTK_FPOINTER func(CCTK_REAL x), \
                    CCTK_REAL xmin, CCTK_REAL xmax)
```

It is assumed that the function pointer argument has the same language as the calling function. The name of the function pointer argument should be short, as the total length of the providing function name and the function pointer argument must be less than 21 characters. Function pointer arguments may not be nested.

- `CCTK_STRING` arguments follow the same conventions as in the previous section. That is, they must appear at the end of the argument list, there must be at most three, and a function with a `CCTK_STRING` argument can only be provided in C, not Fortran, although they may be called from either.

B10.5.4 Examples

- A C function is provided that adds together two real numbers. The `interface.ccl` should read

```
CCTK_REAL FUNCTION SumStuff(CCTK_REAL x, CCTK_REAL y)
PROVIDES FUNCTION SumStuff WITH AddItUp LANGUAGE C
USES FUNCTION SumStuff
```

- A Fortran function is provided that inverts a real number. The `interface.ccl` should read

```
void FUNCTION Invert(CCTK_REAL:ARRAY x)
PROVIDES FUNCTION Invert WITH FindInverse LANGUAGE Fortran
USES FUNCTION Invert
```

- A Fortran function is provided that integrates any function over an interval. The `interface.ccl` should read

```
CCTK_REAL Integrate(CCTK_REAL CCTK_FPOINTER func(CCTK_REAL x), \
                    CCTK_REAL xmin, CCTK_REAL xmax)
PROVIDES FUNCTION Integrate WITH SimpsonsRule LANGUAGE Fortran
USES FUNCTION Integrate
```

B10.6 Naming conventions

- Thorn names must not start with the word “Cactus” (in any case).
- Arrangements will be ignored if their names start with `#` or `.` or end in `~.bak` or `.BAK`.
- Thorns will be ignored if they are called `doc` or start with `#` or `.` or end in `~.bak` or `.BAK`.
- Routine names have to be unique among all thorns.

B10.7 General Naming Conventions

The following naming conventions are followed by the flesh and the supported Cactus arrangements. They are not compulsory, but if followed allow for a homogeneous code.

- Parameters: lower case (except for acronyms) with words separated by an underscore. Examples: `my_first_parameter`, `solve_PDE.equation`.
- Filenames and routine names: Prefixed by thorn name with an underscore, then capitalised words, with no spaces. Examples: `MyThorn_StartUpRoutine`, `BestSolver_InitialDataForPDE`.

B10.8 Data Types and Sizes

Cactus supports the following fixed size data types:

Data Type	Size (bytes)	Variable Type	Fortran Equivalent
CCTK_INT2	2	CCTK_VARIABLE_INT2	<code>integer*2</code>
CCTK_INT4	4	CCTK_VARIABLE_INT4	<code>integer*4</code>
CCTK_INT8	8	CCTK_VARIABLE_INT8	<code>integer*8</code>
CCTK_REAL4	4	CCTK_VARIABLE_REAL4	<code>real*4</code>
CCTK_REAL8	8	CCTK_VARIABLE_REAL8	<code>real*8</code>
CCTK_REAL16	16	CCTK_VARIABLE_REAL16	<code>real*16</code>
CCTK_COMPLEX8	8	CCTK_VARIABLE_COMPLEX8	<code>complex*8</code>
CCTK_COMPLEX16	16	CCTK_VARIABLE_COMPLEX16	<code>complex*16</code>
CCTK_COMPLEX32	32	CCTK_VARIABLE_COMPLEX32	<code>complex*32</code>
CCTK_CHAR	1	CCTK_VARIABLE_CHAR	<code>character</code>

In addition Cactus provides three generic numeric data types which map onto the compilers' native data types used to represent integer, real, and complex values. The size for these generic types can be chosen at configuration time (see [A2.1.2](#)). This is to allow the code to be easily run at different precisions. Note that the effectiveness of running the code at a lower or higher precision depends crucially on all thorns being used making consistent use of these generic data types:

Data Type	Variable Type	Configuration Option
CCTK_INT	CCTK_VARIABLE_INT	INTEGER_PRECISION
CCTK_REAL	CCTK_VARIABLE_REAL	REAL_PRECISION
CCTK_COMPLEX	CCTK_VARIABLE_COMPLEX	Same as real precision

These variable types must be used by thorn writers to declare variables in the thorn interface files, and may be used to declare variables in the thorn routines. Note that variable declarations in thorns should obviously match with definitions in the interface files where appropriate.

Cactus also provides generic data and function pointers, which can be used from either C or Fortran (these may not work yet for thorn interface files):

Data Type	Variable Type	C equivalent
CCTK_POINTER	CCTK_VARIABLE_POINTER	void *data_ptr
CCTK_FPOINTER	CCTK_VARIABLE_FPOINTER	void (*fn_ptr)(void)

Also provided, are a set of macros which are interpreted by the preprocessor at compile time to signify which data size is being used:

Data Type	#define
CCTK_INT2	CCTK_INT_PRECISION_2
CCTK_INT4	CCTK_INT_PRECISION_4
CCTK_INT8	CCTK_INT_PRECISION_8
CCTK_REAL4	CCTK_REAL_PRECISION_4
CCTK_REAL8	CCTK_REAL_PRECISION_8
CCTK_REAL16	CCTK_REAL_PRECISION_16
CCTK_COMPLEX8	CCTK_COMPLEX_PRECISION_8
CCTK_COMPLEX16	CCTK_COMPLEX_PRECISION_16
CCTK_COMPLEX32	CCTK_COMPLEX_PRECISION_32

Note that the availability of these types, and the corresponding C data types are platform dependent.

B10.8.1 Fortran Thorn Writers

Cactus provides a further data type `CCTK_POINTER` for use in Fortran code to declare a pointer passed from C. For example, the variable `cctkGH` is of this type.

Chapter B11

Telling the makesystem what to do

B11.1 Basic Recipe

B11.2 Make Concepts

B11.3 The Four Files

B11.4 How your code is built

Part C

Utility Routines

Chapter C1

Introduction

As well as the high-level `CCTK_*` routines, Cactus also provides a set of lower-level `Util_*` utility routines, which are mostly independent of the rest of Cactus. This chapter gives a general overview of programming with these utility routines.

Chapter C2

Key/Value Tables

C2.1 Motivation

Various Cactus functions need to pass information through a generic interface. In the past we have used various ad-hoc means to do this, and we often had trouble passing "extra" information that wasn't anticipated in the original design. For example, for interpolation `CCTK_InterpLocal()` and `CCTK_InterpGV()` require that any parameters (such as the order of the interpolant) be encoded into the interpolator's character string name. Similarly, elliptic solvers often need to pass various parameters, but we don't have a good way to do this.

Key/value tables ("tables" for short) provide a clean solution to these problems. They're implemented by the `Util_Table*` functions (described in detail in the Reference Manual).

C2.2 The Basic Idea

Basically, a table is an object which maps strings to almost-arbitrary user-defined data. (If you know Perl, a table is very much like a Perl hash table.)¹

More formally, a table is an object which stores a set of "keys" and a corresponding set of "values". We refer to a (key,value) pair as a table "entry".

Keys are C-style null-terminated character strings, with the slash character '/' reserved for future expansion.²

Values are 1-dimensional arrays of any of the usual Cactus data types described in section [B10.8](#). A string can be stored by treating it as a 1-dimensional array of `CCTK_CHAR` (there's an example of this below).

The basic "life cycle" of a table is that you first create it with `Util_TableCreate()` or `Util_TableClone()`, then set entries in it using one or more of the `Util_TableSet*()`, `Util_TableSet*Array()`, `Util_TableSetGeneric()`, `Util_TableSetGenericArray()`, and/or `Util_TableSetString()` functions. Then, at a later time some other piece of code can get (copies of) the values you set, using one or more of the `Util_TableGet*()`, `Util_TableGet*Array()`, `Util_TableGetGeneric()`, `Util_TableGetGenericArray()`, and/or `Util_TableGetString()` functions. Finally, when you're through with a table you destroy it with `Util_TableDestroy()`.

¹However, the present Cactus tables implementation is optimized for a relatively small number of distinct keys in any one table. It will still work ok for huge numbers of keys, but it will be slow.

²Think of hierarchical tables for storing tree-like data structures.

There are also convenience functions `Util_TableSetFromString()` to set entries in a table based on a parameter-file-style string, and `Util_TableCreateFromString()` to create a table and then set entries in it based on a parameter-file-style string.

As well, there are “table iterator” functions `Util_TableIt*()` to allow manipulation of a table even if you don’t know its keys.

A table has an integer “flags word” which may be used to specify various options, via bit flags defined in `"util_Table.h"`. For example, the flags word can be used to control whether keys should be compared as case-sensitive or case-insensitive strings. See the detailed function description of `Util_TableCreate()` in the Reference Manual for a list of the possible bit flags and their semantics.

C2.3 A Simple Example

Here’s a simple example (in C)³ of how to use a table:

```
#include "util_Table.h"
#include "cctk.h"

/* create a table and set some entries in it */
int handle = Util_TableCreate(UTIL_TABLE_FLAGS_DEFAULT);
Util_TableSetInt(handle, 2, "two");
Util_TableSetReal(handle, 3.14, "pi");

...

/* get the values from the table */
CCTK_INT two_value;
CCTK_REAL pi_value;
Util_TableGetInt(handle, &two_value, "two");    /* sets two_value = 2 */
Util_TableGetReal(handle, &pi_value, "pi");      /* sets pi_value = 3.14 */
```

Actually, you shouldn’t write code like this – in the real world errors sometimes happen, and it’s much better to catch them close to their point of occurrence rather than silently produce garbage results or crash your program. So, the *right* thing to do is to always check for errors. To allow this, all the table routines return a status, which is zero or positive for a successful return, but negative if and only if some sort of error has occurred.⁴ So, the above example should be rewritten like this:

```
#include "util_Table.h"

/* create a table and set some entries in it */
int handle = Util_TableCreate(UTIL_TABLE_FLAGS_DEFAULT);
if (handle < 0)
    CCTK_WARN(-1, "couldn't create table!");

/* try to set some table entries */
if (Util_TableSetInt(handle, 2, "two") < 0)
    CCTK_WARN(-1, "couldn't set integer value in table!");
if (Util_TableSetReal(handle, 3.14, "pi") < 0)
```

³All (or almost all) of the table routines are also usable from Fortran. See the full descriptions in the Reference Manual for details.

⁴Often (as in the examples here) you don’t care about the details of which error occurred. But if you do, there are various error codes defined in `"util_Table.h"` and `"util_ErrorCodes.h"`; the detailed function descriptions in the Reference Manual say which error codes each function can return.

```

        CCTK_WARN(-1, "couldn't set real value in table!");

...

/* try to get the values from the table */
CCTK_INT two_value;
CCTK_REAL pi_value;
if (Util_TableGetInt(handle, &two_value, "two") < 0)
    CCTK_WARN(-1, "couldn't get integer value from table!");
if (Util_TableGetReal(handle, &pi_value, "pi") < 0)
    CCTK_WARN(-1, "couldn't get integer value from table!");

/* if we get to here, then two_value = 2 and pi_value = 3.14 */

```

C2.4 Arrays as Table Values

As well as a single numbers (or characters or pointers), tables can also store 1-dimensional arrays of numbers (or characters or pointers).⁵

For example (continuing the previous example):

```

static const CCTK_INT a[3] = { 42, 69, 105 };
if (Util_TableSetIntArray(handle, 3, a, "my array") < 0)
    CCTK_WARN(-1, "couldn't set integer array value in table!");

...

CCTK_INT blah[10];
int count = Util_TableGetIntArray(handle, 10, blah, "my array");
if (count < 0)
    CCTK_WARN(-1, "couldn't get integer array value from table!");
/* now count = 3, blah[0] = 42, blah[1] = 69, blah[2] = 105, */
/*    and all remaining elements of blah[] are unchanged */

```

As you can see, a table entry remembers the length of any array value that has been stored in it.⁶

If you only want the first few values of a larger array, just pass in the appropriate length of your array, that's ok:

```

CCTK_INT blah2[2];
int count = Util_TableGetIntArray(handle, 2, blah2, "my array");
if (count < 0)
    CCTK_WARN(-1, "couldn't get integer array value from table!");
/* now count = 3, blah2[0] = 42, blah2[1] = 69 */

```

You can even ask for just the first value:

```
CCTK_INT blah1;
```

⁵The table makes (stores) a *copy* of the array you pass in, so you probably shouldn't use tables to store really big arrays like grid functions. But it's fine to store things like parameters and coefficients. If you do have a really big array, consider storing a pointer to it.

⁶In fact, actually *all* table values are arrays – setting or getting a single value is just the special case where the array length is 1.

```
int count = Util_TableGetInt(handle, &blah1, "my array");
if (count < 0)
    CCTK_WARN(-1, "couldn't get integer array value from table!");
/* now count = 3, blah1 = 42 */
```

C2.5 Character Strings

One very common thing you might want to store in a table is a character string. While you could do this by explicitly storing an array of CCTK_CHAR, there are also more-convenient routines specially for setting and getting strings:

```
if (Util_TableSetString(handle, "black holes are fun", "bh") < 0)
    CCTK_WARN(-1, "couldn't set string value in table!");

...
char buffer[50];
if (Util_TableGetString(handle, 50, buffer, "bh") < 0)
    CCTK_WARN(-1, "couldn't get string value from table!");

/* now buffer[] contains the string "black holes are fun" */
```

Util_TableGetString() guarantees that the string is terminated by a null character ('\0'), and also returns an error if the string is too long for the buffer.

C2.6 Convenience Routines

There are also convenience routines for the common case of setting values in a table based on a string.

For example, the following code sets up exactly the same table as the example in section C2.3:

```
#include <util_Table.h>

/* create a table and set some values in it */
int handle = Util_TableCreate(UTIL_TABLE_FLAGS_DEFAULT);
if (handle < 0)
    CCTK_WARN(-1, "couldn't create table!");

/* try to set some table entries */
if (Util_TableSetFromString(handle, "two=2 pi=3.14") != 2)
    CCTK_WARN(-1, "couldn't set values in table!");
```

There is also an even higher-level convenience function Util_TableCreateFromString(): this creates a table with the case-insensitive flag set (to match Cactus parameter-file semantics), then (assuming no errors occurred) calls Util_TableSetFromString() to set values in the table.

For example the following code sets up a table (with the case-insensitive flag set) with four entries: an integer number (**two**), a real number (**pi**), a string (**buffer**), and an integer array with three elements (**array**):

```
#include <util_Table.h>
```

```
int handle = Util_TableCreateFromString(" two      = 2 "
                                       " pi       = 3.14 "
                                       " buffer = 'Hello World' "
                                       " array  = { 1 2 3 }");

if (handle < 0)
    CCTK_WARN(-1, "couldn't create table from string!");
```

Note that this code passes a single string to `Util_TableCreateFromString()`⁷ which then gets parsed into key/value pairs, with the key separated from its corresponding value by an equals sign.

Values for numbers are converted into integers (`CCTK_INT`) if possible (no decimal point appears in the value), otherwise into reals (`CCTK_REAL`). Strings must be enclosed in either single or double quotes. String values in single quotes are interpreted literally, strings in double quotes may contain character escape codes which then will be interpreted as in C. Arrays must be enclosed in curly brackets, array elements must be single numbers of the same type (either all integer or all real).

C2.7 Table Iterators

In the examples up to now, the code which wanted to get values from the table knew what the keys were. It's also useful to be able to write generic code which can operate on a table without knowing the keys. "Table iterators" ("iterators" for short) are used for this.

An iterator is an abstraction of a pointer to a particular table entry. Iterators are analogous to the `DIR *` pointers used by the POSIX `opendir()`, `readdir()`, `closedir()`, and similar functions, to Perl hash tables' `each()`, `keys()`, and `values`, and to the C++ Standard Template Library's forward iterators.

At any time the entries in a table may be considered to be in some arbitrary (implementation-defined) order; an iterator may be used to walk through some or all of the table entries in this order. This order is guaranteed to remain unchanged for any given table so long as no changes are made to that table, i.e. so long as no `Util_TableSet*()`, `Util_TableSet*Array()`, `Util_TableSetGeneric()`, `Util_TableSetGenericArray()`, `Util_TableSetString()`, or `Util_TableDeleteKey()` calls are made on that table (making such calls on other tables doesn't matter). The order may change if there is any change in the table, and it may differ even between different tables with identical key/value contents (including those produced by `Util_TableClone()`).⁸

Any change in the table also invalidates all iterators pointing anywhere in the table; using any such iterator is an error. Multiple iterators may point into the same table; they all use the same order, and (unlike in Perl) they're all independent.

The detailed function description in the Reference Manual for `Util_TableItQueryKeyValueInfo()` has an example of using an iterator to print out all the entries in a table.

C2.8 Multithreading and Multiprocessor Issues

At the moment, the table functions are *not* thread-safe in a multithreaded environment! However, this should change in the not-too-distant future: then all the table functions will default to being thread-safe. That is, user code will be able call these functions concurrently from multiple threads,

⁷C automatically concatenates adjacent character string constants separated only by whitespaces.

⁸For example, if tables were implemented by hashing, the internal order could be that of the hash buckets, and the hash function could depend on the internal table address.

with the table functions automatically doing any necessary locking.⁹ (We may add a flags-word bit to suppress this for maximum performance if you know you won't be making concurrent calls from multiple threads.)

Note that tables and iterators will still be process-wide, i.e. all threads see the same tables and iterators (think of them as like the Unix current working directory, with the various routines which modify the table or change iterators acting like a Unix `chdir()` system call).

In a multiprocessor environment, tables are always processor-local.

C2.9 Metadata about All Tables

We have decided that tables do not *themselves* have names or other attributes. However, at some time in the future we may add some special “system tables” to be used by Cactus itself to store this sort of information for those cases where it's needed.

For example, at some time in the future we may add support for a “checkpoint me” bit in a table's flags word, so that if you want a table to be checkpointed, you just need to set this bit. In this case the table will probably get a system-generated name in the checkpoint dump file. But if you want the table to have some other name in the dump file, then you need to tell the checkpointing code that by setting an appropriate entry in a checkpoint table. (You would find the checkpoing table by looking in a special “master system table” that records handles of other interesting tables.)

⁹For the implementation, this means that we will need a reader/writer lock for each table and for each iterator: any number of threads will be able to concurrently read the data structure, but any write will require exclusive access.

Part D

Infrastructure Thorn Writer's Guide

Chapter D1

Introduction

- Concepts and terminology (Overloading and registration of functions)
- The cGH structure — what it is and how to use it
- Extending the cGH structure
- Querying group and variable information
- Providing an IO layer
- Providing a communication layer
- Providing a reduction operator
- Providing an interpolation operator
- Overloadable functions

Chapter D2

Concepts and Terminology

D2.1 Overloading and Registration

The flesh defines a core API which guarantees the presence of a set of functions. Although the flesh guarantees the presence of these functions, they can be provided by thorns. Thorns do this either by the *overloading* or the *registration* of functions.

D2.1.1 Overloading

Some functions can only be provided by one thorn. The first thorn to *overload* this function succeeds, and any later attempt to overload the function fails. For each overloadable function there is a function with a name something like `CCTK_Overload...` which is passed the function pointer.

D2.1.2 Registration

Some functions may be provided by several thorns. The thorns *register* their function with the flesh, and when the flesh-provided function is called, the flesh calls all the registered functions.

D2.2 GH Extensions

A GH extension is a way to associate data with each cGH. This data should be data that is required to be associated with a particular GH by a thorn.

Each GH extension is given a unique handle.

D2.3 IO Methods

An IO method is a distinct way to output data. Each IO method has a unique name, and the flesh-provided IO functions operate on all registered IO methods.

Chapter D3

GH Extensions

A GH extension is created by calling `CCTK_RegisterGHEExtension`, with the name of the extension. This returns a unique handle that identifies the extension. (This handle can be retrieved at any time by a call to `CCTK_GHEExtensionHandle`.)

Associated with a GH extension are three functions

SetupGH	this is used to actually create the data structure holding the extension. It is called when a new cGH is created.
InitGH	this is used to initialise the extension. It is called after the scheduler has been initialised on the cGH.
ScheduleTraverseGH	this is called whenever the schedule tree is due to be traversed on the GH. It should initialise the data on the cGH and the call <code>CCTK_ScheduleTraverse</code> to traverse the schedule tree.

Chapter D4

Overloadable and Registerable Functions in Main

Function	Default
CCTK_Initialise	
CCTK_Evolve	
CCTK_Shutdown	

Chapter D5

Overloadable and Registerable Functions in Comm

Function	Default
CCTK_SyncGroup	
CCTK_EnableGroupStorage	
CCTK_DisableGroupStorage	
CCTK_EnableGroupComm	
CCTK_DisableGroupComm	
CCTK_Barrier	
CCTK_Reduce	
CCTK_Interp	
CCTK_ParallelInit	

Chapter D6

Overloadable and Registerable Functions in IO

Function	Default
CCTK_OutputGH	
CCTK_OutputVarAsByMethod	

Chapter D7

Adding a Driver

The flesh knows nothing about memory allocation for grid variables, or about how to communicate data when synchronisation is called for. It knows nothing about multiple patches or adaptive mesh refinement. All this is the job of a driver.

D7.1 Anatomy

A driver consists of a Startup routine which creates a GH extension and registers its associated functions, and overloads the communication functions. It may optionally register interpolation, reduction, and IO methods.

A driver may also overload the default Initialisation and Evolution routines, although a simple unigrid evolver is supplied in the flesh.

D7.2 Startup

A driver consists of a GH extension, and the following overloaded functions.

1. CCTK_EnableGroupStorage
2. CCTK_DisableGroupStorage
3. CCTK_ArrayGroupSizeB
4. CCTK_QueryGroupStorageB
5. CCTK_SyncGroup
6. CCTK_EnableGroupComm
7. CCTK_DisableGroupComm
8. CCTK_Barrier
9. CCTK_OverloadParallelInit
10. CCTK_OverloadExit
11. CCTK_OverloadAbort

12. CCTK_OverloadMyProc

13. CCTK_OverloadnProcs

D7.3 The GH Extension

The GH extension is where the driver stores all its grid-dependent information. This is stuff like any data associated with a grid variable (e.g. storage and communication state), how many grids if it is AMR, ... It is very difficult to describe in general, but one simple example might be

```
struct SimpleExtension
{
    /* The data associated with each variable */
    /* data[var][timelevel][ijk]                */
    void ***data
} ;
```

with a SetupGH routine like.

```
struct SimpleExtension *SimpleSetupGH(tFleshConfig *config, int conv_level, cGH *GH)
{
    struct SimpleExtension *extension;

    extension = NULL;

    if(conv_level < max_conv_level)
    {
        /* Create the extension */
        extension = malloc(sizeof(struct SimpleExtension));

        /* Allocate data for all the variables */
        extension->data = malloc(num_vars*sizeof(void**));

        for(var = 0 ; var < num_vars; var++)
        {
            /* Allocate the memory for the time levels */
            extension->data[var] = malloc(num_var_time_levels*sizeof(void *));

            for(time_level = 0; time_level < num_var_time_level; time_level++)
            {
                /* Initialise the data to NULL */
                extension->data[var][time_level] = NULL;
            }
        }
    }

    return extension;
}
```

Basically all this example is doing is preparing a data array for use. The function can query the flesh for information on every variable. Note that scalars should always have memory actually assigned to them.

An `InitGH` function isn't strictly necessary, and in this case it could just be a dummy function.

The `ScheduleTraverseGH` function needs to fill out the `cGH` data and then call `CCTK_ScheduleTraverse` to have the functions scheduled at that point executed on the grid.

```
int SimpleScheduleTraverseGH(cGH *GH, const char *where)
{
    int retcode;
    int var;
    int gtype;
    int ntimelevels;
    int level;
    int idir;

    extension = (struct SimpleExtension *)GH->extensions[SimpleExtension];

    for (idir=0; idir<GH->cctk_dim; idir++)
    {
        GH->cctk_levfac[idir] = 1;
        GH->cctk_nghostzones[idir] = extension->nghostzones[idir];
        GH->cctk_lsh[idir] = extension->lsize[idir];
        GH->cctk_gsh[idir] = extension->nsizes[idir];
        GH->cctk_bbox[2*idir] = extension->lb[extension->myproc][idir] == 0;
        GH->cctk_bbox[2*idir+1] = extension->ub[extension->myproc][idir]
                                == extension->nsizes[idir]-1;
        GH->cctk_lbnd[idir] = extension->lb[extension->myproc][idir];
        GH->cctk_ubnd[idir] = extension->ub[extension->myproc][idir];
    }

    for(var = 0; var < extension->nvariables; var++)
    {
        gtype = CCTK_GroupTypeFromVarI(var);
        ntimelevels = CCTK_MaxTimeLevelsVI(var);

        for(level = 0; level < ntimelevels; level++)
        {
            switch(gtype)
            {
                case CCTK_SCALAR :
                    GH->data[var][level] = extension->variables[var][level];
                    break;
                case CCTK_GF :
                    GH->data[var][level] =
                        ((pGF **)(extension->variables))[var][level]->data;
                    break;
                case CCTK_ARRAY :
                    GH->data[var][level] =
                        ((pGA **)(extension->variables))[var][level]->data;
                    break;
                default:
                    CCTK_WARN(1, "Unknown group type in SimpleScheduleTraverse");
            }
        }
    }
}
```

```

    }
}

retcode = CCTK_ScheduleTraverse(where, GH, NULL);

return retcode;
}

```

The third argument to `CCTK_ScheduleTraverse` is actually a function which will be called by the scheduler when it wants to call a function scheduled by a thorn. This function is given some information about the function to call, and is an alternative place where the cGH can be setup.

This function is optional, but a simple implementation might be

```

int SimpleCallFunction(void *function,
                      cFunctionData *fdata,
                      void *data)
{
    void (*standardfunc)(void *);

    int (*noargsfunc)(void);

    switch(fdata->type)
    {
        case FunctionNoArgs:
            noargsfunc = (int (*)(void))function;
            noargsfunc();
            break;
        case FunctionStandard:
            switch(fdata->language)
            {
                case LangC:
                    standardfunc = (void (*)(void *))function;
                    standardfunc(data);
                    break;
                case LangFortran:
                    fdata->FortranCaller(data, function);
                    break;
                default :
                    CCTK_WARN(1, "Unknown language.");
            }
            break;
        default :
            CCTK_WARN(1, "Unknown function type.");
    }

    /* Return 0, meaning didn't synchronise */
    return 0;
}

```

The return code of the function signifies whether or not the function synchronised the groups in this functions synchronisation list of not.

The flesh will synchronise them if the function returns false.

Providing this function is probably the easiest way to do multi-patch or AMR drivers.

D7.4 Memory Functions

These consist of

1. CCTK_EnableGroupStorage
2. CCTK_DisableGroupStorage
3. CCTK_QueryGroupStorageB
4. CCTK_ArrayGroupSizeB

D7.4.1 En/Disable Group Storage

These are responsible for switching the memory for all variables in a group on or off. They should return the former state, e.g. if the group already has storage assigned, they should return 1.

In our simple example above, the enabling routine would look something like

```
int SimpleEnableGroupStorage(cGH *GH, const char *groupname)
{
    extension = (struct SimpleExtension *)GH->extensions[SimpleExtension];

    if(extension->data[first][0][0] == NULL)
    {
        for(var = first; var <= last; var++)
        {
            allocate memory for all time levels;
        }
        retcode = 0;
    }
    else
    {
        retcode = 1;
    }

    return retcode;
}
```

The disable function is basically the reverse of the enable one.

The QueryGroupStorage function basically returns true or false if there is storage for the group, and the ArrayGroupSize returns the size of the grid function or array group in a particular direction.

D7.4.2 En/Disable Group Comm

These are the communication analogues to the storage functions. Basically they flag that communication is to be done on that group or not, and may initialise data structures for the communication.

Chapter D8

Adding an I/O Method

The flesh does not implement output for grid variables and other data by itself. Instead it provides a mechanism for thorns to register their own routines as I/O methods, and to invoke these I/O methods by either the flesh scheduler or by other thorn routines.

D8.1 I/O Method Registration

All I/O methods have to be registered with the flesh before they can be used.

The flesh I/O registration API provides a routine `CCTK_RegisterIOMethod()` to create a handle for a new I/O method which is identified by its name (this name must be unique for all I/O methods). With such a handle, a thorn can then register a set of functions (using the `CCTK_RegisterIOMethod*()` routines from the flesh) for doing periodic, triggered, and/or unconditional output.

The following example shows how a thorn would register an I/O method *"SimpleIO"* with routines to provide all these different types of output.

```
void SimpleIO_Startup (void)
{
    int handle;

    handle = CCTK_RegisterIOMethod ("SimpleIO");
    if (handle >= 0)
    {
        CCTK_RegisterIOMethodOutputGH (handle, SimpleIO_OutputGH);

        CCTK_RegisterIOMethodTimeToOutput (handle, SimpleIO_TimeToOutput);
        CCTK_RegisterIOMethodTriggerOutput (handle, SimpleIO_TriggerOutput);

        CCTK_RegisterIOMethodOutputVarAs (handle, SimpleIO_OutputVarAs);
    }
}
```

D8.2 Periodic Output of Grid Variables

The flesh scheduler will automatically call `CCTK_OutputGH()` at every iteration after the `CCTK_ANALYSIS` time bin. This function loops over all I/O methods and calls their routines registered as `OutputGH()` (see also [B3.3.4](#)).

```
int SimpleIO_OutputGH (const cGH *GH);
```

The `OutputGH()` routine itself should loop over all variables living on the `GH` grid hierarchy, and do all necessary output if requested (this is usually determined by I/O parameter settings).

As its return code it should pass back the number of variables which were output at the current iteration, or zero if no output was done by this I/O method.

D8.3 Triggered Output of Grid Variables

Besides the periodic output at every so many iterations using `OutputGH()`, analysis and output of grid variables can also be done via triggers. For this, a `TimeToOutput()` routine is registered with an I/O method. This routine will be called by the flesh scheduler at every iteration before `CCTK_ANALYSIS` with the triggering variable(s) as defined in the schedule block for all `CCTK_ANALYSIS` routines (see [B6.4](#)).

If the `TimeToOutput()` routine decides that it is now time to do output, the flesh scheduler will invoke the corresponding analysis routine and also request output of the triggering variable(s) using `TriggerOutput()`.

```
int SimpleIO_TimeToOutput (const cGH *GH, int varindex);  
int SimpleIO_TriggerOutput (const cGH *GH, int varindex);
```

Both routines get passed the index of a possible triggering grid variable.

`TimeToOutput()` should return true (a non-zero value) if analysis and output for *varindex* should take place at the current iteration, and false (zero) otherwise.

`TriggerOutput()` should return zero for successful output of variable *varindex*, and a negative value in case of an error.

D8.4 Unconditional Output of Grid Variables

An I/O method's `OutputVarAs()` routine is supposed to do output for a specific grid variable if ever possible. It will be invoked by the flesh I/O API routines `CCTK_OutputVar*()` for unconditional, non-triggered output of grid variables (see also [B8.2](#)).

A function registered as an `OutputVarAs()` routine has the following prototype:

```
int SimpleIO_OutputVarAs (const cGH *GH, const char *varname, const char *alias);
```

The variable to output, *varname*, is given by its full name. In addition to that, an *alias* string can be passed which then serves the purpose of constructing a unique name for the output file.

The `OutputVarAs()` routine should return zero if output for *varname* was done successfully, or a negative error code otherwise.

Chapter D9

Adding a Checkpointing/Recovery Method

Like for I/O methods, checkpointing/recovery functionality must be implemented by thorns. The flesh only provides specific time bins at which the scheduler will call thorns' routines in order to perform checkpointing and/or recovery.

D9.1 Invocation of Checkpointing Routines

Thorns register their checkpointing routines at `CCTK_CPINITIAL` and/or `CCTK_CHECKPOINT`. These time bins are scheduled right after all initial data has been set up and after every evolution timestep resp. (see [B3.3.4](#) for a description of all timebins). Depending on parameter settings, the checkpoint routines decide whether to write an initial data checkpoint, and when to write an evolution checkpoint.

Each checkpoint routine should save all information to persistent storage which is necessary to restart the simulation at a later time from exactly the same state. Such information would include

- the current settings of all parameters
- the contents of all grid variables which have global storage assigned
Note that grid variables should be synced before writing them to disk.
- other relevant information such as the current iteration number and physical time, the number of processors, etc.

D9.2 Invocation of Recovery Routines

Recovering from a checkpoint is a two-phase operation for which the flesh provides distinct timebins for recovery routines to be scheduled at:

`CCTK_RECOVER_PARAMETERS`

This time bin is scheduled right after `CCTK_STARTUP` when the parameter file was parsed. From these parameter settings, the recovery routines should decide whether recovery was requested, and if so, restore all parameters the checkpoint file and overwrite those which aren't steerable.

The flesh scheduler loops over all registered recovery routines to find out whether

recovery was requested. Each recovery routine should therefore return a positive integer value for successful parameter recovery (causing the scheduler to shortcut its loop over all registered recovery routines), zero for no recovery, or a negative value to flag an error.

If recovery was requested but no routine could successfully recover parameters, the flesh will abort the run with an error message.

If parameter recovery was performed successfully, the scheduler will set the `recovered` flag which – in combination with the setting of the `Cactus::recovery_mode` parameter – decides whether any thorn routine scheduled for `CCTK_INITIAL` and `CCTK_POSTINITIAL` will be called. The default is not to execute these initial time bins during recovery because the initial data will be set up from the checkpoint file during the following `CCTK_RECOVER_VARIABLES` time bin.

`CCTK_RECOVER_VARIABLES`

Recovery routines scheduled for this time bin are responsible for restoring the contents of all grid variables with storage assigned from the checkpoint.

Depending on the setting of `Cactus::recovery_mode` they should also decide how to treat errors in recovering individual grid variables. Strict recovery (which is the default) requires all variables to be restored successfully (and should stop the code if not) whereas a relaxed mode could eg. allow for grid variables which are not found in the checkpoint file to be gracefully ignored during recovery.

Chapter D10

Adding a timer

To add a Cactus timer you need to write several functions to provide the timer functionality, and then register these functions with the flesh, with a name for the timer using `CCTK_TimerRegister`.

The functions are registered using a structure `cTimerFuncs` which contains the function pointers. The required functions are:

```
info.n_vals

create          void (*create)(int);
destroy         void (*destroy)(int, void *)
start          void (*start)(int, void *)
stop           void (*stop)(int, void *)
reset          void (*reset)(int, void *)
get            void (*get)(int, void *, cTimerVal *)
set            void (*set)(int, void *, cTimerVal *)
```

Part E

FunctionReference

Chapter E1

Cactus Functions

In this section all CCTK_* Cactus functions are described. These functions are callable from Fortran or C thorns. Note that whereas all functions are available from C, not all are currently available from Fortran.

E1.1 Functions Alphabetically

CCTK_Abort	[E9] Causes abnormal Cactus termination
CCTK_ActiveTimeLevels	[E10] Returns the number of active timelevels from a group name
CCTK_ActiveTimeLevelsGN	[E10] Returns the number of active timelevels from a group name
CCTK_ActiveTimeLevelsGI	[E10] Returns the number of active timelevels from a group index
CCTK_ActiveTimeLevelsVN	[E10] Returns the number of active timelevels from a variable name
CCTK_ActiveTimeLevelsVI	[E10] Returns the number of active timelevels from a variable index
CCTK_ArrayGroupSize	[E11] Returns a pointer to the local size for a group, given by its group name
CCTK_ArrayGroupSizeI	[E12] Returns a pointer to the local size for a group, given by its group index
CCTK_Barrier	[E13] Synchronizes all processors
CCTK_Cmplx	[E14] Turns two real numbers into a complex number (only C)
CCTK_CmplxAbs	[E15] Returns the absolute value of a complex number (only C)
CCTK_CmplxAdd	[E16] Returns the sum of two complex numbers (only C)
CCTK_CmplxConjg	[E17] Returns the complex conjugate of a complex number (only C)
CCTK_CmplxCos	[E18] Returns the Cosine of a complex number (only C) [not yet available]
CCTK_CmplxDiv	[E19] Returns the division of two complex numbers (only C)
CCTK_CmplxExp	[E20] Returns the Exponentiation of a complex number (only C) [not yet available]
CCTK_CmplxImag	[E21] Returns the imaginary part of a complex number (only C)

CCTK_CmplxLog	[E22] Returns the Logarithm of a complex number (only C) [not yet available]
CCTK_CmplxMul	[E23] Returns the multiplication of two complex numbers (only C)
CCTK_CmplxReal	[E24] Returns the real part of a complex number (only C)
CCTK_CmplxSin	[E25] Returns the Sine of a complex number (only C) [not yet available]
CCTK_CmplxSqrt	[E26] Returns the square root of a complex number (only C) [not yet available]
CCTK_CmplxSub	[E27] Returns the subtraction of two complex numbers (only C)
CCTK_CoordDir	[E28] Give the direction for a given coordinate name.
CCTK_CoordIndex	[E29] Give the grid variable index for a given coordinate.
CCTK_CoordRange	[E30] Return the global upper and lower bounds for a given coordinate name on a cctkGH
CCTK_CoordRegisterData	[E31] Register a coordinate as belonging to a coordinate system, with a given name and direction, and optionally with a grid variable
CCTK_CoordRegisterRange	[E32] Saves the global upper and lower bounds for a given coordinate name on a cctkGH
CCTK_CoordRegisterSystem	[E33] Registers a coordinate system with a given dimension
CCTK_CoordSystemDim	[E34] Provides the dimension of a given coordinate system
CCTK_CoordSystemHandle	[E35] Get the handle associated with a registered coordinate system
CCTK_CoordSystemName	[E36] Provides the name of the coordinate system identified by its handle
CCTK_CreateDirectory	[E37] Creates a directory
CCTK_DecomposeName	[E38] Given the full name of a variable/group, separates the name returning both the implementation and the variable/group
CCTK_DisableGroupComm	[E39] Disable the communication for a group
CCTK_DisableGroupCommI	[??] Disable the communication for a group
CCTK_DisableGroupStorage	[E40] Disable the storage for a group
CCTK_DisableGroupStorageI	[??] Disable the storage for a group
CCTK_EnableGroupComm	[E41] Enable the communication for a group
CCTK_EnableGroupCommI	[E41] Enable the communication for a group
CCTK_EnableGroupStorage	[E42] Enable the storage for a group

CCTK_EnableGroupStorage	[E42] Enable the storage for a group
CCTK_Equals	[E43] Check a STRING or KEYWORD parameter for equality with a given string
CCTK_Exit	[E44] Causes normal Cactus termination
CCTK_FirstVarIndex	[E45] Given a group name returns the first variable index in the group
CCTK_FirstVarIndexI	[E46] Given a group index returns the first variable index in the group
CCTK_FortranString	[E47] Changes a C string into a Fortran string
CCTK_FullName	[E48] Given a variable index, returns the full name of the variable
CCTK_GHExtension	[E49] Get the pointer to a registered extension to the Cactus GH structure
CCTK_GHExtensionHandle	[E50] Get the handle associated with a extension to the Cactus GH structure
CCTK_GroupbboxGI	[E51] Given a group index, return an array of the bounding box of the group for each face
CCTK_GroupbboxGN	[E51] Given a group name, return an array of the bounding box of the group for each face
CCTK_GroupbboxVI	[E52] Given a variable index, return an array of the bounding box of the variable for each face
CCTK_GroupbboxVN	[E52] Given a variable name, return an array of the bounding box of the variable for each face
CCTK_GroupData	[E53] Given a group index, returns information about the variables held in the group
CCTK_GroupDynamicData	[E54] Given a group index, returns information about the variables held in the group
CCTK_GroupgshGI	[E55] Given a group index, return an array of the global size of the group in each dimension
CCTK_GroupgshGN	[E55] Given a group name, return an array of the global size of the group in each dimension
CCTK_GroupgshVI	[E57] Given a variable index, return an array of the global size of the variable in each dimension
CCTK_GroupgshVN	[E57] Given a variable name, return an array of the global size of the variable in each dimension
CCTK_GroupIndex	[E59] Get the index number for a group name
CCTK_GroupIndexFromVar	[E60] Given a variable name, returns the index of the associated group
CCTK_GroupIndexFromVarI	[E61] Given a variable index, returns the index of the associated group
CCTK_GrouplbndGI	[E62] Given a group index, return an array of the lower bounds of the group in each dimension
CCTK_GrouplbndGN	[E62] Given a group name, return an array of the lower bounds of the group in each dimension

CCTK_GroupLbndVI	[E64] Given a variable index, return an array of the lower bounds of the variable in each dimension
CCTK_GroupLbndVN	[E64] Given a variable name, return an array of the lower bounds of the variable in each dimension
CCTK_GroupLshGI	[E66] Given a group index, return an array of the local size of the group in each dimension
CCTK_GroupLshGN	[E66] Given a group name, return an array of the local size of the group in each dimension
CCTK_GroupLshVI	[E68] Given a variable index, return an array of the local size of the variable in each dimension
CCTK_GroupLshVN	[E68] Given a variable name, return an array of the local size of the variable in each dimension
CCTK_GroupName	[E70] Given a group index, returns the group name
CCTK_GroupNameFromVarI	[E71] Given a variable index, return the name of the associated group
CCTK_GroupnghostzonesGI	[E72] Given a group index, return an array with the number of ghostzones in each dimension of the group
CCTK_GroupnghostzonesGN	[E72] Given a group name, return an array with the number of ghostzones in each dimension of the group
CCTK_GroupTypeFromVarI	[E74] Provides a group's group type index given a variable index
CCTK_GroupTypeI	[E75] Provides a group's group type index given a group index
CCTK_GroupubndGI	[E76] Given a group index, return an array of the upper bounds of the group in each dimension
CCTK_GroupubndGN	[E76] Given a group name, return an array of the upper bounds of the group in each dimension
CCTK_GroupubndVI	[E78] Given a variable index, return an array of the upper bounds of the variable in each dimension
CCTK_GroupubndVN	[E78] Given a variable name, return an array of the upper bounds of the variable in each dimension
CCTK_ImpFromVarI	[E80] Given a variable index, returns the implementation name
CCTK_INFO	[E81] Macro to print a single string as an information message to screen
CCTK_InterpGridArrays	[E82] Performs an interpolation on a list of CCTK grid arrays, using a chosen external local interpolation operator
CCTK_InterpGV	[E87] Performs an interpolation on a list of CCTK grid arrays, using a chosen build-in local interpolation operator (this function is being phased out; it will eventually be replaced by <code>CCTK_InterpGridArrays()</code>)
CCTK_InterpHandle	[E89] Returns the handle for a given interpolation operator
CCTK_InterpLocal	[E90] Performs an interpolation on a list of processor-local arrays, using a chosen interpolation operator (this function is being phased out; it will eventually be replaced by <code>CCTK_InterpLocalUniform()</code> , <code>CCTK_InterpLocalNonUniform()</code> , and <code>CCTK_InterpLocalWarped()</code> .)

- CCTK_InterpLocalUniform** [\[E93\]](#) Interpolate a list of processor-local arrays which define a uniformly-spaced data grid
- CCTK_InterpRegisterOperatorGV** [\[E98\]](#) Registers a routine as a `CCTK_InterpGV()` interpolation operator
- CCTK_InterpRegisterOperatorLocal** [\[E99\]](#) Registers a routine as a `CCTK_InterpLocal()` interpolation operator
- CCTK_InterpRegisterOpLocalUniform** [\[E100\]](#) Registers a routine as a `CCTK_InterpLocalUniform()` interpolation operator
- CCTK_IsImplementationActive** [\[E102\]](#) Reports whether an implementation was activated in a parameter file
- CCTK_IsImplementationCompiled** [\[E103\]](#) Reports whether an implementation was compiled into a configuration
- CCTK_IsThornActive** [\[E104\]](#) Reports whether a thorn was activated in a parameter file
- CCTK_IsThornCompiled** [\[E104\]](#) Reports whether a thorn was activated in a parameter file
- CCTK_MaxDim** [\[E106\]](#) Get the maximum dimension of any grid variable
- CCTK_MaxTimeLevels** [\[E108\]](#) Gives the maximum number of timelevels for a group
- CCTK_MaxTimeLevelsGI** [\[E111\]](#) Gives the maximum number of timelevels for a group
- CCTK_MaxTimeLevelsGN** [\[??\]](#) Gives the maximum number of timelevels for a group
- CCTK_MaxTimeLevelsVI** [\[??\]](#) Gives the maximum number of timelevels for a variable
- CCTK_MaxTimeLevelsVN** [\[E109\]](#) Gives the maximum number of timelevels for a variable
- CCTK_MyProc** [\[E107\]](#) Get the local processor number
- CCTK_nProcs** [\[E112\]](#) Get the total number of processors used
- CCTK_NullPointer** [\[E113\]](#) Returns a C-style NULL pointer value
- CCTK_NumGroups** [\[E114\]](#) Get the number of groups of variables compiled in the code
- CCTK_NumIOMethods** [\[E115\]](#) Returns the total number of I/O methods registered with the flesh
- CCTK_NumVars** [\[E116\]](#) Get the number of grid variables compiled in the code
- CCTK_NumVarsInGroup** [\[E117\]](#) Provides the number of variables in a group from the group name
- CCTK_NumVarsInGroupI** [\[E118\]](#) Provides the number of variables in a group from the group index
- CCTK_OutputGH** [\[E119\]](#) Conditional output of all variables on a GH by all I/O methods
- CCTK_OutputVar** [\[E120\]](#) Output of a single variable by all I/O methods
- CCTK_OutputVarAs** [\[E121\]](#) Output of a single variable as an alias by all I/O methods
- CCTK_OutputVarAsByMethod** [\[E122\]](#) Output of a single variable as an alias by a single I/O method

CCTK_OutputVarByMethod	[E123] Output of a single variable by a single I/O method
CCTK_ParallelInit	[E124] Initializes the parallel subsystem
CCTK_PARAMWARN	[E125] Prints a warning from parameter checking, and possibly stops the code
CCTK_PointerTo	[E126] Returns the address of a variable passed in by reference from a fortran routine
CCTK_PrintGroup	[E127] Prints a group name from its index
CCTK_PrintString	[E128] Prints a Cactus string to screen (from Fortran)
CCTK_PrintVar	[E129] Prints a variable name from its index
CCTK_QueryGroupStorage	[E130] Queries storage for a group given by its group name
CCTK_QueryGroupStorageB	[E131]
CCTK_QueryGroupStorageI	[E132] Queries storage for a group given by its group index
CCTK_ReductionHandle	[E133] Get the handle for a registered reduction operator
CCTK_RegisterBanner	[E134] Register a banner for a thorn
CCTK_RegisterGHExtension	[E135] Register the name of an extension to the Cactus GH.
CCTK_RegisterGHExtensionInitGH	[E136] Register a routine for providing initialisation for an extension to the Cactus GH
CCTK_RegisterGHExtensionScheduleTraverseGH	[E137] Register a GH extension schedule traversal routine
CCTK_RegisterGHExtensionSetupGH	[E138] Register a routine for setting up an extension to the Cactus GH
CCTK_RegisterIOMethod	[E139] Registers a new I/O method
CCTK_RegisterIOMethodOutputGH	[E140] Registers an I/O method's routine for conditional output
CCTK_RegisterIOMethodOutputVarAs	[E141] Registers an I/O method's routine for unconditional output
CCTK_RegisterIOMethodTimeToOutput	[E142] Register a routine for deciding if it is time to output for an IO method
CCTK_RegisterIOMethodTriggerOutput	[E143] Register a routine for dealing with trigger output for an IO method
CCTK_RegisterReductionOperator	[E144] Register a function as providing a reduction operation
CCTK_SetupGH	[E145] Sets up a CCTK grid hierarchy
CCTK_SyncGroup	[E146] Synchronize the ghost zones for a group of variables
CCTK_VarDataPtr	[E147] Returns the data pointer for a grid variable

CCTK_VarDataPtrB	[E148] Returns the data pointer for a grid variable from the variable index or name
CCTK_VarDataPtrI	[E149] Returns the data pointer for a grid variable from the variable index
CCTK_VarIndex	[E150] Get the index for a variable
CCTK_VarName	[E151] Given a variable index, returns the variable name
CCTK_VarTypeI	[E152] Provides variable type index from the variable index
CCTK_VarTypeSize	[E153] Provides variable type size in bytes from the variable type index
CCTK_VInfo	[E154] Prints a formatted string with a variable argument list as an information message to screen
CCTK_VWarn	[E155] Prints a formatted string with a variable argument list as a warning message to standard error and possibly stops the code
CCTK_WARN	[E156] Macro to print a single string as a warning message to standard error and possibly stop the code

E1.2 Full Description of Functions

CCTK_Abort

Abnormal Cactus termination

Synopsis

C	int istat = CCTK_Abort(cGH * cctkGH, int value)
Fortran	call CCTK_Abort(istat , cctkGH, value)
	integer istat
	CCTK_POINTER cctkGH
	integer value

Parameters

cctkGH - pointer to CCTK grid hierarchy

value - the return code to abort with

Discussion

This routine causes an immediate, abnormal Cactus termination. It never returns to the caller.

CCTK_ActiveTimeLevels

Returns the number of active time levels for a group.

Synopsis

```
C          #include "cctk.h"
          int timelevels = CCTK_ActiveTimeLevels(const cGH *cctkGH,
                                                  const char *groupname);
```

Result

NULL The number of timelevels

Parameters

GH ($\neq$ NULL) Pointer to a valid Cactus grid hierarchy.
groupname Name of the group.

Discussion

This function returns the number of timelevels for which storage has been activated, which is always equal to or less than the maximum number of timelevels which may have storage provided by `CCTK_MaxTimeLevels`.

See Also

CCTK-MaxTimeLevels	Return the maximum number of timelevels
CCTK-GroupStorageIncrease	Base function overloaded by driver which increase the number of timelevels with storage and also returns the number of active timelevels.

CCTK_ArrayGroupSize

Returns a pointer to the processor-local size for variables in a group, specified by its name, in a given dimension.

Synopsis

```
C      #include "cctk.h"
      int *size = CCTK_ArrayGroupSize(const cGH *cctkGH,
                                      int dir,
                                      const char *groupname);
```

Result

NULL A NULL pointer is returned if the group index or the dimension given are invalid.

Parameters

GH ($\neq$ NULL) Pointer to a valid Cactus grid hierarchy.
 dir (≥ 0) Which dimension of array to query.
 groupname Name of the group.

Discussion

For a CCTK_ARRAY or CCTK_GF group, this routine returns a pointer to the processor-local size for variables in that group in a given direction. The direction is counted in C order (zero being the lowest dimension).

This function returns a pointer to the result for technical reasons; so that it will efficiently interface with Fortran. This may change in the future. Consider using CCTK_GroupgshGN instead.

See Also

CCTK_GroupgshGN Returns an array with the array size in all dimensions.
 ... There are many related functions which grab information from the GH, but many are not yet documented.

CCTK_ArrayGroupSizeI

Returns a pointer to the processor-local size for variables in a group, specified by its index, in a given dimension.

Synopsis

```
C      #include "cctk.h"
      int *size = CCTK_ArrayGroupSizeI(const cGH *cctkGH,
                                      int dir,
                                      int groupi);
```

Result

NULL A NULL pointer is returned if the group index or the dimension given are invalid.

Parameters

GH ($\neq$ NULL) Pointer to a valid Cactus grid hierarchy.
 dir (≥ 0) Which dimension of array to query.
 groupi The group index.

Discussion

For a CCTK_ARRAY or CCTK_GF group, this routine returns a pointer to the processor-local size for variables in that group in a given direction. The direction is counted in C order (zero being the lowest dimension).

This function returns a pointer to the result for technical reasons; so that it will efficiently interface with Fortran. This may change in the future. Consider using CCTK_GroupgshGI instead.

See Also

CCTK_GroupgshGI Returns an array with the array size in all dimensions.
 ... There are many related functions which grab information from the GH, but many are not yet documented.

CCTK_Barrier

Synchronizes all processors at a given execution point

Synopsis

C	<code>int istat = CCTK_Barrier(cGH * cctkGH)</code>
Fortran	<code>call CCTK_Barrier(istat , cctkGH)</code>
	integer istat
	CCTK_POINTER cctkGH

Parameters

<code>cctkGH</code>	-	pointer to CCTK grid hierarchy
<code>istat</code>	-	return code

Discussion

This routine synchronizes all processors in a parallel job at a given point of execution. No processor will continue execution until all other processors have called `CCTK_Barrier()`. Note that this is a collective operation – it must be called by all processors otherwise the code will hang.

CCTK_Cmplx

Turns two real numbers into a complex number

Synopsis

```
C      CCTK_COMPLEX cmpno = CCTK_Cmplx( CCTK_REAL realpart, CCTK_REAL imagpart)
```

Parameters

```
      cmpno  -  The complex number  
      realpart -  The real part of the complex number  
      imagpart -  The imaginary part of the complex number
```

Discussion**Examples**

```
C      cmpno = CCTK_Cmplx(re,im);
```

Errors

CCTK_CmplxAbs

Absolute value of a complex number

Synopsis

```
C      CCTK_COMPLEX absval = CCTK_CmplxAbs( CCTK_COMPLEX inval)
```

Parameters

```
absval  - The computed absolute value  
realpart - The complex number who absolute value is to be returned
```

Discussion**Examples**

```
C      absval = CCTK_CmplxAbs(inval);
```

Errors

CCTK_CmplxAdd

Sum of two complex numbers

Synopsis

```
C      CCTK_COMPLEX addval = CCTK_CmplxAdd( CCTK_COMPLEX inval1, CCTK_COMPLEX i
```

Parameters

```
addval  - The computed added value
inval1  - The first complex number to be summed
inval2  - The second complex number to be summed
```

Discussion**Examples**

```
C      addval = CCTK_CmplxAdd(inval1,inval2);
```

Errors

CCTK_CmplxConjg

Complex conjugate of a complex number

Synopsis

```
C      CCTK_COMPLEX conjgval = CCTK_CmplxConjg( CCTK_COMPLEX inval)
```

Parameters

```
    conjgval  -  The computed conjugate  
    inval     -  The complex number to be conjugated
```

Discussion**Examples**

```
C      conjgval = CCTK_CmplxConjg(inval);
```

Errors

CCTK_CmplxCos

Cosine of a complex number

Synopsis

```
C      CCTK_COMPLEX cosval = CCTK_CmplxCos( CCTK_COMPLEX inval)
```

Parameters

`cosval` - The computed cosine
`inval` - The complex number to be cosined

Discussion

NOT YET AVAILABLE

Examples

```
C      cosval = CCTK_CmplxCos(inval);
```

Errors

CCTK_CmplxDiv

Division of two complex numbers

Synopsis

```
C      CCTK_COMPLEX divval = CCTK_CmplxDiv( CCTK_COMPLEX inval1, CCTK_COMPLEX in
```

Parameters

`divval` - The divided value

`inval1` - The enumerator

`inval1` - The denominator

Discussion**Examples**

```
C      divval = CCTK_CmplxDiv(inval1,inval2);
```

Errors

CCTK_CmplxExp

Exponent of a complex number

Synopsis

```
C      CCTK_COMPLEX expval = CCTK_CmplxExp( CCTK_COMPLEX inval)
```

Parameters

`expval` - The computed exponent
`inval` - The complex number to be exponentiated

Discussion

NOT YET AVAILABLE

Examples

```
C      expval = CCTK_CmplxExp(inval);
```

Errors

CCTK_CmplxImag

Imaginary part of a complex number

Synopsis

```
C      CCTK_REAL imval = CCTK_CmplxImag( CCTK_COMPLEX inval)
```

Parameters

```
imval  -  The imaginary part  
inval  -  The complex number
```

Discussion

The imaginary part of a complex number $z = a + bi$ is b .

Examples

```
C      imval = CCTK_CmplxImag(inval);
```

Errors

CCTK_CmplxLog

Logarithm of a complex number

Synopsis

```
C      CCTK_COMPLEX logval = CCTK_CmplxLog( CCTK_COMPLEX inval)
```

Parameters

logval - The computed logarithm

inval - The complex number

Discussion

NOT YET AVAILABLE

Examples

```
C      logval = CCTK_CmplxLog(inval);
```

Errors

CCTK_CmplxMul

Multiplication of two complex numbers

Synopsis

```
C      CCTK_COMPLEX mulval = CCTK_CmplxMul( CCTK_COMPLEX inval1, CCTK_COMPLEX inval2 );
```

Parameters

`mulval` - The product

`inval1` - First complex number to be multiplied

`inval2` - Second complex number to be multiplied

Discussion

The product of two complex numbers $z_1 = a_1 + b_1i$ and $z_2 = a_2 + b_2i$ is $z = (a_1a_2 - b_1b_2) + (a_1b_2 + a_2b_1)i$.

Examples

```
C      mulval = CCTK_CmplxMul(inval1,inval2);
```

Errors

CCTK_CmplxReal

Real part of a complex number

Synopsis

```
C      CCTK_REAL reval = CCTK_CmplxReal( CCTK_COMPLEX inval)
```

Parameters

`reval` - The real part
`inval` - The complex number

Discussion

The real part of a complex number $z = a + bi$ is a .

Examples

```
C      reval = CCTK_CmplxReal(inval);
```

Errors

CCTK_CmplxSin

Sine of a complex number

Synopsis

```
C      CCTK_COMPLEX sinval = CCTK_CmplxSin( CCTK_COMPLEX inval)
```

Parameters

```
sinval  -  The computed sine  
inval   -  The complex number to be Sined
```

Discussion

NOT YET AVAILABLE

Examples

```
C      sinval = CCTK_CmplxSin(inval);
```

Errors

CCTK_CmplxSqrt

Square root of a complex number

Synopsis

```
C      CCTK_COMPLEX sqrtval = CCTK_CmplxSqrt( CCTK_COMPLEX inval)
```

Parameters

`expval` - The computed square root
`inval` - The complex number to be square rooted

Discussion

NOT YET AVAILABLE

Examples

```
C      sqrtval = CCTK_CmplxSqrt(inval);
```

Errors

CCTK_CmplxSub

Subtraction of two complex numbers

Synopsis

```
C      CCTK_COMPLEX subval = CCTK_CmplxSub( CCTK_COMPLEX inval1, CCTK_COMPLEX inval2 );
```

Parameters

`addval` - The computed subtracted value

`inval1` - The complex number to be subtracted from

`inval2` - The complex number to subtract

Discussion

If $z_1 = a_1 + b_1i$ and $z_2 = a_2 + b_2i$ then

$$z_1 - z_2 = (a_1 - a_2) + (b_1 - b_2)i$$

Examples

```
C      subval = CCTK_CmplxSub(inval1,inval2);
```

Errors

CCTK_CoordDir

Give the direction for a given coordinate.

Synopsis

C	<code>int dir = CCTK_CoordDir(const char * coordname, const char * systemname)</code>
Fortran	<code>call CCTK_CoordDir(dir , coordname, systemname)</code>
	integer dir
	character*(*) coordname
	character*(*) systemname

Parameters

<code>dir</code>	-	The direction of the coordinate
<code>coordname</code>	-	The name assigned to this coordinate
<code>systemname</code>	-	The name of the coordinate system

Discussion

The coordinate name is independent of the grid function name.

Examples

C	<code>direction = CCTK_CoordDir("xdir","cart3d");</code>
Fortran	<code>call CCTK_COORDDIR(direction,"radius","spher3d")</code>

Errors

CCTK_CoordIndex

Give the grid variable index for a given coordinate.

Synopsis

C	int index = CCTK_CoordIndex(int direction, const char * coordname, const char * systemname)
Fortran	call CCTK_CoordIndex(index , direction, coordname, systemname)

integer index
integer direction
character*(*) coordname
character*(*) systemname

Parameters

index - The coordinates associated grid variable index

direction - The direction of the coordinate in this coordinate system

coordname - The name assigned to this coordinate

systemname - The coordinate system for this coordinate

Discussion

The coordinate name is independent of the grid variable name. To find the index, the coordinate system name must be given, and either the coordinate direction or the coordinate name. The coordinate name will be used if the coordinate direction is given as less than or equal to zero, otherwise the coordinate name will be used.

Examples

C	index = CCTK_CoordIndex(-1,"xdir","cart3d");
Fortran	one = 1 call CCTK_COORDINDEX(index,one,"radius","spher2d")

Errors

CCTK_CoordRange

Return the global upper and lower bounds for a given coordinate

Synopsis

C	<code>int ierr = CCTK_CoordRange(const cGH * cctkGH, CCTK_REAL * lower, CCTK_REAL * upper,</code>
Fortran	<code>call CCTK_CoordRange(ierr, cctkGH, lower, upper, direction, coordname, systemname</code>
	<code>)</code>
	<code>integer ierr</code>
	<code>CCTK_POINTER cctkGH</code>
	<code>CCTK_REAL lower</code>
	<code>CCTK_REAL upper</code>
	<code>integer direction</code>
	<code>character*(*) coordname</code>
	<code>character*(*) systemname</code>

Parameters

<code>ierr</code>	- Error code
<code>cctkGH</code>	- pointer to CCTK grid hierarchy
<code>lower</code>	- Global lower bound of the coordinate (POINTER in C)
<code>upper</code>	- Global upper bound of the coordinate (POINTER in C)
<code>direction</code>	- Direction of coordinate in coordinate system
<code>coordname</code>	- Coordinate name
<code>systemname</code>	- Coordinate system name

Discussion

The coordinate name is independent of the grid function name. The coordinate range is registered by `CCTK_CoordRegisterRange`. To find the range, the coordinate system name must be given, and either the coordinate direction or the coordinate name. The coordinate direction will be used if is given as a positive value, otherwise the coordinate name will be used.

Examples

C	<code>ierr = CCTK_CoordRange(cctkGH, &xmin, &xmax, -1, "xdir",</code> <code>"mysystem");</code>
Fortran	<code>call CCTK_COORDRANGE(ierr, cctkGH, Rmin, Rmax, -1,</code> <code>"radius", "sphersystem")</code>

Errors

CCTK_CoordRegisterData

Define a coordinate in a given coordinate system.

Synopsis

C	<code>int ierr = CCTK_CoordRegisterData(int direction, const char * gvname, const char * coordname</code>
Fortran	<code>call CCTK_CoordRegisterData(ierr , direction, gvname, coordname, systemname</code>
	<code>)</code>
	<code>integer ierr</code>
	<code>integer direction</code>
	<code>character*(*) gvname</code>
	<code>character*(*) coordname</code>
	<code>character*(*) systemname</code>

Parameters

<code>ierr</code>	- Error code
<code>direction</code>	- Direction of coordinate in coordinate system
<code>gvname</code>	- Name of grid variable associated with coordinate
<code>coordname</code>	- Name of this coordinate
<code>systemname</code>	- Name of this coordinate system

Discussion

There must already be a coordinate system registered, using `CCTK_CoordRegisterSystem`.

Examples

C	<code>ierr = CCTK_CoordRegisterData(1,"coordthorn:myx","x2d","cart2d");</code>
Fortran	<code>two = 2</code>
	<code>call CCTK_COORDREGISTERDATA(ierr,two,"coordthorn:mytheta","spher3d")</code>

Errors

CCTK_CoordRegisterRange

Assign the global maximum and minimum values of a coordinate on a given grid hierarchy

Synopsis

C	<code>int ierr = CCTK_CoordRegisterRange(const cGH * cctkGH, CCTK_REAL min, CCTK_REAL max,</code>
Fortran	<code>call CCTK_CoordRegisterRange(ierr, cctkGH, min, max, direction, coordname, systemname</code>
	<code>)</code>
	<code>integer ierr</code>
	<code>CCTK_POINTER cctkGH</code>
	<code>CCTK_REAL min</code>
	<code>CCTK_REAL max</code>
	<code>integer direction</code>
	<code>character*(*) coordname</code>
	<code>character*(*) systemname</code>

Parameters

<code>ierr</code>	- Error code
<code>dimension</code>	- Pointer to CCTK grid hierarchy
<code>min</code>	- Global minimum of coordinate
<code>max</code>	- Global maximum of coordinate
<code>direction</code>	- Direction of coordinate in coordinate system
<code>coordname</code>	- Name of coordinate in coordinate system
<code>systemname</code>	- Name of this coordinate system

Discussion

There must already be a coordinate registered with the given name, with `CCTK_CoordRegisterData`. The coordinate range can be accessed by `CCTK_CoordRange`.

Examples

C	<code>ierr = CCTK_CoordRegisterRange(cctkGH,-1.0,1.0,1,"x2d","cart2d");</code>
Fortran	<code>min = 0</code>
	<code>max = 3.1415d0/2.0d0</code>
	<code>two = 2</code>
	<code>call CCTK_COORDREGISTERRANGE(ierr,min,max,two,"coordthorn::mytheta","spher3d")</code>

Errors

CCTK_CoordRegisterSystem

Assigns a coordinate system with a chosen name and dimension

Synopsis

C	<code>int ierr = CCTK_CoordRegisterSystem(int dimension, const char * systemname)</code>
Fortran	<code>call CCTK_CoordRegisterSystem(ierr , dimension, systemname)</code>
	<code>integer ierr</code>
	<code>integer dimension</code>
	<code>character*(*) systemname</code>

Parameters

<code>ierr</code>	-	Error code
<code>dimension</code>	-	Dimension of coordinate system
<code>systemname</code>	-	Unique name assigned to coordinate system

Discussion**Examples**

C	<code>ierr = CCTK_CoordRegisterSystem(3,"cart3d");</code>
Fortran	<code>three = 3</code>
	<code>call CCTK_COORDREGISTERSYSTEM(ierr,three,"sphersystem")</code>

Errors

CCTK_CoordSystemDim

Give the dimension for a given coordinate system.

Synopsis

C	<code>int dim = CCTK_CoordSystemDim(const char * systemname)</code>
Fortran	<code>call CCTK_CoordSystemDim(dim , systemname)</code> <code>integer dim</code> <code>character*(*) systemname</code>

Parameters

<code>dim</code>	-	The dimension of the coordinate system
<code>systemname</code>	-	The name of the coordinate system

Discussion**Examples**

C	<code>dim = CCTK_CoordSystemDim("cart3d");</code>
Fortran	<code>call CCTK_COORDSYSTEMDIM(dim,"spher3d")</code>

Errors

CCTK_CoordSystemHandle

Returns the handle associated with a registered coordinate system

Synopsis

C	<code>int handle = CCTK_CoordSystemHandle(const char * systemname)</code>
Fortran	<code>call CCTK_CoordSystemHandle(handle , systemname)</code> <code>integer handle</code> <code>character*(*) systemname</code>

Parameters

<code>handle</code>	-	The coordinate system handle
<code>systemname</code>	-	Name of the coordinate system

Discussion**Examples**

C	<code>handle = CCTK_CoordSystemHandle("my coordinate system");</code>
Fortran	<code>call CCTK_CoordSystemHandle(handle,"my coordinate system")</code>

Errors

A negative return code indicates an invalid coordinate system name.

CCTK_CoordSystemName

Returns the name of a registered coordinate system

Synopsis

```
C      const char * systemname = CCTK_CoordSystemName( int handle)
```

Parameters

```
      handle  -   The coordinate system handle  
  
      systemname  -   The coordinate system name
```

Discussion

No Fortran routine exists at the moment.

Examples

```
C      systemname = CCTK_CoordSystemName(handle);  
      handle = CCTK_CoordSystemHandle(systemname);
```

Errors

A NULL pointer is returned if an invalid handle was given.

CCTK_CreateDirectory

Create a directory with required permissions

Synopsis

C	<code>int ierr = CCTK_CreateDirectory(const char * pathname, int mode)</code>
Fortran	<code>call CCTK_CreateDirectory(ierr , pathname, mode)</code>
	integer ierr
	character*(*) pathname
	integer mode

Parameters

<code>ierr</code>	-	Error code
<code>pathname</code>	-	Directory to create
<code>mode</code>	-	Permission mode for new directory as an octal number

Discussion

To create a directory readable by everyone, but writeable only by the user running the code, the permission mode would be 0755.

Examples

C	<code>ierr = CCTK_CreateDirectory("Results/New",0755) ;</code>
Fortran	<code>call CCTK_CREATEDIRECTORY(ierr,"Results/New",0755)</code>

Errors

- 0 Directory successfully created
- 1 Memory allocation failed
- 2 Failed to create directory
- 3 pathname exists but is not a directory

CCTK_DecomposeName

Given the full name of a variable/group, separates the name returning both the implementation and the variable/group

Synopsis

```
C      int istat = CCTK_DecomposeName( const char * fullname, char ** imp, char ** name)
```

Parameters

`istat` - Status flag returned by routine

`fullname` - The full name of the group/variable

`imp` - The implementation name

`name` - The group/variable name

Discussion

The implementation name and the group/variable name must be explicitly freed after they have been used.

No Fortran routine exists at the moment.

Examples

```
C      istat = CCTK_DecomposeName("evolve::scalars",imp,name)
```

Errors

CCTK_DisableGroupComm

Turn communications off for a group of grid variables

Synopsis

```
C      int istat = CCTK_DisableGroupComm( cGH * cctkGH, const char * group)
```

Parameters

cctkGH - pointer to CCTK grid hierarchy

Discussion

Turning off communications means that ghost zones will not be communicated during a call to CCTK_SyncGroup. By default communications are all off.

Examples**Errors**

CCTK_DisableGroupStorage

Free the storage associated with a group of grid variables

Synopsis

```
C      int istat = CCTK_DisableGroupStorage( cGH * cctkGH, const char * group)
```

Parameters

cctkGH - pointer to CCTK grid hierarchy

Discussion**Examples****Errors**

CCTK_EnableGroupComm

Turn communications on for a group of grid variables

Synopsis

```
C      int istat = CCTK_EnableGroupComm( cGH * cctkGH, const char * group)
```

Parameters

`cctkGH` - pointer to CCTK grid hierarchy

Discussion

Grid variables with communication enabled will have their ghost zones communicated during a call to `CCTK_SyncGroup`. In general, this function does not need to be used, since communication is automatically enabled for grid variables who have assigned storage via the `schedule.ccl` file.

Examples**Errors**

CCTK_EnableGroupStorage

Assign the storage for a group of grid variables

Synopsis

```
C      int istat = CCTK_EnableGroupStorage( cGH * cctkGH, const char * group)
```

Parameters

`cctkGH` - pointer to CCTK grid hierarchy

Discussion

In general this function does not need to be used, since storage assignment is best handled by the Cactus scheduler via a thorn's `schedule.ccl` file.

Examples**Errors**

CCTK_Equals

Checks a STRING or KEYWORD parameter for equality with a given string

Synopsis

C	int istat = CCTK_Equals(const char * param, const char * value)
Fortran	istat = CCTK_Equals(param, value)
	integer istat
	CCTK_POINTER param
	character*(*) value

Parameters

istat	-	returns success or failure of equality
param	-	the STRING or KEYWORD parameter to check
value	-	the string value to compare against

Discussion

This function compares a Cactus parameter of type STRING or KEYWORD against a given string value. The comparison is performed case-independent, returning a non-zero value if the strings are the same, and zero if they differ.

Note that in Fortran code, STRING or KEYWORD parameters are passed as C pointers, and can not be treated as normal Fortran strings. Thus `CCTK_Equals()` should be used to check the value of such a parameter.

CCTK_Exit

Exit the code cleanly

Synopsis

C	int istat = CCTK_Exit(cGH * cctkGH, int value)
Fortran	call CCTK_Exit(istat , cctkGH, value)
	integer istat
	CCTK_POINTER cctkGH
	integer value

Parameters

cctkGH - pointer to CCTK grid hierarchy

value - the return code to abort with

Discussion

This routine causes an immediate, regular termination of Cactus. It never returns to the caller.

CCTK_FirstVarIndex

Given a group name returns the first variable index in the group

Synopsis

C	int firstvar = CCTK_FirstVarIndex(const char * group)
Fortran	call CCTK_FirstVarIndex(firstvar , group) integer firstvar character*(*) group

Parameters

firstvar	-	The first variable index in the given group
group	-	The group name

Discussion**Examples**

C	firstvar = CCTK_FirstVarIndex("evolve::scalars") ;
Fortran	call CCTK_GroupIndex(index,"evolve::scalars")

Errors

CCTK_FirstVarIndexI

Given a group index returns the first variable index in the group

Synopsis

C	int firstvar = CCTK_FirstVarIndexI(int group)
Fortran	call CCTK_FirstVarIndexI(firstvar , group) integer firstvar integer group

Parameters

firstvar	- The first variable index in the given group
group	- The group index

Discussion**Examples**

C	index = CCTK_GroupIndex("evolve::scalars") firstvar = CCTK_FirstVarIndexI(index) ;
Fortran	call CCTK_GroupIndex(index,3)

Errors

CCTK_FortranString

Changes a C string into a Fortran string

Synopsis

C	int nchar = CCTK_FortranString(const char * strout, const char * strin)
Fortran	call CCTK_FortranString(nchar , strout, strin)
	integer nchar
	character*(*) strout
	CCTK_STRING strin

Parameters

- | | | |
|---------------|---|--|
| nchar | - | The number of characters in the C string, not counting the null terminator |
| strout | - | The fortran string which on output contains the C string as the first nchar characters |
| strin | - | The (pointer to the) C string containing the null terminator |

Discussion

String or keyword parameters in Cactus are passed into Fortran routines as pointers to C strings. This means that they cannot be directly used as Fortran strings. This routine allows a Fortran string to be created from such a C string. Note that the Fortran string must be defined to have at least the same expected length as the C string. This routine is only callable from Fortran.

Examples

Fortran

Errors

CCTK_FullName

Given a variable index, returns the full name of the variable

Synopsis

```
C      char * fullname = CCTK_FullName( int index)
```

Parameters

`implementation` - The full variable name

`index` - The variable index

Discussion

The full variable name must be explicitly freed after it has been used.

No Fortran routine exists at the moment. The full variable name is in the form
<implementation>::<variable>

Examples

```
C      index = CCTK_VarIndex("evolve::phi");  
      name = CCTK_FullName(index);  
      printf ("Variable name:  %s", name);  
      free (name);
```

Errors

CCTK_GHExtension

Get the pointer to a registered extension to the Cactus GH structure

Synopsis

```
C void * extension = CCTK_GHExtension( const GH * cctkGH, const char * name)
```

Parameters

extension - The pointer to the GH extension

`cctkGH` - The pointer to the CCTK grid hierarchy

name - The name of the GH extension

Discussion

No Fortran routine exists at the moment.

Examples

```
C    void *extension = CCTK_GHExtension(GH, "myExtension");
```

Errors

NULL A NULL pointer is returned if an invalid extension name was given.

CCTK_GHExtensionHandle

Get the handle associated with a extension to the Cactus GH structure

Synopsis

C	<code>int handle = CCTK_GHExtensionHandle(const char * name)</code>
Fortran	<code>call CCTK_GHExtensionHandle(handle , name)</code> integer handle character*(*) name

Parameters

<code>handle</code>	-	The GH extension handle
<code>group</code>	-	The name of the GH extension

Discussion**Examples**

C	<code>handle = CCTK_GHExtension("myExtension") ;</code>
Fortran	<code>call CCTK_GHExtension(handle,"myExtension")</code>

Errors

CCTK_GroupbboxGI, CCTK_GroupbboxGN

Given a group index or name, return an array of the bounding box of the group for each face

Synopsis

```

C                #include "cctk.h"

                    int status = CCTK_GroupbboxGI(const cGH *cctkGH,
                                                    int dim,
                                                    int *bbox,
                                                    int groupindex);

                    int status = CCTK_GroupbboxGN(const cGH *cctkGH,
                                                    int dim,
                                                    int *bbox,
                                                    const char *groupname);

Fortran          call CCTK_GroupbboxGI(status, cctkGH, dim, bbox, groupindex)

                    call CCTK_GroupbboxGN(status, cctkGH, dim, bbox, groupname)

                    integer          status
                    CCTK_POINTER    cctkGH
                    integer          dim
                    integer          bbox(dim)
                    integer          groupindex
                    character(*)     groupname

```

Result

0	success
-1	incorrect dimension supplied
-2	data not available from driver
-3	called on a scalar group
-4	invalid group index

Parameters

status	Return value.
cctkGH ($\neq$ NULL)	Pointer to a valid Cactus grid hierarchy.
dim (≥ 1)	Number of dimensions of group.
bbox ($\neq$ NULL)	Pointer to array which will hold the return values.
groupindex	Group index.
groupname	Group's full name.

Discussion

The bounding box for a given group is returned in a user-supplied array buffer.

See Also

CCTK_GroupbboxVI, CCTK_GroupbboxVN

Returns the lower bounds for a given variable.

CCTK_GroupbboxVI, CCTK_GroupbboxVN

Given a variable index or name, return an array of the bounding box of the variable for each face

Synopsis

```

C                #include "cctk.h"

                    int status = CCTK_GroupbboxVI(const cGH *cctkGH,
                                                    int dim,
                                                    int *bbox,
                                                    int varindex);

                    int status = CCTK_GroupbboxVN(const cGH *cctkGH,
                                                    int dim,
                                                    int *bbox,
                                                    const char *varname);

Fortran          call CCTK_GroupbboxVI(status, cctkGH, dim, bbox, varindex)

                    call CCTK_GroupbboxVN(status, cctkGH, dim, bbox, varname)

                    integer          status
                    CCTK_POINTER    cctkGH
                    integer          dim
                    integer          bbox(dim)
                    integer          varindex
                    character(*)     varname

```

Result

0	success
-1	incorrect dimension supplied
-2	data not available from driver
-3	called on a scalar group
-4	invalid variable index

Parameters

status	Return value.
cctkGH ($\neq$ NULL)	Pointer to a valid Cactus grid hierarchy.
dim (≥ 1)	Number of dimensions of variable.
bbox ($\neq$ NULL)	Pointer to array which will hold the return values.
varindex	Group index.
varname	Group's full name.

Discussion

The bounding box for a given variable is returned in a user-supplied array buffer.

See Also

CCTK_GroupbboxGI, CCTK_GroupbboxGN

Returns the upper bounds for a given group.

CCTK_GroupData

Given a group index, returns information about the variables held in the group.

Synopsis

```
C      int ierr = CCTK_GroupData( int group, cGroup * pgroup)
```

Parameters

ierr - 0 for success, negative for failure

group - group index

pgroup - returns a pointer to a structure containing group information

Discussion

The cGroup structure contains the information

- **grouptype**: The group type
- **vartype**: The type of variables in the group
- **stagtype**: The type of grid staggering for arrays
- **dim**: The dimension of variables in the group
- **numvars**: The number of variables in the group
- **ntimelevels**: The number of timelevels for variables in the group

No Fortran routine exists at the moment.

Examples

```
C      cGroup pgroup;  
      index = CCTK_GroupIndex("evolve::scalars");  
      ierr = CCTK_GroupData(index,&pgroup);  
      vtype = pgroup.vartype;
```

Errors

CCTK_GroupDynamicData

Given a group index, returns information about the variables held in the group.

Synopsis

```
C      int ierr = CCTK_GroupDynamicData( const cGH * cctkGH, int group, cGroupDynamicData * p
```

Parameters

ierr - 0 for success, negative for failure

cctkGH - pointer to CCTK grid hierarchy

group - group index

pdata - returns a pointer to a structure containing group information

Discussion

The cGroupDynamicData structure contains the information

- **dim**: The dimension of variables in the group
- **gsh[dim]**: The global size of the group in each dimension
- **lsh[dim]**: The local size of the group in each dimension
- **lbnd[dim]**: The lower bounds of the group in each dimension
- **ubnd[dim]**: The upper bounds of the group in each dimension
- **bbox[2*dim]**: The bounding box of the group for each face
- **nghostzones[dim]**: The number of ghostzones in each dimension of the group

No Fortran routine exists at the moment.

Examples

```
C      cGroupDynamicData pdata;
      index = CCTK_GroupIndex("evolve::scalars");
      ierr = CCTK_GroupDynamicData(cctkGH,index,&pgroup);
      vdim = pdata.dim;
```

Errors

CCTK_GroupgshGI, CCTK_GroupgshGN

Given a group index or name, return an array of the global size of the group in each dimension

Synopsis

```

C                #include "cctk.h"

                    int status = CCTK_GroupgshGI(const cGH *cctkGH,
                                                    int dim,
                                                    int *gsh,
                                                    int groupindex);

                    int status = CCTK_GroupgshGN(const cGH *cctkGH,
                                                    int dim,
                                                    int *gsh,
                                                    const char *groupname);

Fortran          call CCTK_GroupgshGI(status, cctkGH, dim, gsh, groupindex)

                    call CCTK_GroupgshGN(status, cctkGH, dim, gsh, groupname)

                    integer          status
                    CCTK_POINTER    cctkGH
                    integer          dim
                    integer          gsh(dim)
                    integer          groupindex
                    character*(*)    groupname

```

Result

0	success
-1	incorrect dimension supplied
-2	data not available from driver
-3	called on a scalar group
-4	invalid group name

Parameters

cctkGH ($\neq$ NULL)	Pointer to a valid Cactus grid hierarchy.
dim (≥ 1)	Number of dimensions of group.
gsh ($\neq$ NULL)	Pointer to array which will hold the return values.
groupindex	Index of the group.
groupname	Name of the group.

Discussion

The global size in each dimension for a given group is returned in a user-supplied array buffer.

See Also

CCTK_GroupgshVI, CCTK_GroupgshVN	Returns the global size for a given variable.
CCTK_GroupplshGI, CCTK_GroupplshGN	Returns the local size for a given group.

CCTK_Group1shVI, CCTK_Group1shVN

Returns the local size for a given variable.

CCTK_GroupgshVI, CCTK_GroupgshVN

Given a variable index or its full name, return an array of the global size of the variable in each dimension

Synopsis

```

C                #include "cctk.h"

                    int status = CCTK_GroupgshVI(const cGH *cctkGH,
                                                    int dim,
                                                    int *gsh,
                                                    int varindex);

                    int status = CCTK_GroupgshVN(const cGH *cctkGH,
                                                    int dim,
                                                    int *gsh,
                                                    const char *varname);

Fortran          call CCTK_GroupgshVI(status, cctkGH, dim, gsh, varindex)

                    call CCTK_GroupgshVN(status, cctkGH, dim, gsh, varname)

                    integer          status
                    CCTK_POINTER     cctkGH
                    integer          dim
                    integer          gsh(dim)
                    integer          varindex
                    character*(*)    varname

```

Result

0	success
-1	incorrect dimension supplied
-2	data not available from driver
-3	called on a scalar group
-4	invalid variable index

Parameters

status	Return value.
cctkGH ($\neq$ NULL)	Pointer to a valid Cactus grid hierarchy.
dim (≥ 1)	Number of dimensions of variable.
gsh ($\neq$ NULL)	Pointer to array which will hold the return values.
varindex	Variable index.
varname	Variable's full name.

Discussion

The global size in each dimension for a given variable is returned in a user-supplied array buffer.

See Also

CCTK_GroupgshGI, CCTK_GroupgshGN

Returns the global size for a given group.

CCTK_Group1shGI, CCTK_Group1shGN

Returns the local size for a given group.

CCTK_Group1shVI, CCTK_Group1shVN

Returns the local size for a given variable.

CCTK_GroupIndex

Get the index number for a group name

Synopsis

C	int index = CCTK_GroupIndex(const char * groupname)
Fortran	call CCTK_GroupIndex(index , groupname)
	integer index
	character*(*) groupname

Parameters

groupname - The name of the group

Discussion

The group name should be the given in its fully qualified form, that is <implementation>::<group> for a public or protected group, and <thornname>::<group> for a private group.

Examples

C	index = CCTK_GroupIndex("evolve::scalars") ;
Fortran	call CCTK_GroupIndex(index,"evolve::scalars")

Errors

CCTK_GroupIndexFromVar

Given a variable name, returns the index of the associated group

Synopsis

C	int groupindex = CCTK_GroupIndexFromVar(const char * name)
Fortran	call CCTK_GroupIndexFromVar(groupindex , name) integer groupindex character*(*) name

Parameters

groupindex	- The index of the group
name	- The full name of the variable

Discussion

The variable name should be in the form <implementation>::<variable>

Examples

C	groupindex = CCTK_GroupIndexFromVar("evolve::phi") ;
Fortran	call CCTK_GROUPINDEXFROMVAR(groupindex,"evolve::phi")

Errors

CCTK_GroupIndexFromVarI

Given a variable index, returns the index of the associated group

Synopsis

C	<code>int groupindex = CCTK_GroupIndexFromVarI(int varindex)</code>
Fortran	<code>call CCTK_GroupIndexFromVarI(groupindex , varindex)</code> integer groupindex integer varindex

Parameters

<code>groupindex</code>	- The index of the group
<code>varindex</code>	- The index of the variable

Discussion**Examples**

C	<code>index = CCTK_VarIndex("evolve::phi");</code> <code>groupindex = CCTK_GroupIndexFromVarI(index) ;</code>
Fortran	<code>call CCTK_VARINDEX("evolve::phi")</code> <code>call CCTK_GROUPINDEXFROMVARI(groupindex,index)</code>

Errors

CCTK_Group1bndGI, CCTK_Group1bndGN

Given a group index or name, return an array of the lower bounds of the group in each dimension

Synopsis

```

C                #include "cctk.h"

                    int status = CCTK_Group1bndGI(const cGH *cctkGH,
                                                    int dim,
                                                    int *lbnd,
                                                    int groupindex);

                    int status = CCTK_Group1bndGN(const cGH *cctkGH,
                                                    int dim,
                                                    int *lbnd,
                                                    const char *groupname);

Fortran          call CCTK_Group1bndGI(status, cctkGH, dim, lbnd, groupindex)

                    call CCTK_Group1bndGN(status, cctkGH, dim, lbnd, groupname)

                    integer          status
                    CCTK_POINTER    cctkGH
                    integer          dim
                    integer          lbnd(dim)
                    integer          groupindex
                    character*(*)    groupname

```

Result

0	success
-1	incorrect dimension supplied
-2	data not available from driver
-3	called on a scalar group
-4	invalid group index

Parameters

status	Return value.
cctkGH ($\neq$ NULL)	Pointer to a valid Cactus grid hierarchy.
dim (≥ 1)	Number of dimensions of group.
lbnd ($\neq$ NULL)	Pointer to array which will hold the return values.
groupindex	Group index.
groupname	Group's full name.

Discussion

The lower bounds in each dimension for a given group is returned in a user-supplied array buffer.

See Also

CCTK_Group1bndVI, CCTK_Group1bndVN

Returns the lower bounds for a given variable.

CCTK_GroupubndGI, CCTK_GroupubndGN

Returns the upper bounds for a given group.

CCTK_GroupubndVI, CCTK_GroupubndVN

Returns the upper bounds for a given variable.

CCTK_Group1bndVI, CCTK_Group1bndVN

Given a variable index or name, return an array of the lower bounds of the variable in each dimension

Synopsis

```

C                #include "cctk.h"

                    int status = CCTK_Group1bndVI(const cGH *cctkGH,
                                                    int dim,
                                                    int *lbnd,
                                                    int varindex);

                    int status = CCTK_Group1bndVN(const cGH *cctkGH,
                                                    int dim,
                                                    int *lbnd,
                                                    const char *varname);

Fortran          call CCTK_Group1bndVI(status, cctkGH, dim, lbnd, varindex)

                    call CCTK_Group1bndVN(status, cctkGH, dim, lbnd, varname)

                    integer          status
                    CCTK_POINTER    cctkGH
                    integer          dim
                    integer          lbnd(dim)
                    integer          varindex
                    character(*)     varname

```

Result

0	success
-1	incorrect dimension supplied
-2	data not available from driver
-3	called on a scalar group
-4	invalid variable index

Parameters

status	Return value.
cctkGH ($\neq$ NULL)	Pointer to a valid Cactus grid hierarchy.
dim (≥ 1)	Number of dimensions of variable.
lbnd ($\neq$ NULL)	Pointer to array which will hold the return values.
varindex	Group index.
varname	Group's full name.

Discussion

The lower bounds in each dimension for a given variable is returned in a user-supplied array buffer.

See Also

CCTK_Group1bndGI, CCTK_Group1bndGN

Returns the lower bounds for a given group.

CCTK_GroupubndGI, CCTK_GroupubndGN

Returns the upper bounds for a given group.

CCTK_GroupubndVI, CCTK_GroupubndVN

Returns the upper bounds for a given variable.

CCTK_Group1shGI, CCTK_Group1shGN

Given a group index or name, return an array of the local size of the group in each dimension

Synopsis

```

C                #include "cctk.h"

                    int status = CCTK_Group1shGI(const cGH *cctkGH,
                                                    int dim,
                                                    int *lsh,
                                                    int groupindex);

                    int status = CCTK_Group1shGN(const cGH *cctkGH,
                                                    int dim,
                                                    int *lsh,
                                                    const char *groupname);

Fortran          call CCTK_Group1shGI(status, cctkGH, dim, lsh, groupindex)

                    call CCTK_Group1shGN(status, cctkGH, dim, lsh, groupname)

                    integer          status
                    CCTK_POINTER     cctkGH
                    integer          dim
                    integer          lsh(dim)
                    integer          groupindex
                    character*(*)    groupname

```

Result

0	success
-1	incorrect dimension supplied
-2	data not available from driver
-3	called on a scalar group
-4	invalid group name

Parameters

cctkGH ($\neq$ NULL)	Pointer to a valid Cactus grid hierarchy.
dim (≥ 1)	Number of dimensions of group.
lsh ($\neq$ NULL)	Pointer to array which will hold the return values.
groupindex	Index of the group.
groupname	Name of the group.

Discussion

The local size in each dimension for a given group is returned in a user-supplied array buffer.

See Also

CCTK_GroupgshVI, CCTK_GroupgshVN	Returns the global size for a given variable.
CCTK_Group1shGI, CCTK_Group1shGN	Returns the local size for a given group.

CCTK_Group1shVI, CCTK_Group1shVN

Returns the local size for a given variable.

CCTK_Group1shVI, CCTK_Group1shVN

Given a variable index or its full name, return an array of the local size of the variable in each dimension

Synopsis

```

C                #include "cctk.h"

                    int status = CCTK_Group1shVI(const cGH *cctkGH,
                                                    int dim,
                                                    int *lsh,
                                                    int varindex);

                    int status = CCTK_Group1shVN(const cGH *cctkGH,
                                                    int dim,
                                                    int *lsh,
                                                    const char *varname);

Fortran          call CCTK_Group1shVI(status, cctkGH, dim, lsh, varindex)

                    call CCTK_Group1shVN(status, cctkGH, dim, lsh, varname)

                    integer          status
                    CCTK_POINTER    cctkGH
                    integer          dim
                    integer          lsh(dim)
                    integer          varindex
                    character(*)    varname

```

Result

0	success
-1	incorrect dimension supplied
-2	data not available from driver
-3	called on a scalar group
-4	invalid variable index

Parameters

status	Return value.
cctkGH ($\neq$ NULL)	Pointer to a valid Cactus grid hierarchy.
dim (≥ 1)	Number of dimensions of variable.
lsh ($\neq$ NULL)	Pointer to array which will hold the return values.
varindex	Variable index.
varname	Variable's full name.

Discussion

The local size in each dimension for a given variable is returned in a user-supplied array buffer.

See Also

CCTK_GroupgshGI, CCTK_GroupgshGN

Returns the global size for a given group.

CCTK_GroupgshVI, CCTK_GroupgshVN

Returns the global size for a given variable.

CCTK_GrouplshGI, CCTK_GrouplshGN

Returns the local size for a given group.

CCTK_GroupName

Given a group index, returns the group name

Synopsis

```
C      char * name = CCTK_GroupName( int index)
```

Parameters

`name` - The group name
`index` - The group index

Discussion

The group name must be explicitly freed after it has been used.

No Fortran routine exists at the moment.

Examples

```
C      index = CCTK_GroupIndex("evolve::scalars");  
      name = CCTK_GroupName(index);  
      printf ("Group name:  %s", name);  
      free (name);
```

Errors

CCTK_GroupNameFromVarI

Given a variable index, return the name of the associated group

Synopsis

```
C      char * group = CCTK_GroupNameFromVarI( int varindex)
```

Parameters

```
      group  -   The name of the group  
      varindex - The index of the variable
```

Discussion**Examples**

```
C      index = CCTK_VarIndex("evolve::phi");  
      group = CCTK_GroupNameFromVarI(index) ;
```

Errors

CCTK_GroupnghostzonesGI, CCTK_GroupnghostzonesGN

Given a group index or name, return an array with the number of ghostzones in each dimension of the group

Synopsis

```

C                #include "cctk.h"

                    int status = CCTK_GroupnghostzonesGI(const cGH *cctkGH,
                                                            int dim,
                                                            int *nghostzones,
                                                            int groupindex)

                    int status = CCTK_GroupnghostzonesGN(const cGH *cctkGH,
                                                            int dim,
                                                            int *nghostzones,
                                                            const char *groupname)

Fortran          call CCTK_GroupnghostzonesGI(status, cctkGH, dim, nghostzones, groupindex)

                    call CCTK_GroupnghostzonesGN(status, cctkGH, dim, nghostzones, groupname)

                    integer      status
                    CCTK_POINTER cctkGH
                    integer      dim
                    integer      nghostzones(dim)
                    integer      groupindex
                    character*(*) groupname

```

Result

0	success
-1	incorrect dimension supplied
-2	data not available from driver
-3	called on a scalar group

Parameters

status	Return value.
cctkGH ($\neq$ NULL)	Pointer to a valid Cactus grid hierarchy.
dim (≥ 1)	Number of dimensions of group.
nghostzones ($\neq$ NULL)	Pointer to array which will hold the return values.
groupindex	Group index.
groupname	Group name.

Discussion

The number of ghostzones in each dimension for a given group is returned in a user-supplied array buffer.

See Also

CCTK_GroupnghostzonesVI, CCTK_GroupnghostzonesVN

Returns the number of ghostzones for a given variable.

CCTK_GroupnghostzonesVI, CCTK_GroupnghostzonesVN

Given a variable index or its full name, return an array with the number of ghostzones in each dimension of the variable

Synopsis

C

```
#include "cctk.h"

int status = CCTK_GroupnghostzonesVI(const cGH *cctkGH,
                                     int dim,
                                     int *nghostzones,
                                     int varindex)

int status = CCTK_GroupnghostzonesVN(const cGH *cctkGH,
                                     int dim,
                                     int *nghostzones,
                                     const char *varname)
```

Fortran

```
call CCTK_GroupnghostzonesVI(status, cctkGH, dim, nghostzones, varindex)

call CCTK_GroupnghostzonesVN(status, cctkGH, dim, nghostzones, varname)

integer      status
CCTK_POINTER cctkGH
integer      dim
integer      nghostzones(dim)
integer      varindex
character(*) varname
```

Result

0	success
-1	incorrect dimension supplied
-2	data not available from driver
-3	called on a scalar group

Parameters

status	Return value.
cctkGH ($\neq$ NULL)	Pointer to a valid Cactus grid hierarchy.
dim (≥ 1)	Number of dimensions of group.
nghostzones ($\neq$ NULL)	Pointer to array which will hold the return values.
varindex	Variable index.
varname	Variable's full name.

Discussion

The number of ghostzones in each dimension for a given variable is returned in a user-supplied array buffer.

See Also

CCTK_GroupnghostzonesGI, CCTK_GroupnghostzonesGN

Returns the number of ghostzones for a given group.

CCTK_GroupTypeFromVarI

Provides a group's group type index given a variable index

Synopsis

C	<code>int type = CCTK_GroupTypeFromVarI(int index)</code>
Fortran	<code>call CCTK_GroupTypeFromVarI(type , index)</code> integer type integer index

Parameters

<code>type</code>	- The group's group type index
<code>group</code>	- The variable index

Discussion

The group's group type index indicates the type of variables in the group. Either scalars, grid functions or arrays. The group type can be checked with the Cactus provided macros for `CCTK_SCALAR`, `CCTK_GF`, `CCTK_ARRAY`.

Examples

C	<code>index = CCTK_GroupIndex("evolve::scalars")</code> <code>array = (CCTK_ARRAY == CCTK_GroupTypeFromVarI(index)) ;</code>
Fortran	<code>call CCTK_GROUPTYPEFROMVARI(type,3)</code>

Errors

CCTK_GroupTypeI

Provides a group's group type index given a group index

Synopsis

```
C          #include "cctk.h"
          int group_type = CCTK_GroupTypeI(int group);
```

Result

-1 -1 is returned if the given group index is invalid.

Parameters

group Group index.

Discussion

A group's group type index indicates the type of variables in the group. The three group types are scalars, grid functions, and grid arrays. The group type can be checked with the Cactus provided macros for `CCTK_SCALAR`, `CCTK_GF`, `CCTK_ARRAY`.

See Also

`CCTK_GroupTypeFromVarI` This function takes a variable index rather than a group index as its argument.

CCTK_GroupubndGI, CCTK_GroupubndGN

Given a group index or name, return an array of the upper bounds of the group in each dimension

Synopsis

```

C                #include "cctk.h"

                    int status = CCTK_GroupubndGI(const cGH *cctkGH,
                                                    int dim,
                                                    int *ubnd,
                                                    int groupindex);

                    int status = CCTK_GroupubndGN(const cGH *cctkGH,
                                                    int dim,
                                                    int *ubnd,
                                                    const char *groupname);

Fortran          call CCTK_GroupubndGI(status, cctkGH, dim, ubnd, groupindex)

                    call CCTK_GroupubndGN(status, cctkGH, dim, ubnd, groupname)

                    integer          status
                    CCTK_POINTER    cctkGH
                    integer          dim
                    integer          ubnd(dim)
                    integer          groupindex
                    character*(*)    groupname

```

Result

0	success
-1	incorrect dimension supplied
-2	data not available from driver
-3	called on a scalar group
-4	invalid group index

Parameters

status	Return value.
cctkGH ($\neq$ NULL)	Pointer to a valid Cactus grid hierarchy.
dim (≥ 1)	Number of dimensions of group.
ubnd ($\neq$ NULL)	Pointer to array which will hold the return values.
groupindex	Group index.
groupname	Group's full name.

Discussion

The upper bounds in each dimension for a given group is returned in a user-supplied array buffer.

See Also

CCTK_GrouplbndGI, CCTK_GrouplbndGN

Returns the lower bounds for a given group.

CCTK_Group1bndVI, CCTK_Group1bndVN

Returns the lower bounds for a given variable.

CCTK_GroupubndVI, CCTK_GroupubndVN

Returns the upper bounds for a given variable.

CCTK_GroupubndVI, CCTK_GroupubndVN

Given a variable index or name, return an array of the upper bounds of the variable in each dimension

Synopsis

```

C                #include "cctk.h"

                    int status = CCTK_GroupubndVI(const cGH *cctkGH,
                                                    int dim,
                                                    int *ubnd,
                                                    int varindex);

                    int status = CCTK_GroupubndVN(const cGH *cctkGH,
                                                    int dim,
                                                    int *ubnd,
                                                    const char *varname);

Fortran          call CCTK_GroupubndVI(status, cctkGH, dim, ubnd, varindex)

                    call CCTK_GroupubndVN(status, cctkGH, dim, ubnd, varname)

                    integer      status
                    CCTK_POINTER cctkGH
                    integer      dim
                    integer      ubnd(dim)
                    integer      varindex
                    character(*) varname

```

Result

0	success
-1	incorrect dimension supplied
-2	data not available from driver
-3	called on a scalar group
-4	invalid variable index

Parameters

status	Return value.
cctkGH ($\neq$ NULL)	Pointer to a valid Cactus grid hierarchy.
dim (≥ 1)	Number of dimensions of variable.
ubnd ($\neq$ NULL)	Pointer to array which will hold the return values.
varindex	Group index.
varname	Group's full name.

Discussion

The upper bounds in each dimension for a given variable is returned in a user-supplied array buffer.

See Also

CCTK_GrouplbndGI, CCTK_GrouplbndGN

Returns the lower bounds for a given group.

CCTK_Group1bndVI, CCTK_Group1bndVN

Returns the lower bounds for a given variable.

CCTK_GroupubndGI, CCTK_GroupubndGN

Returns the upper bounds for a given group.

CCTK_ImpFromVarI

Given a variable index, returns the implementation name

Synopsis

```
C      char * implementation = CCTK_ImpFromVarI( int index)
```

Parameters

```
implementation  -  The implementation name  
index          -  The variable index
```

Discussion

No Fortran routine exists at the moment

Examples

```
C      index = CCTK_VarIndex("evolve::phi");  
      implementation = CCTK_ImpFromVarI(index);
```

Errors

CCTK_INFO

Macro to print a single string as an information message to screen

Synopsis

```

C          #include "cctk.h"
              #include "cctk_WarnLevel.h"

              CCTK_INFO(const char *message);

Fortran    #include "cctk.h"

              call CCTK_INFO(message)
              character*(*) message
  
```

Parameters

message The string to print as an info message

Discussion

This macro can be used by thorns to print a single string as an info message to screen. `CCTK_INFO(level, message)` expands to a call to the underlying function `CCTK_Info()`: `CCTK_Info(CCTK_THORNSTRING, message)`. So the macro automatically includes the name of the originating thorn in the info message. It is recommended that the macro `CCTK_INFO` is used to print a message rather than calling `CCTK_Info()` directly. To include variables in an info message from C, you can use the routine `CCTK_VInfo()` which accepts a variable argument list. To include variables from Fortran, a string must be constructed and passed in a `CCTK_INFO` macro.

See Also

`CCTK_VInfo()` prints a formatted string with a variable argument list as an info message to screen

Examples

```

C          #include "cctk.h"
              #include "cctk_WarningLevel.h"

              CCTK_INFO("Output is disabled");

Fortran    #include "cctk.h"

              integer      myint
              real         myreal
              character*200 message

              write(message, '(A32, G12.7, A5, I8)')
              &      'Your info message, including ', myreal, ' and ', myint
              call CCTK_INFO(message)
  
```


CCTK_InterpGridArrays

Interpolate a list of distributed grid arrays

(This function will eventually replace `CCTK_InterpGV()`; see the Cactus web pages “Development” section for further details.)

The computation is optimized for the case of interpolating a number of grid arrays at a time; in this case all the interprocessor communication can be done together, and the same interpolation coefficients can be used for all the arrays.

Synopsis

```

C          #include "cctk.h"

              int status =
                  CCTK_InterpGridArrays(const cGH *cctkGH,
                                          int N_dims,
                                          int local_interp_handle, int param_table_handle,
                                          int coord_system_handle,
                                          int N_interp_points,
                                          const int interp_coords_type_code,
                                          const void *const interp_coords[],
                                          int N_input_arrays,
                                          const CCTK_INT input_array_variable_indices[],
                                          int N_output_arrays,
                                          const CCTK_INT output_array_type_codes[],
                                          void *const output_arrays[]);

Fortran    call CCTK_InterpGridArrays(status,
              .          cctkGH,
              .          N_dims,
              .          local_interp_handle, param_table_handle,
              .          coord_system_handle,
              .          N_interp_points,
              .          interp_coords_type_code, interp_coords,
              .          N_input_arrays, input_array_variable_indices,
              .          N_output_arrays, output_array_type_codes,
              .          output_arrays)
              integer      status
              CCTK_POINTER cctkGH
              integer      local_interp_handle, param_table_handle, coord_system_handle
              integer      N_dims, N_interp_points, N_input_arrays, N_output_arrays
              CCTK_POINTER interp_coords(N_dims)
              integer      interp_coords_type_code
              CCTK_INT      input_array_variable_indices(N_input_arrays)
              CCTK_INT      output_array_type_codes(N_output_arrays)
              CCTK_POINTER output_arrays(N_output_arrays)

```

Result

0 success
 < 0 indicates an error condition (see **Errors**)

Parameters

cctkGH ($\neq$ NULL)
 Pointer to a valid Cactus grid hierarchy.

- N_dims** (≥ 1) Number of dimensions in which to interpolate. This must be $\leq$ the dimensionality of the coordinate system defined by `coord_system_handle`. The default case is that it's $=$; see the discussion of the `interpolation_hyperslab_handle` parameter-table entry for the $<$ case.
- local_interp_handle** (≥ 0) Handle to the local interpolation operator as returned by `CCTK_InterpHandle()`.
- param_table_handle** (≥ 0) Handle to a key-value table containing zero or more additional parameters for the interpolation operation. The table is allowed to be modified by the local and/or global interpolation routine(s).
- coord_system_handle** (≥ 0) Cactus coordinate system handle defining the mapping between (usually floating-point) coordinates and integer grid subscripts, as returned by `CCTK_CoordSystemHandle()`.
- N_interp_points** (≥ 0) The number of interpolation points requested by this processor.
- interp_coords_type_code** One of the `CCTK_VARIABLE_*` type codes, giving the data type of the interpolation-point coordinate arrays pointed to by `interp_coords[]`. All interpolation-point coordinate arrays must be of the same data type.
- interp_coords** ($\neq \text{NULL}$) (Pointer to) an array of **N_dims** pointers to 1-D arrays giving the coordinates of the interpolation points requested by this processor. These coordinates are with respect to the coordinate system defined by `coord_system_handle`.
- N_input_arrays** (≥ 0) The number of input arrays to be interpolated. If **N_input_arrays** is zero then no interpolation is done; such a call may be useful for setup, interpolator querying, etc. Note that if the parameter table entry `operand_indices` is used to specify a nontrivial (eg. 1-to-many) mapping of input arrays to output arrays, only the unique set of input arrays should be given here.
- input_array_variable_indices** ($\neq \text{NULL}$) (Pointer to) an array of **N_input_arrays** CCTK grid array indices (as returned by `CCTK_VarIndex()`) specifying the input grid arrays for the interpolation. For any element with an index value of -1 in the grid array indices array, that interpolation is skipped. This may be useful if the main purpose of the call is eg. to do some query or setup computation.
- N_output_arrays** (≥ 0) The number of output arrays to be returned from the interpolation. If **N_output_arrays** is zero then no interpolation is done; such a call may be useful for setup, interpolator querying, etc. Note that **N_output_arrays** may differ from **N_input_arrays**, eg. if the `operand_indices` parameter-table entry is used to specify a nontrivial (eg. many-to-1) mapping of input arrays to output arrays. If such a mapping is specified, only the unique set of output arrays should be given in the `output_arrays` argument.
- output_array_type_codes** ($\neq \text{NULL}$) (Pointer to) an array of **N_output_arrays** `CCTK_VARIABLE_*` type codes giving the data types of the 1-D output arrays pointed to by `output_arrays[]`.
- output_arrays** ($\neq \text{NULL}$) (Pointer to) an array of **N_output_arrays** pointers to the (user-supplied) 1-D output arrays for the interpolation. If any of the pointers in the `output_arrays` array is `NULL`, then that interpolation is skipped. This may be useful if the main purpose of the call is eg. to do some query or setup computation.

Discussion

This function interpolates a list of CCTK grid arrays (in a multiprocessor run these are generally distributed over processors) on a list of interpolation points. The grid

topology and coordinates are implicitly specified via a Cactus coordinate system. The interpolation points may be anywhere in the global Cactus grid. In a multiprocessor run they may vary from processor to processor; each processor will get whatever interpolated data it asks for. The routine `CCTK_InterpGridArrays()` does not do the actual interpolation itself but rather takes care of whatever interprocessor communication may be necessary, and – for each processor’s local patch of the domain-decomposed grid arrays – calls `CCTK_InterpLocalUniform()` to invoke an external local interpolation operator (as identified by an interpolation handle).

Additional parameters for the interpolation operation of both `CCTK_InterpGridArrays()` and `CCTK_InterpLocalUniform()` can be passed in via a handle to a key/value options table. All interpolation operators should check for a parameter table entry with the key `"suppress_warnings"` which – if present – indicates that the caller wants the interpolator to be silent in case of an error condition and only return an appropriate error code. One common parameter-table option, which a number of interpolation operators are likely to support, is `order`, a `CCTK_INT` specifying the order of the (presumably polynomial) interpolation (1=linear, 2=quadratic, 3=cubic, etc). As another example, a table might be used to specify that the local interpolator should take derivatives, by specifying

```
const CCTK_INT operand_indices[N_output_arrays];
const CCTK_INT operation_codes[N_output_arrays];
```

Also, the global interpolator will typically need to specify some options of its own for the local interpolator.¹ These will overwrite any entries with the same keys in the `param_table_handle` table. Finally, the parameter table can be used to pass back arbitrary information by the local and/or global interpolation routine(s) by adding/modifying appropriate key/value pairs.

Note that `CCTK_InterpGridArrays()` is a collective operation, so in the multiprocessor case you *must* call this function in parallel on *each* processor, passing identical arguments except for the number of interpolation points, the interpolation coordinates, and the output array pointers. You may (and typically will) specify a different set of interpolation points on each processor’s call – you may even specify an empty set on some processors. The interpolation points may be “owned” by any processors (this function takes care of all interprocessor-communication issues), though it may be more efficient to have most or all of the interpolation points “owned” by the current processor.

The semantics of `CCTK_InterpGridArrays()` are mostly independent of which Cactus driver is being used, but an implementation will most likely depend on, and make use of, driver-specific internals. For that reason, `CCTK_InterpGridArrays()` is made an overloadable function. The Cactus flesh will supply only a dummy routine for it which – if called – does nothing but print a warning message saying that it wasn’t overloaded by another thorn, and stop the code. So one will always need to compile in and activate a driver-specific thorn which provides an interpolation routine for CCTK grid arrays and properly overloads `CCTK_InterpGridArrays()` with it at startup.

Details of the operation performed, and what (if any) inputs and/or outputs are specified in the parameter table, depend on which driver-specific interpolation thorn and interpolation operator (provided by a local interpolation thorn) you use. See the documentation on individual interpolator thorns (eg. `PUGHInterp` in the `CactusPUGH` arrangement, `LocalInterp` in the `CactusBase` arrangement, and/or `AEILocalInterp` in the `AEIThorns` arrangement) for details.

Note that in a multiprocessor Cactus run, it’s the user’s responsibility to choose the interprocessor ghost-zone size (`driver::ghost_size`) large enough so that the local interpolator never has to off-center its molecules near interprocessor boundaries. (This

¹ It is the caller’s responsibility to ensure that the specified local interpolator supports any optional parameter-table entries that `CCTK_InterpGridArrays()` passes to it. Each thorn providing a `CCTK_InterpLocalUniform()` interpolator should document what options it requires from the global interpolator.

ensures that the interpolation results are independent of the interprocessor decomposition, at least up to floating-point roundoff errors.) If the ghost-zone size is too small, the interpolator should return the `CCTK_ERROR_INTERP_GHOST_SIZE_TOO_SMALL` error code.

See Also

<code>CCTK_InterpHandle()</code>	Get the interpolator handle for a given character-string name.
<code>CCTK_InterpLocalUniform()</code>	Interpolate a list of processor-local arrays which define a uniformly-spaced data grid
<code>CCTK_InterpGV()</code>	Older API to interpolate a list of Cactus grid arrays.
<code>CCTK_InterpLocal()</code>	Older API to interpolate a list of processor-local arrays.

Errors

The following list of error codes indicates specific error conditions. For the complete list of possible error return codes you should refer to the ThornGuide's chapter of the corresponding interpolation thorn(s) you are using. To find the numerical values of the error codes (or more commonly, to find which error code corresponds to a given numerical value), look in the files `cctk_Interp.h`, `util_ErrorCodes.h`, and/or `util_Table.h` in the `src/include/` directory in the Cactus flesh.

<code>CCTK_ERROR_INTERP_POINT_OUTSIDE</code>	one or more of the interpolation points is out of range (in this case additional information about the out-of-range point may be reported through the parameter table; see the Thorn Guide for whatever thorn provides the local interpolation operator for further details)
<code>CCTK_ERROR_INTERP_GRID_TOO_SMALL</code>	one or more of the dimensions of the input arrays is/are smaller than the molecule size chosen by the interpolator (based on the parameter-table options, e.g. the interpolation order)
<code>CCTK_ERROR_INTERP_GHOST_SIZE_TOO_SMALL</code>	for a multi-processor run, the size of the interprocessor boundaries (the <i>ghostzone</i> size) is smaller than the molecule size chosen by the interpolator (based on the parameter-table options, e.g. the interpolation order). This error code is also returned if a processor's chunk of the global grid is smaller than the actual molecule size.
<code>UTIL_ERROR_BAD_INPUT</code>	one or more of the input arguments is invalid (eg. NULL pointer)
<code>UTIL_ERROR_NO_MEMORY</code>	unable to allocate memory
<code>UTIL_ERROR_BAD_HANDLE</code>	parameter table handle is invalid
other error codes	this function may also return any error codes returned by the <code>Util_Table*()</code> routines used to get parameters from (and/or set results in) the parameter table

Examples

C Here's a simple example to do quartic 3-D interpolation of a real and a complex grid array, at 1000 interpolation points:

```
#include "cctk.h"
#include "util_Table.h"

#define N_DIMS      3
#define N_INTERP_POINTS 1000
```

```

#define N_INPUT_ARRAYS 2
#define N_OUTPUT_ARRAYS 2

const cGH *GH;
int operator_handle, coord_system_handle;

/* interpolation points */
CCTK_REAL interp_x[N_INTERP_POINTS],
            interp_y[N_INTERP_POINTS],
            interp_z[N_INTERP_POINTS];
static const CCTK_INT interp_coord_type_codes[N_DIMS]
    = { CCTK_VARIABLE_REAL, CCTK_VARIABLE_REAL, CCTK_VARIABLE_REAL };
const void *interp_coords[N_DIMS];

/* input and output arrays */
CCTK_INT input_array_variable_indices[N_INPUT_ARRAYS];
static const CCTK_INT output_array_type_codes[N_OUTPUT_ARRAYS]
    = { CCTK_VARIABLE_REAL, CCTK_VARIABLE_COMPLEX };
void *output_arrays[N_OUTPUT_ARRAYS];
CCTK_REAL output_for_real_array [N_INTERP_POINTS];
CCTK_COMPLEX output_for_complex_array[N_INTERP_POINTS];

operator_handle = CCTK_InterpHandle("generalized polynomial interpolation");
if (operator_handle < 0)
{
    CCTK_WARN(-1, "can't get operator handle!");
}

coord_system_handle = CCTK_CoordSystemHandle("cart3d");
if (coord_system_handle < 0)
{
    CCTK_WARN(-1, "can't get coordinate-system handle!");
}

interp_coords[0] = (const void *) interp_x;
interp_coords[1] = (const void *) interp_y;
interp_coords[2] = (const void *) interp_z;
input_array_variable_indices[0] = CCTK_VarIndex("my_thorn::real_array");
input_array_variable_indices[1] = CCTK_VarIndex("my_thorn::complex_array");
output_arrays[0] = (void *) output_for_real_array;
output_arrays[1] = (void *) output_for_complex_array;

if (CCTK_InterpGridArrays(GH, N_DIMS,
                          operator_handle,
                          Util_TableCreateFromString("order=4"),
                          coord_system_handle,
                          N_INTERP_POINTS, interp_coord_type_codes,
                          interp_coords,
                          N_INPUT_ARRAYS, input_array_variable_indices,
                          N_OUTPUT_ARRAYS, output_array_type_codes,
                          output_arrays) < 0)
{
    CCTK_WARN(-1, "error return from interpolator!");
}

```

CCTK_InterpGV

Perform an interpolation on a list of distributed CCTK grid variables, using a chosen interpolation operator

This function is being phased out; it will eventually be replaced by `CCTK_InterpGridArrays()`.

Synopsis

C	<code>int ierr = CCTK_InterpGV(cGH * cctkGH, int operator_handle, int coord_system_handle, int num_points, num_in_array_indices, num_out_arrays, ...)</code>
Fortran	<code>call CCTK_InterpGV(ierr, cctkGH, operator_handle, coord_system_handle, num_points, num_in_array_indices, num_out_arrays, ...)</code>

integer ierr
CCTK_POINTER cctkGH
integer operator_handle
integer coord_system_handle
integer num_points
integer num_in_array_indices
integer num_out_arrays
 ...
num_dims * **CCTK_POINTER** interp_coord_arrays
num_dims * **integer** interp_coord_array_types
num_array_indices * **integer** in_array_indices
num_out_arrays * **CCTK_POINTER** out_arrays
num_out_arrays * **integer** out_array_types

Parameters

<code>cctkGH</code>	- Pointer to the CCTK grid hierarchy
<code>operator_handle</code>	- Handle for the interpolation operator
<code>coord_system_handle</code>	- Handle for the coordinate system. This handle denotes the coordinate system to use for locating the points to interpolate at.
<code>num_points</code>	- Number of points to be interpolated on this processor
<code>num_in_array_indices</code>	- Number of passed input array indices
<code>num_out_arrays</code>	- Number of passed output arrays
<code>...</code>	- Variable argument list, with arguments as following
<code>interp_coord_arrays</code>	- List of coordinate arrays for points to interpolate at (<i>num_dims</i> is the number of dimensions of the coordinate system)
<code>interp_coord_array_types</code>	- List of CCTK datatypes for coordinate arrays
<code>in_array_indices</code>	- List of CCTK grid variables to interpolate (given by their indices)
<code>out_arrays</code>	- List of output arrays
<code>out_array_types</code>	- List of CCTK datatypes for output arrays

Discussion

Examples

```

C      int interp_handle, coord_system_handle;
      CCTK_REAL coord_x[NUM_POINTS], coord_y[NUM_POINTS];
      int my_grid_fn1, my_grid_fn2;
      CCTK_REAL my_out_array1[NUM_POINTS];
      CCTK_COMPLEX my_out_array2[NUM_POINTS];

      interp_handle =
          CCTK_InterpHandle("my interpolation operator");
      coord_system_handle =
          CCTK_CoordSystemHandle("my 2D coordinate system");
      my_grid_fn1 = CCTK_VarIndex("myThorn:myGF1");
      my_grid_fn2 = CCTK_VarIndex("myThorn:myGF2");

      ierr = CCTK_InterpGV(cctkGH,
          interp_handle, coord_system_handle,
          NUM_POINTS, 2, 2,
          coord_x, coord_y,
          CCTK_VARIABLE_REAL, CCTK_VARIABLE_REAL,
          my_grid_fn1, my_grid_fn2,
          my_out_array1, my_out_array2,
          CCTK_VARIABLE_REAL, CCTK_VARIABLE_COMPLEX);

```

```

Fortran  integer interp_handle, coord_system_handle
          CCTK_REAL coord(NUM_POINTS)
          integer my_grid_fn1, my_grid_fn2, my_grid_fn3
          CCTK_REAL my_out_array1(NUM_POINTS)
          CCTK_COMPLEX my_out_array2(NUM_POINTS)
          CCTK_INT my_out_array3(NUM_POINTS)

          call CCTK_InterpHandle(interp_handle,
              "my interpolation operator")
          call CCTK_CoordSystemHandle(coord_system_handle,
              "my 1D coordinate system")
          call CCTK_VarIndex(my_grid_fn1, "myThorn:myGF1")
          call CCTK_VarIndex(my_grid_fn2, "myThorn:myGF2")
          call CCTK_VarIndex(my_grid_fn2, "myThorn:myGF3")

          call CCTK_InterpGV(ierr, cctkGH,
              interp_handle, coord_system_handle,
              NUM_POINTS, 3, 3,
              coord,
              CCTK_VARIABLE_REAL,
              my_grid_fn1, my_grid_fn2, my_grid_fn3,
              my_out_array1, my_out_array2, my_out_array3,
              CCTK_VARIABLE_REAL, CCTK_VARIABLE_COMPLEX,
              CCTK_VARIABLE_INT)

```

Errors

A negative return code indicates an error condition:

- 1 Invalid interpolation operator handle passed
- 2 Invalid coordinate system handle passed

CCTK_InterpHandle

Return the handle for a given interpolation operator

Synopsis

C	<code>int handle = CCTK_InterpHandle(const char * operator)</code>
Fortran	<code>call CCTK_InterpHandle(handle , operator)</code> <code>integer handle</code> <code>character*(*) operator</code>

Parameters

<code>handle</code>	-	Handle for the interpolation operator
<code>operator</code>	-	Name of interpolation operator

Discussion**Examples**

C	<code>handle = CCTK_InterpHandle("my interpolation operator");</code>
Fortran	<code>call CCTK_InterpHandle(handle,"my interpolation operator")</code>

Errors

A negative value is returned for invalid/unregistered interpolation operator names.

CCTK_InterpLocal

Perform an interpolation on a list of processor-local arrays, using a chosen interpolation operator

This function is being phased out; it will eventually be replaced by `CCTK_InterpLocalUniform()` et al.

Synopsis

C	<code>int ierr = CCTK_InterpLocal(cGH * cctkGH, int operator_handle, int num_points, int num_dims,</code>
Fortran	<code>call CCTK_InterpLocal(ierr , cctkGH, operator_handle, num_points, num_dims, num_in_arrays, num_out_arrays,</code>
	<code>)</code>
	<code>integer ierr</code>
	<code>CCTK_POINTER cctkGH</code>
	<code>integer operator_handle</code>
	<code>integer num_points</code>
	<code>integer num_dims</code>
	<code>integer num_in_arrays</code>
	<code>integer num_out_arrays</code>
	<code>...</code>
	<code>num_dims * integer dims</code>
	<code>num_dims * CCTK_POINTER coord_arrays</code>
	<code>num_dims * integer coord_array_types</code>
	<code>num_dims * CCTK_POINTER interp_coord_arrays</code>
	<code>num_dims * integer interp_coord_array_types</code>
	<code>num_in_arrays * CCTK_POINTER in_arrays</code>
	<code>num_in_arrays * integer in_array_types</code>
	<code>num_out_arrays * CCTK_POINTER out_arrays</code>
	<code>num_out_arrays * integer out_array_types</code>

Parameters

<code>cctkGH</code>	- Pointer to the CCTK grid hierarchy
<code>operator_handle</code>	- Handle for the interpolation operator
<code>num_points</code>	- Number of points to be interpolated on this processor
<code>num_dims</code>	- Number of dimensions of the coordinate system
<code>num_in_arrays</code>	- Number of passed input arrays
<code>num_out_arrays</code>	- Number of passed output arrays
<code>...</code>	- Variable argument list, with arguments as following
<code>dims</code>	- Dimensions of the underlying coordinate system
<code>coord_arrays</code>	- List of coordinate arrays describing the coordinate system
<code>coord_array_types</code>	- List of CCTK datatypes for coordinate arrays
<code>interp_coord_arrays</code>	- List of interpolation coordinate arrays
<code>interp_coord_array_types</code>	- List of CCTK datatypes for interpolation coordinate arrays
<code>in_arrays</code>	- List of input arrays to interpolate
<code>in_array_types</code>	- List of CCTK datatypes for input arrays
<code>out_arrays</code>	- List of output arrays

Discussion

Examples

```

C      int interp_handle;
      CCTK_REAL coord_x[XDIM], coord_y[YDIM];
      CCTK_REAL interp_coord_x[NUM_POINTS],
      interp_coord_y[NUM_POINTS];
      CCTK_REAL my_in_array1[NUM_POINTS];
      CCTK_COMPLEX my_in_array2[NUM_POINTS];
      CCTK_REAL my_out_array1[NUM_POINTS];
      CCTK_COMPLEX my_out_array2[NUM_POINTS];

      interp_handle =
          CCTK_InterpHandle("my interpolation operator");
      ierr = CCTK_InterpLocal(cctkGH,
          interp_handle, NUM_POINTS, 2, 2, 2,
          XDIM, YDIM, coord_x, coord_y,
          CCTK_VARIABLE_REAL, CCTK_VARIABLE_REAL,
          interp_coord_x, interp_coord_y,
          CCTK_VARIABLE_REAL, CCTK_VARIABLE_REAL,
          my_in_array1, my_in_array2,
          CCTK_VARIABLE_REAL, CCTK_VARIABLE_COMPLEX);
      my_out_array1, my_out_array2,
      CCTK_VARIABLE_REAL, CCTK_VARIABLE_COMPLEX);

Fortran integer interp_handle
      CCTK_REAL coord(XDIM)
      CCTK_REAL interp_coord(NUM_POINTS)
      CCTK_REAL my_in_array1(NUM_POINTS), my_out_array1(NUM_POINTS)
      CCTK_COMPLEX my_in_array2(NUM_POINTS),
      my_out_array2(NUM_POINTS)
      CCTK_INT my_in_array3(NUM_POINTS), my_out_array3(NUM_POINTS)

      call CCTK_InterpHandle(interp_handle,
          "my interpolation operator")
      call CCTK_InterpLocal(ierr, cctkGH,
          interp_handle, NUM_POINTS, 1, 3, 3,
          XDIM, coord,
          CCTK_VARIABLE_REAL,
          interp_coord,
          CCTK_VARIABLE_REAL,
          my_in_array1, my_in_array2, my_in_array3,
          CCTK_VARIABLE_REAL, CCTK_VARIABLE_COMPLEX,
          CCTK_VARIABLE_INT,
          my_out_array1, my_out_array2, my_out_array3,
          CCTK_VARIABLE_REAL, CCTK_VARIABLE_COMPLEX,
          CCTK_VARIABLE_INT)

```

Errors

A negative return code indicates an error condition:

-1 Invalid interpolation operator handle passed

in the `src/include/` directory in the Cactus flesh.

CCTK_InterpLocalUniform

Interpolate a list of processor-local arrays which define a uniformly-spaced data grid

(CCTK_InterpLocalNonUniform(), CCTK_InterpLocalUniform(), and CCTK_InterpLocalWarped() will eventually replace CCTK_InterpLocal().)

The computation is optimized for the case of interpolating a number of arrays at a time; in this case the same interpolation coefficients can be used for all the arrays.

Synopsis

```

C      #include "util_ErrorCodes.h"
      #include "cctk.h"
      int status
        = CCTK_InterpLocalUniform(int N_dims,
                                   int operator_handle,
                                   int param_table_handle,
                                   const CCTK_REAL coord_origin[],
                                   const CCTK_REAL coord_delta[],
                                   int N_interp_points,
                                   int interp_coords_type_code,
                                   const void *const interp_coords[],
                                   int N_input_arrays,
                                   const CCTK_INT input_array_dims[],
                                   const CCTK_INT input_array_type_codes[],
                                   const void *const input_arrays[],
                                   int N_output_arrays,
                                   const CCTK_INT output_array_type_codes[],
                                   void *const output_arrays[]);

```

Result

0 success

Parameters

N_dims (≥ 1) Number of dimensions in which to interpolate. Note that this may be less than the number of dimensions of the input arrays if the storage is set up appropriately. For example, we might want to interpolate along 1-D lines or in 2-D planes of a 3-D input array; here **N_dims** would be 1 or 2 respectively. For details, see the section on “Non-Contiguous Input Arrays” in the Thorn Guide for thorn AEILocalInterp.

operator_handle (≥ 0)

Handle to the interpolation operator as returned by CCTK_InterpHandle().

param_table_handle (≥ 0)

Handle to a key-value table containing additional parameters for the interpolator.

One common parameter-table option, which a number of interpolation operators are likely to support, is **order**, a CCTK_INT specifying the order of the (presumably polynomial) interpolation (1=linear, 2=quadratic, 3=cubic, etc).

See the Thorn Guide for the AEILocalInterp thorn for other parameters.

coord_origin ($\neq$ NULL)

(Pointer to) an array giving the coordinates of the data point with integer array subscripts 0, 0, ..., 0, or more generally (if the actual array bounds don't include the all-zeros-subscript point) the coordinates which this data point would have if it existed. See the “Discussion” section below for more on how **coord_origin[]** is

actually used.

`coord_delta` ($\neq$ NULL)

(Pointer to) an array giving the coordinate spacing of the data arrays. See the “Discussion” section below for more on how `coord_delta[]` is actually used.

`N_interp_points` (≥ 0)

The number of points at which interpolation is to be done.

`interp_coords_type_code`

One of the `CCTK_VARIABLE_*` type codes, giving the data type of the 1-D interpolation-point-coordinate arrays pointed to by `interp_coords[]`.

`interp_coords` ($\neq$ NULL)

(Pointer to) an array of `N_dims` pointers to 1-D arrays giving the coordinates of the interpolation points. These coordinates are with respect to the coordinate system defined by `coord_origin[]` and `coord_delta[]`.

`N_input_arrays` (≥ 0)

The number of input arrays to be interpolated. Note that if the parameter table entry `operand_indices` is used to specify a 1-to-many mapping of input arrays to output arrays, only the unique set of input arrays should be given here.

`input_array_dims` ($\neq$ NULL)

(Pointer to) an array of `N_dims` integers giving the dimensions of the `N_dims`-D input arrays. By default all the input arrays are taken to have these dimensions, with `[0]` the most contiguous axis and `[N_dims-1]` the least contiguous axis, and array subscripts in the range $0 \leq \text{subscript} < \text{input_array_dims}[\text{axis}]$. See the discussion of the `input_array_strides` optional parameter (passed in the parameter table) for details of how this can be overridden.

`input_array_type_codes` ($\neq$ NULL)

(Pointer to) an array of `N_input_arrays` `CCTK_VARIABLE_*` type codes giving the data types of the `N_dims`-D input arrays pointed to by `input_arrays[]`.

`input_arrays` ($\neq$ NULL)

(Pointer to) an array of `N_input_arrays` pointers to the `N_dims`-D input arrays for the interpolation. If any `input_arrays[in]` pointer is NULL, that interpolation is skipped.

`N_output_arrays` (≥ 0)

The number of output arrays to be returned from the interpolation.

`output_array_type_codes` ($\neq$ NULL)

(Pointer to) an array of `N_output_arrays` `CCTK_VARIABLE_*` type codes giving the data types of the 1-D output arrays pointed to by `output_arrays[]`.

`output_arrays` ($\neq$ NULL)

(Pointer to) an array of `N_output_arrays` pointers to the (user-supplied) 1-D output arrays for the interpolation. If any `output_arrays[out]` pointer is NULL, that interpolation is skipped.

Discussion

`CCTK_InterpLocalUniform()` is a generic API for interpolating processor-local arrays when the data points’ *xyz* coordinates are *linear* functions of the integer array subscripts *ijk* (we’re describing this for 3-D, but the generalization to other numbers of

dimensions should be obvious). The `coord_origin[]` and `coord_delta[]` arguments specify these linear functions:

```
x = coord_origin[0] + i*coord_delta[0]
y = coord_origin[1] + j*coord_delta[1]
z = coord_origin[2] + k*coord_delta[2]
```

The (x, y, z) coordinates are used for the interpolation (i.e. the interpolator may internally use polynomials in these coordinates); `interp_coords[]` specifies coordinates in this same coordinate system.

Details of the operation performed, and what (if any) inputs and/or outputs are specified in the parameter table, depend on which interpolation operator you use. See the Thorn Guide for the `AEILocalInterp` thorn for further discussion.

See Also

<code>CCTK_InterpHandle()</code>	Get the interpolator handle for a given character-string name.
<code>CCTK_InterpGridArrays()</code>	Interpolate a list of Cactus grid arrays
<code>CCTK_InterpGV()</code>	Older API to Interpolate a list of Cactus grid arrays
<code>CCTK_InterpLocal()</code>	Older API to interpolate a list of processor-local arrays
<code>CCTK_InterpRegisterOpLocalUniform()</code>	Register a <code>CCTK_InterpLocalUniform()</code> interpolation operator
<code>CCTK_InterpLocalNonUniform()</code>	Interpolate a list of processor-local arrays, with non-uniformly spaced data points.

Errors

To find the numerical values of the error codes (or more commonly, to find which error code corresponds to a given numerical value), look in the files `cctk_Interp.h`, `util_ErrorCodes.h`, and/or `util_Table.h` in the `src/include/` directory in the Cactus flesh.

<code>CCTK_ERROR_INTERP_POINT_OUTSIDE</code>	one or more of the interpolation points is out of range (in this case additional information about the out-of-range point may be reported through the parameter table; see the Thorn Guide for the <code>AEILocalInterp</code> thorn for further details)
<code>CCTK_ERROR_INTERP_GRID_TOO_SMALL</code>	one or more of the dimensions of the input arrays is/are smaller than the molecule size chosen by the interpolator (based on the parameter-table options, e.g. the interpolation order)
<code>UTIL_ERROR_BAD_INPUT</code>	one or more of the inputs is invalid (eg. <code>NULL</code> pointer)
<code>UTIL_ERROR_NO_MEMORY</code>	unable to allocate memory
<code>UTIL_ERROR_BAD_HANDLE</code>	parameter table handle is invalid
other error codes	this function may also return any error codes returned by the <code>Util_Table*()</code> routines used to get parameters from (and/or set results in) the parameter table

Examples

C	Here's a simple example of interpolating a <code>CCTK_REAL</code> and a <code>CCTK_COMPLEX</code> 10×20 2-D array, at 5 interpolation points, using cubic interpolation. Note that since C allows arrays to be initialized only if the initializer values are compile-time constants, we have to declare the <code>interp_coords[]</code> , <code>input_arrays[]</code> ,
----------	---

and `output_arrays[]` arrays as non-`const`, and set their values with ordinary (run-time) assignment statements. In C++, there's no restriction on initializer values, so we could declare the arrays `const` and initialize them as part of their declarations.

```

#define N_DIMS      2
#define N_INTERP_POINTS  5
#define N_INPUT_ARRAYS    2
#define N_OUTPUT_ARRAYS   2

/* (x,y) coordinates of data grid points */
#define X_ORIGIN  ...
#define X_DELTA   ...
#define Y_ORIGIN  ...
#define Y_DELTA   ...
const CCTK_REAL origin[N_DIMS] = { X_ORIGIN, Y_ORIGIN };
const CCTK_REAL delta [N_DIMS] = { X_DELTA,  Y_DELTA  };

/* (x,y) coordinates of interpolation points */
const CCTK_REAL interp_x[N_INTERP_POINTS];
const CCTK_REAL interp_y[N_INTERP_POINTS];
const void *interp_coords[N_DIMS];          /* see note above */

/* input arrays */
/* ... note Cactus uses Fortran storage ordering, i.e.\ X is contiguous */
#define NX      10
#define NY      20
const CCTK_REAL  input_real  [NY][NX];
const CCTK_COMPLEX input_complex[NY][NX];
const CCTK_INT  input_array_dims[N_DIMS] = { NX, NY };
const CCTK_INT  input_array_type_codes[N_INPUT_ARRAYS]
    = { CCTK_VARIABLE_REAL, CCTK_VARIABLE_COMPLEX };
const void *input_arrays[N_INPUT_ARRAYS];    /* see note above */

/* output arrays */
CCTK_REAL  output_real  [N_INTERP_POINTS];
CCTK_COMPLEX output_complex[N_INTERP_POINTS];
const CCTK_INT output_array_type_codes[N_OUTPUT_ARRAYS]
    = { CCTK_VARIABLE_REAL, CCTK_VARIABLE_COMPLEX };
void *const output_arrays[N_OUTPUT_ARRAYS];  /* see note above */

int operator_handle, param_table_handle;
operator_handle = CCTK_InterpHandle("my interpolation operator");
if (operator_handle < 0)
    CCTK_WARN(-1, "can't get interpolation handle!");
param_table_handle = Util_TableCreateFromString("order=3");
if (param_table_handle < 0)
    CCTK_WARN(-1, "can't create parameter table!");

/* initialize the rest of the parameter arrays */
interp_coords[0] = (const void *) interp_x;
interp_coords[1] = (const void *) interp_y;
input_arrays[0] = (const void *) input_real;
input_arrays[1] = (const void *) input_complex;
output_arrays[0] = (void *) output_real;
output_arrays[1] = (void *) output_complex;

/* do the actual interpolation, and check for error returns */

```

```
if (CCTK_InterpLocalUniform(N_DIMS,
                             operator_handle, param_table_handle,
                             origin, delta,
                             N_INTERP_POINTS,
                             CCTK_VARIABLE_REAL,
                             interp_coords,
                             N_INPUT_ARRAYS,
                             input_array_dims,
                             input_array_type_codes,
                             input_arrays,
                             N_OUTPUT_ARRAYS,
                             output_array_type_codes,
                             output_arrays) < 0)
    CCTK_WARN(-1, "error return from interpolator!");
```

in the `src/include/` directory in the Cactus flesh.

CCTK_InterpRegisterOperatorGV

Register a routine as a `CCTK_InterpGV()` interpolation operator

Synopsis

```
C      int ierr = CCTK_InterpRegisterOperatorGV( cInterpOperatorGV operator, const char * name)
```

Parameters

`operator` - Routine to be registered as the interpolation operator

`name` - Name of interpolation operator

Discussion

Only C routines can be registered as interpolation operators.

Examples

```
C      extern int my_operator (cGH *GH,
                              const char *coord_system,
                              int num_points,
                              int num_in_array_indices,
                              int num_out_arrays,
                              void *interp_coord_arrays[],
                              int interp_coord_array_types[],
                              int in_array_indices[],
                              void *out_arrays[],
                              int out_array_types[]);

      ierr = CCTK_InterpRegisterOperatorGV(my_operator,
      "my interpolation operator");
```

Errors

A negative return code indicates an error condition:

- 1 NULL pointer was passed as interpolation operator routine
- 2 interpolation handle could not be allocated
- 3 Interpolation operator with this name already exists

CCTK_InterpRegisterOperatorLocal

Register a routine as a `CCTK_InterpLocal()` interpolation operator

Synopsis

```
C      int ierr = CCTK_InterpRegisterOperatorLocal( cInterpOperatorLocal operator, const char * name
```

Parameters

`operator` - Routine to be registered as the interpolation operator

`name` - Name of interpolation operator

Discussion

Only C routines can be registered as interpolation operators.

Examples

```
C      extern int my_operator (cGH *GH,
                              int num_points,
                              int num_dims,
                              int num_in_arrays,
                              int num_out_arrays,
                              int coord_dims[],
                              void *coord_arrays[],
                              int coord_array_types[],
                              void *interp_coord_arrays[],
                              int interp_coord_array_types[],
                              void *in_arrays[],
                              int in_array_types[],
                              void *out_arrays[],
                              int out_array_types[]);

      ierr = CCTK_InterpRegisterOperatorLocal(my_operator,
        "my interpolation operator");
```

Errors

A negative return code indicates an error condition:

- 1 NULL pointer was passed as interpolation operator routine
- 2 interpolation handle could not be allocated
- 3 Interpolation operator with this name already exists

CCTK_InterpRegisterOpLocalUniform

Register a CCTK_InterpLocalUniform() interpolation operator.

Synopsis

```
C      #include "cctk.h"
      int CCTK_InterpRegisterOpLocalUniform(cInterpOpLocalUniform operator_ptr,
                                             const char *operator_name,
                                             const char *thorn_name);
```

Result

handle (≥ 0) A cactus handle to refer to all interpolation operators registered under this operator name.

Parameters

operator_ptr ($\neq$ NULL)
 Pointer to the CCTK_InterpLocalUniform() interpolation operator. This argument must be a C function pointer of the appropriate type; the typedef can be found in `src/include/cctk_Interp.h` in the Cactus source code.

operator_name ($\neq$ NULL)
 (Pointer to) a (C-style null-terminated) character string giving the name under which to register the operator.

thorn_name ($\neq$ NULL)
 (Pointer to) a (C-style null-terminated) character string giving the name of the thorn which provides the interpolation operator.

Discussion

Only C functions (or other routines with C-compatible calling sequences) can be registered as interpolation operators.

See Also

CCTK_InterpHandle()	Get the interpolator handle for a given character-string name.
CCTK_InterpLocalUniform()	Interpolate a list of processor-local arrays, with uniformly spaced data points.

Errors

-1	NULL pointer was passed as interpolation operator routine
-2	interpolation handle could not be allocated
-3	Interpolation operator with this name already exists

Examples

```
C      /* prototype for function we want to register */
      int AEILocalInterp_InterpLocalUniform(int N_dims,
                                             int param_table_handle,
                                             /***** coordinate system *****/
                                             const CCTK_REAL coord_origin[],
                                             const CCTK_REAL coord_delta[],
                                             /***** interpolation points *****/
```

```
        int N_interp_points,
        int interp_coords_type_code,
        const void *const interp_coords[],
    /**** input arrays *****/
        int N_input_arrays,
        const CCTK_INT input_array_dims[],
        const CCTK_INT input_array_type_codes[],
        const void *const input_arrays[],
    /**** output arrays *****/
        int N_output_arrays,
        const CCTK_INT output_array_type_codes[],
        void *const output_arrays[]);

/* register it! */
CCTK_InterpRegisterOpLocalUniform(AEILocalInterp_InterpLocalUniform,
    "generalized polynomial interpolation",
    CCTK_THORNSTRING);
```

CCTK_IsImplementationActive

Reports whether an implementation was activated in a parameter file

Synopsis

C	int istat = CCTK_IsImplementationActive(const char * implementationname)
Fortran	istat = CCTK_IsImplementationActive(implementationname)
	integer istat
	character*(*) implementationname

Parameters

istat	-	the return status
	-	the name of the implementation to check
implementationname		

Discussion

This function returns a non-zero value if the implementation given by **implementationname** was activated in a parameter file, and zero otherwise.

CCTK_IsImplementationCompiled

Reports whether an implementation was compiled into the configuration

Synopsis

C	<code>int istat = CCTK_IsImplementationCompiled(const char * implementationname)</code>
Fortran	<code>istat = CCTK_IsImplementationCompiled(implementationname)</code> <code>integer istat</code> <code>character*(*) implementationname</code>

Parameters

<code>istat</code>	- the return status
<code>implementationname</code>	- the name of the implementation to check

Discussion

This function returns a non-zero value if the implementation given by `implementationname` was compiled into the configuration, and zero otherwise.

CCTK_IsThornActive

Reports whether a thorn was activated in a parameter file

Synopsis

C	int istat = CCTK_IsThornActive(const char * thornname)
Fortran	istat = CCTK_IsThornActive(thornname)
	integer istat
	character*(*) thornname

Parameters

istat	-	the return status
thornname	-	the name of the thorn to check

Discussion

This function returns a non-zero value if the thorn given by **thornname** was activated in a parameter file, and zero otherwise.

CCTK_IsThornCompiled

Reports whether a thorn was activated in a parameter file

Synopsis

C	int istat = CCTK_IsThornCompiled(const char * thornname)
Fortran	istat = CCTK_IsThornCompiled(thornname)
	integer istat
	character*(*) thornname

Parameters

istat	-	the return status
thornname	-	the name of the thorn to check

Discussion

This function returns a non-zero value if the implementation given by **thornname** was compiled into the configuration, and zero otherwise.

CCTK_MaxDim

Get the maximum dimension of any grid variable

Synopsis

C	int dim = CCTK_MaxDim()
Fortran	call CCTK_MaxDim(dim)
	integer dim

Parameters

dim - The maximum dimension

Discussion

Note that the maximum dimension will depend on the compiled thorn list, and not the active thorn list.

Examples

C	dim = CCTK_MaxDim() ;
Fortran	call CCTK_MaxDim(dim)

Errors

CCTK_MyProc

Returns the number of the local processor for a parallel run

Synopsis

C	<code>int myproc = CCTK_MyProc(const cGH * cctkGH)</code>
Fortran	<code>myproc = CCTK_MyProc(cctkGH)</code> <code>integer myproc</code> <code>CCTK_POINTER cctkGH</code>

Parameters

`cctkGH` - pointer to CCTK grid hierarchy

Discussion

For a single processor run this call will return zero. For multiprocessor runs, this call will return $0 \leq myproc < \text{CCTK_nProcs}(\text{cctkGH})$.

Examples**Errors**

CCTK_MaxTimeLevels

Gives the number of timelevels for a group

Synopsis

C	int numlevels = CCTK_MaxTimeLevels(const char * name)
Fortran	call CCTK_MaxTimeLevels(numlevels , name) integer numlevels character*(*) name

Parameters

name	-	The full group name
numlevels	-	The number of timelevels

Discussion

The group name should be in the form <implementation>::**group**>

Examples

C	numlevels = CCTK_MaxTimeLevels("evolve::phivars") ;
Fortran	call CCTK_MAXTIMELEVELS(numlevels,"evolve::phivars")

Errors

CCTK_MaxTimeLevelsVN

Gives the number of timelevels for a variable

Synopsis

C	int numlevels = CCTK_MaxTimeLevelsVN(const char * name)
Fortran	call CCTK_MaxTimeLevelsVN(numlevels , name)
	integer numlevels
	character*(*) name

Parameters

name	-	The full variable name
numlevels	-	The number of timelevels

Discussion

The variable name should be in the form <implementation>::<variable>

Examples

C	numlevels = CCTK_MaxTimeLevelsVN("evolve::phi") ;
Fortran	call CCTK_MAXTIMELEVELSVN(numlevels,"evolve::phi")

Errors

CCTK_MaxTimeLevelsVI

Gives the number of timelevels for a variable

Synopsis

C	<code>int numlevels = CCTK_MaxTimeLevelsVI(int index)</code>
Fortran	<code>call CCTK_MaxTimeLevelsVI(numlevels , index)</code> integer numlevels integer index

Parameters

<code>numlevels</code>	-	The number of timelevels
<code>index</code>	-	The variable index

Discussion**Examples**

C	<code>index = CCTK_VarIndex("evolve::phi")</code> <code>numlevels = CCTK_MaxTimeLevelsVI(index) ;</code>
Fortran	<code>call CCTK_MAXTIMELEVELSVI(numlevels,3)</code>

Errors

CCTK_MaxTimeLevelsGI

Gives the number of timelevels for a group

Synopsis

C	int numlevels = CCTK_MaxTimeLevelsGI(int index)
Fortran	call CCTK_MaxTimeLevelsGI(numlevels , index) integer numlevels integer index

Parameters

numlevels	-	The number of timelevels
index	-	The group index

Discussion**Examples**

C	index = CCTK_GroupIndex("evolve::phivars") numlevels = CCTK_MaxTimeLevelsGI(index) ;
Fortran	call CCTK_MAXTIMELEVELSGI(numlevels,3)

Errors

CCTK_nProcs

Returns the number of processors being used for a parallel run

Synopsis

C	<code>int nprocs = CCTK_nProcs(const cGH * cctkGH)</code>
Fortran	<code>nprocs = CCTK_nProcs(cctkGH)</code> <code>integer nprocs</code> <code>CCTK_POINTER cctkGH</code>

Parameters

`cctkGH` - pointer to CCTK grid hierarchy

Discussion

For a single processor run this call will return one.

Examples**Errors**

CCTK_NullPointer

Returns a C-style NULL pointer value.

Synopsis

```
Fortran          #include "cctk.h"

                  CCTK_POINTER pointer_var

                  pointer_var = CCTK_NullPointer()
```

Result

pointer_var a CCTK_POINTER type variable which is initialized with a C-style NULL pointer

Discussion

Fortran doesn't know the concept of pointers so problems arise when a C function is to be called which expects a pointer as one (or more) of its argument(s).

In order to pass a NULL pointer from fortran to C, a local CCTK_POINTER variable should be used which has been initialized before with CCTK_NullPointer().

Note that there is only a fortran wrapper available for CCTK_NullPointer().

See Also

CCTK_PointerTo() Returns the address of a variable passed in by reference from a fortran routine.

Examples

```
Fortran          #include "cctk.h"

                  integer      ierror, table_handle
                  CCTK_POINTER pointer_var

                  pointer_var = CCTK_NullPointer()

                  call Util_TableCreate(table_handle, 0)
                  call Util_TableSetPointer(ierror, table_handle, pointer_var, "NULL pointer")
```


CCTK_NumGroups

Get the number of groups of variables compiled in the code

Synopsis

C	<code>int number = CCTK_NumGroups()</code>
Fortran	<code>call CCTK_NumGroups(number)</code> <code>integer number</code>

Parameters

`number` - The number of groups compiled from the thorns `interface.ccl` files

Discussion**Examples**

C	<code>number = CCTK_NumGroups() ;</code>
Fortran	<code>call CCTK_NumGroups(number);</code>

Errors

CCTK_NumIOMethods

Find the total number of I/O methods registered with the flesh

Synopsis

C	<code>int num_methods = CCTK_NumIOMethods (void);</code>
Fortran	<code>call CCTK_NumIOMethods (num_methods)</code> <code>integer num_methods</code>

Parameters

<code>num_methods</code>	number of registered IO methods
--------------------------	---------------------------------

Discussion

Returns the total number of IO methods registered with the flesh.

CCTK_NumVars

Get the number of grid variables compiled in the code

Synopsis

C	<code>int number = CCTK_NumVars()</code>
Fortran	<code>call CCTK_NumVars(number)</code> integer number

Parameters

`number` - The number of grid variables compiled from the thorn's `interface.ccl` files

Discussion**Examples**

C	<code>number = CCTK_NumVars() ;</code>
Fortran	<code>call CCTK_NumVars(number)</code>

Errors

CCTK_NumVarsInGroup

Provides the number of variables in a group from the group name

Synopsis

C	<code>int num = CCTK_NumVarsInGroup(const char * name)</code>
Fortran	<code>call CCTK_NumVarsInGroup(num , name)</code> <code>integer num</code> <code>character*(*) name</code>

Parameters

<code>num</code>	-	The number of variables in the group
<code>group</code>	-	The full group name

Discussion

The group name should be given in the form `<implementation>::<group>`

Examples

C	<code>numvars = CCTK_NumVarsInGroup("evolve::scalars") ;</code>
Fortran	<code>call CCTK_NUMVARSINGROUP(numvars,"evolve::scalars")</code>

Errors

CCTK_NumVarsInGroupI

Provides the number of variables in a group from the group index

Synopsis

C	<code>int num = CCTK_NumVarsInGroupI(int index)</code>
Fortran	<code>call CCTK_NumVarsInGroupI(num , index)</code> <code>integer num</code> <code>integer index</code>

Parameters

<code>num</code>	-	The number of variables in the group
<code>group</code>	-	The group index

Discussion**Examples**

C	<code>index = CCTK_GroupIndex("evolve::scalars")</code> <code>firstvar = CCTK_NumVarsInGroupI(index) ;</code>
Fortran	<code>call CCTK_NUMVARSINGROUPI(firstvar,3)</code>

Errors

CCTK_OutputGH

Output all variables living on the GH looping over all registered IO methods.

Synopsis

C `int istat = CCTK_OutputGH (const cGH *cctkGH);`
Fortran `call CCTK_OutputGH (istat, cctkGH)`
 `integer istat`
 `CCTK_POINTER cctkGH`

Parameters

`istat` total number of variables for which output was done by all IO methods
`cctkGH` pointer to CCTK grid hierarchy

Discussion

The IO methods decide themselves whether it is time to do output now or not.

Errors

0 it wasn't time to output anything yet by any IO method
-1 if no IO methods were registered

CCTK_OutputVar

Output a single variable by all I/O methods

Synopsis

```
C      int istat = CCTK_OutputVar (const cGH *cctkGH,
                                   const char *variable);
```

```

Fortran      call CCTK_OutputVar (istat, cctkGH, variable)
               integer istat
               CCTK_POINTER cctkGH
               character(*) variable

```

Parameters

istat	return status
cctkGH	pointer to CCTK grid hierarchy
variable	full name of variable to output

Discussion

The output should take place if at all possible. If the appropriate file exists the data is appended, otherwise a new file is created.

Errors

0	for success
negative	for some error condition (eg. IO method is not registered)

CCTK_OutputVarAs

Output a single variable as an alias by all I/O methods

Synopsis

```

C          int istat = CCTK_OutputVarAs (const cGH *cctkGH,
                                           const char *variable,
                                           const char *alias);

Fortran    call CCTK_OutputVarAsByMethod (istat, cctkGH, variable, alias)
integer istat
CCTK_POINTER cctkGH
character(*) variable
character(*) alias

```

Parameters

<code>istat</code>	return status
<code>cctkGH</code>	pointer to CCTK grid hierarchy
<code>variable</code>	full name of variable to output
<code>alias</code>	alias name to base the output filename on

Discussion

The output should take place if at all possible. If the appropriate file exists the data is appended, otherwise a new file is created. Uses `alias` as the name of the variable for the purpose of constructing a filename.

Errors

<code>positive</code>	the number of IO methods which did output of <code>variable</code>
<code>0</code>	for success
<code>negative</code>	if no IO methods were registered

CCTK_OutputVarAsByMethod

Synopsis

C	<pre> int istat = CCTK_OutputVarAsByMethod (const cGH *cctkGH, const char *variable, const char *method, const char *alias); </pre>
Fortran	<pre> call CCTK_OutputVarAsByMethod (istat, cctkGH, variable, method, alias) integer istat CCTK_POINTER cctkGH character(*) variable character(*) method character(*) alias </pre>

Parameters

istat	return status
cctkGH	pointer to CCTK grid hierarchy
variable	full name of variable to output
method	method to use for output
alias	alias name to base the output filename on

Discussion

Output a variable **variable** using the method **method** if it is registered. Uses **alias** as the name of the variable for the purpose of constructing a filename. The output should take place if at all possible. If the appropriate file exists the data is appended, otherwise a new file is created.

Errors

0	for success
negative	indicating some error (eg. IO method is not registered)

CCTK_OutputVarByMethod

Synopsis

[illegible]

```

Fortran      call CCTK_OutputVarByMethod (istat, cctkGH, variable, method)
               integer istat
               CCTK_POINTER cctkGH
               character(*) variable
               character(*) method

```

Parameters

istat	return status
cctkGH	pointer to CCTK grid hierarchy
variable	full name of variable to output
method	method to use for output

Discussion

Output a variable **variable** using the IO method **method** if it is registered. The output should take place if at all possible. if the appropriate file exists the data is appended, otherwise a new file is created.

Errors

0	for success
negative	indicating some error (eg. IO method is not registered)

CCTK_ParallelInit

Initialize the parallel subsystem

Synopsis

```
C      int istat = CCTK_ParallelInit( cGH * cctkGH)
```

Parameters

`cctkGH` - pointer to CCTK grid hierarchy

Discussion

Initializes the parallel subsystem.

CCTK_PARAMWARN

Prints a warning from parameter checking, and possibly stops the code

Synopsis

C	= CCTK_PARAMWARN(const char * message)
Fortran	call CCTK_PARAMWARN(, message)
	character*(*) message

Parameters

message - The warning message

Discussion

The call should be used in routines registered at the schedule point `CCTK_PARAMCHECK` to indicate that there is parameter error or conflict and the code should terminate. The code will terminate only after all the parameters have been checked.

Examples

C	CCTK_PARAMWARN("Mass cannot be negative") ;
Fortran	call CCTK_PARAMWARN("Inside interpolator")

Errors

CCTK_PointerTo

Returns the address of a variable passed in by reference from a fortran routine.

Synopsis

```
Fortran          #include "cctk.h"

                  CCTK_POINTER addr, var

                  addr = CCTK_PointerTo(var)
```

Result

addr the address of variable *var*

Parameters

var variable in the fortran context from which to take the address

Discussion

Fortran doesn't know the concept of pointers so problems arise when a C function is to be called which expects a pointer as one (or more) of its argument(s).

To obtain the pointer to a variable in fortran, one can use `CCTK_PointerTo()` which takes the variable itself as a single argument and returns the pointer to it.

Note that there is only a fortran wrapper available for `CCTK_PointerTo()`.

See Also

`CCTK_NullPointer()` Returns a C-style NULL pointer value.

Examples

```
Fortran          #include "cctk.h"

                  integer      ierror, table_handle
                  CCTK_POINTER addr, var

                  addr = CCTK_PointerTo(var)

                  call Util_TableCreate(table_handle, 0)
                  call Util_TableSetPointer(ierror, table_handle, addr, "variable")
```

CCTK_PrintGroup

Prints a group name from its index

Synopsis

C	= CCTK_PrintGroup(int index)
Fortran	call CCTK_PrintGroup(, index)
	integer index

Parameters

index - The group index

Discussion

This routine is for debugging purposes for Fortran programmers.

Examples

C	CCTK_PrintGroup(1) ;
Fortran	call CCTK_PRINTGROUP(1)

Errors

CCTK_PrintString

Prints a Cactus string

Synopsis

C	= CCTK_PrintString(char * string)
Fortran	call CCTK_PrintString(, string)
	CCTK_STRING string

Parameters

string - The string to print

Discussion

This routine can be used to print Cactus string variables and parameters from Fortran.

Examples

C	CCTK_PrintString(string_param) ;
Fortran	call CCTK_PRINTSTRING(string_param)

Errors

CCTK_PrintVar

Prints a variable name from its index

Synopsis

C	<code>= CCTK_PrintVar(int index)</code>
Fortran	<code>call CCTK_PrintVar(, index)</code> integer index

Parameters

`index` - The variable index

Discussion

This routine is for debugging purposes for Fortran programmers.

Examples

C	<code>CCTK_PrintVar(1) ;</code>
Fortran	<code>call CCTK_PRINTVAR(1)</code>

Errors

CCTK_QueryGroupStorage

Query storage for a group given by its group name

Synopsis

C	int istat = CCTK_QueryGroupStorage(const cGH * cctkGH, const char * groupname)
Fortran	call CCTK_QueryGroupStorage(istat , cctkGH, groupname)
	integer istat
	CCTK_POINTER cctkGH
	character*(*) groupname

Parameters

cctkGH	-	pointer to CCTK grid hierarchy
groupname	-	the group to query, given by its full name
istat	-	the return code

Discussion

This routine queries whether the variables in a group have storage assigned. If so it returns true (a non-zero value), otherwise false (zero).

Errors

A negative error code is returned for an invalid group name.

CCTK_QueryGroupStorageB

Synopsis

```
C      int storage = CCTK_QueryGroupStorageB( const cGH * cctkGH, int groupindex, const char * groupname );
```

Parameters

`cctkGH` - pointer to CCTK grid hierarchy

Discussion

Examples

Errors

CCTK_QueryGroupStorageI

Query storage for a group given by its group index

Synopsis

C	int istat = CCTK_QueryGroupStorageI(const cGH * cctkGH, int groupindex)
Fortran	call CCTK_QueryGroupStorageI(istat , cctkGH, groupindex)
	integer istat
	cctkGH
	integer groupindex

Parameters

cctkGH	-	pointer to CCTK grid hierarchy
groupindex	-	the group to query, given by its index
istat	-	the return code

Discussion

This routine queries whether the variables in a group have storage assigned. If so it returns true (a non-zero value), otherwise false (zero).

Errors

A negative error code is returned for an invalid group name.

CCTK_ReductionHandle

Handle for given reduction method

Synopsis

C	<code>int handle = CCTK_ReductionHandle(const char * reduction)</code>
Fortran	<code>handle = CCTK_ReductionHandle(reduction)</code> integer handle character*(*) reduction

Parameters

<code>handle</code>	-	handle returned for this method
<code>name</code>	-	name of the reduction method required

Discussion

Reduction methods should be registered at `CCTK_STARTUP`. Note that integer reduction handles are used to call `CCTK_Reduce` to avoid problems with passing Fortran strings. Note that the name of the reduction operator is case dependent.

Examples

C	<code>handle = CCTK_ReductionHandle("maximum") ;</code>
Fortran	<code>call CCTK_ReductionHandle(handle,"maximum")</code>

Errors

CCTK_RegisterBanner

Register a banner for a thorn

Synopsis

C	<code>void = CCTK_RegisterBanner(const char * message)</code>
Fortran	<code>call CCTK_RegisterBanner(, message)</code> <code>character*(*) message</code>

Parameters

`message` - String which will be displayed as a banner

Discussion

The banner must be registered during `CCTK_STARTUP`. The banners are displayed in the order in which they are registered.

Examples

C	<code>CCTK_RegisterBanner("My Thorn: Does Something Useful");</code>
Fortran	<code>call CCTK_REGISTERBANNER("*** MY THORN ***")</code>

Errors

CCTK_RegisterGHEExtension

Register an extension to the CactusGH

Synopsis

```
C      int istat = CCTK_RegisterGHEExtension( const char * name)
```

Parameters**Discussion****Examples****Errors**

CCTK_RegisterGHExtensionInitGH

Register a function which will initialise a given extension to the Cactus GH

Synopsis

```
C      int istat = CCTK_RegisterGHExtensionInitGH( int handle, void * (*func)(cGH *))
```

Parameters**Discussion****Examples****Errors**

CCTK_RegisterGHExtensionScheduleTraverseGH

Register a GH extension schedule traversal routine

Synopsis

```
C      int istat = CCTK_RegisterGHExtensionScheduleTraverseGH( int handle, int (*func)(cGH *,const
```

Parameters**Discussion****Examples****Errors**

CCTK_RegisterGHExtensionSetupGH

Register a function which will set up a given extension to the Cactus GH

Synopsis

```
C      int istat = CCTK_RegisterGHExtensionSetupGH( int handle, void * (*func)(tFleshConfig *, int, c
```

Parameters**Discussion****Examples****Errors**

CCTK_RegisterIOMethod

Register a new I/O method

Synopsis

C	int handle = CCTK_RegisterIOMethod(const char * name)
Fortran	handle = CCTK_RegisterIOMethod(name)
	integer handle
	name

Parameters

handle	-	handle returned by registration
name	-	name of the I/O method

Discussion

IO methods should be registered at CCTK.STARTUP.

Examples**Errors**

CCTK_RegisterIOMethodOutputGH

Register a routine for an I/O method which will be called from CCTK_OutputGH.

Synopsis

```
C      int istat = CCTK_RegisterIOMethodOutputGH( int handle, int (* func)(const cGH *))
```

Parameters**Discussion****Examples****Errors**

CCTK_RegisterIOMethodOutputVarAs

Register a routine for an I/O method which will provide aliased variable output

Synopsis

```
C      int istat = CCTK_RegisterIOMethodOutputVarAs( int handle, int (* func)(const cGH *,const cha
```

Parameters**Discussion****Examples****Errors**

CCTK_RegisterIOMethodTimeToOutput

Register a routine for an I/O method which will decide if it is time for the method to output.

Synopsis

```
C      int istat = CCTK_RegisterIOMethodTimeToOutput( int handle, int (* func)(const cGH *,int))
```

Parameters**Discussion****Examples****Errors**

CCTK_RegisterIOMethodTriggerOutput

Register a routine for an I/O method which will handle trigger output

Synopsis

```
C      int istat = CCTK_RegisterIOMethodTriggerOutput( int handle, int (* func)(const cGH *,int))
```

Parameters**Discussion****Examples****Errors**

CCTK_RegisterReductionOperator

Synopsis

C CCTK_RegisterReductionOperator()

Parameters**Discussion****Examples****Errors**

CCTK_SetupGH

Setup a new GH

Synopsis

C `cGH * cctkGH = CCTK_SetupGH( tFleshConfig config, int convlevel)`

Parameters**Discussion****Examples****Errors**

CCTK_SyncGroup

Synchronise the ghostzones for a group of grid variables

Synopsis

C	int istat = CCTK_SyncGroup(cGH * cctkGH, const char * group)
Fortran	call CCTK_SyncGroup(istat , cctkGH, group)
	integer istat
	CCTK_POINTER cctkGH
	character*(*) group

Parameters

`cctkGH` - pointer to CCTK grid hierarchy

Discussion

Only those grid variables which have communication enabled will be synchronised. This is usually equivalent to the variables which have storage assigned, unless communication has been explicitly turned off with a call to `CCTK_DisableGroupComm`.

Note that an alternative to calling `CCTK_SyncGroup` explicitly from within a thorn, is to use the `SYNC` keyword in a thorns `schedule.ccl` file to indicate which groups of variables need to be synchronised on exit from the routine. This latter method is the preferred method from synchronising variables.

Examples**Errors**

CCTK_VarDataPtr

Returns the data pointer for a grid variable

Synopsis

```
C      void * ptr = CCTK_VarDataPtr( const cGH * cctkGH, int timelevel, char * name)
```

Parameters

```
      cctkGH  -  pointer to CCTK grid hierarchy
      timelevel -  The timelevel of the grid variable
      name    -  The full name of the variable
```

Discussion

The variable name should be in the form `<implementation>::<variable>`.

Examples

```
C      myVar = (CCTK_REAL *) (CCTK_VarDataPtr(GH,0,"imp::realvar"))
```

Errors

CCTK_VarDataPtrB

Returns the data pointer for a grid variable from the variable index or the variable name

Synopsis

```
C      void * ptr = CCTK_VarDataPtrB( const cGH * cctkGH, int timelevel, int index, char * name)
```

Parameters

```
      ptr    - a void pointer to the grid variable data
      cctkGH -
      timelevel - The timelevel of the grid variable
      index  - The index of the variable
      name   - The full name of the variable
```

Discussion

If the name is NULL the index will be used, if the index is negative the name will be used.

Examples

```
C      myVar = (CCTK_REAL *) (CCTK_VarDataPtrB(GH,0,CCTK_VarIndex("imp:realvar"),NULL))
```

Errors

CCTK_VarDataPtrI

Returns the data pointer for a grid variable from the variable index

Synopsis

```
C      void * ptr = CCTK_VarDataPtrI( const cGH * cctkGH, int timelevel, int index)
```

Parameters

```
      cctkGH  -  
      timelevel - The timelevel of the grid variable  
      index   - The index of the variable
```

Discussion**Examples**

```
C      myVar = (CCTK_REAL *) (CCTK_VarDataPtr(GH,0,CCTK_VarIndex("imp::realvar")))
```

Errors

CCTK_VarIndex

Get the index for a variable.

Synopsis

```

C          #include "cctk.h"
              int index = CCTK_VarIndex(const char *varname);

Fortran    call CCTK_VarIndex(index, varname)
              integer index
              character*(*) varname

```

Parameters

varname The name of the variable.

Discussion

The variable name should be the given in its fully qualified form, that is `<implementation>::<variable>` for a public or protected variable, and `<thornname>::<variable>` for a private variable.

Errors

-1	no variable of this name exists
-2	failed to catch error code from <code>Util_SplitString</code>
-3	given full name is in wrong format
-4	memory allocation failed

Examples

```

C          index = CCTK_VarIndex("evolve::phi");

Fortran    call CCTK_VarIndex(index,"evolve::phi")

```

CCTK_VarName

Given a variable index, returns the variable name

Synopsis

```
C      const char * name = CCTK_VarName( int index)
```

Parameters

`name` - The variable name
`index` - The variable index

Discussion

No Fortran routine exists at the moment.

Examples

```
C      index = CCTK_VarIndex("evolve::phi");  
      name = CCTK_VarName(index);
```

Errors

CCTK_VarTypeI

Provides variable type index from the variable index

Synopsis

C	int type = CCTK_VarTypeI(int index)
Fortran	call CCTK_VarTypeI(type , index) integer type integer index

Parameters

type	- The variable type index
group	- The variable index

Discussion

The variable type index indicates the type of the variable. Either character, int, complex or real. The group type can be checked with the Cactus provided macros for CCTK_VARIABLE_INT, CCTK_VARIABLE_REAL, CCTK_VARIABLE_COMPLEX or CCTK_VARIABLE_CHAR.

Examples

C	index = CCTK_VarIndex("evolve::phi") real = (CCTK_VARIABLE_REAL == CCTK_VarTypeI(index)) ;
Fortran	call CCTK_VARTYPEI(type,3)

Errors

CCTK_VarTypeSize

Provides variable type size in bytes from the variable type index

Synopsis

```
C          #include "cctk.h"

          int CCTK_VarTypeSize (int vtype);
```

Parameters

vtype Variable type index.

Discussion

This function returns the size in bytes of any of the Cactus variable types CCTK_INT, CCTK_REAL, CCTK_COMPLEX, etc.

CCTK_VInfo

Prints a formatted string with a variable argument list as an info message to screen

Synopsis

```
C      #include "cctk.h"
      #include "cctk_WarnLevel.h"

      int status = CCTK_VInfo(const char *thorn,
                              const char *format,
                              ...);
```

Result

0 ok

Parameters

thorn	The name of the thorn printing this info message. You can use the <code>CCTK_THORNSTRING</code> macro here (defined in <code>cctk.h</code>).
format	The <code>fprintf(3)</code> -like format string to use for printing the info message.
...	The variable argument list.

Discussion

This routine can be used by thorns to print a formatted string with a variable argument list as an info message to screen. The message will include the name of the originating thorn, otherwise its semantics is equivalent to `fprintf(3)`.

See Also

CCTK_INFO	macro to print an info message with a single string argument
-----------	--

Examples

```
C      #include "cctk.h"
      #include "cctk_WarningLevel.h"

      const char *outdir;

      CCTK_VInfo(CCTK_THORNSTRING, "Output files will go to '%s'", outdir);
```

CCTK_VWarn

Prints a formatted string with a variable argument list as warning message and possibly stops the code

Synopsis

```
C      #include "cctk.h"
      #include "cctk_WarnLevel.h"

      int status = CCTK_VWarn(int level,
                             int line,
                             const char *file,
                             const char *thorn,
                             const char *format,
                             ...);
```

Result

0 ok

Parameters

level (≥ 0)	The warning level for the message to print, with level 0 being the severest level.
line	The line number in the originating source file where the <code>CCTK_VWarn()</code> call occurred. You can use the standardized <code>__LINE__</code> preprocessor macro here.
file	The file name of the originating source file where the <code>CCTK_VWarn()</code> call occurred. You can use the standardized <code>__FILE__</code> preprocessor macro here.
thorn	The thorn name of the originating source file where the <code>CCTK_VWarn()</code> call occurred. You can use the <code>CCTK_THORNSTRING</code> macro here (defined in <code>cctk.h</code>).
format	The <code>fprintf(3)</code> -like format string to use for printing the warning message.
...	The variable argument list.

Discussion

This routine can be used by thorns to print a formatted string followed by a variable argument list as a warning message to `stderr`.

By default Cactus prints any warning with warning levels ≤ 1 to standard error, and would stop the code on a level 0 warning. This behaviour can be changed on the command line using the flags `-W` and `-E` (see the Users' Guide for full details).

The boolean flesh parameter `cctk.full_warnings` determines whether all the details about the warning origin (processor ID, line number, source file, source thorn) are shown. The default is to omit the line number and name of the source file.

See Also

`CCTK_WARN` macro to print a warning message with a single string argument

Examples

```
C      #include "cctk.h"
      #include "cctk_WarningLevel.h"

      const char *outdir;

      CCTK_VWarn(1, __LINE__, __FILE__, CCTK_THORNSTRING,
                 "Output directory '%s' could not be created", outdir);
```

CCTK_WARN

Macro to print a single string as a warning message and possibly stop the code

Synopsis

C `#include "cctk.h"`
 `#include "cctk_WarnLevel.h"`

`CCTK_WARN(int level, const char *message);`

Fortran `#include "cctk.h"`

`call CCTK_WARN(level, message)`
 `integer            level`
 `character*(*) message`

Parameters

`level` The warning level to use

`message` The warning message to print

Discussion

This macro can be used by thorns to print a single string as a warning message to `stderr`.

`CCTK_WARN(level, message)` expands to a call to the underlying function `CCTK_Warn()`:
`CCTK_Warn(level, __LINE__, __FILE__, CCTK_THORNSTRING, message)`

So the macro automatically includes details about the origin of the warning (the thorn name, the source code file name and the line number where the macro occurs). It is recommended that the macro `CCTK_WARN` is used to print a warning message rather than calling `CCTK_Warn()` directly.

To include variables in a warning message from C, you can use the routine `CCTK_VWarn()` which accepts a variable argument list. To include variables from Fortran, a string must be constructed and passed in a `CCTK_WARN` macro.

See Also

`CCTK_VWarn()` prints a warning message with a variable argument list

Examples

C `#include "cctk.h"`
 `#include "cctk_WarningLevel.h"`

`CCTK_WARN(0, "Divide by 0");`

Fortran `#include "cctk.h"`

`integer            myint`
 `real               myreal`
 `character*200 message`

`write(message, '(A32, G12.7, A5, I8)')`
 `&        'Your warning message, including ', myreal, ' and ', myint`
 `call CCTK_WARN(1, message)`

Chapter E2

Utility Functions

In this section all `Util_*` Cactus utility functions are described. These are low-level functions mainly for more complicated programming, which are used by the rest of Cactus, but don't depend heavily on it. Some of them are callable from Fortran or C, but many are C-only.

E2.1 Functions Alphabetically

Here the functions are listed alphabetically within each section.

E2.1.1 String Functions

- `Util_StrCmpi` [\[E160\]](#) Compare two strings, ignoring upper/lower case.
- `Util_Strdup` [\[E162\]](#) “Duplicate” a string, i.e. copy it to a newly-~~malloced~~ buffer.
- `Util_Strlcat` [\[E164\]](#) Concatenate two strings safely.
- `Util_Strlcpy` [\[E166\]](#) Copy a string safely.

E2.1.2 Table Functions

- `Util_TableClone` [\[E168\]](#) Create a new table which is a “clone” (exact copy) of an existing table
- `Util_TableCreate` [\[E170\]](#) Create a new (empty) table
- `Util_TableCreateFromString`
 [\[E172\]](#) Create a new table (with the case-insensitive flag set) and sets values in it based on a string argument (interpreted with “parameter-file” semantics)
- `Util_TableDeleteKey` [\[E174\]](#) Delete a specified key/value entry from a table
- `Util_TableDestroy` [\[E175\]](#) Destroy a table
- `Util_TableGet*` [\[E176\]](#) This is a family of functions, one for each Cactus data type, to get the single (1-element array) value, or more generally the first array element of the value, associated with a specified key in a key/value table.
- `Util_TableGet*Array` [\[E178\]](#) This is a family of functions, one for each Cactus data type, to get a copy of the value associated with a specified key, and store it (more accurately, as much of it as will fit) in a specified array

- Util_TableGetGeneric [E180] Get the single (1-element array) value, or more generally the first array element of the value, associated with a specified key in a key/value table; the value's data type is generic
- Util_TableGetGenericArray [E182] Get a copy of the value associated with a specified key, and store it (more accurately, as much of it as will fit) in a specified array; the array's data type is generic
- Util_TableGetString [E184] Gets a copy of the character-string value associated with a specified key in a table, and stores it (more accurately, as much of it as will fit) in a specified character string
- Util_TableItAdvance [E186] Advance a table iterator to the next entry in the table
- Util_TableItClone [E187] Creates a new table iterator which is a "clone" (exact copy) of an existing table iterator
- Util_TableItCreate [E189] Create a new table iterator
- Util_TableItDestroy [E190] Destroy a table iterator
- Util_TableItQueryIsNonNull [E191] Query whether a table iterator is *not* in the "null-pointer" state
- Util_TableItQueryIsNull [E192] Query whether a table iterator is in the "null-pointer" state
- Util_TableItQueryKeyValueInfo [E193] Query the key and the type and number of elements of the value corresponding to that key, of the table entry to which an iterator points
- Util_TableItQueryTableHandle [E196] Query what table a table iterator iterates over
- Util_TableItResetToStart [E197] Reset a table iterator to point to the starting table entry
- Util_TableItSetToKey [E198] Set a key/value iterator to point to a specified entry in the table.
- Util_TableItSetToNull [E199] Set a key/value iterator to the "null-pointer" state.
- Util_TableQueryFlags [E200] Query a table's flags word
- Util_TableQueryValueInfo [E201] Query whether or not a specified key is in the table, and optionally the type and/or number of elements of the value corresponding to this key
- Util_TableQueryMaxKeyLength [E203] Query the maximum key length in a table
- Util_TableQueryNKeys [E204] Query the number of key/value entries in a table
- Util_TableSet* [E205] This is a family of functions, one for each Cactus data type, to set the value associated with a specified key to be a specified single (1-element array) value
- Util_TableSet*Array [E207] This is a family of functions, one for each Cactus data type, to set the value associated with a specified key to be a copy of a specified array
- Util_TableSetFromString [E209] Sets values in a table based on a string argument (interpreted with "parameter-file" semantics)
- Util_TableSetGeneric [E211] Set the value associated with a specified key to be a specified single (1-element array) value, whose data type is generic

Util_TableSetGenericArray

[\[E213\]](#) Set the value associated with a specified key to be a copy of a specified array, whose data type is generic

Util_TableSetString

[\[E215\]](#) Sets the value associated with a specified key in a table, to be a copy of a specified C-style null-terminated character string

E2.2 Full Descriptions of String Functions

Util_StrCmpi

Compare two strings, ignoring upper/lower case.

Synopsis

```
C          #include "util_String.h"
          int cmp = Util_StrCmpi(const char *str1, const char *str2);
```

Result

cmp An integer which is:

- < 0 if **str1** < **str2** in lexicographic order ignoring upper/lower case distinctions
- 0 if **str1** = **str2** ignoring upper/lower case distinctions
- > 0 if **str1** > **str2** in lexicographic order ignoring upper/lower case distinctions

Parameters

str1 A non-NULL pointer to a (C-style NUL-terminated) string to be compared.

str2 A non-NULL pointer to a (C-style NUL-terminated) string to be compared.

Discussion

The standard C library `strcmp()` function does a *case-sensitive* string comparison, i.e. `strcmp("cactus", "Cactus")` will find the two strings not equal. Sometimes it's useful to do *case-insensitive* string comparison, where upper/lower case distinctions are ignored. Many systems provide a `strcascmp()` or `strcmpi()` function to do this, but some systems don't, and even on those that do, the name isn't standardised. So, Cactus provides its own version, `Util_StrCmpi()`.

Notice that the return value of `Util_StrCmpi()`, like that of `strcmp()`, is zero (logical "false" in C) for equal strings, and nonzero (logical "true" in C) for non-equal strings. Code of the form

```
if (Util_StrCmpi(str1, str2))
    { /* strings differ */ }

or

if (!Util_StrCmpi(str1, str2))
    { /* strings are identical apart from case distinctions */ }
```

may be confusing to readers, because the sense of the comparison isn't immediately obvious. Writing an explicit comparison against zero makes things clearer:

```
if (Util_StrCmpi(str1, str2) != 0)
    { /* strings differ */ }

or

if (Util_StrCmpi(str1, str2) == 0)
    { /* strings are identical apart from case distinctions */ }
```

Unfortunately, the basic concept of "case-insensitive" string operations doesn't generalize well to non-English character sets,¹ where lower-case ↔ upper-case mappings may be context-dependent, many-to-one, and/or time-dependent.² At present Cactus basically ignores these issues. :(

¹Hawaiian and Swahili are apparently the only other living languages that use solely the 26-letter "English" Latin alphabet.

²For example, the (lower-case) German "ß" doesn't have a unique upper-case equivalent: "ß" usually maps to "SS" (for example "groß" ↔ "GROSS"), *but* if that would conflict with another word, then "ß" maps to "SZ" (for example "maße" ↔ "MASZE" because there's a different word "MASSE"). Or at least that's the way it was prior to 1998. The 1998 revisions to German orthography removed the SZ rule, so now (post-1998) the two distinct German words "masse" (English "mass") and "maße" ("measures") have identical upper-case forms "MASSE". To further complicate matters, (the German-speaking parts of) Switzerland have a slightly different orthography, which never had the SZ rule.

See Also

`strcmp()` Standard C library function (prototype in `<string.h>`) to compare two strings.

Examples

```
C                   #include "util_String.h"

                    /* does the Cactus parameter driver specify the PUGH driver? */
                    /* (Cactus parameters are supposed to be case-insensitive) */
                    if (Util_StrCmpi(driver, "pugh") == 0)
                        { /* PUGH code */ }
                    else
                        { /* non-PUGH code */ }
```

French provides another tricky example: In France “é” ↔ “É” and “è” ↔ “È”, whereas in (the French-speaking parts of) Canada there are no accents on upper-case letters, so “é” ↔ “E” and “è” ↔ “E”.

Util_Strdup

“Duplicate” a string, i.e. copy it to a newly-allocated buffer.

Synopsis

```
C          #include "util_String.h"
          char* copy = Util_Strdup(const char *str);
```

Result

copy A pointer to a buffer obtained from `malloc()`, which this function sets to a copy of the (C-style NUL-terminated) string `str`. This buffer should be freed with `free()` when it's not needed any more.

Parameters

str A non-NULL pointer to a (C-style NUL-terminated) string.

Discussion

Many systems have a C library function `strdup`, which `mallocs` sufficient memory for a copy of its argument string, does the copy, and returns a pointer to the copy. However, some systems lack this function, so Cactus provides its own version, `Util_Strdup()`.

See Also

<code><stdlib.h></code>	System header file containing prototypes for <code>malloc()</code> and <code>free()</code> .
<code>strcpy()</code>	Standard C library function (prototype in <code><string.h></code>) to copy a string to a buffer. <i>This does not check that the buffer is big enough to hold the string, and is thus very dangerous. Use <code>Util_Strncpy()</code> instead!</i>
<code>Util_Strncpy()</code>	Safely copy a string.

Errors

NULL `malloc()` was unable to allocate memory for the buffer.

Examples

```
C          #include "util_String.h"

          /*
           * return the (positive) answer to a question,
           * or negative if an error occurred
           */
          int answer_question(const char* question)
          {
              /*
               * we need to modify the question string in the process of parsing it
               * but we must not destroy the input ==> copy it and modify the copy
               *
               * ... note the const qualifier on question_copy says that
               *      the pointer question_copy won't itself change, but
               *      we can modify the string that it points to
               */
              char* const question_copy = Util_Strdup(question);
```

```
if (question_copy == NULL)
    { return -1; }      /* couldn't get memory for copy buffer */

/* code that will modify question_copy */

free(question_copy);
return 42;
}
```

Util_Strlcat

Concatenate strings safely.

Synopsis

```
C          #include "util_String.h"
          size_t result_len = Util_Strlcat(char *dst, const char *src, size_t size);
```

Result

result_len The size of the string the function tried to create, i.e. the initial `strlen(dst)` plus `strlen(src)`.

Parameters

dst A non-NULL pointer to the (C-style NUL-terminated) destination string.
src A non-NULL pointer to the (C-style NUL-terminated) source string.
size The size of the destination buffer.

Discussion

The standard `strcat()` and `strcpy()` functions provide no way to specify the size of the destination buffer, so code using these functions is often vulnerable to buffer overflows. The standard `strncat()` and `strncpy()` functions can be used to write safe code, but their API is cumbersome, error-prone, and sometimes surprisingly inefficient:

- Their **size** arguments are the number of characters *remaining* in the destination buffer, which must often be calculated at run-time, and is prone to off-by-one errors.
- `strncpy()` doesn't always NUL-terminate the destination string.
- `strncpy()` NUL-fills the remainder of the buffer not used for the source string; this NUL-filling can be *very* expensive.

To solve these problems, the OpenBSD project developed the `strlcat()` and `strlcpy()` functions. See <http://www.openbsd.org/papers/strlcpy-paper.ps> for a history and general discussion of these functions. Some other Unix systems (notably Solaris) now provide these, but many don't, so Cactus provides its own versions, `Util_Strlcat()` and `Util_Strlcpy()`.

`Util_Strlcat()` appends the NUL-terminated string **src** to the end of the NUL-terminated string **dst**. It will append at most `size - strlen(dst) - 1` characters (hence it never overflows the destination buffer), and it always leaves **dst** string NUL-terminated.

See Also

strcat() Standard C library function (prototype in `<string.h>`) to concatenate two strings. *This does not check that the buffer is big enough to hold the result, and is thus very dangerous. Use `Util_Strlcat()` instead!*

Util_Strlcpy() Safely copy a string.

Examples

```
C          #include "util_String.h"
```

C

```

/*
 * safely concatenate strings s1,s2,s3 into buffer:
 * ... this code is safe (it will never overflow the buffer), but
 *    quick-n-dirty in that it doesn't give any error indication
 *    if the result is truncated to fit in the buffer
 */
#define BUFFER_SIZE      1024
char buffer[BUFFER_SIZE];

Util_Strncpy(buffer, s1, sizeof(buffer));
Util_Strncat(buffer, s2, sizeof(buffer));
Util_Strncat(buffer, s3, sizeof(buffer));

#include "util_String.h"

#define OK                0
#define ERROR_TRUNC      1

/*
 * safely concatenate strings s1,s2,s3 into buffer[N_buffer];
 * return OK if ok, ERROR_TRUNC if result was truncated to fit in buffer
 */
int cat3(int N_buffer, char buffer[],
         const char s1[], const char s2[], const char s3[])
{
    int length;

    length = Util_Strncpy(buffer, s1, N_buffer);
    if (length >= N_buffer)
        return ERROR_TRUNC;                /*** ERROR EXIT ***/

    length = Util_Strncat(buffer, s2, N_buffer);
    if (length >= N_buffer)
        return ERROR_TRUNC;                /*** ERROR EXIT ***/

    length = Util_Strncat(buffer, s3, N_buffer);
    if (length >= N_buffer)
        return ERROR_TRUNC;                /*** ERROR EXIT ***/

    return OK;                             /*** NORMAL RETURN ***/
}

```

Util_Strlcpy

Copies a string safely.

Synopsis

```
C          #include "util_String.h"
          size_t result_len = Util_Strlcpy(char *dst, const char *src, size_t size);
```

Result

result_len The size of the string the function tried to create, i.e. `strlen(src)`.

Parameters

dst A non-NULL pointer to the (C-style NUL-terminated) destination string.

src A non-NULL pointer to the (C-style NUL-terminated) source string.

size The size of the destination buffer.

Discussion

The standard `strcat()` and `strcpy()` functions provide no way to specify the size of the destination buffer, so code using these functions is often vulnerable to buffer overflows. The standard `strncat()` and `strncpy()` functions can be used to write safe code, but their API is cumbersome, error-prone, and sometimes surprisingly inefficient:

- Their **size** arguments are the number of characters *remaining* in the destination buffer, which must often be calculated at run-time, and is prone to off-by-one errors.
- `strncpy()` doesn't always NUL-terminate the destination string.
- `strncpy()` NUL-fills the remainder of the buffer not used for the source string; this NUL-filling can be *very* expensive.

To solve these problems, the OpenBSD project developed the `strlcat()` and `strlcpy()` functions. See <http://www.openbsd.org/papers/strlcpy-paper.ps> for a history and general discussion of these functions. Some other Unix systems (notably Solaris) now provide these, but many don't, so Cactus provides its own versions, `Util_Strlcat()` and `Util_Strlcpy()`.

`Util_Strlcpy()` copies up to **size-1** characters from the source string to the destination string, followed by a NUL character (so **dst** is always NUL-terminated). Unlike `strncpy()`, `Util_Strlcpy()` does *not* fill any left-over space at the end of the destination buffer with NUL characters.

See Also

strcpy() Standard C library function (prototype in `<string.h>`) to copy a string to a buffer. *This does not check that the buffer is big enough to hold the string, and is thus very dangerous. Use Util_Strlcpy() instead!*

Util_Strdup() “Duplicate” a string, i.e. copy it to a newly-allocated buffer.

Util_Strlcat() Safely concatenates two strings.

Examples

```
C          #include "util_String.h"
```

```

/*
 * safely concatenate strings s1,s2,s3 into buffer:
 * ... this code is safe (it will never overflow the buffer), but
 *    quick-n-dirty in that it doesn't give any error indication
 *    if the result is truncated to fit in the buffer
 */
#define BUFFER_SIZE      1024
char buffer[BUFFER_SIZE];

Util_Strncpy(buffer, s1, sizeof(buffer));
Util_Strncat(buffer, s2, sizeof(buffer));
Util_Strncat(buffer, s3, sizeof(buffer));
#include "util_String.h"

#define OK                0
#define ERROR_TRUNC      1

/*
 * safely concatenate strings s1,s2,s3 into buffer[N_buffer];
 * return OK if ok, ERROR_TRUNC if result was truncated to fit in buffer
 */
int cat3(int N_buffer, char buffer[],
         const char s1[], const char s2[], const char s3[])
{
    int length;

    length = Util_Strncpy(buffer, s1, N_buffer);
    if (length >= N_buffer)
        return ERROR_TRUNC;                /*** ERROR EXIT ***/

    length = Util_Strncat(buffer, s2, N_buffer);
    if (length >= N_buffer)
        return ERROR_TRUNC;                /*** ERROR EXIT ***/

    length = Util_Strncat(buffer, s3, N_buffer);
    if (length >= N_buffer)
        return ERROR_TRUNC;                /*** ERROR EXIT ***/

    return OK;                             /*** NORMAL RETURN ***/
}

```

E2.3 Full Descriptions of Table Functions

Util_TableClone

Creates a new table which is a “clone” (exact copy) of an existing table

Synopsis

```
C          #include "util_ErrorCodes.h"
           #include "util_Table.h"
           int clone_handle = Util_TableClone(int handle);

Fortran    call Util_TableClone(clone_handle, handle)
           integer clone_handle, handle
```

Result

clone_handle (≥ 0)
A handle to the clone table

Parameters

handle Handle to the table to be cloned

Discussion

Viewing a table as a set of key/value pairs, this function creates a new table (with the same flags word as the original) containing copies of all the original table’s key/value pairs. The two tables are completely independent, i.e. future changes to one won’t affect the other.

Note that table iterators are *not* guaranteed to sequence through the original and clone tables in the same order. (This is a special case of the more general “non-guarantee” in the Section of table iterators in the Users’ Guide: the order of table iterators may differ even between different tables with identical key/value contents.)

See Also

Util_TableCreate()	create a table
Util_TableCreateFromString()	convenience routine to create a table and set key/value entries in it based on a parameter-file-like character string
Util_TableDestroy()	destroy a table

Errors

UTIL_ERROR_NO_MEMORY	unable to allocate memory
UTIL_ERROR_TABLE_BAD_FLAGS	flags word is negative in the to-be-cloned table (this indicates an internal error in the table routines, and should never happen)

Examples

```
C          #include "util_ErrorCodes.h"
           #include "util_Table.h"

           /*
            * This function is passed (a handle to) a table containing some entries.
            * It needs to set some additional entries and pass the table to some
            * other function(s), but it also needs to leave the original table
            * intact for other use by the caller. The solution is to clone the
            * original table and work on the clone, leaving the original table
            * unchanged.
```

```
    */
    int my_function(int handle, int x, int y)
    {
        int status;

        /* clone the table */
        const int clone_handle = Util_TableClone(handle)
        if (clone_handle < 0)
            return clone_handle;                /* error in cloning table */

        /* now set our entries in the clone table */
        status = Util_TableSetInt(clone_handle, x, "x");
        if (status < 0)
            return status;                      /* error in setting x */
        status = Util_TableSetInt(clone_handle, y, "y");
        if (status < 0)
            return status;                      /* error in setting y */

        /* ... code to use the clone table ... */
        /* ... eg pass clone_handle to other functions ... */

        /* we're done with the clone now */
        Util_TableDestroy(clone_handle);
        return 0;
    }
```

Util_TableCreate

Creates a new (empty) table

Synopsis

C	<pre>#include "util_ErrorCodes.h" #include "util_Table.h" int handle = Util_TableCreate(int flags);</pre>
Fortran	<pre>call Util_TableCreate(handle, flags) integer handle, flags</pre>

Result

handle (≥ 0)	A handle to the newly-created table
---------------------	-------------------------------------

Parameters

flags (≥ 0)	A flags word for the table. This should be the inclusive-or of zero or more of the UTIL_TABLE_FLAGS_* bit masks (defined in "util_Table.h"). For Fortran users, note that inclusive-or is the same as sum here, since the bit masks are all disjoint.
--------------------	---

Discussion

We require the flags word to be non-negative so that other functions can distinguish flags from (negative) error codes.

Any User-defined flag words should use only bit positions at or above UTIL_TABLE_FLAGS_USER_DEFINED_BASE, i.e. all bit positions below this are reserved for present or future Cactus use.

At present there is only a single flags-word bit mask defined in "util_Table.h":

UTIL_TABLE_FLAGS_CASE_INSENSITIVE

By default keys are treated as C-style character strings, and the table functions compare them with the standard C `strcmp()` function. However, by setting the UTIL_TABLE_FLAGS_CASE_INSENSITIVE bit in the flags word, this table's keys may be made case-insensitive, i.e. the table routines then compare this table's keys with `Util_StrCmpi()`. Note that keys are still *stored* exactly as the caller specifies them (i.e. they are *not* forced into a canonical case); it's only their *comparison* that's affected by this flag.

See Also

Util_StrCmpi()	compare two strings, ignoring upper/lower case
Util_TableClone()	create a new table which is a "clone" (exact copy) of an existing table
Util_TableCreateFromString()	convenience routine to create a table and set key/value entries in it based on a parameter-file-like character string
Util_TableDestroy()	destroy a table

Errors

UTIL_ERROR_NO_MEMORY	unable to allocate memory
UTIL_ERROR_TABLE_BAD_FLAGS	flags word is negative

Examples

C	<pre>#include "util_ErrorCodes.h"</pre>
----------	---

```
#include "util_Table.h"

/* create a table, simplest case */
int handle = Util_TableCreate(UTIL_TABLE_FLAGS_DEFAULT);

/* create a table whose keys will be treated as case-insensitive */
int handle2 = Util_TableCreate(UTIL_TABLE_FLAGS_CASE_INSENSITIVE);
```

Util_TableCreateFromString

Creates a new table (with the case-insensitive flag set) and sets values in it based on a string argument (interpreted with “parameter-file” semantics)

Synopsis

```
C          #include "util_ErrorCodes.h"
          #include "util_Table.h"
          int handle = Util_TableCreateFromString(const char *string);

Fortran    call Util_TableCreateFromString(handle, string)
          integer      handle
          character*(*) string
```

Result

handle (≥ 0) a handle to the newly-created table

Parameters

string a pointer to a C-style null-terminated string specifying the table contents; see the description for `Util_TableSetFromString()` for a full description of the syntax and semantics of this string

See Also

<code>Util_TableClone()</code>	Create a new table which is a “clone” (exact copy) of an existing table
<code>Util_TableCreate()</code>	create a table
<code>Util_TableSetFromString()</code>	sets values in a table based on a string argument
<code>Util_TableSetInt()</code>	store a CCTK_INT value in a table
<code>Util_TableSetReal()</code>	store a CCTK_REAL value in a table

Errors

<code>UTIL_ERROR_NO_MEMORY</code>	unable to allocate memory
<code>UTIL_ERROR_BAD_KEY</code>	invalid input: key contains invalid character
<code>UTIL_ERROR_BAD_INPUT</code>	invalid input: can’t parse input string
other error codes	this function may also return any error codes returned by <code>Util_TableCreate()</code> or <code>Util_TableSetFromString()</code>

Examples

```
C          #include "util_ErrorCodes.h"
          #include "util_Table.h"

          int handle = Util_TableCreateFromString("order = 3\t"
                                                  "myreal = 42.314159\t"
                                                  "mystring = 'hello'\t"
                                                  "myarray = { 0 1 2 3 }");

          /* equivalent code to the above */
          int handle = Util_TableCreate(UTIL_TABLE_FLAGS_CASE_INSENSITIVE);
          Util_TableSetFromString(handle, "order = 3\t"
                                     "myreal = 42.314159\t"
                                     "mystring = 'hello'")
```

```
                                "myarray = { 0 1 2 3 }");

/* also equivalent to the above */
int handle = Util_TableCreate(UTIL_TABLE_FLAGS_CASE_INSENSITIVE);
CCTK_INT array[] = {0, 1, 2, 3};

Util_TableSetInt(handle, 3, "order");
Util_TableSetReal(handle, 42.314159, "myreal");
Util_TableSetString(handle, "hello", "mystring");
Util_TableSetIntArray(handle, 4, array, "myarray");
```

Util_TableDeleteKey

Deletes a specified key/value entry from a table

Synopsis

C	<pre>#include "util_ErrorCodes.h" #include "util_Table.h" int key_exists = Util_TableDeleteKey(int handle, const char *key);</pre>
Fortran	<pre>call Util_TableDeleteKey(key_exists, handle, key) integer key_exists, handle character(*) key</pre>

Result

0	ok (key existed before this call, and has now been deleted)
---	---

Parameters

handle (≥ 0)	handle to the table
key	a pointer to the key (a C-style null-terminated string)

Discussion

This function invalidates any iterators for the table which are not in the “null-pointer” state.

Errors

UTIL_ERROR_BAD_HANDLE	handle is invalid
UTIL_ERROR_TABLE_BAD_KEY	key contains '/' character
UTIL_ERROR_TABLE_NO_SUCH_KEY	no such key in table

Util_TableDestroy

Destroys a table

Synopsis

```
C          #include "util_ErrorCodes.h"
           #include "util_Table.h"
           int status = Util_TableDestroy(int handle);

Fortran    call Util_TableDestroy(status, handle)
           integer status, handle
```

Result

0 ok

Parameters

handle (≥ 0) handle to the table

Discussion

Of course, this function invalidates any and all iterators for the table. :)

See Also

Util_TableClone()	Create a new table which is a “clone” (exact copy) of an existing table
Util_TableCreate()	create a table
Util_TableCreateFromString()	convenience routine to create a table and set key/value entries in it based on a parameter-file-like character string

Errors

UTIL_ERROR_BAD_HANDLE handle is invalid

Examples

```
C          #include "util_ErrorCodes.h"
           #include "util_Table.h"

           /* create a table */
           int handle = Util_TableCreate(UTIL_TABLE_FLAGS_DEFAULT);

           /* do things with the table: put values in it, */
           /* pass its handle to other functions, etc etc */
           /* ... */

           /* at this point we (and all other functions we */
           /* may call in the future) are done with the table */
           Util_TableDestroy(handle);
```

Util_TableGet*

This is a family of functions, one for each Cactus data type, to get the single (1-element array) value, or more generally the first array element of the value, associated with a specified key in a key/value table.

Synopsis

C `#include "util_ErrorCodes.h"`
 `#include "util_Table.h"`
 `int N_elements = Util_TableGetXxx(int handle,`
 `CCTK_XXX *value,`
 `const char *key);`

where XXX is one of POINTER, FPOINTER³, CHAR, INT, INT2, INT4, INT8, REAL, REAL4, REAL8, REAL16, COMPLEX, COMPLEX8, COMPLEX16, COMPLEX32 (not all of these may be supported on any given system)

Fortran `call Util_TableGetXxx(N_elements, handle, value, key)`
 `integer            N_elements, handle`
 `CCTK_XXX        value`
 `character*(*)    key`

Result

N_elements the number of array elements in the value

Parameters

handle (≥ 0) handle to the table

value a pointer to where this function should store a copy of the value (or more generally the first array element of the value) associated with the specified key, or NULL pointer to skip storing this

key a pointer to the key (a C-style null-terminated string)

Discussion

Note that it is *not* an error for the value to actually have > 1 array elements; in this case only the first element is stored. The rationale for this design is that the caller may know or suspect that the value is a large array, but may only want the first array element; in this case this design avoids the caller having to allocate a large buffer unnecessarily.

In contrast, it *is* an error for the value to actually be an empty (0-length) array, because then there is no “first array element” to get.

It is also an error for the value to actually have a different type than CCTK_XXX.

If any error code is returned, the user’s value buffer (pointed to by **value** if this is non-NULL) is unchanged.

See Also

Util_TableCreateFromString()	convenience routine to create a table and set key/value entries in it based on a parameter-file-like character string
Util_TableGet*Array()	get an array value
Util_TableGet*String()	get a character-string value
Util_TableSet*()	set a single (1-element array) value
Util_TableSet*Array()	set an array value

³For historical reasons the corresponding function is called **Util_TableGetFnPointer**. This should be changed soon.

<code>Util_TableSetGeneric()</code>	set a single (1-element array) value with generic data type
<code>Util_TableSetGenericArray()</code>	set an array value with generic data type
<code>Util_TableSetFromString()</code>	convenience routine to set key/value entries in a table based on a parameter-file-like character string
<code>Util_TableSetString()</code>	set a character-string value

Errors

<code>UTIL_ERROR_BAD_HANDLE</code>	handle is invalid
<code>UTIL_ERROR_TABLE_BAD_KEY</code>	key contains '/' character
<code>UTIL_ERROR_TABLE_NO_SUCH_KEY</code>	no such key in table
<code>UTIL_ERROR_TABLE_WRONG_DATA_TYPE</code>	value has data type other than <code>CCTK_TYPE</code>
<code>UTIL_ERROR_TABLE_VALUE_IS_EMPTY</code>	value is an empty (0-element) array

Examples

```
C      #include "util_ErrorCodes.h"
      #include "util_Table.h"

      #define N_DIGITS      5
      static const CCTK_INT pi_digits[N_DIGITS] = {3, 14, 159, 2653, 58979};

      int N;
      CCTK_INT x;
      int handle = Util_TableCreate(UTIL_TABLE_FLAGS_DEFAULT);

      Util_TableSetIntArray(handle, N_DIGITS, pi_digits, "digits of pi");
      Util_TableSetIntArray(handle, 0, pi_digits, "empty array");

      /* gets N = 5, x = 3 */
      N = Util_TableGetInt(handle, &x, "digits of pi");

      /* gets N = UTIL_ERROR_TABLE_VALUE_IS_EMPTY */
      N = Util_TableGetInt(handle, &x, "empty array");
```


Util_TableGet*Array

This is a family of functions, one for each Cactus data type, to get a copy of the value associated with a specified key, and store it (more accurately, as much of it as will fit) in a specified array

Synopsis

```
C
#include "util_ErrorCodes.h"
#include "util_Table.h"
int N_elements = Util_TableGetXxxArray(int handle,
                                         int N_array, CCKT_XXX array[],
                                         const char *key);
```

where XXX is one of POINTER, FNPOINTER ⁴, CHAR, INT, INT2, INT4, INT8, REAL, REAL4, REAL8, REAL16, COMPLEX, COMPLEX8, COMPLEX16, COMPLEX32 (not all of these may be supported on any given system)

```

Fortran      call Util_TableGetXxxArray(N_elements, handle, N_array, array, key)
                integer          N_elements, handle, N_array
                CCTK_XXX(*)      array
                character*(*)    key

```

Result

N_elements	the number of array elements in the value
------------	---

Parameters

<code>handle</code> (≥ 0)	handle to the table
<code>N_array</code>	the number of array elements in <code>array[]</code> (must be ≥ 0 if <code>array</code> $\neq$ NULL)
<code>array</code>	a pointer to where this function should store (up to <code>N_array</code> elements of) a copy of the value associated with the specified key, or NULL pointer to skip storing this
<code>key</code>	a pointer to the key (a C-style null-terminated string)

Discussion

Note that it is *not* an error for the value to actually have `> N_array` array elements; in this case only the first `N_array` elements are stored. The caller can detect this by comparing the return value with `N_array`. The rationale for this design is that the caller may know or suspect that the value is a large array, but may only want the first few array elements; in this case this design avoids the caller having to allocate a large buffer unnecessarily.

It is also *not* an error for the value to actually have `< N_array` array elements; again the caller can detect this by comparing the return value with `N_array`.

It *is* an error for the value to actually have a different type than CCTLK_XXX.

If any error code is returned, the user's value buffer (pointed to by `array` if this is non-NULL) is unchanged.

See Also

<code>Util_TableCreateFromString()</code>	convenience routine to create a table and set key/value entries in it based on a parameter-file-like character string
<code>Util_TableGet*()</code>	get a single (1-element array) value, or more generally the first array element of an array value
<code>Util_TableGetGeneric()</code>	get a single (1-element array) value with generic data type

⁴Use `Util_TableGetFnPointer`, for historical reasons.

<code>Util_TableGetGenericArray()</code>	get an array value with generic data type
<code>Util_TableGet*String()</code>	get a character-string value
<code>Util_TableSet*()</code>	set a single (1-element array) value
<code>Util_TableSet*Array()</code>	set an array value
<code>Util_TableSetGeneric()</code>	set a single (1-element array) value with generic data type
<code>Util_TableSetGenericArray()</code>	set an array value with generic data type
<code>Util_TableSetFromString()</code>	convenience routine to set key/value entries in a table based on a parameter-file-like character string
<code>Util_TableSetString()</code>	set a character-string value

Errors

<code>UTIL_ERROR_BAD_HANDLE</code>	handle is invalid
<code>UTIL_ERROR_TABLE_BAD_KEY</code>	key contains '/' character
<code>UTIL_ERROR_BAD_INPUT</code>	<code>array != NULL</code> and <code>N_array < 0</code>
<code>UTIL_ERROR_TABLE_NO_SUCH_KEY</code>	no such key in table
<code>UTIL_ERROR_TABLE_WRONG_DATA_TYPE</code>	value has data type other than <code>CCTK_TYPE</code>

Examples

```

C      #include "util_ErrorCodes.h"
      #include "util_Table.h"

      #define N_STUFF          3
      static const CCTK_REAL stuff[N_STUFF] = {42.0, 69.0, 105.5};

      #define N_OUTPUT          2
      CCTK_INT output[N_OUTPUT];

      int N;
      int handle = Util_TableCreate(UTIL_TABLE_FLAGS_DEFAULT);

      Util_TableSetRealArray(handle, N_STUFF, stuff, "blah blah blah");

      /* gets N = 3, output[0] = 42.0, output[1] = 69.0 */
      N = Util_TableGetRealArray(handle, N_OUTPUT, output, "blah blah blah");

```

Util_TableGetGeneric

Get the single (1-element array) value, or more generally the first array element of the value, associated with a specified key in a key/value table; the value's data type is generic. That is, the value is specified by a CCTK_VARIABLE_* type code and a void * pointer.

Synopsis

```
C          #include "util_ErrorCodes.h"
          #include "util_Table.h"
          int N_elements = Util_TableGetGeneric(int handle,
                                              int type_code,
                                              void *value,
                                              const char *key);

Fortran    call Util_TableGetGeneric(N_elements, handle, type_code, value, key)
          integer      N_elements, handle, type_code
          CCTK_POINTER  value
          character*(*) key
```

Result

N_elements the number of array elements in the value

Parameters

handle (≥ 0) handle to the table

type_code the value's type code (one of the CCTK_VARIABLE_* constants from "cctk_Constants.h")

value a pointer to where this function should store a copy of the value (or more generally the first array element of the value) associated with the specified key, or NULL pointer to skip storing this

key a pointer to the key (a C-style null-terminated string)

Discussion

Note that it is *not* an error for the value to actually have > 1 array elements; in this case only the first element is stored. The rationale for this design is that the caller may know or suspect that the value is a large array, but may only want the first array element; in this case this design avoids the caller having to allocate a large buffer unnecessarily.

In contrast, it *is* an error for the value to actually be an empty (0-length) array, because then there is no "first array element" to get.

It is also an error for the value to actually have a different type than that specified by type_code.

If any error code is returned, the user's value buffer (pointed to by value if this is non-NULL) is unchanged.

See Also

Util_TableCreateFromString()	convenience routine to create a table and set key/value entries in it based on a parameter-file-like character string
Util_TableGet*()	get a single (1-element array) value
Util_TableGet*Array()	get an array value
Util_TableGet*String()	get a character-string value
Util_TableItQueryValueInfo()	query key present/absent in table, and optionally type and/or number of elements, using an iterator

<code>Util_TableQueryValueInfo()</code>	query key present/absent in table, and optionally type and/or number of elements
<code>Util_TableSet*()</code>	set a single (1-element array) value
<code>Util_TableSet*Array()</code>	set an array value
<code>Util_TableSetGeneric()</code>	set a single (1-element array) value with generic data type
<code>Util_TableSetGenericArray()</code>	set an array value with generic data type
<code>Util_TableSetFromString()</code>	convenience routine to set key/value entries in a table based on a parameter-file-like character string
<code>Util_TableSetString()</code>	set a character-string value

Errors

<code>UTIL_ERROR_BAD_HANDLE</code>	handle is invalid
<code>UTIL_ERROR_TABLE_BAD_KEY</code>	key contains '/' character
<code>UTIL_ERROR_TABLE_NO_SUCH_KEY</code>	no such key in table
<code>UTIL_ERROR_TABLE_WRONG_DATA_TYPE</code>	value has data type other than <code>CCTK_TYPE</code>
<code>UTIL_ERROR_TABLE_VALUE_IS_EMPTY</code>	value is an empty (0-element) array

Examples

```

C      #include "util_ErrorCodes.h"
      #include "util_Table.h"
      #include "cctk_Constants.h"

      #define N_DIGITS      5
      static const CCTK_INT pi_digits[N_DIGITS] = {3, 14, 159, 2653, 58979};

      int N;
      CCTK_INT x;
      void *xptr = (void *) &x;
      int handle = Util_TableCreate(UTIL_TABLE_FLAGS_DEFAULT);

      Util_TableSetIntArray(handle, N_DIGITS, pi_digits, "digits of pi");
      Util_TableSetIntArray(handle, 0, pi_digits, "empty array");

      /* gets N = 5, x = 3 */
      N = Util_TableGetGeneric(handle, CCTK_VARIABLE_INT, &x, "the answer");

      /* gets N = UTIL_ERROR_TABLE_VALUE_IS_EMPTY, leaves x unchanged */
      N = Util_TableGetGeneric(handle, CCTK_VARIABLE_INT, &x, "empty array");

```

Util_TableGetGenericArray

Get a copy of the value associated with a specified key, and store it (more accurately, as much of it as will fit) in a specified array; the array's data type is generic. That is the array is specified by a CCTK_VARIABLE_* type code, a count of the number of array elements, and a void * pointer.

Synopsis

```
C      #include "util_ErrorCodes.h"
      #include "util_Table.h"
      int N_elements = Util_TableGetGenericArray(int handle,
                                                int type_code,
                                                int N_array, void *array,
                                                const char *key);

Fortran  call Util_TableGetGenericArray(N_elements,
      .           handle,
      .           type_code,
      .           N_array, array,
      .           key)
integer      N_elements, handle, type_code, N_array
CCTK_POINTER array
character(*) key
```

Result

N_elements the number of array elements in the value

Parameters

handle (≥ 0) handle to the table

type_code the value's type code (one of the CCTK_VARIABLE_* constants from "cctk_Constants.h")

N_array the number of array elements in array[] (must be ≥ 0 if array != NULL)

array a pointer to where this function should store (up to N_array elements of) a copy of the value associated with the specified key, or NULL pointer to skip storing this

key a pointer to the key (a C-style null-terminated string)

Discussion

Note that it is *not* an error for the value to actually have $> N_array$ array elements; in this case only the first N_array elements are stored. The caller can detect this by comparing the return value with N_array. The rationale for this design is that the caller may know or suspect that the value is a large array, but may only want the first few array elements; in this case this design avoids the caller having to allocate a large buffer unnecessarily.

It is also *not* an error for the value to actually have $< N_array$ array elements; again the caller can detect this by comparing the return value with N_array.

It *is* an error for the value to actually have a different type than that specified by type_code.

If any error code is returned, the user's value buffer (pointed to by array if this is non-NULL) is unchanged.

See Also

Util_TableCreateFromString() convenience routine to create a table and set key/value entries in it based on a parameter-file-like character string

<code>Util_TableGet*()</code>	get a single (1-element array) value, or more generally the first array element of an array value
<code>Util_TableGetGeneric()</code>	get a single (1-element array) value with generic data type
<code>Util_TableGetGenericArray()</code>	get an array value with generic data type
<code>Util_TableGet*String()</code>	get a character-string value
<code>Util_TableItQueryValueInfo()</code>	query key present/absent in table, and optionally type and/or number of elements, using an iterator
<code>Util_TableQueryValueInfo()</code>	query key present/absent in table, and optionally type and/or number of elements
<code>Util_TableSet*()</code>	set a single (1-element array) value
<code>Util_TableSet*Array()</code>	set an array value
<code>Util_TableSetGeneric()</code>	set a single (1-element array) value with generic data type
<code>Util_TableSetGenericArray()</code>	set an array value with generic data type
<code>Util_TableSetFromString()</code>	convenience routine to set key/value entries in a table based on a parameter-file-like character string
<code>Util_TableSetString()</code>	set a character-string value

Errors

<code>UTIL_ERROR_BAD_HANDLE</code>	handle is invalid
<code>UTIL_ERROR_TABLE_BAD_KEY</code>	key contains '/' character
<code>UTIL_ERROR_BAD_INPUT</code>	<code>array != NULL</code> and <code>N_array < 0</code>
<code>UTIL_ERROR_TABLE_NO_SUCH_KEY</code>	no such key in table
<code>UTIL_ERROR_TABLE_WRONG_DATA_TYPE</code>	value has data type other than <code>CCTK_TYPE</code>

Examples

```

C      #include "util_ErrorCodes.h"
      #include "util_Table.h"

      #define N_STUFF          3
      static const CCTK_REAL stuff[N_STUFF] = {42.0, 69.0, 105.5};

      #define N_OUTPUT          2
      CCTK_INT output[N_OUTPUT];

      int N;
      int handle = Util_TableCreate(UTIL_TABLE_FLAGS_DEFAULT);

      Util_TableSetRealArray(handle, N_STUFF, stuff, "stuff");

      /* gets N = UTIL_ERROR_TABLE_WRONG_DATA_TYPE, output[] unchanged */
      N = Util_TableGetGenericArray(handle,
                                   CCTK_VARIABLE_INT,
                                   N_OUTPUT, output,
                                   "stuff");

      /* gets N = 3, output[0] = 42.0, output[1] = 69.0 */
      N = Util_TableGetGenericArray(handle,
                                   CCTK_VARIABLE_REAL,
                                   N_OUTPUT, output,
                                   "stuff");

```

Util_TableGetString

Gets a copy of the character-string value associated with a specified key in a table, and stores it (more accurately, as much of it as will fit) in a specified character string

Synopsis

```
C          #include "util_ErrorCodes.h"
          #include "util_Table.h"
          int length = Util_TableGetString(int handle,
                                          int buffer_length, char buffer[],
                                          const char *key);
```

Result

Results are the same as all the other `Util_TableGet*` functions:

length the length of the string (C `strlen()` semantics, i.e. *not* including the terminating null character)

Parameters

handle (≥ 0) handle to the table

buffer_length the length (`sizeof()`) of `buffer[]` (must be ≥ 1 if `buffer != NULL`)

buffer a pointer to a buffer into which this function should store (at most `buffer_length-1` characters of) the value, terminated by a null character as usual for C strings, or NULL pointer to skip storing this

key a pointer to the key (a C-style null-terminated string)

Discussion

This function assumes that the string is stored as an array of `CCTK_CHARS`, *not* including a terminating null character.

This function differs from `Util_TableGetCharArray()` in two ways: It explicitly provides a terminating null character for C-style strings, and it explicitly checks for the string being too long to fit in the buffer (in which case it returns `UTIL_ERROR_TABLE_STRING_TRUNCATED`).

If the error code `UTIL_ERROR_TABLE_STRING_TRUNCATED` is returned, then the first `buffer_length-1` characters of the string are returned in the user's buffer (assuming `buffer` is non-NULL), followed by a null character to properly terminate the string in the buffer. If any other error code is returned, the user's value buffer (pointed to by `buffer` if this is non-NULL) is unchanged.

See Also

<code>Util_TableCreateFromString()</code>	convenience routine to create a table and set key/value entries in it based on a parameter-file-like character string
<code>Util_TableGet*()</code>	get a single (1-element array) value, or more generally the first array element of an array value
<code>Util_TableGet*Array()</code>	get an array value
<code>Util_TableGetCharArray()</code>	get an array-of- <code>CCTK_CHAR</code> value
<code>Util_TableGetGeneric()</code>	get a single (1-element array) value with generic data type
<code>Util_TableGetGenericArray()</code>	get an array value with generic data type
<code>Util_TableSet*()</code>	set a single (1-element array) value
<code>Util_TableSet*Array()</code>	set an array value

<code>Util_TableSetGeneric()</code>	set a single (1-element array) value with generic data type
<code>Util_TableSetGenericArray()</code>	set an array value with generic data type
<code>Util_TableSetString()</code>	set a character-string value
<code>Util_TableSetFromString()</code>	convenience routine to set key/value entries in a table based on a parameter-file-like character string
<code>Util_TableSetCharArray()</code>	set an array-of-CCTK_CHAR value

Errors

<code>UTIL_ERROR_BAD_HANDLE</code>	handle is invalid
<code>UTIL_ERROR_TABLE_BAD_KEY</code>	key contains '/' character
<code>UTIL_ERROR_BAD_INPUT</code>	<code>buffer != NULL</code> and <code>buffer_length ≤ 0</code>
<code>UTIL_ERROR_TABLE_NO_SUCH_KEY</code>	no such key in table
<code>UTIL_ERROR_TABLE_WRONG_DATA_TYPE</code>	value has data type other than CCTK_CHAR
<code>UTIL_ERROR_TABLE_STRING_TRUNCATED</code>	<code>buffer != NULL</code> and value was truncated to fit in <code>buffer[]</code>

Examples

```
C      #include "util_ErrorCodes.h"
      #include "util_Table.h"

      #define N_BUFFER      100
      char buffer[N_BUFFER];

      int handle = Util_TableCreate(UTIL_TABLE_FLAGS_DEFAULT);
      Util_TableSetString(handle, "relativity", "Einstein");

      /* get length of string (= 10 here) */
      int length = Util_TableGetString(handle, 0, NULL, "Einstein");

      /* get null-terminated string into buffer, also returns 10 */
      Util_TableGetString(handle, N_BUFFER, buffer, "Einstein");
```


Util_TableItAdvance

Advance a table iterator to the next entry in the table

Synopsis

```
C          #include "util_ErrorCodes.h"
          #include "util_Table.h"
          int is_nonnull = Util_TableItAdvance(int ihandle);
```

Result

1	ok (iterator now points to some table entry)
0	ok (iterator has just advanced past the last table entry, and is now in the "null-pointer" state)

Parameters

ihandle (≥ 0)	handle to the iterator
----------------------	------------------------

Discussion

If we view an iterator as an abstraction of a pointer into the table, then this function is the abstraction of the C++ operation applied to the pointer, except that this function automatically sets the iterator to the "null-pointer" state when it advances past the last table entry.

Note that bad things (garbage results, core dumps) may happen if you call this function on an iterator which has been invalidated by a change in the table's contents.

Errors

UTIL_ERROR_BAD_HANDLE	iterator handle is invalid
-----------------------	----------------------------

Examples

```
C          /* walk through all entries of a table */
          int ihandle;

          for ( ihandle = Util_TableItCreate(handle) ;
              Util_TableItQueryIsNonNull(ihandle) > 0 ;
              Util_TableItAdvance(ihandle) )
          {
              /* do something with the table entry */
          }

          Util_TableItDestroy(ihandle);
```

Util_TableItClone

Creates a new table iterator which is a “clone” (exact copy) of an existing table iterator

Synopsis

```
C          #include "util_ErrorCodes.h"
          #include "util_Table.h"
          int clone_ihandle = Util_TableItClone(int ihandle);
```

Result

clone_ihandle (≥ 0)
A handle to the clone table iterator

Parameters

ihandle Handle to the table iterator to be cloned

Discussion

This function creates a new iterator which points to the same place in the same table as the original iterator. If the original iterator is in the “null-pointer” state, then the clone is also in this state.

Note that bad things (garbage results, core dumps) may happen if you call this function on an iterator which has been invalidated by a change in the table’s contents.

See Also

Util_TableClone()	create a new table which is a “clone” (exact copy) of an existing table
Util_TableItCreate()	create a table iterator
Util_TableItDestroy()	destroy a table iterator

Errors

UTIL_ERROR_BAD_HANDLE	iterator handle to be cloned, is invalid
UTIL_ERROR_NO_MEMORY	unable to allocate memory

Examples

```
C          #include "util_ErrorCodes.h"
          #include "util_Table.h"

          /*
           * Apart from efficiency and slight differences in error return codes,
           * Util_TableItClone() could be simulated by the following code.
           */
          int Util_TableItClone(int ihandle)
          {
              int status;

              /* to what table does the to-be-cloned iterator point? */
              const int handle = Util_TableQueryTableHandle(ihandle);
              if (handle < 0)
                  return handle;                      /* error in querying table handle */

              /* create the to-be-cloned iterator */
```

```

/* (pointing into the same table as the original iterator) */
{
    const int clone_ihandle = Util_TableItCreate(handle);
    if (clone_ihandle < 0)
        return clone_ihandle;          /* error in creating clone iterator */

    /* how long is the key to which the to-be-cloned iterator points? */
    {
        const int key_length = Util_TableItQueryKeyValueInfo(ihandle,
                                                             0, NULL,
                                                             NULL, NULL);

        if (key_length == UTIL_TABLE_ITERATOR_IS_NULL)
        {
            /* to-be-cloned iterator is in "null-pointer" state */
            Util_TableItSetToNull(clone_ihandle);
            return clone_ihandle;          /* normal return */
        }
        if (key_length < 0)
            return key_length;          /* error in querying to-be-cloned iterator */

        /* to what key does the to-be-cloned iterator point? */
        {
            const int key_buffer_length = key_length + 1;
            char *const key_buffer = (char *) malloc(key_buffer_length);
            if (key_buffer == NULL)
                return UTIL_ERROR_NO_MEMORY;
            status = Util_TableItQueryKeyValueInfo(ihandle,
                                                  key_buffer_length, key_buffer);

            if (status < 0)
                return status;          /* error in querying to-be-cloned iterator */

            /* set the clone iterator to point to the same key as the original */
            status = Util_TableItSetToKey(clone_ihandle, key_buffer);
            free(key_buffer);
            return clone_ihandle;          /* normal return */
        }
    }
}

```

Util_TableItCreate

Create a new table iterator

Synopsis

```
C          #include "util_ErrorCodes.h"
          #include "util_Table.h"
          int ihandle = Util_TableItCreate(int handle);
```

Result

ihandle (≥ 0) handle to the iterator

Parameters

handle (≥ 0) handle to the table over which the iterator should iterate

Discussion

This function creates a new table iterator. The iterator initially points at the starting table entry.

See Also

Util_TableItDestroy() destroy a table iterator

Errors

UTIL_ERROR_BAD_HANDLE	table handle is invalid
UTIL_ERROR_NO_MEMORY	unable to allocate memory

Examples

```
C          /* walk through all entries of a table */
          int ihandle;

          for ( ihandle = Util_TableItCreate(handle) ;
              Util_TableItQueryIsNonNull(ihandle) > 0 ;
              Util_TableItAdvance(ihandle) )
          {
              /* do something with the table entry */
          }

          Util_TableItDestroy(ihandle);
```

Util_TableItDestroy

Destroy a table iterator

Synopsis

```
C          #include "util_ErrorCodes.h"
          #include "util_Table.h"
          int status = Util_TableItDestroy(int ihandle);
```

Result

0 ok

Parameters

ihandle (≥ 0) handle to the iterator

Discussion

See Also

Util_TableItCreate() create a table iterator

Errors

UTIL_ERROR_BAD_HANDLE	iterator handle is invalid
UTIL_ERROR_NO_MEMORY	unable to allocate memory

Examples

```
C          /* walk through all entries of a table */
          int ihandle;

          for ( ihandle = Util_TableItCreate(handle) ;
              Util_TableItQueryIsNonNull(ihandle) > 0 ;
              Util_TableItAdvance(ihandle) )
          {
              /* do something with the table entry */
          }

          Util_TableItDestroy(ihandle);
```

Util_TableItQueryIsNotNull

Query whether a table iterator is *not* in the “null-pointer” state

Synopsis

```
C          #include "util_ErrorCodes.h"
          #include "util_Table.h"
          int status = Util_TableItQueryIsNotNull(int ihandle);
```

Result

1	iterator is <i>not</i> in the “null-pointer” state, i.e. iterator points to some table entry
0	iterator is in the “null-pointer” state

Parameters

ihandle (≥ 0)	handle to the iterator
----------------------	------------------------

Discussion

If no errors occur, `Util_TableItQueryIsNotNull(ihandle)` is the same as `1 - Util_TableItQueryIsNull(ihandle)`.

Note that bad things (garbage results, core dumps) may happen if you call this function on an iterator which has been invalidated by a change in the table’s contents.

See Also

<code>Util_TableItQueryIsNull()</code>	query whether a table iterator is in the “null-pointer” state
--	---

Errors

<code>UTIL_ERROR_BAD_HANDLE</code>	iterator handle is invalid
------------------------------------	----------------------------

Examples

```
C          /* walk through all entries of a table */
          int ihandle;

          for ( ihandle = Util_TableItCreate(handle) ;
              Util_TableItQueryIsNotNull(ihandle) > 0 ;
              Util_TableItAdvance(ihandle) )
          {
              /* do something with the table entry */
          }

          Util_TableItDestroy(ihandle);
```

Util_TableItQueryIsNull

Query whether a table iterator is in the “null-pointer” state

Synopsis

```
C          #include "util_ErrorCodes.h"
          #include "util_Table.h"
          int status = Util_TableItQueryIsNull(int ihandle);
```

Result

1	iterator is in the “null-pointer” state
0	iterator is <i>not</i> in the “null-pointer” state, i.e. iterator points to some table entry

Parameters

`ihandle` (≥ 0) handle to the iterator

Discussion

If no errors occur, `Util_TableItQueryIsNull(ihandle)` is the same as `1 - Util_TableItQueryIsNonNull(ihandle)`.
Note that bad things (garbage results, core dumps) may happen if you call this function on an iterator which has been invalidated by a change in the table’s contents.

See Also

`Util_TableItQueryIsNonNull()` query whether a table iterator is *not* in the “null-pointer” state, i.e. whether the iterator points to some table entry

Errors

`UTIL_ERROR_BAD_HANDLE` iterator handle is invalid

Examples

```
C          /* variant code to walk through all entries of a table */
          int ihandle;

          for ( ihandle = Util_TableItCreate(handle) ;
              Util_TableItQueryIsNull(ihandle) == 0 ;
              Util_TableItAdvance(ihandle) )
          {
              /* do something with the table entry */
          }

          Util_TableItDestroy(ihandle);
```

Util_TableItQueryKeyValueInfo

Query the key and the type and number of elements of the value corresponding to that key, of the table entry to which an iterator points

Synopsis

```
C          #include "util_ErrorCodes.h"
          #include "util_Table.h"
          int key_length =
            Util_TableItQueryKeyValueInfo(int ihandle,
                                          int key_buffer_length, char key_buffer[],
                                          CCTK_INT *type_code, CCTK_INT *N_elements)
```

Result

key_length The string length of the key (this has C `strlen()` semantics, i.e. it does *not* include a terminating null character)

Parameters

handle (≥ 0) handle to the table iterator

key_buffer_length the length (`sizeof()`) of `key_buffer[]` (must be ≥ 1 if `key_buffer != NULL`)

key_buffer a pointer to a buffer into which this function should store (at most `key_buffer_length-1` characters of) the key, terminated by a null character as usual for C strings, or NULL pointer to skip storing this

type_code a pointer to where this function should store the value's type code (one of the `CCTK_VARIABLE_*` constants from "`cctk_Constants.h`"), or a NULL pointer to skip storing this.

N_elements a pointer to where this function should store the number of array elements in the value, or a NULL pointer to skip storing this.

Discussion

The usual use of an iterator is to iterate through all the entries of a table, calling this function on each entry, then taking further action based on the results.

Note that bad things (garbage results, core dumps) may happen if you call this function on an iterator which has been invalidated by a change in the table's contents.

If the error code `UTIL_ERROR_TABLE_STRING_TRUNCATED` is returned, then the first `key_buffer_length-1` characters of the key are returned in the user's key buffer (assuming `key_buffer` is non-NULL), followed by a null character to properly terminate the string in the buffer. If any other error code is returned, the user's key buffer (pointed to by `key_buffer` if this is non-NULL) is unchanged.

See Also

`Util_TableQueryValueInfo()` query key present/absent in table, and optionally type and/or number of elements, but using the key instead of an iterator

Errors

`UTIL_ERROR_BAD_HANDLE` handle is invalid

`UTIL_ERROR_TABLE_ITERATOR_IS_NULL` iterator is in "null-pointer" state

`UTIL_ERROR_TABLE_STRING_TRUNCATED` `key_buffer != NULL` and key was truncated to fit in `key_buffer`

Examples

```

C      /* print out all entries in a table */
      /* return 0 for ok, type code for any types we can't handle, */
      /*      -ve for other errors */
      #include <stdio.h>
      #include <stdlib.h>
      #include "util_ErrorCodes.h"
      #include "util_Table.h"
      #include "cctk.h"

      int print_table(int handle)
      {
        int max_key_length, N_key_buffer, ihandle;
        char *key_buffer;

        max_key_length = Util_TableQueryMaxKeyLength(handle);
        if (max_key_length < 0)
            return max_key_length;

        N_key_buffer = max_key_length + 1;
        key_buffer = (char *) malloc(N_key_buffer);
        if (key_buffer == NULL)
            return UTIL_ERROR_NO_MEMORY;

        for ( ihandle = Util_TableItCreate(handle) ;
              Util_TableItQueryIsNonNull(ihandle) > 0 ;
              Util_TableItAdvance(ihandle) )
        {
          CCTK_INT type_code, N_elements;
          CCTK_INT value_int;
          CCTK_REAL value_real;

          Util_TableItQueryKeyValueInfo(ihandle,
                                         N_key_buffer, key_buffer,
                                         &type_code, &N_elements);
          printf("key = \"%s\"\n", key_buffer);

          switch (type_code)
          {
            case CCTK_VARIABLE_INT:
              Util_TableGetInt(handle, &value_int, key_buffer);
              printf("value[int] = %d\n", (int)value_int);
              break;
            case CCTK_VARIABLE_REAL:
              Util_TableGetReal(handle, &value_real, key_buffer);
              printf("value[real] = %g\n", (double)value_real);
              break;
            default:
              /* we don't know how to handle this type */
              Util_TableItDestroy(ihandle);
              free(key_buffer);
              return type_code;
          }
        }

        Util_TableItDestroy(ihandle);

```

```
free(key_buffer);  
return 0;  
}
```

Util_TableItQueryTableHandle

Query what table a table iterator iterates over

Synopsis

```
C          #include "util_ErrorCodes.h"
          #include "util_Table.h"
          int handle = Util_TableItQueryTableHandle(int ihandle);
```

Result

handle (≥ 0) handle to the table over which the iterator iterates

Parameters

ihandle (≥ 0) handle to the iterator

Discussion

Note that it is always ok to call this function, regardless of whether or not the iterator is in the “null-pointer” state.

It’s also ok to call this function even when the iterator has been invalidated by a change in the table’s contents.

See Also

Util_TableItCreate() create an iterator (which iterates over a specified table)

Errors

UTIL_ERROR_BAD_HANDLE iterator handle is invalid

Util_TableItResetToStart

Reset a table iterator to point to the starting table entry

Synopsis

```
C          #include "util_ErrorCodes.h"
          #include "util_Table.h"
          int status = Util_TableItResetToStart(int ihandle);
```

Result

Results are the same as calling `Util_TableItQueryIsNotNull()` on the iterator after the reset:

1	iterator is <i>not</i> in the “null-pointer” state, i.e. iterator points to some table entry
0	iterator is in the “null-pointer” state (this happens if and only if the table is empty)

Parameters

`ihandle` (≥ 0) handle to the iterator

Discussion

Note that it is always ok to call this function, regardless of whether or not the iterator is in the “null-pointer” state.

It’s also ok to call this function even when the iterator has been invalidated by a change in the table’s contents.

See Also

<code>Util_TableItSetToNull()</code>	set an iterator to the “null-pointer” state
<code>Util_TableItSetToKey()</code>	set an iterator to point to a specified table entry

Errors

<code>UTIL_ERROR_BAD_HANDLE</code>	iterator handle is invalid
------------------------------------	----------------------------

Util_TableItSetToKey

Set a table iterator to point to a specified table entry

Synopsis

```
C          #include "util_ErrorCodes.h"
          #include "util_Table.h"
          int status = Util_TableItSetToKey(int ihandle, const char *key);
```

Result

0 ok

Parameters

ihandle (≥ 0) handle to the iterator

Discussion

This function has the same effect as calling `Util_TableItResetToStart()` followed by calling `Util_TableItAdvance()` zero or more times to make the iterator point to the desired table entry. However, this function will typically be (much) more efficient than that sequence.

Note that it is always ok to call this function, regardless of whether or not the iterator is in the “null-pointer” state.

It’s also ok to call this function even when the iterator has been invalidated by a change in the table’s contents.

See Also

<code>Util_TableItResetToStart()</code>	reset an iterator to point to the starting table entry
<code>Util_TableItSetToNull()</code>	set a table iterator to the “null-pointer” state

Errors

<code>UTIL_ERROR_BAD_HANDLE</code>	iterator handle is invalid
<code>UTIL_ERROR_TABLE_BAD_KEY</code>	key contains ‘/’ character
<code>UTIL_ERROR_TABLE_NO_SUCH_KEY</code>	no such key in table

Util_TableItSetToNull

Set a table iterator to the "null-pointer" state

Synopsis

```
C          #include "util_ErrorCodes.h"
           #include "util_Table.h"
           int handle = Util_TableItSetToNull(int ihandle);
```

Result

0 ok

Parameters

ihandle (≥ 0) handle to the iterator

Discussion

Note that it is always ok to call this function, regardless of whether or not the iterator is already in the "null-pointer" state.

It's also ok to call this function even when the iterator has been invalidated by a change in the table's contents.

See Also

<code>Util_TableItResetToStart()</code>	reset an iterator to point to the starting table entry
<code>Util_TableItSetToKey()</code>	set an iterator to point to a specified table entry

Errors

<code>UTIL_ERROR_BAD_HANDLE</code>	iterator handle is invalid
------------------------------------	----------------------------

Util_TableQueryFlags

Query a table's flags word

Synopsis

```
C          #include "util_ErrorCodes.h"
          #include "util_Table.h"
          int flags = Util_TableQueryFlags(int handle);

Fortran    call Util_TableQueryFlags(flags, handle)
          integer flags, handle
```

Result

flags (≥ 0) the flags word

Parameters

handle (≥ 0) handle to the table

Discussion

See `Util_TableCreate()` for further discussion of the semantics of flag words.

See Also

<code>Util_TableClone()</code>	create a new table which is a “clone” (exact copy) of an existing table
<code>Util_TableCreate()</code>	create a table (flags word specified explicitly)
<code>Util_TableCreateFromString()</code>	convenience routine to create a table (with certain default flags) and set key/value entries in it based on a parameter-file-like character string

Errors

UTIL_ERROR_BAD_HANDLE handle is invalid

Examples

```
C          #include <string.h>
          #include "util_ErrorCodes.h"
          #include "util_String.h"
          #include "util_Table.h"

          /* compare two strings, doing the comparison with the same */
          /* case-sensitive/insensitive semantics as a certain table uses */
          int compare_strings(int handle, const char *str1, const char *str2)
          {
              int flags = Util_TableQueryFlags(handle);
              return (flags & UTIL_TABLE_FLAGS_CASE_INSENSITIVE)
                  ? Util_StrCmpi(str1, str2)
                  :      strcmp (str1, str2);
          }
```

Util_TableQueryValueInfo

Query whether or not a specified key is in the table, and optionally the type and/or number of elements of the value corresponding to this key

Synopsis

```
C      #include "util_ErrorCodes.h"
      #include "util_Table.h"
      int key_exists =
        Util_TableQueryValueInfo(int handle,
                                CCTK_INT *type_code, CCTK_INT *N_elements,
                                const char *key);

Fortran call Util_TableQueryValueInfo(key_exists,
.                                     handle,
.                                     type_code, N_elements,
.                                     key)
integer      key_exists, handle
CCTK_INT      type_code, N_elements
character*(*) key
```

Result

```
1      ok (key is in table)
0      ok (no such key in table)
      (in this case nothing is stored in *type_code and *N_elements)
```

Parameters

handle (≥ 0)	handle to the table
type_code	a pointer to where this function should store the value's type code (one of the CCTK_VARIABLE_* constants from "cctk_Constants.h"), or a NULL pointer to skip storing this.
N_elements	a pointer to where this function should store the number of array elements in the value, or a NULL pointer to skip storing this.
key	a pointer to the key (a C-style null-terminated string)

Discussion

Unlike all the other table query functions, this function returns 0 for “no such key in table”. The rationale for this design is that by passing NULL pointers for **type_code** and **N_elements**, this function is then a Boolean “is key in table?” predicate.

If any error code is returned, the user's buffers (pointed to by **type_code** and **N_elements** if these are non-NULL) are unchanged.

See Also

Util_TableIterQueryKeyValueInfo()
query key present/absent in table, and optionally type and/or number of elements, but using an iterator instead of the key

Errors

UTIL_ERROR_BAD_HANDLE	handle is invalid
UTIL_ERROR_TABLE_BAD_KEY	key contains ‘/’ character

Examples


```
C      #include <stdio.h>
      #include <assert.h>
      #include "util_ErrorCodes.h"
      #include "util_Table.h"

      static const int data[] = {314, 159, 265};
      #define N_DATA  (sizeof(data) / sizeof(data[0]))

      CCTK_INT type_code, N_elements;

      /* see whether or not "key" is in table */
      if (Util_TableQueryValueInfo(handle, NULL, NULL, "key"))
      {
          /* key is in the table */
      }
      else {
          /* key is not in the table */
      }

      /* put "data" in table as 3-element integer array */
      Util_TableSetIntArray(handle, N_DATA, data, "data");

      /* query info about "data" value */
      assert( Util_TableQueryValueInfo(handle,
                                      &type_code, &N_elements,
                                      "data") == 1 );
      assert( type_code == CCTK_VARIABLE_INT );
      assert( N_elements == N_DATA );
```

Util_TableQueryMaxKeyLength

Query the maximum key length in a table

Synopsis

```
C          #include "util_ErrorCodes.h"
          #include "util_Table.h"
          int max_key_length = Util_TableQueryMaxKeyLength(int handle);

Fortran    call Util_TableQueryMaxKeyLength(max_key_length, handle)
          integer max_key_length, handle
```

Result

max_key_length (≥ 0)

The string length of the longest key in the table (this has C `strlen()` semantics, i.e. it does *not* include a terminating null character)

Parameters

handle (≥ 0) handle to the table

Discussion

This function is useful if you're going to iterate through a table, and you want to allocate a buffer which is guaranteed to be big enough to hold any key in the table.

Errors

UTIL_ERROR_BAD_HANDLE handle is invalid

Examples

```
C          #include <stdlib.h>
          #include "util_ErrorCodes.h"
          #include "util_Table.h"
          #include "cctk.h"

          int max_key_length = Util_TableQueryMaxKeyLength(handle);
          int N_buffer = max_key_length + 1;
          char *const buffer = (char *) malloc(N_buffer);
          if (buffer == NULL)
          {
              CCTK_WARN(-1, "couldn't allocate memory for table key buffer!");
              abort();          /* CCTK_Abort() would be better */
                               /* if we have a cGH* available */
          }

          /* now buffer is guaranteed to be */
          /* big enough for any key in the table */
```

Util_TableQueryNKeys

Query the number of key/value entries in a table

Synopsis

```
C           #include "util_ErrorCodes.h"
              #include "util_Table.h"
              int N_Keys = Util_TableQueryNKeys(int handle);

Fortran     call Util_TableQueryNKeys(N_Keys, handle)
              integer N_Keys, handle
```

Result

N_Keys (≥ 0) the number of key/value entries in the table

Parameters

handle (≥ 0) handle to the table

Errors

UTIL_ERROR_BAD_HANDLE handle is invalid

Util_TableSet*

This is a family of functions, one for each Cactus data type, to set the value associated with a specified key to be a specified single (1-element array) value

Synopsis

C `#include "util_ErrorCodes.h"`
 `#include "util_Table.h"`
 `int status = Util_TableSetXxx(int handle,`
 `CCTK_XXX value,`
 `const char *key);`

 where XXX is one of POINTER, FPOINTER, CHAR, INT, INT2, INT4, INT8, REAL, REAL4, REAL8, REAL16, COMPLEX, COMPLEX8, COMPLEX16, COMPLEX32 (not all of these may be supported on any given system)

Fortran `call Util_TableSetXxx(status, handle, value, key)`
 `integer            status, handle`
 `CCTK_XXX        value`
 `character*(*)    key`

Result

1 ok (key was already in table before this call, old value was replaced)
 (it doesn't matter what the old value's `type_code` and `N_elements` were, i.e. these do *not* have to match the new value)

0 ok (key was not in table before this call)

Parameters

`handle` (≥ 0) handle to the table

`value` the value to be associated with the key

`key` a pointer to the key (a C-style null-terminated string)

Discussion

The key may be any C character string which does not contain a slash character (`'/'`).

The value is stored as a 1-element array.

This function invalidates any iterators for the table which are not in the "null-pointer" state.

See Also

<code>Util_TableCreateFromString()</code>	convenience routine to create a table and set key/value entries in it based on a parameter-file-like character string
<code>Util_TableGet*()</code>	get a single (1-element array) value, or more generally the first array element of an array value
<code>Util_TableGet*Array()</code>	get an array value
<code>Util_TableGetGeneric()</code>	get a single (1-element array) value with generic data type
<code>Util_TableGetGenericArray()</code>	get an array value with generic data type
<code>Util_TableGet*String()</code>	get a character-string value
<code>Util_TableSet*Array()</code>	set an array value
<code>Util_TableSetGeneric()</code>	set a single (1-element array) value with generic data type
<code>Util_TableSetGenericArray()</code>	set an array value with generic data type

<code>Util_TableSetFromString()</code>	convenience routine to set key/value entries in a table based on a parameter-file-like character string
<code>Util_TableSetString()</code>	set a character-string value

Errors

<code>UTIL_ERROR_BAD_HANDLE</code>	handle is invalid
<code>UTIL_ERROR_TABLE_BAD_KEY</code>	key contains '/' character
<code>UTIL_ERROR_NO_MEMORY</code>	unable to allocate memory

Examples

```
C      #include <math.h>
      #include "util_ErrorCodes.h"
      #include "util_Table.h"

      CCTK_COMPLEX16 z;
      int handle = Util_TableCreate(UTIL_TABLE_FLAGS_DEFAULT);

      Util_TableSetInt(handle, 42, "the answer");
      Util_TableSetReal(handle, 299792458.0, "speed of light");

      z.Re = cos(0.37);      z.Im = sin(0.37);
      Util_TableSetComplex16(handle, z, "my complex number");
```

Util_TableSet*Array

This is a family of functions, one for each Cactus data type, to set the value associated with a specified key to be a copy of a specified array

Synopsis

```
C
#include "util_ErrorCodes.h"
#include "util_Table.h"
int status = Util_TableSetXxxArray(int handle,
                                   int N_elements,
                                   const CCTK_XXX array[],
                                   const char *key);
```

where XXX is one of POINTER, FPOINTER, CHAR, INT, INT2, INT4, INT8, REAL, REAL4, REAL8, REAL16, COMPLEX, COMPLEX8, COMPLEX16, COMPLEX32 (not all of these may be supported on any given system)

```

Fortran      call Util_TableSetXxxArray(status, handle, N_elements, array, key)
               integer      status, handle, N_elements
               CCTK_XXX(*)   array
               character*(*) key

```

Result

1	ok (key was already in table before this call, old value was replaced) (it doesn't matter what the old value's <code>type_code</code> and <code>N_elements</code> were, i.e. these do <i>not</i> have to match the new value)
0	ok (key was not in table before this call)

Parameters

handle (≥ 0)	handle to the table
N_elements (≥ 0)	the number of array elements in array []
array	a pointer to the array (a copy of which) is to be associated with the key
key	a pointer to the key (a C-style null-terminated string)

Discussion

The key may be any C character string which does not contain a slash character (`'/'`). Note that empty (0-element) arrays are ok.

This function invalidates any iterators for the table which are not in the “null-pointer” state.

See Also

<code>Util_TableCreateFromString()</code>	convenience routine to create a table and set key/value entries in it based on a parameter-file-like character string
<code>Util_TableGet*()</code>	get a single (1-element array) value, or more generally the first array element of an array value
<code>Util_TableGet*Array()</code>	get an array value
<code>Util_TableGetGeneric()</code>	get a single (1-element array) value with generic data type
<code>Util_TableGetGenericArray()</code>	get an array value with generic data type
<code>Util_TableGet*String()</code>	get a character-string value
<code>Util_TableSet*()</code>	set a single (1-element array) value

<code>Util_TableSetGeneric()</code>	set a single (1-element array) value with generic data type
<code>Util_TableSetGenericArray()</code>	set an array value with generic data type
<code>Util_TableSetFromString()</code>	convenience routine to set key/value entries in a table based on a parameter-file-like character string
<code>Util_TableSetString()</code>	set a character-string value

Errors

<code>UTIL_ERROR_BAD_HANDLE</code>	handle is invalid
<code>UTIL_ERROR_TABLE_BAD_KEY</code>	key contains '/' character
<code>UTIL_ERROR_BAD_INPUT</code>	<code>N_elements < 0</code>
<code>UTIL_ERROR_NO_MEMORY</code>	unable to allocate memory

Examples

```
C      #include "util_ErrorCodes.h"
      #include "util_Table.h"

      #define N_DIGITS      5
      static const CCTK_INT pi_digits[N_DIGITS] = {3, 14, 159, 2653, 58979};
      int handle = Util_TableCreate(UTIL_TABLE_FLAGS_DEFAULT);

      Util_TableSetIntArray(handle, N_DIGITS, pi_digits, "digits of pi");
```

Util_TableSetFromString

Sets values in a table based on a string argument, which is interpreted with “parameter-file” semantics

Synopsis

```

C          #include "util_ErrorCodes.h"
              #include "util_Table.h"
              int count = Util_TableSetFromString(int handle, const char *string);

Fortran    call Util_TableSetFromString(count, handle, string)
              integer      count, handle
              character*(*) string

```

Result

count (≥ 0) the number of key/value entries set

Parameters

string a pointer to a C-style null-terminated string specifying the table entries to be set (see below for details on the string contents)

Discussion

The string should contain a sequence of zero or more **key=value** “assignments”, separated by whitespace. This function processes these assignments in left-to-right order, setting corresponding key/value entries in the table.

The present implementation only recognises integer, real, and character-string values (not complex), and integer and real arrays. To be precise, the string must match the following BNF:

```

string      → assign*
assign      → whitespace*
assign      → whitespace* key whitespace* = whitespace* value delimiter
key         → any string not containing '/' or '=' or whitespace
value       → array | int_value | real_value | string_value
array       → { int_value* } | { real_value }
int_value   → anything recognized as a valid integer by strtod(3) in base
              10
real_value  → anything not recognized as a valid integer by strtol(3) but
              recognized as valid by strtod(3)
string_value → a C-style string enclosed in "double quotes" (C-style
              character escape codes are allowed ie. bell ('\a'),
              backspace ('\b'), form-feed ('\f'), newline ('\n'),
              carriage-return ('\r'), tab ('\t'), vertical-tab ('\v'),
              backslash ('\'), single-quote ('\'''), double-quote ('\\"'),
              question-mark ('\?'))
string_value → a C-style string enclosed in 'single quotes' (C-style character
              escape codes are not allowed, ie. every character within the
              string is interpreted literally)
delimiter   → end-of-string | whitespace
whitespace  → blank (' ') | tab ('\t') | newline ('\n') | carriage-
              return ('\r') | form-feed ('\f') | vertical-tab ('\v')

```

where * denotes 0 or more repetitions and | denotes logical or.

Notice also that the keys allowed by this function are somewhat more restricted than those allowed by the other **Util_TableSet***() functions, in that this function disallows keys containing '=' and/or whitespace.

If any error code is returned, assignments lexicographically earlier in the input string than where the error was detected will already have been made in the table. Unfortunately, there is no easy way to find out where the error was detected. :(

See Also

<code>Util_TableCreateFromString()</code>	convenience routine to create a table and set key/value entries in it based on a parameter-file-like character string
<code>Util_TableGet*()</code>	get a single (1-element array) value, or more generally the first array element of an array value
<code>Util_TableGet*Array()</code>	get an array value
<code>Util_TableGetGeneric()</code>	get a single (1-element array) value with generic data type
<code>Util_TableGetGenericArray()</code>	get an array value with generic data type
<code>Util_TableGet*String()</code>	get a character-string value
<code>Util_TableSet*()</code>	set a single (1-element array) value
<code>Util_TableSet*Array()</code>	set an array value
<code>Util_TableSetGeneric()</code>	set a single (1-element array) value with generic data type
<code>Util_TableSetGenericArray()</code>	set an array value with generic data type
<code>Util_TableSetString()</code>	set a character-string value

Errors

<code>UTIL_ERROR_NO_MEMORY</code>	unable to allocate memory
<code>UTIL_ERROR_BAD_KEY</code>	invalid input: key contains invalid character
<code>UTIL_ERROR_BAD_INPUT</code>	invalid input: can't parse input string
<code>UTIL_ERROR_NO_MIXED_TYPE_ARRAY</code>	invalid input: different array values have different datatypes
other error codes	this function may also return any error codes returned by <code>Util_TableSetString()</code> , <code>Util_TableSetInt()</code> , <code>Util_TableSetReal()</code> , <code>Util_TableSetIntArray()</code> , or <code>Util_TableSetRealArray()</code> .

Examples

```
C
#include "util_ErrorCodes.h"
#include "util_Table.h"

/* suppose we have a table referred to by handle */

/* then the call... */
int count = Util_TableSetFromString(handle, "n = 6\t"
                                     "dx = 4.0e-5\t"
                                     "pi = 3.1\t"
                                     "s = 'my string'\t"
                                     "array = { 1 2 3 }");

/* ... will return count=5 ... */

/* ... and is otherwise equivalent to the five calls ... */
CCTK_INT array[] = {1, 2, 3};

Util_TableSetInt(handle, 6, "n");
Util_TableSetReal(handle, 4.0e-5, "dx");
Util_TableSetReal(handle, 3.1, "pi");
Util_TableSetString(handle, "my string", "s");
Util_TableSetIntArray(handle, 3, array, "array");
```

Util_TableSetGeneric

Set the value associated with a specified key to be a specified single (1-element array) value, whose data type is generic. That is, the value is specified by a `CCTK_VARIABLE_*` type code and a `void *` pointer.

Synopsis

```
C          #include "util_ErrorCodes.h"
          #include "util_Table.h"
          int status = Util_TableSetGeneric(int handle,
                                           int type_code, const void *value,
                                           const char *key);

Fortran    call Util_TableSetGeneric(status, handle, type_code, value, key)
          integer      status, handle, type_code
          CCTK_POINTER  value
          character*(*) key
```

Result

```
1          ok (key was already in table before this call, old value was replaced)
          (it doesn't matter what the old value's type_code and N_elements were, i.e. these
          do not have to match the new value)

0          ok (key was not in table before this call)
```

Parameters

<code>handle</code> (≥ 0)	handle to the table
<code>type_code</code>	the array elements' type code (one of the <code>CCTK_VARIABLE_*</code> constants from " <code>cctk_Constants.h</code> ")
<code>value_ptr</code>	a pointer to the value to be associated with the key
<code>key</code>	a pointer to the key (a C-style null-terminated string)

Discussion

The key may be any C character string which does not contain a slash character (`'/'`).

The value is stored as a 1-element array.

This function invalidates any iterators for the table which are not in the “null-pointer” state.

See Also

<code>Util_TableCreateFromString()</code>	convenience routine to create a table and set key/value entries in it based on a parameter-file-like character string
<code>Util_TableGet*()</code>	get a single (1-element array) value, or more generally the first array element of an array value
<code>Util_TableGet*Array()</code>	get an array value
<code>Util_TableGetGeneric()</code>	get a single (1-element array) value with generic data type
<code>Util_TableGetGenericArray()</code>	get an array value with generic data type
<code>Util_TableGet*String()</code>	get a character-string value
<code>Util_TableSet*()</code>	set a single (1-element array) value
<code>Util_TableSet*Array()</code>	set an array value
<code>Util_TableSetGeneric()</code>	set a single (1-element array) value with generic data type
<code>Util_TableSetGenericArray()</code>	set an array value with generic data type

<code>Util_TableSetFromString()</code>	convenience routine to set key/value entries in a table based on a parameter-file-like character string
<code>Util_TableSetString()</code>	set a character-string value

Errors

<code>UTIL_ERROR_BAD_HANDLE</code>	handle is invalid
<code>UTIL_ERROR_TABLE_BAD_KEY</code>	key contains '/' character
<code>UTIL_ERROR_NO_MEMORY</code>	unable to allocate memory

Examples

```
C      #include "util_Table.h"
      #include "cctk_Constants.h"

      const CCTK_INT i = 42;
      const void *iptr = (void *) &i;
      CCTK_INT icopy;

      const CCTK_REAL x = 299792458.0;
      const void *xptr = (void *) &x;
      CCTK_REAL xcopy;

      const int handle = Util_TableCreate(UTIL_TABLE_FLAGS_DEFAULT);

      Util_TableSetGeneric(handle, CCTK_VARIABLE_INT, iptr, "the answer");
      Util_TableSetGeneric(handle, CCTK_VARIABLE_REAL, xptr, "speed of light");

      /* gets icopy to 42 */
      Util_TableGetInt(handle, &icopy, "the answer");

      /* gets xcopy to 299792458.0 */
      Util_TableGetReal(handle, &xcopy, "speed of light");
```

Util_TableSetGenericArray

Set the value associated with a specified key to be a copy of a specified array, whose data type is generic. That is, the array is specified by a `CCTK_VARIABLE_*` type code, a count of the number of array elements, and a `void *` pointer.

Synopsis

```
C      #include "util_ErrorCodes.h"
      #include "util_Table.h"
      int status = Util_TableSetGenericArray(int handle,
                                             int type_code,
                                             int N_elements, const void *array,
                                             const char *key);

Fortran  call Util_TableSetGenericArray(status,
      .           handle,
      .           type_code,
      .           N_elements, array,
      .           key)
integer      status, handle, type_code, N_elements
CCTK_POINTER(*)  array
character*(*)   key
```

Result

1	ok (key was already in table before this call, old value was replaced) (it doesn't matter what the old value's <code>type_code</code> and <code>N_elements</code> were, i.e. these do <i>not</i> have to match the new value)
0	ok (key was not in table before this call)

Parameters

<code>handle</code> (≥ 0)	handle to the table
<code>type_code</code>	the array elements' type code (one of the <code>CCTK_VARIABLE_*</code> constants from " <code>cctk_Constants.h</code> ")
<code>N_elements</code> (≥ 0)	the number of array elements in <code>array[]</code>
<code>value_ptr</code>	a pointer to the value to be associated with the key
<code>key</code>	a pointer to the key (a C-style null-terminated string)

Discussion

The key may be any C character string which does not contain a slash character (`'/'`).

The value is stored as a 1-element array.

This function invalidates any iterators for the table which are not in the "null-pointer" state.

See Also

<code>Util_TableCreateFromString()</code>	convenience routine to create a table and set key/value entries in it based on a parameter-file-like character string
<code>Util_TableGet*()</code>	get a single (1-element array) value, or more generally the first array element of an array value
<code>Util_TableGet*Array()</code>	get an array value
<code>Util_TableGetGeneric()</code>	get a single (1-element array) value with generic data type

<code>Util_TableGetGenericArray()</code>	get an array value with generic data type
<code>Util_TableGetString()</code>	get a character-string value
<code>Util_TableSet*()</code>	set a single (1-element array) value
<code>Util_TableSet*Array()</code>	set an array value
<code>Util_TableSetGeneric()</code>	set a single (1-element array) value with generic data type
<code>Util_TableSetFromString()</code>	convenience routine to set key/value entries in a table based on a parameter-file-like character string
<code>Util_TableSetString()</code>	set a character-string value

Errors

<code>UTIL_ERROR_BAD_HANDLE</code>	handle is invalid
<code>UTIL_ERROR_TABLE_BAD_KEY</code>	key contains '/' character
<code>UTIL_ERROR_NO_MEMORY</code>	unable to allocate memory

Examples

```

C      #include "util_Table.h"
      #include "cctk_Constants.h"

      #define N_IARRAY    3
      const CCTK_INT iarray[N_IARRAY] = {42, 69, 105};
      const void *iarray_ptr = (void *) iarray;
      CCTK_INT iarray2[N_IARRAY];

      #define N_XARRAY    2
      const CCTK_REAL xarray[N_XARRAY] = {6.67e-11, 299792458.0};
      const void *xarray_ptr = (void *) xarray;
      CCTK_REAL xarray2[N_XARRAY];

      const int handle = Util_TableCreate(UTIL_TABLE_FLAGS_DEFAULT);

      Util_TableSetGenericArray(handle,
                                CCTK_VARIABLE_INT,
                                N_IARRAY, iarray_ptr,
                                "my integer array");
      Util_TableSetGenericArray(handle,
                                CCTK_VARIABLE_REAL,
                                N_XARRAY, xarray_ptr,
                                "my real array");

      /* gets iarray2[0] = 42, iarray2[1] = 69, iarray2[2] = 105 */
      Util_TableGetIntArray(handle, N_IARRAY, iarray2, "my integer array");

      /* gets xarray2[0] = 6.67e-11, xarray2[1] = 299792458.0 */
      Util_TableGetRealArray(handle, N_XARRAY, xarray2, "my real array");

```

Util_TableSetString

Sets the value associated with a specified key in a table, to be a copy of a specified C-style null-terminated character string

Synopsis

```
C          #include "util_ErrorCodes.h"
          #include "util_Table.h"
          int status = Util_TableSetString(int handle,
                                          const char *string,
                                          const char *key);

Fortran    call Util_TableSetString(status, handle, string, key)
          integer          status, handle
          character*(*)    string, key
```

Result

Results are the same as all the other `Util_TableSet*` functions:

```
1          ok (key was already in table before this call, old value was replaced)
          (it doesn't matter what the old value's type_code and N_elements were, i.e. these
          do not have to match the new value)

0          ok (key was not in table before this call)
```

Parameters

`handle` (≥ 0) handle to the table
`string` a pointer to the string (a C-style null-terminated string)
`key` a pointer to the key (a C-style null-terminated string)

Discussion

The key may be any C character string which does not contain a slash character (`'/'`).

The string is stored as an array of `strlen(string)` `CCTK_CHARS`. It does *not* include a terminating null character.

This function is very similar to `Util_TableSetCharArray()`.

This function invalidates any iterators for the table which are not in the “null-pointer” state.

See Also

<code>Util_TableCreateFromString()</code>	convenience routine to create a table and set key/value entries in it based on a parameter-file-like character string
<code>Util_TableGet*()</code>	get a single (1-element array) value, or more generally the first array element of an array value
<code>Util_TableGet*Array()</code>	get an array value
<code>Util_TableGetGeneric()</code>	get a single (1-element array) value with generic data type
<code>Util_TableGetGenericArray()</code>	get an array value with generic data type
<code>Util_TableGet*String()</code>	get a character-string value
<code>Util_TableSetCharArray()</code>	get an array-of- <code>CCTK_CHAR</code> value
<code>Util_TableSet*()</code>	set a single (1-element array) value
<code>Util_TableSet*Array()</code>	set an array value

<code>Util_TableSetGeneric()</code>	set a single (1-element array) value with generic data type
<code>Util_TableSetGenericArray()</code>	set an array value with generic data type
<code>Util_TableSetCharArray()</code>	set an array-of-CCTK_CHAR value

Errors

<code>UTIL_ERROR_BAD_HANDLE</code>	handle is invalid
<code>UTIL_ERROR_TABLE_BAD_KEY</code>	key contains '/' character
<code>UTIL_ERROR_NO_MEMORY</code>	unable to allocate memory

Examples

```
C      #include "util_ErrorCodes.h"
      #include "util_Table.h"

      static const CCTK_CHAR array[]
          = {'r', 'e', 'l', 'a', 't', 'i', 'v', 'i', 't', 'y'};
      #define N_ARRAY (sizeof(array) / sizeof(array[0]))
      int handle = Util_TableCreate(UTIL_TABLE_FLAGS_DEFAULT);

      Util_TableSetString(handle, "relativity", "Einstein");

      /* this produces the same table entry as the Util_TableSetString() */
      Util_TableSetCharArray(handle, N_ARRAY, array, "Einstein");
```

Part F

Appendices

Chapter F1

Glossary

API	Applications Programming Interface, the externally-visible interface provided by some software component. (An API usually consists of a set of subroutine/function calls, but may also include data passed via Cactus key-value tables, grid arrays, or other means.) For example, the Reference Manual documents most of the Cactus flesh APIs.
arrangement	A collection of thorns, normally stored in the Cactus <code>arrangements</code> directory.
autoconf	A GNU program which builds a configuration script which can be used to make a Makefile.
boundary zone	A boundary zone is a set of points added to the edge of a grid to contain boundary data. E.g Dirichlet or Von Neumann data. (See also: symmetry zone, ghost zone.)
Cactus	A green fleshy plant with lots of thorns, usually painful if touched.
CCTK	Cactus Computational Tool Kit (The Cactus flesh and computational thorns).
CCL	The Cactus Configuration Language , this is the language that the thorn configuration files are written in.
configuration	The combination of a set of thorns, and all the Cactus configure options which affect what binary will be produced when compiling Cactus. For example, the choice of compilers (Cactus <code>CC</code> , <code>CXX</code> , <code>F77</code> , and <code>F90</code> configure options) and the compiler optimization settings (<code>OPTIMISE/OPTIMIZE</code> and <code>*_OPTIMISE_FLAGS</code> configure options) are part of a configuration (these flags change what binary is produced), but the Cactus <code>SILENT</code> and <code>WARN</code> configure options aren't part of a configuration (they don't change what binary will be produced).
checkout	Get a copy of the code or thorns from CVS.
checkpointing	Save the entire state of a run to file so that it can be restarted at a later time.
convergence	
CST	This stands for Cactus Specification Tool, which is the perl scripts which parse the thorns' configuration files (those that have a <code>.ccl</code> extension).
CVS	The " <i>Concurrent Versioning System</i> " is the favoured distribution system for Cactus and can be downloaded from your favorite GNU site.
driver	A thorn which creates and handles grid hierachies.
flesh	The routines which hold all the thorns together, this is what you get if you check out Cactus from our CVS repository.

friend	
GF	Shorthand for a <code>grid function</code> .
gmake	GNU version of make utility.
ghostzone	A ghostzone is a set of points added for parallelisation purposes to enable inter-processor communication of data from processor boundaries. In single processor runs there are no ghost zones. Ghost zones should not be confused with symmetry or boundary zones.
grid array	A <i>grid variable</i> whose global size need not be that of the computational grid; instead, the size is declared explicitly in an <code>interface.ccl</code> file.
grid function	A <i>grid variable</i> whose global size is the size of the computational grid
grid hierarchy	A computational grid, and the grid variables associated with it.
grid scalar	A <i>grid variable</i> which has zero indices, i.e. it's just a number on each processor.
grid variable	Any variable which can be passed between thorns, or routines belonging to the same thorn, but passed through the defined flesh interface; implicitly implies it is related to the computational grid rather than being an internal variable of the thorn or one of its routines. <code>grid scalar</code> , <code>grid function</code> , and <code>grid array</code> are all examples of <code>grid variables</code> .
GNATS	The GNU program we use for reporting and tracking bugs, comments and suggestions.
handle	A signed integer value ≥ 0 . Many Cactus routines return or accept a handle to represent a dynamic data or code object. Handles for the same object class should not be trusted to comprise a consecutive sequence of integer values.
GV	Shorthand for a <i>grid variable</i>
HDF5	Hierarchical Data Format version 5, an API, subroutine library, and file format for storing multidimensional data. An HDF5 file can store both data (for example Cactus grid variables), and metadata (data describing the other data, for example Cactus coordinate systems).
implementation	Defines the interface that a thorn presents to the outside world. See Section B2.3 .
inherit	
interpolation	Given N <i>grid arrays</i> and C interpolation points (in the grid array coordinate space), return M values on each requested processor at each interpolation point. (See also <i>local interpolation</i>)
local interpolation	Given N arrays on the current processor with a given coordinate system, and C interpolation points (in the array coordinate space) return M values at each interpolation point. (See also <i>interpolation</i>)
MPI	Message Passing Interface, an API and software library for sending messages between processors in a multiprocessor system.
mutual recursion	See <i>recursion, mutual</i> .
NUL character	The C programming language uses a “NUL character” to terminate character strings. A NUL character has the integer value zero, but it's useful to write it as <code>'\0'</code> to emphasize to human readers that this has type <code>char</code> rather than <code>int</code> .

NULL pointer	C defines a “NULL pointer” which is guaranteed not to point to any object. You get a NULL pointer by converting the integer constant 0 to a pointer type, e.g. <code>int* ptr = 0;</code>. Note that if you have an expression which has the value zero, but which isn’t an integer constant, converting this to a pointer type is <i>not</i> guaranteed to give a NULL pointer, e.g.: <pre>int i = 0; int* ptr = i; /* ptr is NOT guaranteed to be a NULL pointer! */</pre> Many programmers prefer to use the predefined macro NULL (defined in <code><stdlib.h></code> and <code><stdio.h></code>) to create NULL pointers, e.g. <code>int* ptr = NULL;</code>, to emphasize to human readers that this is a NULL <i>pointer</i> rather than “just” the integer zero. Note that it is explicitly <i>not</i> defined whether a NULL pointer is represented by a bit pattern of all zero bits – this varies from system to system, and there are real-world systems where NULL pointers are in fact <i>not</i> represented this way.
parameter	A variable which remains unchanged throughout the execution of a Cactus executable. Parameters all have default values which can be changed in a parameter file.
processor topology	
PUGH	The default parallel driver for Cactus which uses MPI.
PVM	Parallel Virtual Machine, provides interprocessor communication.
recursion, mutual	See <i>mutual recursion</i>.
reduction	Given N grid variables return M output values on each requested processor.
symmetry zone	A symmetry zone is a set of points added to the edge of a grid to contain data which is obtained, by some symmetry operation, from another part of the grid. (See also: boundary zone, ghost zone.)
TAGS	
target	
thorn	A collection of subroutines with a definite interface and purpose.
WMPI	Win32 Message Passing Interface.

Chapter F2

Configuration file syntax

F2.1 General Concepts

Each thorn is configured by three compulsory files in the top level thorn directory:

- `interface.ccl`
- `param.ccl`
- `schedule.ccl`

These files are written in the *Cactus Configuration Language* which is case insensitive.

F2.2 `interface.ccl`

The interface configuration file consists of

- a header block giving details of the thorn's relationship with other thorns.
- a block detailing which include files are used from other thorns, and which include files are provided by this thorn
- blocks detailing aliased functions provided or used by this thorn
- a series of blocks listing the thorn's global variables.

(For a more extensive discussion of Cactus variables see Chapter [B4](#).)

F2.2.1 Header block

The header block has the form:

```
implements: <implementation>
inherits: <implementation>, <implementation>
friend: <implementation>, <implementation>
```

where

- The implementation name must be unique among all thorns, except between thorns which have the same public and protected variables and global and restricted parameters.
- Inheriting from another implementation makes all that implementation's public variables available to your thorn. At least one thorn providing any inherited implementation must be present at compile time. A thorn cannot inherit from itself. Inheritance is transitive, but not commutative.
- Being a friend of another implementation makes all that implementation's protected variables available to your thorn. At least one thorn providing an implementation for each friend must be present at compile time. A thorn cannot be its own friend. Friendship is associative and commutative. Note that your thorn is also a friend of all your thorn's friends' friends.

F2.2.2 Include files

The include file section has the form:

```
USES INCLUDE [SOURCE|HEADER]: <file_name>
INCLUDE[S] [SOURCE|HEADER]: <file_to_include> in <file_name>
```

The former is used when a thorn wishes to use an include file from another thorn. The latter indicates that this thorn adds the code in <file_to_include> to the include file <file_name>. If the include file is described as `SOURCE`, the included code is only executed if the providing thorn is active. Both default to `HEADER`.

F2.2.3 Function aliasing

If any aliased function is to be used or provided by the thorn then the prototype must be declared with the form:

```
<return_type> FUNCTION <alias>(<arg1_type> <arg1>, ...)
```

The <return_type> must be either `void`, `CCTK_INT` or `CCTK_REAL`. The name of the aliased function <alias> must be 21 characters or less, contain at least one uppercase and one lowercase letter, and follow the C standard for function names. The type of each argument, <arg*_type> must be a CCTK type in `INT`, `REAL`, `STRING`, `POINTER`, `INT:ARRAY`, `REAL:ARRAY`. Arguments may only be modified if they have the `ARRAY` suffix.

If the argument <arg*> is a function pointer then the argument itself (which will be preceded by the return type) should be

```
CCTK_FPOINTER <function_arg1>(<arg1_type> <arg1>, ...)
```

Function pointers may not be nested. The combined character length of <alias> and <function_arg*> must be 21 characters or less.

If an aliased function is to be used then the block

```
USES FUNCTION <alias>
```

is required.

If a function is provided then the block

```
PROVIDES FUNCTION <alias> WITH <provider> LANGUAGE <providing_language>
```

is required. As with the alias name, <provider> must be 21 characters or less, contain at least one uppercase and one lowercase letter, and follow the C standard for function names. Currently the only supported values of <providing_language> are C and Fortran.

F2.2.4 Variable blocks

The thorn's variables are collected into groups. This is not only for convenience, but for collecting like variables together. Storage assignment, communication assignment, and ghostzone synchronization take place for groups only.

The thorn's variables are defined by:

```
[<access>:]

<data_type> <group_name>[[<number>]] [TYPE=<group_type>] [DIM=<dim>]
[TIMELEVELS=<num>]
[SIZE=<size in each direction>] [DISTRIB=<distribution_type>]
[GHOSTSIZE=<ghostsize>] [STAGGER=<stagger-specification>]
[TAGS=<string>] ["<group_description>"]
[{
  [ <variable_name>[,]<variable_name>
    <variable_name> ]
}] ["<group_description>"] ]
```

(The options TYPE, DIM, etc. following <group_name> must all appear on one line.) Note that the beginning brace ({) must sit on a line by itself; the ending brace (}) must be preceded by a carriage return.

- **access** defines which thorns can use the following groups of variables. **access** can be either **public**, **protected** or **private**.
- **data_type** defines the data type of the variables in the group. Supported data types are **CHAR**, **BYTE**, **INT**, **REAL** and **COMPLEX**.
- **group_name** must be an alpha-numeric name (which may also contain underscores) which is unique across group and variable names within the scope of the thorn. A group name is compulsory.
- **[number]**, if present, indicates that this is a **vector** group. Such a group does not have named variables (so the { } block should not be present) but instead appears as a one-dimensional array of grid variables. The first index in the array is the number-index, and any other indices are the normal spatial indices for a group of this type and dimension. The number can be any valid arithmetical expression consisting of integers or integer-valued parameters.
- **TYPE** designates the kind of variables held by the group. The choices are **GF**, **ARRAY** or **SCALAR**. This field is optional, with the default variable type being **SCALAR**.
- **DIM** defines the spatial dimension of the **ARRAY** or **GF**. The default value is **DIM=3**.
- **TIMELEVELS** defines the number of timelevels a group has if the group is of type **ARRAY** or **GF**, and can take any positive value. The default is one timelevel.

- **SIZE** defines the number grid-points an **ARRAY** has in each direction. This should be a comma-separated list of valid arithmetical expressions consisting of integers or integer-valued parameters.
- **DISTRIB** defines the processor decomposition of an **ARRAY**. **DISTRIB=DEFAULT** distributes **SIZE** grid-points across all processors. **DISTRIB=CONSTANT** implies that **SIZE** grid-points should be allocated on each processor. The default value is **DISTRIB=DEFAULT**.
- **GHOSTSIZE** defines number of ghost-zones in each dimension of an **ARRAY**.
- **STAGGER** defines position of grid-points of a **GF** with respect to the underlying grid. It consists of a string made up of a combination **DIM** of the letters **M**, **C**, **P** depending on whether the layout in that direction is on the Minus face, Centre, or Plus face of the cell in that dimension.
- **TAGS** defines an optional string which is used to create a set of key-value pairs associated with the group. The keys are case independent.
- The (optional) block following the group declaration line contains a list of variables contained in the group. All variables in a group have the same data type, variable type, dimension and distribution. The list can be separated by spaces, commas, or new lines. The variable names must be unique within the scope of the thorn. A variable can only be a member of one group. The block must be delimited by brackets on new lines. If no block is given after a group declaration line, a variable with the same name as the group is created. Apart from this case, a group name cannot be the same as the name of any variable seen by this thorn.
- An optional description of the group can be given on the last line. If the variable block is omitted, this description can be given at the end of the declaration line.

The process of sharing code among thorns using include files is discussed in Section [B10.2](#).

F2.3 param.ccl

The parameter configuration file consists of a list of *parameter object specification items* (OSIs) giving the type and range of the parameter separated by optional *parameter data scoping items* (DSIs) which detail access to the parameter. (For a more extensive discussion of Cactus parameters see Chapter [B5](#).)

F2.3.1 Parameter data scoping items

<access>:

The key word **access** designates that all parameter object specification items up to the next parameter data scoping item are in the same protection or scoping class. **access** can take the values:

global	all thorns have access to global parameters
restricted	other thorns can have access to these parameters if they specifically request it in their own param.ccl
private	only your thorn has access to private parameters
shares	in this case an implementation name must follow the colon. It declares that all the parameters in the following scoping block are restricted variables from the specified implementation . (Note: only one implementation can be specified on this line.)

F2.3.2 Parameter object specification items

```
[EXTENDS|USES] <parameter_type> <parameter name>[[<integer>]] "<parameter description>"
[AS <alias>] [STEERABLE=<NEVER|ALWAYS|RECOVER>]
[ACCUMULATOR=<expression>] [ACCUMULATOR-BASE=<parameter-name>]
{
  <PARAMETER_VALUES>
} <default value>
```

Where the options AS, STEERABLE, etc. following <parameter_description> must all appear on one line. Note that the beginning brace ({) must sit on a line by itself; the ending brace (}) must be preceded by a carriage return.

- Allowed `parameter_types` are

INT	The specification of <code>parameter_values</code> takes the form of any number of comma-separated blocks of the form: <pre>[<low-range>][:<high-range>][:<step>]][: "<range description>"]</pre> Where an empty field, or a * in the place of <code>low-range</code> or <code>high-range</code> indicates $-\infty$ and ∞ respectively. The default value for <code>step</code> is 1. A number by itself denotes that this number is the only acceptable value.
REAL	The range specification is the same as integers, except that here, no <code>step</code> implies a continuum of values. Note that numeric constants should be expressed as in C (e.g. <code>1e-10</code>). Note also that one can use the Cactus types such as <code>CCTK_REAL4</code> to specify the precision of the parameter.
KEYWORD	Each entry in the list of acceptable values for a keyword has the form <pre>"<keyword value>", "<keyword value>" :: "<description>"</pre>
STRING	Allowed values for strings should be specified using regular expressions. To allow any string, the regular expression <code>".*"</code> should be used.
BOOLEAN	No <code>PARAMETER_VALUES</code> should be specified for a boolean parameter. The default value for a boolean can be – True: 1, yes, y, t, true – False: 0, no, n, f, false

- The `parameter name` must be unique within the scope of the thorn.
- A thorn can declare that it `EXTENDS` a parameter of another thorn. This allows it to declare additional acceptable values. By default it is acceptable for two thorns to declare the same value as acceptable.
- If the thorn wants to simply use a parameter from another thorn, without declaring additional values, use `USES` instead.
- `[integer]`, if present, specifies that this is an **array** parameter. (Note that the notation used above for the parameter specification breaks down here, there must be square brackets around the given integer). The parameter is then a one-dimensional array of values of the specified type.
- `alias` allows a parameter to appear as a different name in this thorn than its original name in another thorn. The name as seen in the parameter file is unchanged.
- `STEERABLE` specifies when a parameter value may be changed. By default parameters may not be changed after the parameter file has been read, or on restarting from checkpoint. This option relaxes this restricting, specifying that the parameter may be changed at recovery time or at any time.

- **ACCUMULATOR** specifies that this is an **accumulator** parameter. Such parameters cannot be set directly, but are set by other parameters who specify this one as an **ACCUMULATOR-BASE**. The expression is a two parameter arithemetical expression of X and y . Setting the parameter consists of evaluating this expression successively, with x being the current value of the parameter (at the first iteration this is the default value) and y the value of the setting parameter. This procedure is repeated, starting from the default value of the parameter, each time one of the setting parameters changes.
- **ACCUMULATOR-BASE** specifies the name of an **ACCUMULATOR** parameter which this parameter sets.

F2.4 schedule.ccl

(A more extensive discussion of Cactus scheduling is provided in Chapter B6.) A schedule configuration file consists of

- *assignment statements* to switch on storage for grid variables for the entire duration of program execution
- *schedule blocks* to schedule a subroutine from a thorn to be called at specific times during program execution in a given manner
- *conditional statements* for both assignment statements and schedule blocks to allow them to be processed depending on parameter values

F2.4.1 Assignment Statements

Assignment statements, currently only assign storage.

These lines have the form:

```
[STORAGE: <group>[timelevels], <group>[timelevels]]
```

If the thorn is active, storage will be allocated for the given groups for the duration of program execution (unless storage is explicitly switched off by some call to `CCTK_DisableGroupStorage` within a thorn).

The storage line includes the number of timelevels to activate storage for, this number can be from 1 up to the maximum number or timelevels for the group as specified in the defining `interface.ccl` file. If the maximum number of timelevels is 1 (the default), this number may be omitted.

The behaviour of an assignment statement is independent of its position in the schedule file (so long as it is outside a schedule block).

F2.4.2 Schedule Blocks

Each *schedule block* in the file `schedule.ccl` must have the syntax:

```
schedule [GROUP] <function name|group name> AT|IN <time> \
    [BEFORE|AFTER <function name>] [WHILE <variable>] [AS <alias>]
{
    [LANG: <language>]
```

```

[STORAGE:      <group>[timelevels],<group>[timelevels]...]
[TRIGGER:      <group>,<group>...]
[SYNCHRONISE:  <group>,<group>...]
[OPTIONS:      <option>,<option>...]
} "Description of function"

```

GROUP Schedule a schedule group with the same options as a schedule function. The schedule group will be created if it doesn't exist.

<function name|group name>

The name of a function or a schedule group to be scheduled. Function and schedule group names are case sensitive

<group>

A group of grid variables. Variable groups inherited from other thorns may be used, but they must then be fully qualified with the implementation name.

AT

Functions can be scheduled to run at the Cactus schedule bins, for example `CCTK_EVOL`, `CCTK_STARTUP`. A complete list and description of these is provided in Appendix F3. The initial letters `CCTK_` are optional. Grid variables cannot be used in the `CCTK_STARTUP` and `CCTK_SHUTDOWN` timebins.

IN

Schedules a function or schedule group to run in a schedule group rather than in a Cactus timebin.

BEFORE/AFTER

Takes either a function name, a function alias, or a schedule group name. If no function, alias or schedule group with this name is provided by an active thorn this directive is ignored.

WHILE

Executes a function or schedule group until the given variable (which must be a fully qualified integer grid scalar) has the value zero.

AS

Provides an alias for a function or schedule group which should be used for scheduling before, after or in. This can be used to provide thorn independence for other thorns scheduling functions or schedule groups relative to this one.

LANG

The code language for the function (either `C` or `FORTRAN`). No language should be specified for a schedule group.

STORAGE

List of variable groups which should have storage switched on for the duration of the function or schedule group. Each group must specify how many timelevels to activate storage for, from 1 up to the maximum number for the group as specified in the defining `interface.ccl` file. If the maximum is 1 (the default) this number may be omitted.

TRIGGER

List of grid variables or groups to be used as triggers for causing an analysis function or analysis group to be executed. Any schedule block for an analysis function or analysis group must contain a non-trivial `TRIGGER` line.

SYNCHRONISE

List of groups to be synchronised as soon as the function or schedule group is exited.

OPTIONS

List of additional options (see Section F2.4.2) for the scheduled function or group of functions

Allowed Options

GLOBAL

This routine will only be called once on a grid hierarchy, not for all subgrids making up the hierarchy. This can be used, for example, for analysis routines which use global reduction or interpolation routines rather than the local subgrid passed to them, and hence should only be called once.

LEVEL

This routine will only be called once on any "level" of grid hierarchy. I.e. it will only be called once for any set of sub-grids which have the same `cctk_levfac` numbers.

F2.4.3 Conditional Statements

Any schedule block or assignment statements can be optionally surrounded by conditional `if-elseif-else` constructs using the parameter data base. These can be nested, and have the general form:

```
if (CCTK_Equals(<parameter>,<string>))
{
    [<assignments>]
    [<schedule blocks>]
}
```

Conditional constructs cannot be used inside a schedule block.

Chapter F3

Schedule bins

Using the `schedule.ccl` files, thorn functions can be scheduled to run in the different timebins which are executed by the Cactus flesh. This chapter describes these standard timebins, and shows the flow of program execution through them.

CCTK_STARTUP	Run before any grids are constructed, this is the timebin for example where grid independent information (e.g. output methods, reduction operators) is registered. Note that since no grids are setup at this point, grid variables cannot be used in routines scheduled here.
CCTK_RECOVER_PARAMETERS	Used by thorns with relevant IO methods as the point to read parameters when recovering from checkpoint files.
CCTK_PARAMCHECK	This timebin is for thorns to check the validity of parameter combinations. This bin is also executed before the grid hierarchy is made, so that routines scheduled here only have access to the global grid size and the parameters.
CCTK_BASEGRID	The first timebin to be executed after a driver thorn constructs grid, this bin should only be used to set up coordinate systems on the newly created grids.
CCTK_INITIAL	This is the place to set up any required initial data. This timebin is not run when recovering from a checkpoint file.
CCTK_POSTINITIAL	
CCTK_RECOVER_VARIABLES	Used by thorns with relevant IO methods as the point to read in all the grid variables when recovering from checkpoint files.
CCTK_POST_RECOVER_VARIABLES	This timebin exists for scheduling any functions which need to modify grid variables after recovery.
CCTK_CPINITIAL	Used by thorns with relevant IO methods as the point to checkpoint initial data if required.
CCTK_CHECKPOINT	Used by thorns with relevant IO methods as the point to checkpoint data during the iterative loop when required.
CCTK_PRESTEP	The timebin for scheduling any routines which need to be executed before any routines in the main evolution step. This timebin exists for thorn writers convenience, the BEFORE , AFTER etc. functionality of the <code>schedule.ccl</code> file should allow all functions to be scheduled in the main <code>CCTK_EVOL</code> timebin.

CCTK_EVOL	The timebin for the main evolution step.
CCTK_POSTSTEP	The timebin for scheduling any routines which need to be executed after all the routines in the main evolution step. This timebin exists for thorn writers convenience, the BEFORE , AFTER etc. functionality of the <code>schedule.cc1</code> file should allow all functions to be scheduled in the main CCTK_EVOL timebin.
CCTK_ANALYSIS	The analysis timebin is special, in that it is closely coupled with output, and routines which are scheduled here are typically only executed if output of analysis variables is required. Routines which perform analysis should be independent of the main evolution loop (that is, it should not matter for the results of a simulation whether routines in this timebin are executed or not).
CCTK_TERMINATE	Called after the main iteration loop when Cactus terminates. Note that sometime in this timebin a driver thorn should be destroying the grid hierarchy and removing grid variables.
CCTK_SHUTDOWN	Cactus final shutdown routines, after the grid hierarchy has been destroyed. Grid variables are no longer available.

Chapter F4

Flesh parameters

The flesh parameters are defined in the file `src/param.ccl`.

F4.1 Private parameters

<code>allow_mixeddim_gfs</code>	Allow use of GFs from different dimensions [no]
<code>cctk_brief_output</code>	Give only brief output [no]
<code>cctk_full_warnings</code>	Give detailed information for each warning statement [yes]
<code>cctk_run_title</code>	Description of this simulation ["]
<code>cctk_show_banners</code>	Show any registered banners for the different thorns [yes]
<code>cctk_show_schedule</code>	Print the scheduling tree to standard output [yes]
<code>cctk_strong_param_check</code>	Die on parameter errors in CCTK_PARAMCHECK [yes]
<code>cctk_timer_output</code>	Give timing information [off] {off, full}
<code>manual_cache_setup</code>	Set the cache size manually [no]
<code>manual_cache_size</code>	The size to set the cache to if not done automatically (bytes) [0]
<code>manual_cacheline_bytes</code>	The size of a cacheline if not set automatically (bytes) [0]
<code>recovery_mode</code>	How to behave when recovering from a checkpoint [strict] {strict, relaxed}

F4.2 Restricted parameters

<code>cctk_final_time</code>	Final time for evolution, overridden by <code>cctk_itlast</code> unless it is positive [-1.0]
<code>cctk_initial_time</code>	Initial time for evolution [0.0]
<code>cctk_itlast</code>	Final iteration number [10]

<code>max_runtime</code>	Terminate evolution loop after a certain elapsed runtime (in minutes); set to zero to disable this termination condition [0]
<code>terminate</code>	Condition on which to terminate evolution loop [iteration] {never, iteration, time, runtime, any, all}
<code>terminate_next</code>	Terminate on next iteration ? [no]

Chapter F5

Using GNATS

GNATS is a freely redistributable set of tools for tracking bug reports. It allows users to categorize their problem report and submit them to the GNATS. The bugtracker will assign appropriate maintainers to analyze and solve the problem. We are currently supporting a web based interface at <http://www.cactuscode.org> which lets you interactively file a bug report. Here, we briefly describe the main categories when creating a Cactus problem report.

Reporters email	Your email address so we can get in contact with you.
Category	there is currently a category for each of the Cactus thorns and arrangements, also a category for the old Cactus3.x and some general subjects like Web,etc. Select whatever category fits best.
Synopsis	A brief and informative subject line.
Confidential	Unused, all PRs are public.
Severity	Pick one three levels.
Class	In the selected category, specify what kind of problem you are dealing with.
submitter ID	Unused
Originator	Your name. Anonymous is OK, but you real name would be best.
Release	The Cactus release you are using. You can find this out, for example, from an executable by typing <code>cactus.<config> -v</code> .
Environment	Very important: specify the environment, e.g. <code>uname -a</code> , MPI/non-MPI, etc.
Description	Describe your problem precisely, if you get a core dump, include the stack trace, give the minimal number of thorns, this problems occurs with.
How-To-Repeat	Tell us how to repeat the problem if it is software related.
Fix	If you can provide a fix, let us know.

We also provide the customized `send-pr` and `send-pr.el` programs at our website. These commands are compiled to submit Cactus problem reports in your shell and from within emacs, respectively.

Chapter F6

Using CVS

CVS is a version control system, which allows you to keep old versions of files (usually source code), log of when, and why changes occurred, and who made them, etc. Unlike the simpler systems, CVS does not just operate on one file at a time or one directory at a time, but operates on hierarchical collections of directories consisting of version controlled files. CVS helps to manage releases and to control the concurrent editing of source files among multiple authors. CVS can be obtained from <http://www.cyclic.com>.

A *CVS repository* located on a *server* may consist of an arbitrary number of *modules*, which can be checked out (that is downloaded) independently. The Cactus flesh and the Cactus arrangements are organized as modules, their *CVS server* is cvs.cactuscode.org.

F6.1 Essential CVS commands

cvs login Logs into the repository. You will be prompted for a *password*. This cvs command leaves a file `.cvspass` in your home directory. There is no need to login every time you issue a cvs command, as long as this file exists. For a Cactus checkout, you have to log into the CVS server, using the cvs option **-d** to specify CVSROOT:
`cvs -d :pserver:cvs_anon@cvs.cactuscode.org:/cactus login`

Note: that there is no "logout" command: if you log in with administrative rights from an arbitrary account, you should be aware that the password file enables subsequent administrative logins from that account. *Delete the file if necessary.*

cvs checkout modules ...

This command creates your private copy of the source for *modules*. You can work with this copy without interfering with others' work. At least one subdirectory level is always created: it does not write all files into your current directory but creates a directory. For Cactus, you need to either include the **-d** options to specify the CVSROOT directory and the CVS server, or specify them with an environment variable (see below). Once you have checked out the repository there is no need to include the **-d** option and its rather lengthy argument: the necessary information is contained in the local **CVS/** directories.

cvs update Execute this command from *within* your private source directory when you wish to update your copies of source files from changes that other developers have made to the source in the repository. Merges are performed automatically when possible, a warning is issued if manual resolution is required for conflicting changes. If your local copy is several versions behind the actual repository copy, CVS will *refetch* the whole file instead of applying multiple patches.

cvs add file	Use this command to enroll new files in cvs records of your working directory. The files will be added to the repository the next time you run 'cvs commit'.
cvs commit file	Use this command to add your local changes to the source to the repository and thereby making it publically available to checkouts and updates by other users. You cannot commit a newly created file unless you have <i>added</i> it.
cvs diff file	Show differences between a file in your working directory and a file in the source repository, or between two revisions in source repository. (Does not change either repository or working directory.) For example, to see the difference between versions 1.8 and 1.9 of a file <code>foobar.c</code> : <pre>cvs diff -r 1.8 1.9 foobar.c</pre>
cvs remove file	Remove files from the source repository, pending a <code>cvs commit</code> on the same files.
cvs status [file	] This command returns the current status of your local copy relative to the repository: e.g. it indicates local modifications and possible updates.
cvs import repository tag1 tag2	Import adds an entire source distribution (starting from the directory you issue the command in) to the repository directory. Use this command to add new arrangements to the Cactus4.0 repository. The repository argument is a directory name (or a path to a directory) and the CVS root directory for repositories; to obtain this directory on the CVS server, send a request to <code>cactus@cactuscode.org</code> . tag1 and tag2 are two tags (vendor and release tags) that have to be supplied. For example, to add <code>MyThorn</code> to the <code>MyArrangement</code> arrangement, which may or may not already exist on the CVS repository <pre>cvs -d :pserver:<name>@cvs.cactuscode.org:/cactus import MyArrangement/MyThorn start v1</pre> After you import a thorn, you should check it out from the repository straight away, and only edit this version.

F6.2 CVS Options

The CVS command line can include the following:

cvs options	which apply to the overall cvs program
a cvs command	which defines a particular task carried out by cvs
command options	to specify certain working modes for the cvs command.
command arguments	to specify which file to act on.

The options must be put *relative* to the *cvs command* as the same option name can mean different things: *cvs options* go to the *left* of the cvs command, *command options* go to the *right* of the cvs command. Here is a list of essential *cvs options*

-d cvs_root_directory	Use <i>cvs_root_directory</i> as the root directory pathname of the master source repository. Overrides the setting of the <code>CVSROOT</code> environment variable. This value should be specified as an absolute pathname. In the Cactus checkout procedure, you specify the Cactus CVS server: <pre>-d :pserver:cvs_anon@cvs.cactuscode.org:/cactus/</pre>
------------------------------	---

- z *compression-level*** When transferring files across the network use **gzip** with compression level *compression-level* to compress and decompress data as it is transferred. Requires the presence of the GNU gzip program in the current search path at both ends of the link.
- n** Do not change any file. Attempt to execute the *cvs command* but only to issue reports. Does not remove, update, etc. any files. Very effective for testing.
- v** Displays version information of the installed CVS.
- H *cvs-command*** Displays usage information about the specified cvs command. Without *cvs-command*, a list of all available commands is returned.

Here is a list of essential *command options* with the commands they are used with. They go after the cvs command. For a more complete list of all options, please refer to the manual page.

- P** Prune (remove) directories that are empty after being updated, on **checkout**, or **update**. Normally, an empty directory (one that is void of revision controlled files) is left alone. Specifying **-P** will cause these directories to be silently removed from your checked-out sources. This does not remove the directory from the repository, only from your checked out copy.
- m "*Text*"** Specify a logging message explaining changes, etc. on **commit**, **import**. If you do not specify a message, your default editor is invoked to allow you to enter one.
- d** Use this option with the **update** command to create any directories if they are missing from your local copy. This is normally the case if another user has added files or directories to the repository. By default the **update** command only acts on files in your local copy. Note that omitting this option is a frequent cause of files missing during compilation. (You can change this default behavior of cvs by putting a *.cvsrc* in your home directory with the contents "update -d".)

F6.3 CVS Examples

We list some sample CVS commands to treat the most typical Cactus4.0 CVS situations.

Logging into the server

```
cvs -d :pserver:cvs_anon@cvs.cactuscode.org:/cactus login
```

You will be asked for the password for user *cvs_anon*, which is *anon*.

Checking out the code

```
cvs -d :pserver:cvs_anon@cvs.cactuscode.org:/cactus checkout Cactus
```

check out a CVS module named "Cactus", in this case it checks out the Cactus Computational Toolkit. A directory *./Cactus* is created if it doesn't already exist. If you perform a checkout on an already existing and locally modified copy of the module, CVS will try to merge the files with your local copy.

Updating a file or directory

Assuming that you have a file *./foobar* in your checked out copy, you may perform a

```
cvs status ./foobar
```

to inform yourself about the necessary updates, etc. To update the file issue

```
cvs update ./foobar
```

If that was file was locally modified, CVS will try to merge the changes. Manual merging might be necessary and will be indicated by a CVS warning.

Updating a directory

To recursively update the current directory and all subdirectories, type

```
cvsv update .
```

To update a directory `./mysources`, type

```
cvsv update ./path/to/mysources
```

Committing a changed file

To commit changes you have applied to your local copy, your file must be in sync with the repository: your changes must be done to the latest version, otherwise CVS will instruct you to perform an **update** first. To commit changes made to a file `./foobar`, type

```
cvsv commit -m "Reason for the change" ./foobar
```

You may specify several files to commit.

Adding and committing a new file

Adding a new file to the repository is a two fold procedure you first schedule the file for addition, then you commit it:

```
cvsv add ./newfoo
```

```
cvsv commit -m "new few message" ./newfoo
```

Creating a new thorn

To add a new **module** (e.g. an arrangement) to a Cactus repository we first have to create a directory for you with the right permissions. Please contact `cactus@cactuscode.org` providing the name of the requested module, and who should be able to commit changes to the module.

To add the new module, change directory so that you are in the first directory that you want to commit to the repository. (e.g. if you want to commit a new arrangement called **MyArrange** then change directory to **MyArrange**). Then type

```
cvsv -d :pserver:your_login@cvsv.cactuscode.org:<repository name> import module_name start V1
```

(where **start** and **V1** are the vendor and release tags, which you could change to something different).

To add a new **directory** `<new dir>` to an existing module (that you have write permissions for), either add the directory using

```
cvsv add <new dir>
```

and then recursing down adding all the new files and directories contained inside, or import the directory by changing directory to sit inside it, and then using

```
cvsv -d :pserver:your_login@cvsv.cactuscode.org:<repository name> import <relative name> start V1
```

Where `<relative name>` means the position of the directory within the module. (For example, if you have a module called *AMod* which contains a directory *BMod*, and you want to add *CMod* inside *BMod*, then change directory to *BMod*, and use *AMod/BMod* for the *relative name*).

F6.4 Checking out flesh and thorns with CVS

Login

Prior to any CVS operation, you need to log into the Cactus repository. For an anonymous checkout, type:

```
cvsv -d :pserver:cvsv_anon@cvsv.cactuscode.org:/cactus login
```

You will be prompted for a password which is **anon**.

Checkout

To obtain a fresh copy of Cactus, move to a directory which does not contain a previously checked out version, and type

```
cvsv -d :pserver:cvsv_anon@cvsv.cactuscode.org:/cactus checkout Cactus
```

The CVS checkout procedure will create a directory called **Cactus** and install the code inside this directory. From now on we will reference all directory names relative to **Cactus**.

If you want to compile Cactus with thorns, you now need to checkout separately the required arrangement (or *arrangements*) into the **arrangements** directory. To see the available Cactus arrangements and thorns type

```
cvs -d :pserver:cvs_anon@cvs.cactuscode.org:/cactus checkout -s
```

To check out an arrangement or thorn go to the arrangements directory, `cd arrangements`, and for an arrangement type

```
cvs checkout <arrangement_name>
```

or for just one thorn

```
cvs checkout <arrangement_name/thorn_name>
```

To simplify this procedure you may use `gmake checkout` in the Cactus home directory which provides menus to pick arrangements and thorns from.

Update

To update an existing Cactus checkout (to patch in possible changes, etc.), do the following *within* the **Cactus** directory.

```
cvs update
```

The update process will operate recursively downwards from your current position within the Cactus tree. To update only on certain directories, change into these directories and issue the update command.

CVS status

To obtain a status report on the “age” of your Cactus or arrangement routines (from your current directory position downward), type

```
cvs status
```

non-anonymous CVS

If you have an account at the central repository (`cvs.cactuscode.org`) and would like to perform any of the operation above *non-anonymously*, replace `cvs_anon` by your login name and provide the appropriate password during the CVS login process. Depending on your permissions, you may then make commits to Cactus or its arrangements.

Commits

You need to perform a personalized login and have proper permissions to commit code to the repository.

Chapter F7

Using TAGS

Finding your way around in the Cactus structure can be pretty difficult to handle. To make life easier there is support for TAGS, which lets you browse the code easily from within Emacs/XEmacs or vi. A TAGS database can be generated with gmake:

F7.1 TAGS with emacs

`gmake TAGS` will create a database for a routine reference table to be used within Emacs. This database can be accessed within emacs if you add either of the following lines to your `.emacs` file:

```
(setq tags-file-name "CACTUS_HOME/TAGS") XOR
(setq tag-table-alist '(("CACTUS_HOME" . "CACTUS_HOME/TAGS")))
```

where `CACTUS_HOME` is your Cactus directory.

You can now easily navigate your Cactus Flesh and Toolkits by searching for functions or “tags”:

1. **Alt.** will find a tag
2. **Alt**, will find the next matching tag
3. **Alt*** will go back to the last matched tag

If you add the following lines to your `.emacs` file, the files found with TAGS will opened in *read-only* mode:

```
(defun find-tag-readonly (&rest a)
  (interactive)
  (call-interactively 'find-tag a)
  (if (eq nil buffer-read-only) (setq buffer-read-only t)) )

(defun find-tag-readonly-next (&rest a)
  (interactive)
  (call-interactively 'tags-loop-continue a)
  (if (eq nil buffer-read-only) (setq buffer-read-only t)) )

(global-set-key [(control meta \.)] 'find-tag-readonly)
(global-set-key [(control meta \,)] 'find-tag-readonly-next)
```

The Key strokes to use when you want to browse in read-only mode are:

1. CTRL Alt. will find a tag and open the file in read-only mode
2. CTRL Alt, will find the next matching tag in read-only mode

F7.2 TAGS with vi

The commands available are highly dependent upon the version of **vi**, but the following is a selection of commands which may work.

1. **vi -t tag** Start vi and position the cursor at the file and line where 'tag' is defined.
2. **Control-]** Find the tag under the cursor.
3. **:ta tag** Find a tag.
4. **:tnext** Find the next matching tag.
5. **:pop** Return to previous location before jump to tag.
6. **Control-T** Return to previous location before jump to tag (not widely implemented).

Note: Currently some of the CCKT.FILEVERSION() macros at the top of every source file don't have a trailing semicolon, which confuses the etags and ctags programs, so TAGS does not find the first subroutine in any file with this problem.

Chapter F8

ThornLists

This section still needs to be written.